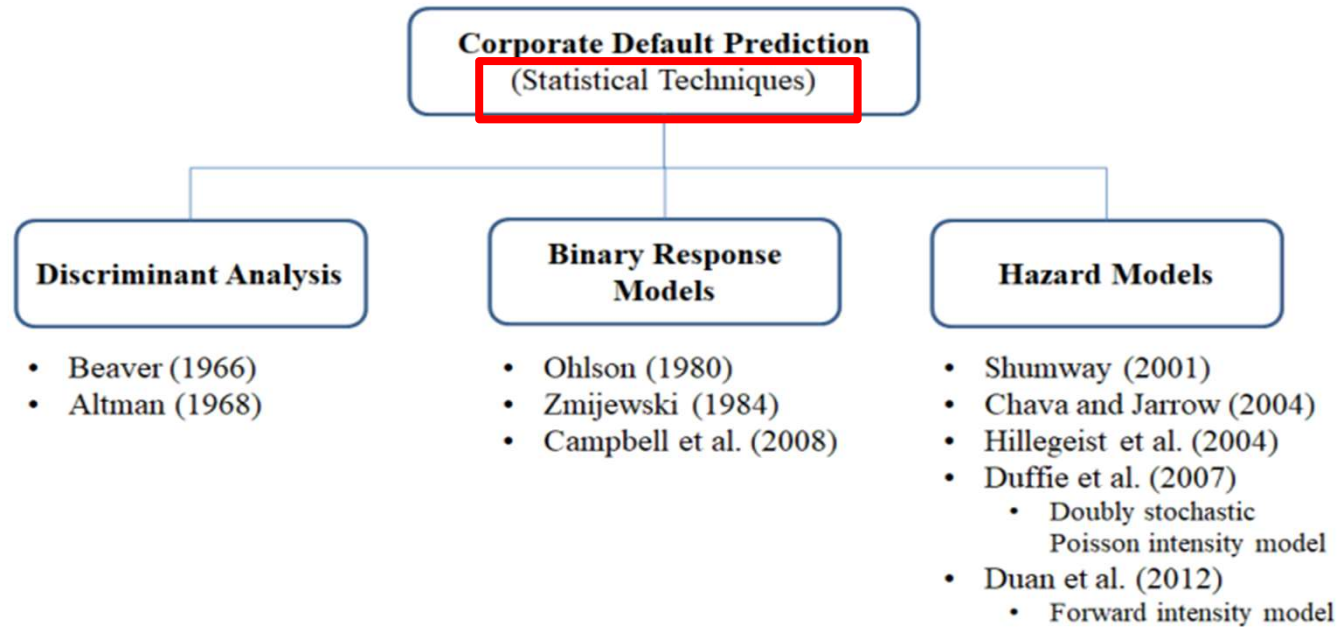


빅데이터를 활용한 디지털금융전문가 과정

부실예측모형연구 3-2

(Machine Learning)

통계적 모형 기업부도예측



Cox-PH Hazard

공변량의 특성에 따른
생존기간 예측 모형

로지스틱 회귀분석 (Logit)

金融工學研究, 제18권 제3호 (2019. 9.) 131 ~ 152 내용 수정

1. W.H. Beaver 판별분석(1966)

W.H. Beaver 예측모형(1966)

- **단변량분석**에 의해 부도기업과 비부도기업 비교 분석 → 일변량분석
- Beaver의 연구는 **최초로 재무비율을 이용하여** 기업 부도를 예측했다는 점에서 의의
- (한계) 예측모델은 단일변수로만 분석했다는 면에서 한계가 있다. 단일변량예측이 어느 정도의 예측력을 지닐 수 있는가에 대한 의문 제기

단일변량분석 한계

- 1) 변수별로 상반되거나 모순된 예측결과를 얻을 가능성이 있다는 점
- 2) 판별점을 선택하는데 있어서 대칭적인 오류비용, 즉 1종오류 비용과 2종오류비용이 동일하다는 가정을 하고 있다.
- 3) 도산기업과 비도산기업의 특성을 결정짓는데 다수의 변수가 복합적으로 작용할 가능성이 높음에도 불구하고 이점을 간과하고 있다.

비슷한 규모의 비도산기업을 동수 **matched-paired sampling**을 적용

→ 도산기업과 비도산기업의 회귀변수 비교.

1. W.H. Beaver 판별분석(1966)

단변량 부실예측 방법 이해 해

- 부실예측모형에 한 개의 변수 만을 포함시켜 예측하는 것
 - 1) 여러 재무비율 중에서 부실 예측을 잘 설명하는 특정 재무비율을 찾아야 함
 - 2) 특정된 재무비율(변수)의 어느 수준을 기준(최적 분류기준점)으로 하여 부실기업을 예측 할 것인가를 결정
- 두 집단 (정상기업, 부실기업) 간 평균 차이가 0과 의미 있는 차이가 있는지에 대한 통계적 분석 → t 검증

$$(\bar{X}_N - \bar{X}_F) \pm t_{\alpha/2} \cdot \sqrt{\frac{S_F^2}{N_F} + \frac{S_N^2}{N_N}} \quad \text{----- (1)}$$

여기서, $\bar{X}_N$: 정상기업군의 재무비율 X 의 평균
 $\bar{X}_F$: 도산기업군의 재무비율 X 의 평균
 t : t통계치(유의수준 $\alpha\%$, 자유도가 $(N_F + N_N - 2)$)
 S_N : 정상기업군의 재무비율 X 의 표준편차
 S_F : 도산기업군의 재무비율 X 의 표준편차
 N_N : 정상기업군의 표본 수
 N_F : 도산기업군의 표본 수

유의수준 : 잘못 판단 할 확률, (1-신뢰수준) → 통상 0.05

- 신뢰구간내에 0 값이 포함 된 경우는 집단 간 평균 차이가 있지 않으므로 해당 변수는 부실예측에 적합하지 않음을 의미
- 신뢰구간 내에 0 값이 포함되어 있지 않은 경우는 변수에 대한 평균 차이가 통계적으로 유의하므로 부실예측 변수로 적절

1. W.H. Beaver 판별분석(1966)

*이원분류법: 최적 분류기준점 결정

정상기업집단과 부실기업 집단을 2개 집단(이원)분류하기 위하여 예측오류를 최소화하는 절사점을 정하는 방법
(수행 절차)

- 1) 기업별 특정 재무비율에 대하여 크기 순으로 배열한다.
- 2) 각 중간점에 대하여 예측오류 계산 : 예측오류(제1종 오류, 제2종 오류)
- 3) **예측오류가 최소가 되는 중간점을 최적분류기준점으로 결정**

이원분류법 사례

기업명	이자보상배율	실제 결과	기업명	이자보상배율	실제 결과
A	10.5	정상기업	G	2.5	정상기업
B	8.5	정상기업	H	2.0	정상기업
C	7.0	정상기업	I	1.5	정상기업
D	5.5	정상기업	J	1.0	부실기업
E	4.5	부실기업	K	0.5	부실기업
F	3.0	부실기업	L	0.25	부실기업

- 제1종 오류 : 부실기업(실제)인데 정상기업(예측)으로 분류하는 오류
- 제2종 오류 : 정상기업(실제)인데 부실기업(예측)으로 분류하는 오류

1. 최적분류점 4.5와 3.0 중간점 : 3.75로 정한 경우.
→ 제1종 오류 : 1개(E), 제2종 오류 : 3개
2. 최적분류점 5.5와 4.5 중간점 : 5로 정한 경우.
→ 제1종 오류 : 0개, 제2종 오류 : 3개
3. **최적분류점 1.5와 1.0 중간점 : 1.25로 정한 경우.**
→ **1종 오류 : 2개, 2종 오류 : 0개 (오류합계 최소)**

2. Altman(1968) MDA : Z –SCORE

ALTMAN, E. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." Journal of Finance (September 1968)

- 재무비율을 활용한 다변량 판별분석(MDA) 방법
- 2개 이상의 모집단에서 추출된 자료가 어느 그룹으로 분류되는지를 판별하는 방법
- (부도기업 정의) 부도의 범위를 파산으로 한정
- (표본)
- **부실기업군**: 1946년~ 1965년 사이에 파산신청을 낸 **제조업체**를 모집단으로 하여, 이 가운데 일정 자산규모를 갖춘 33개 기업 선정
- **건전기업군**: 1966년까지 존속하고 있는 기업 중 **부실기업군에 대응하는 산업별, 규모별로 층화임의추출(Stratified Random Sampling)**하여 **부실기업에 속한 연도가 같은 자료를 수집.**
- (feature) **22개의 비율을** 선택(후보변수) 하여 **MDA를 통하여 기업부실 예측력이 가장 높은 5개의 최종 변수**→ 판별 함수 도출

$$Z = 0.012X_1 + 0.014X_2 + 0.033X_3 + 0.006X_4 + 0.999X_5$$

Z: 부도가능성을 나타내는 종합점수

X_1 : 운전자본/총자산
 X_2 : 이익잉여금/총자산
 X_3 : 영업이익/총자산
 X_4 : 자기자본/총부채
 X_5 : 매출액/총자산

상장기업 대상

- (a) $z < 1.81$: 부실기업으로 판정
- (b) $1.81 \leq z \leq 2.99$: 판정 유보
- (c) $z > 2.99$: 건전기업으로 판정

예측력 테스트 결과

부도 1년전에는 95%, 2년전에는 72%, 3년전에는 48%, 4년전에는 29%, 5년전에는 36%의 정확성을 보였다.

예측력 테스트 결과 부도 2년전까지는 높은 정확성을 보이다가 3년전부터는 예측력이 크게 떨어지고 있다.

2. Altman(1968) MDA : Z -SCORE

분석대상기업이 비상장기업인 경우

: 자기자본 시장가치를 구할 수 없으므로 x_4 를 계산할 때 자기자본 장부가치를 사용

: 비상기업에 대하여는 Altman이 x_4 를 『자기자본 장부가치/부채 장부가치』로 측정

$$Z' = 0.717X_1 + 0.847 X_2 + 3.107 X_3 + 0.420 X_4 + 0.998 X_5$$

비상장기업

(a) $Z' < 1.23$: 부실기업으로 판정

(b) $1.23 \leq Z' \leq 2.90$: 판정 유보

(c) $Z' > 2.90$: 건전기업으로 판정

상장기업

(a) $z < 1.81$: 부실기업으로 판정

(b) $1.81 \leq z \leq 2.99$: 판정 유보

(c) $z > 2.99$: 건전기업으로 판정

Altman : 한국기업 대상 (1996) , K1 / K2 모형

- (대상기업)
- 1989~1992년 : 부실화된 34개 기업과 61개 건전기업간 비교 (한국기업 대상)
- (feature)
- **20개의** 재무변수들을 검토 후 → 최종 기업규모, 총자산회전율, 누적수익성, 재무구조 → **4개 최종 변수 선정**

- **K1-score 모형 (상장기업+ 비상장기업 적용 모형)** : X_4 자기자본 장부가치 사용
- **K2-score 모형 (상장기업 적용 모형)** : X_4 자기자본을 시장가격으로 사용 → 『자기자본 시장가격/총부채』를 각각 계산

$$K1 - score = -17.862 + 1.472X_1 + 3.041X_2 + 14.389X_3 + 1.516X_4$$

$$K2 - score = -18.696 + 1.501X_1 + 2.706X_2 + 19.760X_3 + 1.146X'_4$$

X_1 : log총자산(단위 사용)

X_2 : log(매출액/총자산)

X_3 : 이익잉여금/총자산)

X_4 : 자기자본 장부가치/총부채 (**K1모형**)

X'_4 : 자기자본 시장가격/총부채 (**K2 모형**)

K1모형 부도 판별

- $K1 < -2.0$: 부실가능성 심각
- $-2.0 \leq K1 \leq 0.75$: 판정보류
- $K1 > 0.75$: 건전판정

K2모형 부도 판별

- $K2 < -2.30$: 부실가능성 심각
- $-2.30 \leq K2 \leq 0.75$: 판정보류
- $K2 > 0.75$: 건전판정

K1-score 모형 : 도산예측 정확도는 도산 1년 전에는 97.1%, 2년 전에는 88.2%로 나타내고 있으며 , 5년 전으로 돌아가면 68.8%로 떨어진다.

K2-score 모형 : 경우: 도산 1년 전에 96.6%, 2년 전에는 85.2%, 그리고 5년 전에는 75.0%로 나타나고 있다.

* 다변량 판별분석 (MDA : Multiple Discriminant Analysis)

판별분석 모형설계

$$Z = W_1X_1 + W_2X_2 + W_3X_3 + \dots + W_nX_n$$

판별점수(종속변수 Z), 판별상수(절편), 판별변수(설명변수 X1), 판별계수(W1 회귀계수)

1. 판별변수 (discriminant variable)

상관관계가 높은 두 독립변수를 선택하는 것보다는 두 독립변수 중 하나를 판별변수로 선택하고, 그것과 상관관계가 적은 독립변수를 선택하는 게 효과적 → **적절한 재무비율만 추출** (상관계수 비교, 비모수 검정 등 다양한 방법)

2. 판별함수 (discriminant function)

판별변수들의 선형조합으로 '집단의 수 - 1' (집단 수 : 정상, 부도, 보류) 과 독립변수의 수 중 작은 값만큼 도출
목적은 종속변수의 집단을 정확하게 분류할 수 있는 예측력을 높이는 데 있다. 회귀계수 값

3. 유의성 검증 : Wilks' Lambda, 카이제곱, 유의확률. p-value (0.05 비교)

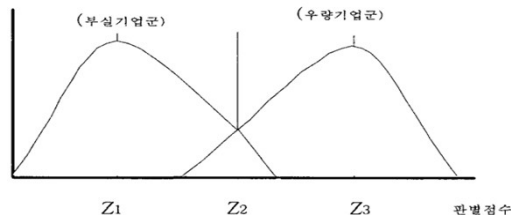
4. 판별점수 (discriminant score)

어떤 대상이 어떤 집단에 속하는지 판별하기 위하여 그 대상의 판별변수들의 값을 판별함수에 대입하여 구한 값

5. Cutoff -point* 도출 → (정상기업/부도기업 판단기준)

각 기업에 대해 계산된 판별점수를 절사점(cut-off point)과 비교 → 어떤 집단에 속하는지 결정.

6. 판별분석 결과 검증 (acc, F1)



- 부실기업인데 정상기업으로 판단하는 경우 (제1종 오류)
- 정상기업이지만 부실기업으로 판단하는 (제2종 오류),
- 오류 합을 최소화하는 회귀계수(판별계수)를 추정하는 과정 진행

판별함수 한계점 : 판별변수의 정규화 분포 가정

* 다변량 판별분석 (MDA : Multiple Discriminant Analysis)

최적절사점(Cutoff -point)= 판별기준점 도출

1. 판별함수(Z)의 집단별 정상기업 판별점수 평균과 부도기업 판별점수 평균을 각각 계산

- 1) 만약 **정상기업 판별점수 평균 0.5**, **부도기업 판별점수 평균 -0.5** 인 경우로 가정
- 2) 정상기업 표본수 500개, 부도기업 표본수 500개 가정

최적절사점(Cutoff -point*)

$$\begin{aligned} &= [(\text{부도기업 표본수} \times 0.5) + (\text{정상기업 표본수} \times -0.5)] / (\text{정상기업 표본수} + \text{부도기업 표본수}) \\ &= [(500 \times 0.5) + (500 \times -0.5)] / (500 + 500) = 0 \end{aligned}$$

결국 판별점수의 값은 0 이 된다.

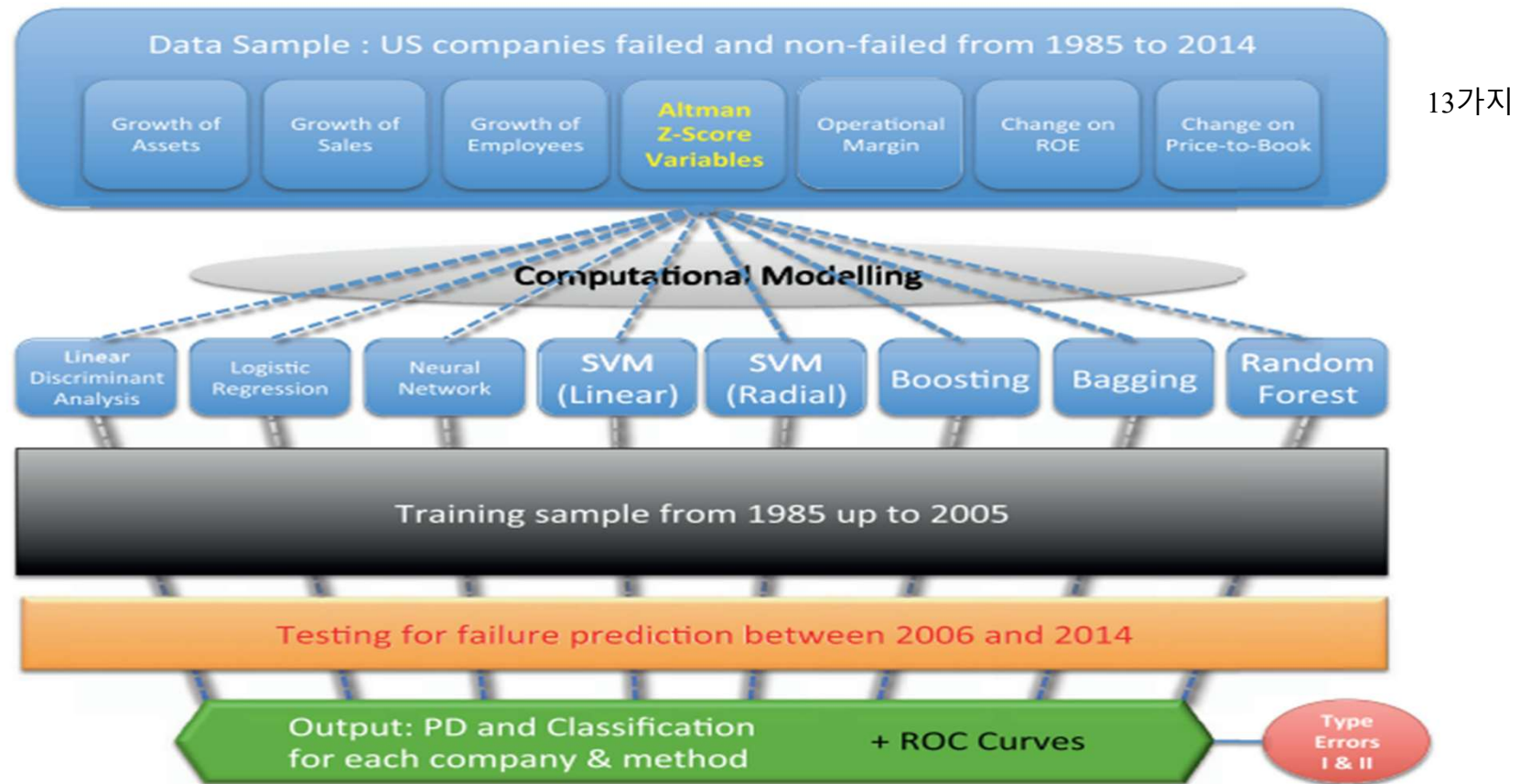
→ 새로운 기업이 주어지면 기업의 **판별점수(Z) 값이 0** 보다 크면 정상기업, 0보다 작으면 부도기업으로 판별한다.

최적절사점 (Cutoff -point)

$$= \frac{(\text{부도기업수} \times \text{정상기업수 판별점수 평균}) + (\text{정상기업수} \times \text{부도기업 판별점수 평균})}{(\text{정상기업수} + \text{부도기업수})}$$

Altman(2017)

Machine learning models and bankruptcy prediction Flavio Barboza*, Herbert Kimura b , Edward Altman



Altman + Beneish M-score (2017)

- Using Altman Z-score and Beneish M-score Models to Detect Financial Fraud and Corporate Failure:
- <https://developers.refinitiv.com/en/article-catalog/article/Beneish-M-Score-and-Altman-Z-Score-for-analyzing-stock-returns-of-the-companies-listed-in-the-SP500>
- <https://github.com/Roobiyakhan/Bankruptcy-Prediction-using-Machine-Learning-Algorithms-in-Python>
- Bankruptcy forecasting: An empirical comparison of AdaBoost and neural networks ☆ Esteban Alfaro a,*, Noelia García a , Matías Gámez a , David Elizondo 2008

3. 이항반응모형 (Binary Response Models)

1) 의의

기업의 상태를 정상(=0)과 부도(=1)로 나누고 부도사건이 발생할 확률을 설명변수를 사용하여 추정하는 모형

→ 확률값으로 반환 됨.

2) 사례

Ohlson(1980) 이 최초로 로짓분석을 이용해 도산예측에 관한 연구 Ohlson O-score가 대표적

구분	분류기법
통계기반	로지스틱 회귀, 나이브 베이지안
트리 기반	Decision Tree
비선형 함수기반	SVM
D.L	RNN, DNN, CNN

3. 이항반응모형 (Binary Response Models)

로지스틱 회귀분석 (Logistic Regression Analysis)

- 실패의 사전확률(prior probability)이나 예측변수(predictor variable)의 분포에 대한 가정을 필요로 하지 않는다.
- 개별 독립변수의 유의성을 검정할 수 있고, 다음 기간의 부도확률을 계산 가능

다음 기간 동안 부도가 발생할 확률

$$P_n(y_n = 1) = \frac{1}{(1 + e^{-Z})}$$

$$= \frac{1}{\{1 + \exp[-(\beta_0 + \beta_1 X_{1,n} + \beta_2 X_{2,n} + \dots + \beta_m X_{m,n})]\}}$$

Logit분석은 판별분석에 비해 관련변수들이 정규분포이어야 한다는 가정이 전제될 필요가 없으며, 다만 선택확률이 logistic 함수를 취한다는 가정이 필요하다

3.로지스틱 회귀분석 (Logistic Regression Analysis)

로지스틱 회귀분석은 현재 금융권의 평가 모형 개발 시 보편적으로 활용되는 모형

→ 모형 개발 과정 및 결과에 대한 이해가 용이하고 모형 개발시 변별력이 높은 것이 장점임.

$$D = \beta_0 + \beta_1 \text{Age} + \beta_2 \text{Income} + \beta_3 \text{Employed}$$

로지스틱 회귀분석에서는 로짓Logit 변환 → 부도확률이 0과 1사이의 값을 가져야 하므로

$$P(D = 1 | \text{Age, Income, Employed}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{Age} + \beta_2 \text{Income} + \beta_3 \text{Employed})}}$$

확률(P) 0과 1 사이로 제한

일반 회귀식과 다르게 피설명변수(여기서는 P(D)에 설명변수 (x_i)가 미치는 영향이 일정하지 않기 때문이다.
 x_i 를 1에서 한 단위 증가할때와, 100에서 한 단위 증가할 때, P(D)가 증가하는 수준이 일정하지 않다는 것을 의미!!!

→ 비선형함수이므로 이를 선형으로 변환이 필요

개별 변수가 미치는 일관된 영향 (β_j)를 살펴보고자 하는 것이 Odds에 Logit을 감안

$$\ln \left(\frac{P(D = 1 | x_1, \dots, x_N)}{P(D = 0 | x_1, \dots, x_N)} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_N x_N$$

Odds (Ratio)

미국의 라스베가스에서는 포커에서 내가 이길 확률을 계산할 때 주로 거론되는 용어

Odds = P / (1-P) =(사건이 발생확률)/(사건 발생안 할 확률)

$$\frac{P(D = 1 | x_1, \dots, x_N)}{P(D = 0 | x_1, \dots, x_N)} = e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_N x_N)}$$

Wald신뢰구간 안에 Odds값이 1을 포함하고 있으면 통계적 유의성이 없는 것이고, 1에서 벗어나면 유의성이 있다고 해석 가능

3.로지스틱 회귀분석 (Logistic Regression Analysis)

1. 회귀계수 추정 Maximum Likelihood Estimation
2. 회귀계수 통계적 유의성 검정 : Wald-test()
3. 변수 제거 : 후진 제거법 (backward elimination), 전진, 스텝 등 → AIC 낮은 값
4. test data 이용한 모델 성능 평가 : ROC curve, AUC

1.회귀계수 추정 방법: Maximum Likelihood Estimation

Maximum Likelihood Estimation

- 어떤 확률변수에서 표집한 값들을 토대로 그 확률변수의 모수를 구하는 방법
- 어떤 모수가 주어졌을 때, 원하는 값들이 나올 가능성을 최대로 만드는 모수를 선택하는 방법

동전을 던질 때 앞면이 나올 확률을 추측하는 것

특정 사람 i에 대해서 우리가 샘플에서 나온 결과를 관찰할 가능성 Likelihood

전체 사람 n명에 대해서 우리가 샘플에서 나온 결과를 관찰할 가능성 Likelihood

$$P(D = 1 | x_{1i}, \dots, x_{Ni})^{D_i} (1 - P(D = 1 | x_{1i}, \dots, x_{Ni}))^{1-D_i}$$

$$\prod_{i=1}^n P(D = 1 | x_{1i}, \dots, x_{Ni})^{D_i} (1 - P(D = 1 | x_{1i}, \dots, x_{Ni}))^{1-D_i}$$

→ 식을 최대화하는 $\beta_1, \beta_2, \dots, \beta_N$ 을 찾는 것이 : Maximum Likelihood Estimation

$$\ln \left(\frac{P(D = 1 | x_1, \dots, x_N)}{P(D = 0 | x_1, \dots, x_N)} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_N x_N$$

https://ko.wikipedia.org/wiki/%EB%A1%9C%EC%A7%80%EC%8A%A4%ED%8B%B1_%ED%9A%8C%EA%B7%80

3.로지스틱 회귀분석 (Logistic Regression Analysis)

2. 회귀계수 통계적 유의성 검정

Wald-test() → p 0.05 비교

Wald유의도, $0 < \text{우도비율지수} < 1$

3. 변수 선정

➤ 모델링 이전에 불필요한 변수 필터링

→ 종속변수(D = 0, 1) 와 피어슨 상관관계를 측정하여 불필요한 변수는 모델링 과정 전에 쳐 낸다.

모델링 과정에서 변수 선정

➤ 로지스틱 분석에서 독립변수를 줄여가며 모델의 성능을 향상시키는 방식인 Backward stepwise selection 방법에 의해 변수를 선택

➤ 로지스틱은 비선형모델로서 다중공선성 문제를 고려하지 않는 모형

$$|\rho_p| > 0.50.$$

➤ 로지스틱 분석의 경우 정상기업, 부도기업 데이터의 수가 각각 설명변수 수의 최소 10배 이상이 되어야 다중 로지스틱 회귀분석을 이용하는 것이 적절하다고 알려 짐.

3.로지스틱 회귀분석 (Logistic Regression Analysis)

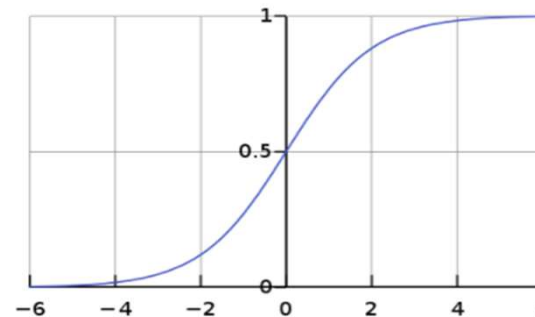
4. 모형 설계 및 예측력 검증

- 최종 변수 선택 결과 로짓분석 모형(Wald 통계량, 유의확률, 로짓함수 계수)
- 제1종 오류, 제2종 오류 체크
- 도산 1년전, 도산2년전, 도산 3년전
- 표본의 상대비율(정상기업의 수와 부도기업의 수)을 달리하면 변한다!!!!
- 분류기준점 (cut-off point) 을 0.1, 0.2, 0.3.....0.9 등 달리하면 1종정확도, 2종정확도 달라진다
- Ohlson(1980)은 제1종 오류와 제2종 오류의 합을 최소화 시키는 분류기준점 (cut-off point) 0.038을 제시함.

로지스틱 함수 그래프

역 S자 그래프

→설명변수가 한 개인 경우 해당 회귀 계수의 부호가 0보다 작을 때 표현되는 그래프



3.로지스틱 회귀분석 (Logistic Regression Analysis)

기업부도예측모형 분석 시 : 정상기업 수 > 부도기업 수

Classification Threshold (임계값, 기준치)

로지스틱 회귀 값을 이진 카테고리에 매핑(Mapping)하려면 **분류 임계값(Classification Threshold, 결정 임계값)**을 정의
→ 값의 범위 : 0~1 사이 갖는다.

→ 로지스틱 회귀 알고리즘의 결과 값은 '분류 확률'이고, 그래서 이 확률이 특정 수준 이상 확보되면 훈련데이터가 해당 집단에 포함될지 여부를 결정

→ 대부분의 알고리즘에서 기본 임계값은 **0.5**

→ (그러나 **기업수가 불균형(정상기업 수 와 부도기업 수)인 경우 세분화하여 결과 확인 필요**)

- 임계값 (Threshold) 가 1이면

- 확률이 1일때만 True 라고 예측하기 때문에 FP (실제값 False, 예측값 True 경우) 가 0, fall out = 0

- 임계값 (Threshold) 가 0이면

- 확률이 모두 True 라고 예측하기 때문에 TN (실제값 False, 예측값 False 경우) 가 0, fall out = 1

→ 따라서, 임계값을 1~0 에서 거꾸로 조절이 필요하게 됨.

→ 일반적으로 Threshold 기준을 [0.025, 0.05, 0.075, ..., 0.3]와 같이 12가지로 세분화 하여 분석 결과 확인

* 프로빗 모형(Probit Model과 로짓 모형(Logit Model)의 차이점

프로빗 모형

선택 확률 p 가 $[0,1]$ 의 구간에 놓이도록 하기 위해서는 설명변수와 p 사이 에 선형이 아닌 비선형의 관계가 요구됨
프로빗모형(probit model)은 확률분포가 normal distribution을 따른다는 가정에 기초를 한다.

→ 이론적으로는 타당성을 인정받고 있지만 선택가능한 대안이 3개 이상인 다항 프로빗모형(multinomial probit model)의 경우 계산의 어려움(computational difficulties)

→ 프로빗 모형의 추정 - 최우추정법(주어진 표본의 값이 관측될 확률 또는 우도를 극대화하는 값)

로짓 모형

로짓모형(logit model)은 프로빗모형의 약점을 보완하기 위해서 개발된 것으로 확률분포가 독립적이고 동일하게 분포되어(independently and identically distributed)있다고 가정

→ 와이블 분포(베이불 분포)를 가정

이 분포는 모양이 정규분포와 비슷하면서 계산이 편리하여 가장 널리 활용

프로빗 모형의 대안으로 흔히 사용됨 → 유일한 차이점은 사용되는 확률분포함수 로지스틱(logistic) 분포의 확률분포함수가 사용됨

→ 프로빗 모형에서와 마찬가지로 방법으로 최우추정량을 구할 수 있으며, 그 결과에 대한 해석 역시 정규분포 확률밀도함수를 로지스틱 확률밀도함수로 대체하면 마찬가지임

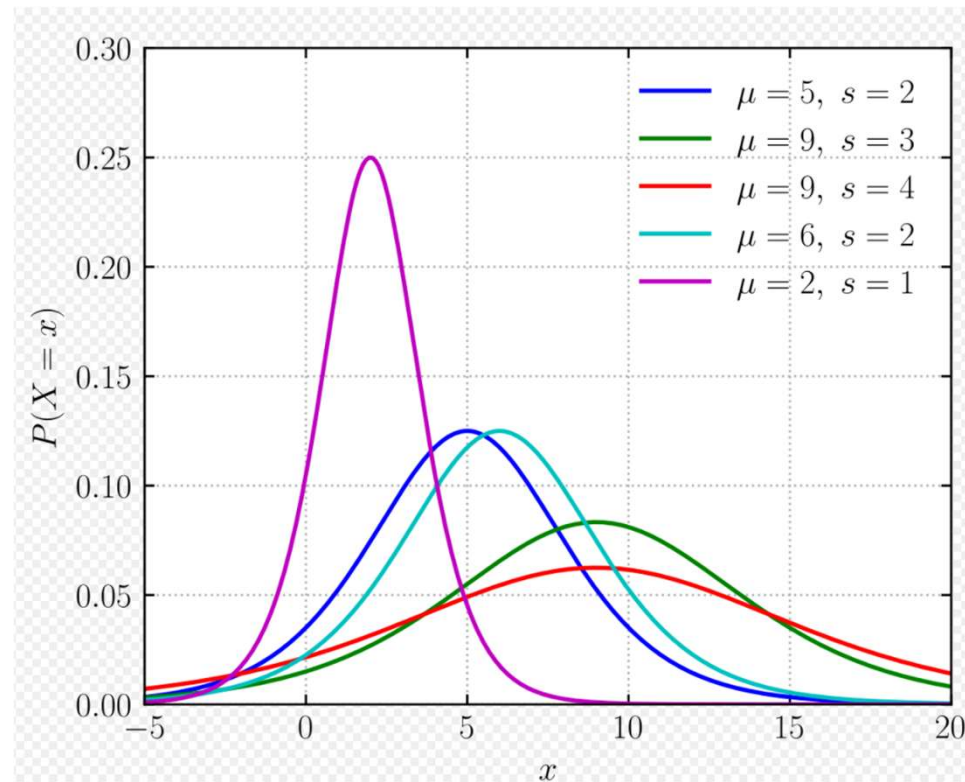
Weibull distribution

- 가장 많이 쓰이는 확률로 이론적으로 가장 설득력이 있는 정규분포와 비슷한 모양의 확률밀도함수(probability density function)를 가지면서도 동시에 계산이 편리하기 때문
- 지수분포를 보다 일반화시켜, 여러 다양한 확률분포 형태를 모두 나타낼 수 있도록 고안됨
- 로짓모형은 확률적 효용이 와이블분포임을 가정하는 확률선택모형
- 와이블 분포와 다른 분포와의 관계 : $\beta = 3.5$: 정규분포에 근사적

** 로지스틱 분포

로지스틱 분포(logistic distribution)

누적분포함수가 로지스틱 함수가 되는 확률분포

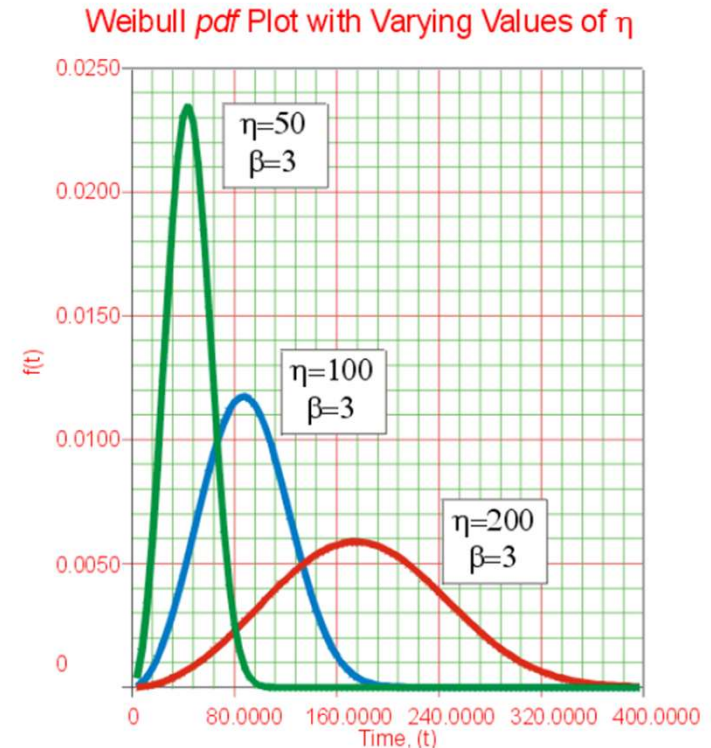


https://en.wikipedia.org/wiki/Logistic_distribution

*** 베이불 분포

베이불 분포(Weibull distribution)

- 연속 확률 분포
- 발로디 베이불(스웨덴어: Waloddi Weibull)의 이름에서 유래
- 데이터 셋을 모형화하는데 충분히 유연하기(flexible) 때문에 와이불분포를 활용
- 데이터가 우측으로 치우쳤던, 좌측으로 치우쳤던 와이불분포로 적합시킬 수 있음
- → 척도모수(scale parameter)와 형상모수(shape parameter)에 따라 분포의 모양 변함.
- 척도모수(Scale Parameter): lambda는 분포의 축소 및 확장을 의미하는 스케일 모수
- 형상모수(Shape Parameter): beta는 와이불 확률분포의 형태를 결정짓는 확률분포에서 확률분포의 기울기
- 형상모수가 1인 경우 지수 분포(exponential distribution)
- 형상모수가 2인 경우 라이레히 분포(Rayleigh distribution)



$$f(t) = \frac{dF(t)}{dt} = \frac{\beta}{\eta^\beta} t^{\beta-1} \exp\left[-\left(\frac{t}{\eta}\right)^\beta\right]$$

$$\mu = \eta \Gamma\left(1 + \frac{1}{\beta}\right)$$

4. Merton(1974) : 부도거리 모형(distance to default, DD 모형)

시장모형(market-based model)

Merton(1974)의 부도거리 모형(distance to default, DD 모형)

→ DD 모형에서는 만기에 기업이 상환해야 할 부채가 자산보다 큰 경우에 부도가 발생

Merton(1974)

도산 : 특정시점의 자산가치 < 부채가치

→ 시차 존재하므로 단기부채는 그대로, 장기부채는 일부만 반영 : 부도점 계산

옵션모형 리뷰

$$V^E = V^A \cdot N(d_1) - e^{-r_T T} \cdot V^D \cdot N(d_2)$$

식 (1)

V^E 는 자기자본의 시장가치 = 시가총액

V^A 는 자산의 시장가치

V^D 는 부채의 장부가치

σ_A 는 자산가치의 변동성

T 는 부채의 만기

r_T 는 만기가 T인 무위험이자율

$N(\cdot)$ 는 표준누적정규분포 값

$$d_1 = \frac{\log\left(\frac{V^A}{V^D}\right) + \left(r + \frac{\sigma_A^2}{2}\right) \cdot T}{\sigma_A \cdot \sqrt{T}}$$

$$d_2 = \frac{\log\left(\frac{V^A}{V^D}\right) + \left(r - \frac{\sigma_A^2}{2}\right) \cdot T}{\sigma_A \cdot \sqrt{T}} = d_1 - \sigma_A \cdot \sqrt{T}$$

4. Merton(1974) : 부도거리 모형(distance to default, DD 모형)

시장모형(market-based model)

1. 부도점 (도산지점, 임계점 : DP)

$$DP_t = SD_t + 0.5 \cdot LD_t$$

DP : 부도임계점(default point)

SD_t : t일의 단기부채의 양

LD_t : t일의 장기부채의 양

2. 부도거리(DD :distance to default)

$$\frac{(\text{기업가치} - \text{부도점})}{\text{기업가치} \times \text{기업가치 변동성}}$$

- DD는 과거자료에 의해 산출
- Merton (1974)에서 부도확률 d_2

자산가치가 기하적인 브라운 운동을 따를 경우 부도를 예측하고자 하는 시점이 T 일 때,

$$DD = \frac{\ln\left(\frac{V_A}{X}\right) + (\mu - 0.5\sigma_A^2)T}{\sigma_A \sqrt{T}}$$

Vassalou and Xing(2004), Bharath and Shumway(2008)

→ DD 계산을 위하여 자산의 시장가치 (V_A)와 자산의 변동성 (σ_A)을 반복갱신법 (iterative method*)으로 추정하여 DD값을 계산

3. 부도기대확률(EDF :expected default frequency)

부도기대확률(EDF :expected default frequency)= 예상부도확률 : 기업의 부도가능성을 확률로 표시한 것

→ 값이 클수록 부도가능성이 높은 기업

→ 부도거리와 반비례, $EDF = 1/DD$

1) 역사적 EDF *

2) 정규분포를 가정한 EDF * → Merton (1974)에서 $N(-d_2)$, 결국 $N(-DD)$

DD 모형은 만기에 자산의 가치가 부도 임계점(DP : default point)으로부터 평균적으로 얼마나 떨어져 있는가를 측정!!!
DD 모형은 값이 클수록 기업의 부도 발생가능성이 감소하는 것을 의미하기 때문에 부도위험을 측정할 수 있는 지표로 활용

4. Merton(1974) : 부도거리 모형(distance to default, DD 모형)

반복갱신법* (iterative method) : Newton- Raphson Method

- 1) DD값을 계산하기 위해 과거 1년 동안의 일별 주가수익률로 주식의 변동성을 구한다.
 - → 이 값을 이용하여 자산변동성(σ_A) 초기값(initial value)을 설정
- 2) 자산변동성(σ_A)을 아래 식에 대입하여 자산의 시장가치(V_A), 내재수익률(implied log return), 자산의 변동성(σ_A), 자산증가율의 기댓값(μ)을 순차적으로 계산
- 3) 이상의 과정은 자산의 시장가치(V_A)가 일정 오차범위(10^{-3}) 내의 값으로 수렴할 때까지 반복 시행

$$V_E = V_A N(d_1) - e^{-r_f T} X N(d_2)$$

$$\sigma_E = \left(\frac{V_A}{V_E} \right) N(d_1) \sigma_A$$

$$d_1 = \frac{\log\left(\frac{V_A}{V_D}\right) + \left(r + \frac{\sigma_A^2}{2}\right) \cdot T}{\sigma_A \cdot \sqrt{T}}$$

$$d_2 = \frac{\log\left(\frac{V_A}{V_D}\right) + \left(r - \frac{\sigma_A^2}{2}\right) \cdot T}{\sigma_A \cdot \sqrt{T}} = d_1 - \sigma_A \cdot \sqrt{T}$$

V_E = 자본의 시장가치, → t 일의 증가 × t일 주식수, 로그수익률 전환

V_A = 기업(총자산)의 시장가치,

r_f = 무위험이자율,

X = 부채의 액면가

$N()$: 표준정규누적 분포,

σ_E : 자본의 변동성

σ_A : 자산의 변동성

T : 만기까지 남은 기간

- σ_E (자본 변동성) ; 구간을 20일에서 250일에 이르기까지 다양하게 변형시켜 가면서 train data(부도기업과 정상기업) 에서 반복적 계산을 통해 변별력을 가질 수 있도록 선정해야 함.

$$\sigma_{E,t} = \sqrt{\frac{252 \cdot \left(\sum_{k=t-251}^t LY_k^2 \right) - \left(\sum_{k=t-251}^t LY_k \right)^2}{251 \cdot 252}}$$

$$LY_t = \ln\left(\frac{V_{E,t}}{V_{E,t-1}}\right)$$

LY_t : t일의 로그수익률

4. Merton(1974) : 부도거리 모형(distance to default, DD 모형)

부도기대확률(EDF)= 예상부도확률 : 기업의 부도가능성을 확률로 표시한 것

- 1) 역사적 EDF * : 검정 → 비모수통계방법 : Kolmogorov- Smirnov Test
- 2) 정규분포를 가정한 EDF * → Merton (1974)에서 $N(-d_2)$ 검정 → 모수적통계 방법 대응표본 t 검정

1) 역사적 EDF(expected default frequency)

$$= \frac{\text{특정 DD를 가진 기업 중 1년 이내에 파산한 기업 수}}{\text{특정 DD를 가진 기업의 전체 모집단}} \times 100$$

(예시1) 예를 들어, 1년 후 자산가치 평균과 도산점 사이의 거리가 2×표준편차 크기로 멀어질 때의 도산확률을 구하기 위해 KMV는 과거 자료를 통해 EDF를 구한다.

$$= \frac{\text{DD=2 가진 기업 중 1년 이내에 파산한 기업 수}}{\text{DD 2 기업의 전체수}} \times 100,$$

분석기간 초에 기업가치가 도산점에서 2×표준편차 만큼 떨어진 기업의 수가 2,000개 인데 이중 실제로 도산한 기업수가 20개이면 실증적 EDF는 1%가 된다.

2) 정규분포 가정한 EDF

$$P_T = P(V_{A,T} \leq V_{D,T} \mid V_{A,t} = V_A)$$

$$= P(\ln V_{A,T} \leq \ln V_{D,T} \mid V_{A,t} = V_A)$$

$$= P\left\{ -\frac{\ln \frac{V_{A,t}}{V_{D,T}} + \left(\mu - \frac{\sigma_A^2}{2}\right) \cdot T}{\sigma_A \cdot \sqrt{T}} \leq w \right\}$$

$$= N(-d_2) = \text{부도확률}$$

$N(-d_2)$ 또는 $1 - N(d_2)$ 는 T시점에 자산가치가 부채가치보다 작을 확률을 의미

(예시2) 자산가치가 정규분포를 따른다고 가정할 경우

DD가 2일 경우 자산가치가 도산점 이하로 줄어들어 도산사태가 발생할 확률은 표준정규분포 확률변수가 2 보다 작은 확률인 2.27%이다. 따라서 정규분포를 이용한 예상도산확률 (EDF) 추정값은 2.27%가 된다.

4. Merton(1974) : 부도거리 모형(distance to default, DD 모형)

$N(-d_2)$ 기업의 도산 가능성을 나타내는 지표로 활용 문제점 ?

- 자본 일부 잠식을 부도로 인식?
- 자산가치가 높다고 부채상환 능력이 높다?
- Wiener's process 과정을 따른다.
- 수준별 도산 빈도를 고려 해야 한다.
- 도산가능성을 $N(-d_2)$ 대신에 예상도산확률을 이용 (행사가격에 부채가치 대신에 부도점으로,
- 자산가치를 부채상환에 사용될 수 있는 부분과 그렇지 않은 부분으로 구분
- 도산가능성 지표로 d_2 대신에 도산거리(DD) 활용
- 표준정규분포 대신에 과거 도산빈도를 활용하여 예상도산확률 계산

논문 list 참조

- 부채구조 문제점 : 고정부채 가중치 0.5??
- 한국 기업 특수성 감안

* 시장 모형 , KMV 모형

- Merton(1974) KMV 모형 : 시장정보인 주가로서 기업의 일정기간 동안의 부도확률을 예측하는 모형
- Merton은 옵션가격 결정모형인 블랙-숄즈-머튼 옵션 가격 모형(Black-Sholes-Merton Option Pricing Model)을 사용 기업의 부채 구조를 분석
- **KMV : Kealhofer , McQuown , and Vasicek** 설립

Merton(1974)

- 시가총액을 이용해 자산 또는 부채의 시장가치를 산출
- 옵션가격결정모형을 이용하면 만기시점에 콜옵션이 행사되지 않을 확률 $\rightarrow$ 만기시점의 자산가치가 부채가치보다 적을 확률을 의미
- 특정시점의 자산가치가 부채가치보다 적을 경우 자산을 모두 매각한다 하더라도 부채를 모두 상환할 수 없기 때문에 기업은 도산상태에 이르게 될 것이다.
- 이러한 확률을 통해 실제 도산가능성을 산출하고자 하는 것이 예상도산확률모형의 기본적인 개념
- 옵션가격결정모형의 $-N(d_2)$ 는 만기시점에 T에 콜옵션이 가치를 가질 확률을 의미
- $N(-d_2)$: T시점에 자산가치가 부채가치보다 작을 확률을 의미하고 이는 기업의 도산 가능성을 나타내는 지표로 활용

* 시장 모형 , KMV 모형

기업의 부도확률을 결정하는 요소

첫째, 기업자산의 시장가치를 나타내는 자산가치(Value of Assets)로 기업의 자산으로부터 산출되는 미래의 현금흐름을 할인한 현재가치이다. 기업의 미래의 전망과 기업이 속한 산업 및 경제와 관련된 정보를 내포한다.

둘째, 자산가치의 불확실성 혹은 위험도를 나타내는 자산의 위험도(Asset Risk)로 기업의 영업활동이 나 산업위험도를 측정

셋째, 기업의 계약상의 부채를 나타내는 레버리지(Leverage)이다. 기업의 자산은 시장가치로 측정되는 반면, 기업이 상환해야 하는 부채는 장부가치로 계산되므로 부채의 장부가치가 기업의 레버리지를 측정하는 적절한 지표

- 기업의 자산가치가 부채의 장부가치에 접근함에 따라 기업의 부도위험은 증가 하며, 자산의 시장가치가 부채를 상환하는 데 충분치 못한 시점에 다다르면 기업 의 부도가 발생한다.
- 기업의 부도가 발생하는 시점에서의 자산가치, 즉 **부도점(default point)**에서의 자산가치는 총부채와 경상 혹은 단기 부채의 중간 정도에 위치하는 것으로 알려져 있다.
- 따라서 기업의 순자산가치(net worth)는 기업자산의 시장가치에서 부도점을 차감한 값이고, 순자산의 시장가치가 '0' 이 될 때 기업의 부도가 발생한다

순자산가치 = [자산의 시장가치] - [부도점]

자산가치, 영업활동의 위험도 및 레버리지 통합하여 순자산의 시장가치를 자산가치의 1표준편차 규모에 비교하는 단일의 부도위험지표를 산출할 수 있다. → **부도거리(distance- to-default)**

$$\frac{(\text{기업가치} - \text{부도점})}{\text{기업가치} \times \text{기업가치 변동성}}$$

* 시장 모형

시뮬레이션을 이용한 예측 D.D.의 산출

KMV모형 부도거리(DD)

자산가치가 부도점 간의 간격을 자산가치의 표준편차를 단위로 측정

1. 자산수익률 변동성 도출

$$\sigma_E = d_1 - \sigma_A^* \sqrt{T}$$

2. KMV 모형을 통하여 부도 확률을 예측하기 위해서는 D.D.(Default to distance)의 산출
→ D.D.를 몬테카를로 시뮬레이션 사용

$$DD = \frac{\ln(V_A/K_T) + (\mu - \sigma_A^2/2)T}{\sigma_A \sqrt{T}}$$

Kt : 부채의 장부가, Va : 자산가치
u : 자산가치 성장률

시뮬레이션을 통하여 주식의 미래 가격 분포를 예측하여 식에 대입 한다면 D.D.의 예측 분포 또한 구할 수 있고, 그에 따라 기업의 EDF를 추정

DD 도출과정 아래 논문 참고 할 것

옵션모형을 이용한 은행부실의 조기경보모형 구축 전선애* . 김봉한**

* 시장 모형

시장정보를 이용한 부도예측

D.D.(Default to distance:부도거리)를 통한 표본 집단 간 부도 확률 비교

각 표본 기업은 추정된 D.D.를 통하여 현재 상황이 지속될 경우 1년 후에 얼마나 부도 확률이 나타나는지 추정할 수 있다.
두 표본 집단의 D.D.를 비교하여 두 집단이 통계적으로 부도 위험성이 차이가 나는지 분석

- DD 모형은 만기에 자산의 가치가 부도 발생점(default point)으로부터 평균적으로 얼마나 떨어져 있는가를 측정
- DD 모형은 값이 클수록 기업의 부도 발생가능성이 감소하는 것을 의미
- 부도위험을 측정할 수 있는 지표로 활용

예측 모형의 유용성 평가

부도기업과 건전기업 D.D.의 평균차이 t-검정

* 시장 모형

EDF(Expected Default Frequency : 예상도산확률)

KMV는 주어진 부도거리로부터 PD를 과거 자료를 이용하여 산출하였고 이를 EDF라 부른다.
 자산가치가 기하적인 브라운 운동을 따를 경우 부도를 예측하고자 하는 시점이 T일 때, 이 시점에서 부채의 장부가를 K_t 라 하면 EDF는 다음과 같다

$$\begin{aligned} EDF &= P[V_T^A \leq K_T | V_0^A = V_A] \\ &= P\left[Z \leq -\frac{\ln(V_A/K_T) + (\mu - \sigma_A^2/2)T}{\sigma_A \sqrt{T}}\right] = N(-DD) \end{aligned}$$

$Z \sim N(0,1)$: 표준정규분포, DD : 부도거리

EDF(Expected Default Frequency:예상도산확률) 를 추정하여 기업의 부도를 추정

(장점)

매 시점에서 움직이는 주가 정보로서 EDF를 도출함으로 이를 보완하여 보다 빠르게 기업 부도위험을 인지할 수 있다는 것이 최대 장점 → 적시성

.KMV 모형은 또한 EDF를 구하기 위한 과정이 매우 간단하면서도, 블랙-숄즈-머튼 옵션 가격 모형을 사용하였기 때문에 이론적으로 기반이 확실하다는 장점

기존의 재무제표 변수는 회계정보의 기간 단위 보고의 특성상 즉각적인 정보의 적용이 어렵다는 단점이 있으나 KMV 모형은 매 시점에서 움직이는 주가 정보로서 EDF를 도출함으로 이를 보완하여 보다 빠르게 기업 부도 위험을 인지할 수 있다는 것이 최대 장점

5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

생존분석(Survival Analysis)

1) 의의

개체들로 구성된 집단의 수명(Lifetime)과 관련된 자료들을 분석하는, **특정의 사건이 일어나기까지의 시간을 분석하는 방법론**

→ **관심있는 이벤트가 발생하는 데 걸리는 시간을 결정하는 데 사용되는 일련의 통계적 접근 방식**

→ 어떤 사건의 발생 확률을 **시간**이라는 변수와 함께 생각하는 통계 분석 및 예측 기법

→ 생존분석(Survival Analysis) 기법은 모집단을 가정하지 않아도 생존주기(Lifecycle)라는 또 다른 형태의 변수로서 자료의 해석이 가능하므로 **보다 동태적인 분석이 가능한 장점을** 가지고 있다.

2) 활용

- 보통 의료계 임상 실험(의학)에서 주로 사용되는 이론- 투약효과
- 서비스 고객의 이탈확률을 구하고자 할 때 사용

• **시간 (time)** : 생존분석을 시행할 때 주로 시간 경과에 따른 위험도나 생존도를 구하는데 이 때 두는 독립변수로 시간이 있다. 상대적 시간이며, 분석하고자 하는 대상을 관찰하기 시작한 시점부터 0으로 카운트 됨

• **사건 (event)** : 보통 생존의 반대인 이탈, 죽음 등을 가리키며, 이것들이 생존분석으로 분석하려는 대상이 되어 시간과 함께 종속 관계를 맺음. 사건은 한번만 일어나게 되며 0과 1로 보통 구분이 됨

• **중도절단 (censoring / censored) ****

- 우측 중도절단 (Right censored) : 어떤 특정 기간 내에도 사건이 발생을 하지 않았거나, 기타 다양한 이유로 관찰이 종료된 것을 의미
- 좌측 중도절단 (Left censored) : 대상 관찰 전에 사건이 일어났거나 **(event occurred 또는 failed to the event)** 기대했던 최소한의 기간보다 생존 시간이 더 짧았던 경우

• **생존함수 (Survival function)** : 어떤 관찰 대상이 특정 기준 시간보다 더 늦게 사건이 발생하거나 일어나지 않을 확률을 계산하는 함수

• **위험함수 (Hazard function)** : 특정 기간 t에 대상에 사건이 발생할 확률. t 전 시간까지는 사건이 발생해서는 안됨

• **누적위험함수 (Cumulative hazard function)** : 0에서 t 시간 사이의 위험함수를 적분한 값이며, 이는 t시점까지 사건이 발생할 확률을 모두 더한 것과 같다.

5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

생존분석 분류

1) 모수적 방법

와이블(베이블 Weibull)분포, 지수(Exponential)분포를 모수로서 이용하여 비례적으로 분석

지수 분포(Exponential Distribution)

- 지수 분포는 이벤트 사이의 시간(이벤트 A와 이벤트 B 사이에 걸린 시간)을 모델링하는데 많이 이용
- 각 이벤트는 포아송분포에 의해서 생성
- 지수 분포의 모수(parameter)는 람다(λ)
- 람다는 단위 시간동안 평균 이벤트 발생횟수를 의미 → 포아송 분포의 모수와 동일

베이블 분포(Weibull distribution)

- 연속확률 분포로서 고장 확률에 대한 예측에 대하여 많이 사용
- 신뢰도를 측정하는데 많이 사용
- 시스템이 작동을 시작하여 그 시점까지 고장 나지 않고 여전히 작동되고 있을 확률

2) 비모수적 방법

: Cox 비례위험 모형(Cox PH Regression),가속화 실패시간 모형(AFT Model)

생존함수에 영향을 주는 다른 변수들의 효과를 회귀식의 형태로 추정하여,어떠한 요인이 생존확률에 영향을 얼마만큼 미치는지 계량적으로 분석하는 방법

3) 공변량 적용

Kaplan-Meier추정량 ,Mantel-Haenszel검정 절차 이용

분포의 정의 없이 순위(rank)를 설정하여 분석하거나 표본의 분포를 이용하여 생존함수를 추정하고 검정하는 방법

5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

생존분석을 수행할 경우 유의할 점

- 생존분석이 자료의 사망 시점을 분석하는 방법이므로 자료의 절단 방법과 시기의 인식이 매우 중요한 특성을 가지고 있다는 점.
- 만약 분석에서 실험 설계자가 임의의 종료 시점을 설정할 경우,종료 시각 전에 사망이 관측되는 자료에 대해서는 그 시점에 생존 기간을 절단해야 한다.
- 이러한 절단을 형태-I 절단(type-I censoring)이라 한다.
- 반면 실험 시작 단계에서 종료 시점을 결정하는 대신 사망이 관측 되는 개체의 개수를 미리 정하여 절단하는 것을 형태-II 절단(type-II censoring)이라 한다.
- 이러한 절단의 형태 차이는 생존분석 방법론의 형태를 결정하는데 중요한 요인으로 작용한다.

5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

생존 함수 추정법 : 단변량 추정법/ 다변량 추정법

1.Univariate method (단변량 추정법)

1.1 Terminology: Univariate

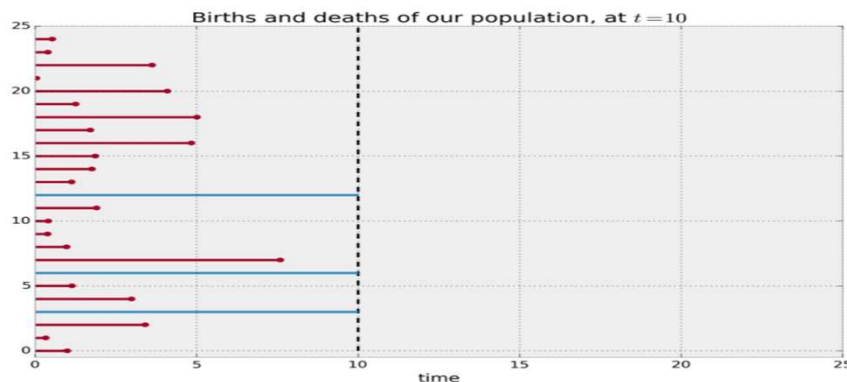
- 종속변수가 하나일 때 이를 단변량
- 종속변수가 두개 이상이면 다변량 분석

생존분석에서의 단변량은 시간 변화에 따라 하나의 결과 변수(종속 변수)가 생존에서 나타나는 것
→ kaplan-Meier 추정법이 있다.

5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

1.2 Kaplan-Meier 추정법 (KM)

- 어떤 데이터가 주어졌을 때 이 데이터의 생존확률과 위험확률은 아직 구해지지 않은 미정의 truth이다.
따라서 데이터에 생존 분석 기법들을 사용하여 이 확률들을 구하기 위한 함수를 추정해야 하는데 이 때 기본적으로 사용하는 것이 미국의 통계학자 Paul Meier와 Edward L. Kaplan이 개발한 Kaplan-Meier Estimation, (카플란-마이어 추정)
- 이 추정법은 관찰 시간이 짧을 때 부터 긴 순서로 나열해놓고 각 사건이 발생한 시점에서의 사건이 일어날 확률을 계산하는 비모수적(non-parametric) 방법
- 각 시점마다 구간 생존율을 구하여 이들을 누적시킴으로써 생성된 empirical cumulative density function (eCDF)을 사용하여 누적 생존확률을 추정
- 표본 수가 적을 때도 충분히 사용할 수 있어 생존분석법들 중 널리 사용되고 있다. Kaplan-Meier를 이용한 생존분석은 생존곡선들 간의 차이 비교 및 검정, 중앙값이나 평균 생존 기간의 차이 판별에 사용
- 아래 그래프는 데이터가 주어졌을 때 이를 시간 축에 따라 사건 발생을 나타낸 것이며, 파란색 선은 (right) censored data를 의미한다.



5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

(특성)

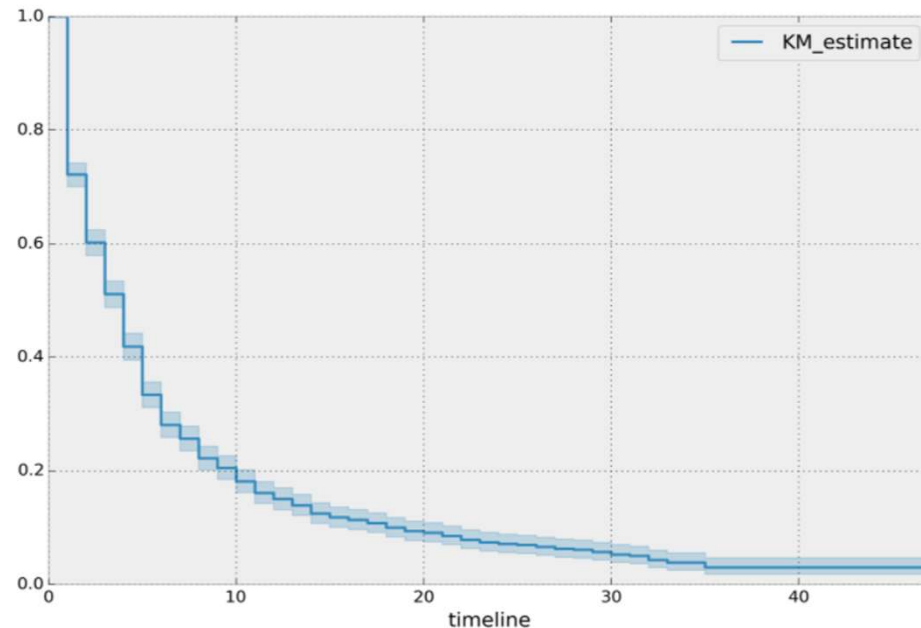
케플란 마이어 생존곡선(Kaplan Meier Estimate Curve)이라는 곡선을 만들 수 있으며, 곡선을 만들 때 좌표는 사건 발생 시간을 x축으로, 추정된 생존 확률을 y축으로 갖는다.

생존 확률 y는 0에서 1사이의 값을 가지며 보통 하향곡선을 그린다.

생존 곡선 두개가 있을 때 이 곡선을 비교하는 것은 임상에서 중요하다.

단순히 시각화만 하는 것이 아닌, 통계적인 유의성을 검정하기 위해 곡선간의 차이를 정량화 할 필요가 있다.

→ 이를 위한 방법 중 하나가 로그순위법(log-rank method)이다.



5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

1.3 로그 순위법 (Log-rank method)

'비교 대상간의 생존곡선에 큰 차이가 없다'라고 귀무가설을 설정하고, 실제 관측값 개수(observed count)와 사건이 일어날 법한 수(expected count)를 각 그룹별 사건 발생 시간의 카이-제곱(chi-square)을 계산하여 비교, 결과를 합산하는 방법.[Junyong I. 2018]

위험비(hazard ratio)를 구하는 것도 케플란 마이어 생존곡선들을 비교하는데에 도움이 된다. 그러나 위험비의 경우, 로그순위법을 사용하기 위해 조정해야 할 제약 조건 중에 하나인데, 실험 기간 동안 그 위험비는 항상 일정해야 한다. 이 조건(가정)을 "**비례위험가정(Proportional hazard assumption)**"이라고 함.

5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

케플란 마이어 생존곡선을 분석할 때 먼저 사건과 측정 단위를 명확히 정의하고, 사망 등의 사건이 일어났음을 나타내는 계단이 많은 곡선인지, 제한된 수만을 대상으로 하여 폭이 큰 단계만 갖는 곡선인지 확인한다.

중도절단된 대상과 분포도 해석하는데에 고려해야 할 중요한 요소이다.

중도절단 된 것은 곡선에서 눈금으로 나타낸다.

어떤 모집단에서 개인의 사건까지 걸리는 시간은 전체적인 수준에서 매우 중요하다.

그러나 실제 상황에서는 사건 발생만큼 공변량(covariate, 독립변수)의 값이 중요하게 작용한다.

이 공변량 값을 미리 안다면, 개별 생존 확률을 예측하는데에 도움을 얻을 수 있다.

공변량을 고려했을 때 조건부 생존 함수를 다음과 같이 세울 수 있다.

$$S(t|x) = P(T > t|x)$$

T 는 사건이 발생했을 때의 시간을 나타내며, x 가 공변량(covariate)에 해당한다.

5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

2. Multi-variate method (다변량 추정법)

2.1 Terminology : Multivariate

보통 이 두 가지 이상의 함수 해(근)를 해석하기 힘들기 때문에 결과 변수(종속변수)를 시간 하나로 두고 여러 독립변수들에서 결과 변수에 미치는 영향을 분석하는 방법을 쓰고 이를 생존분석에서는 다변량이라고 주로 칭하며, 대표적으로 시간과 사건 사이의 예측 회귀모형을 만드는 통계방법→ 콕스 비례위험모형(Cox Proportional Hazard model) **2.2 Cox PH Model (CPH)**

KM 분석은 하나의 타겟 특성 이외의 다른 factor들을 통제할 수 없다는 점에서 한계를 지녔다.

→ 이를 보완하는 것이 cox 모형이며, 다양한 관측치들을 동시에 통제하여 사건 발생에 미치는 영향을 분석

→ David Cox가 제안한 **Cox 비례 위험 모형(Cox Proportional Hazard model)** 은 공변량과 생존 함수를 결합한 모델이며, 특정 분포만을 고려하지 않고 데이터의 변량(변수) 여러개가 사건 발생에 영향을 미치는 다변량(Multivariate) 생존 회귀분석법이라고 하나, 정확히는 시간이라는 종속 변수 하나만을 갖는 **단변수 생존 분석법**

→ 시간 종속 변수에서 어떤 시점 t까지 생존한 사람이 직후에 사망할 조건부확률은 다음과 같이 나타낼 수 있다.

$$h(t|X = x) = h_0(t)exp(x^T \beta)$$

여기서 β 는 각 공변량들의 계수(coefficient)로 구성된 벡터이고 $h_0(t)$ 는 공변량들이 0일 때 **기저위험함수(baseline hazard function)** 이다. 그렇게 해서 나온 위험함수 Cox의 비례위험 모형은 특별한 가정(assumption) 두 가지를 지니는데 앞서 로그순위법에서 언급하였듯 위험비가 일정하다는 **비례위험가정**과, 관측치(observed data)는 서로 독립적이며, 공변량이 위험 함수에 선형 곱셈 효과를 갖는 지수함수를 따라 특정 시점의 생존함수가 위험비에 대한 **지수함수로 나타난다**는 것이다.[Junyong I. 2018]

5. 위험모형(Hazard Model) ,생존분석(Survival Analysis)

- 지수함수를 따른다는 Cox PH assumption은 개인의 위험을 로그로 만든 로그 위험이 정적인 공변량의 선형 함수여서 위험함수와 변수사이에 로그-선형관계를 가진다는 말과 같고, 시간이 지남에 따라 변하는 모집단 수준의 기저 위험이라는 것, Hazard Ratio(위험비, HR)는 시간이 경과해도 일정하다 것은 비례 위험 가정을 강조한다.
- 여기서 비교되는 두 실험군간의 HR=1이라는 귀무가설을 설정하고, HR이 1보다 크면 사망 위험이 증가한다는 것, 1보다 작으면 사망 위험이 감소함을 뜻하며 이 때는 대립가설을 채택한다. HR은 수식으로 나타낼 때 보통 기호 λ 를 많이 쓴다.

(비례위험모형을 추정한 이후로는 비례위험 가정을 충족 여부 검정)

- 1) 케플란 메이어 곡선을 통해 위험비를 비교하는 것이고,
 - 2) cumulative hazard function으로 martingale residual을 구하여 log minus log(LML) plot을 그려서 log-linear 관계를 검정하는 것이다. LML plot 방법은 추정된 생존함수를 두 번 로그로 변환을 한다는 것인데, 생존함수의 값이 0과 1사이의 확률이다보니 로그를 씌웠을 때 음수가 되어 여기에 minus를 취하고 다시 로그 변환을 실시하는 아이디어에 근거한다.
 - 3) 적합도 검정(Goodness of fit test)으로, 관측값과 추정된 생존함수에 의한 값의 차이를 비교하여 가정을 확인할 수 있다.[Junyong I. 2019]
- 이러한 검정들을 거쳐 비례위험가정을 만족하는 Cox Proportional Hazard function은 다변수 처리가 가능하다는 것과, 지수적인 회귀 생존 모형을 사용한다는 것, 상대 위험을 도출해낸다는 것과 범주형 변수, 연속형 변수 둘 다 처리가 가능하다는 특징을 지니게 된다.
- 만약 언급한 비례위험가정들을 만족하지 않는다면, CoxPH model로 접근해서는 정확한 예측값을 얻기 어려우므로, 대신 time-dependent CoxPH model 이나 층화된(stratified) 콕스 모델, Gehan's generalized Wilcoxon test 등의 non-proportional hazard model을 사용해야 한다.

5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

파이썬의 lifelines 모듈

```
# lifelines 모듈 import  
from lifelines import CoxPHFitter
```

5. 위험모형(Hazard Model) , 생존분석(Survival Analysis)

위험모형(Hazard model)

- 회계정보와 시장 정보를 통합하여 부도를 예측하는 모형으로 부도 발생시점까지의 시간을 고려하는 방법론
- 위험모형은 기업의 상태를 정상(=0)과 부도(=1)로 구분하고, 부도 사건이 발생하면 기업의 상태에 대한 관측을 종료한다.
- 위험함수(hazard function) $\lambda(t)$ 시점에 부도가 발생하는 순간의 실패율 (instantaneous failure rate)
- 추적오차가 적은 기업 의 재무정보와 시장정보는 생존분석의 신뢰성을 한층 높여주어 보다 좋은 결과를 도출할 것으로 기대되는 또 하나의 요소이다

- 추가, 거시경제, 비정형정보 등 Hazard 함수 설명 변수로 반영 가능
- 산업별 생존함수를 추정하여 산업별 특성 반영 • 변수 선택(Stepwise) 필요
- F-value(P-value) 및 R^2 로 모형 평가

→ 시장상황이 그때 그때 다르다.

5. 위험모형(Hazard Model)

Cox 비례위험 모형(Cox PH Regression)

종속변수가 부도 여부를 판별하는 이진 분석 방법론에 비하여 기업 생존 주기에 따른 부도 발생 확률이라는 추가적인 정보를 적용할 수 있다는 장점

Cox 비례 위험 모형을 사용하기 위해서는 우선 위험 함수의 추정이 필요 하다.
어떠한 생존분석 실험에서 실험 시작 시점을 t_0 사망 시점을 T 라고 가정하면, 생존기간은 $T-t_0$ 이 된다. 이와 같은 가정하
여 실험 시작 시점(t_0)을 0으로 가정하면 생존기간은 아래의 확률 밀도함수를 가지는 확률 변수가 된다.

$$\lambda(t) = \lim_{\Delta t \rightarrow 0^+} \frac{\Pr(t \leq T < t + \Delta t | T \geq t)}{\Delta t}$$

시점 t 이전에 사망이 일어나지 않을 확률로서 생존함수 (survival function)

$$S(t) = P(T > t) = 1 - F(t)$$

$$h(t) = \frac{f(t)}{S(t)}$$

Kaplan-Meier 플롯 : 생존 곡선을 시각화

Nelson-Aalen : 누적 위험을 시각화하기 위해 플롯

두 개 이상의 그룹의 생존 곡선을 비교하기 위한 로그 순위 테스트

Cox 비례 위험 회귀 : 연령, 성별 및 체중과 같은 다양한 변수가 생존에 미치는 영향

Kaplan-Meier Estimator 이론 및 예

카플란 - 마이어 추정기 수명 데이터로부터 생존 기능 (사람의 생존 확률)을 추정하기 위해 사용되는 비모수 통계이다.
의학 연구에서 종종 치료 또는 진단 후 특정 시간 동안 살아가는 환자의 비율을 측정하는 데 사용

생존함수

비교 검정 통계량

생존함수 비교에 가장 많이 사용되는 방법

로그-순위 검정(Log-Rank)과 Wilcoxon방법

test	카이제곱	자유도	P-value
Log-Rank	92.8077	1	<.0001
Wilcoxon	87.9121	1	<.0001

정상기업과 부실기업에 대한 생존율 차이

- 생존시간만을 고려할 경우 Log-Rank 검정, Wilcoxon검증을 실시한 결과
- 유의수준 0.05 에서 정상기업과 부실기업간에 유의한 차이 여부 확인

(사례)

모두 유의 확률(P-value)이 0에 매우 근사하는 것으로 나타나서 두 생존함수는 차이가 명확함을 알 수 있다. 즉, 표본 집단과 부도 기업 표본 집단으로부터 각각 도출한 생존함수 간에는 통계적으로 차이가 있다.
대립가설을 채택

헤저드 모형 → 확장

Campbell et al.(2008)

부도예측력이 개선된 헤저드모형을 제시

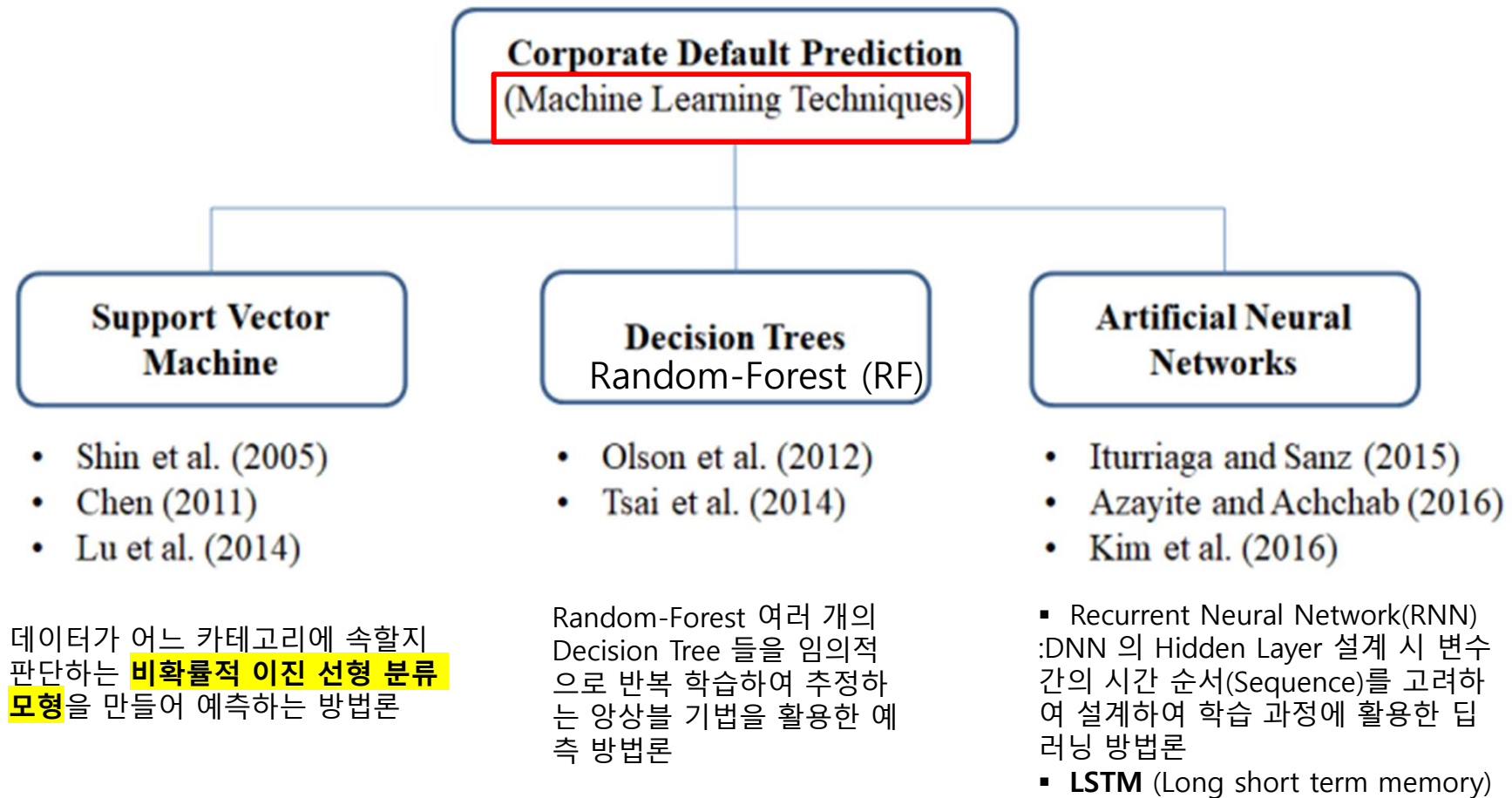
- 자본의 시장가치와 부채의 장부가치의 합을 시장총자산(market total asset)으로 정의

- 회계비율에 포함된 총자산(장부가치)을 시장총자산으로 대체

수익성 관련 설명변수는 이전 10개월의 가중평균값을 이용하여 기업의 동태적 변화(dynamic change)를 포착하도록 하였다.

기계학습을 이용한 기업부도예측

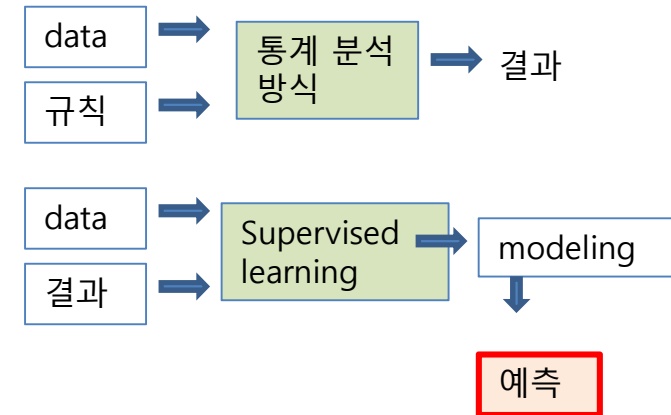
Machine Learning 부도모형 구분



* 머신러닝 구분

1. 지도 학습(Supervised learning)

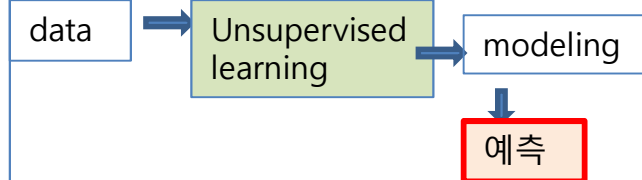
- 학습 데이터에 입력값과 그의 정답인 출력값이 같이 포함되어 있는 상황에서 학습하는 형태 → 회귀분석, 분류
 - 과거의 입력값과 출력값을 가지고 패턴을 학습하여 모델링 한 후, 새로운 입력값이 들어오면 이에 맞는 출력값을 예측하는 방식 (알고리즘)
- MDA , penalized regression, SVM, KNN, **Decision Tree**, 랜덤 포레스트, 나이브 베이지안 등



2. 비지도 학습(Unsupervised learning)

입력값만으로 학습 데이터가 구성되어 있고 레이블이 없는 경우의 학습방식 군집화, 차원 축소 등 (알고리즘)

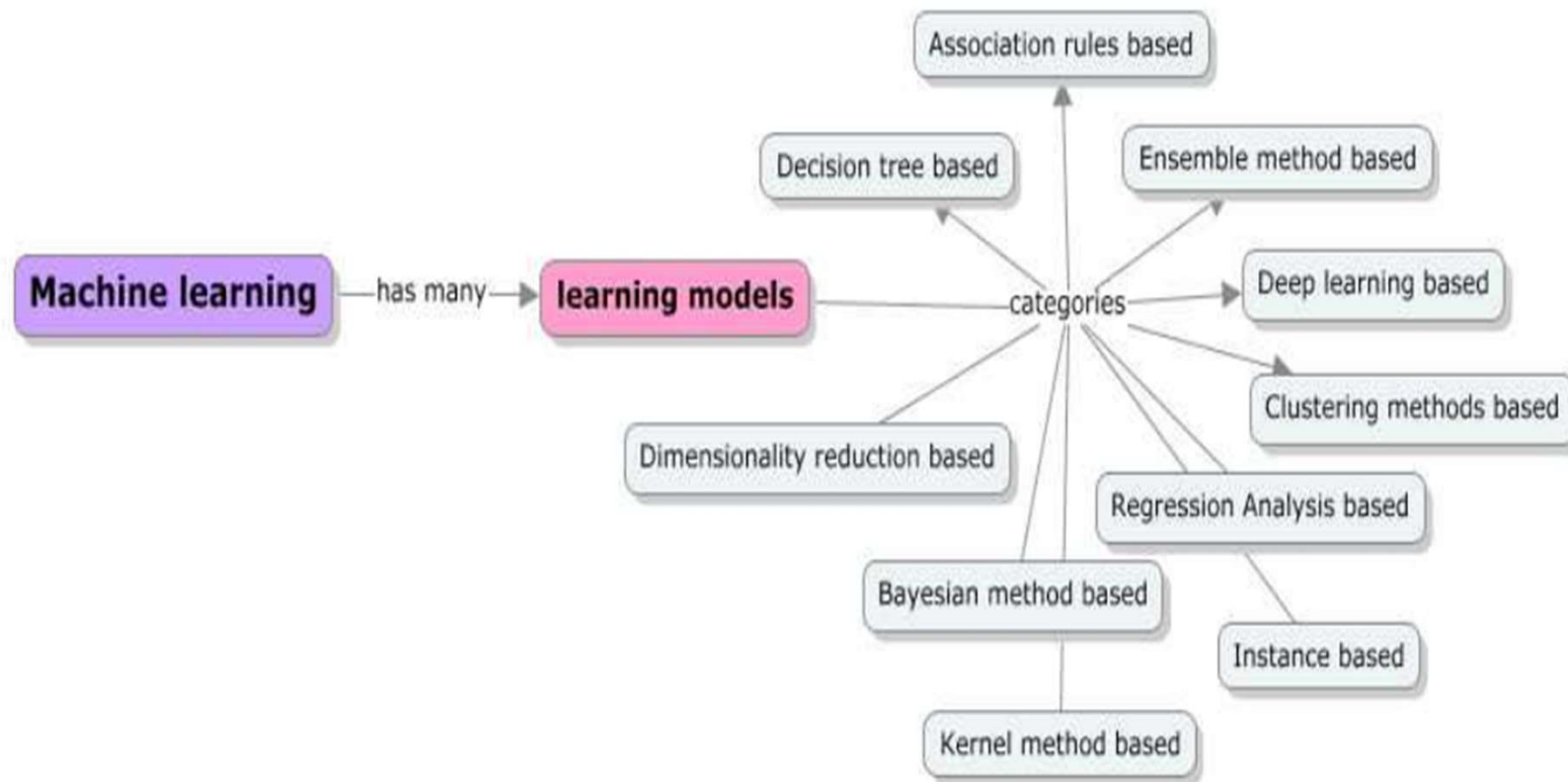
PCA, k-mean 군집 등



3. 강화 학습(Reinforcement learning)

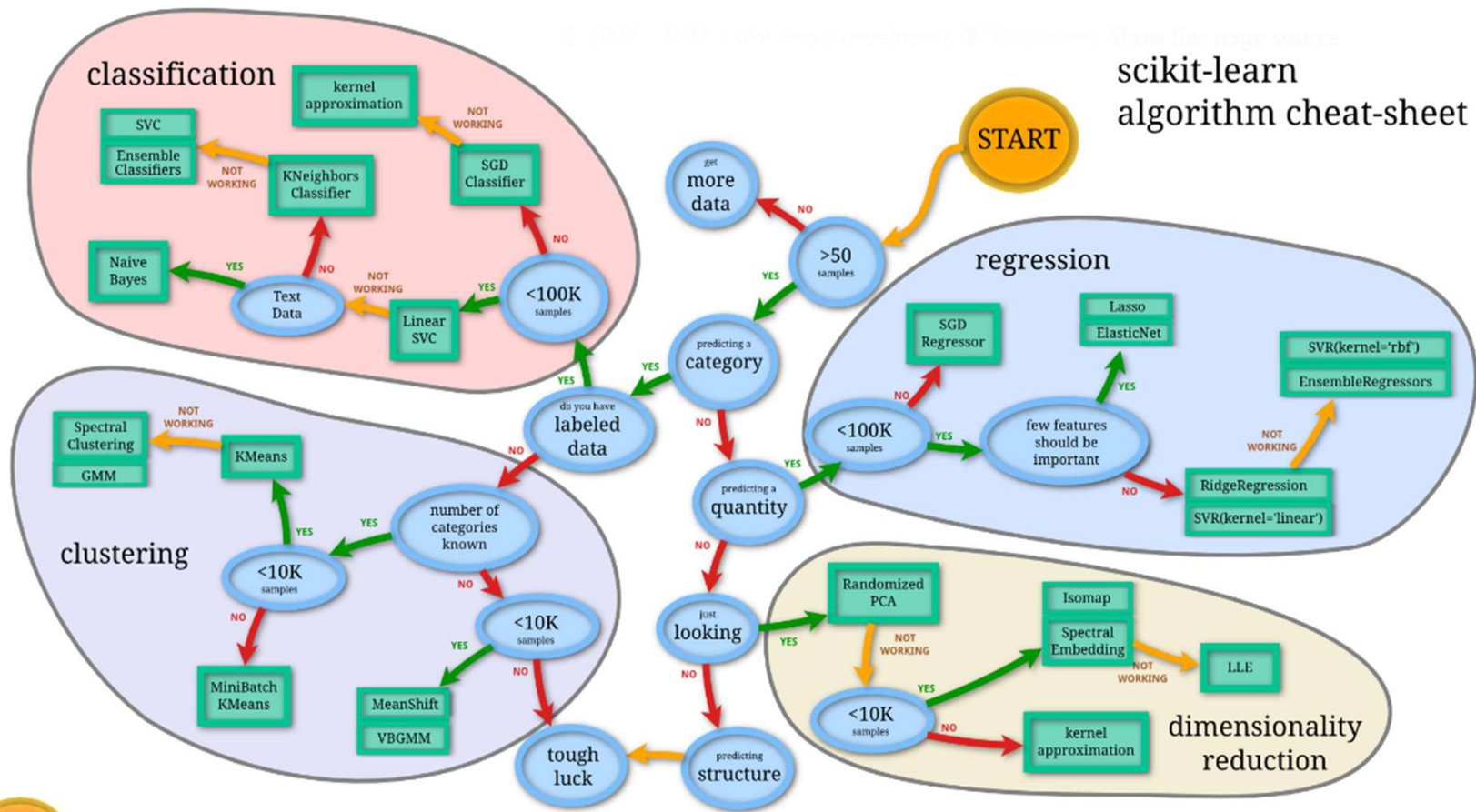
에이전트가 환경을 관찰해서 행동을 실행하고 그 결과로 보상을 받는 환경을 만들어서, 가장 큰 보상을 얻기 위해 최상의 전략을 스스로 학습하게 만드는 알고리즘

* Machine_learning_map



Gollapudi, 2016

* Machine_learning_map



https://scikit-learn.org/stable/tutorial/machine_learning_map/index.html

* 기계학습을 이용한 기업부도예측

Machine Learning 절차



기계학습을 이용한 기업부도예측 **한계점**

- 알고리즘에 따라 예측결과의 계산과정이 **블랙박스(Black box)**로 남는 경우가 있어 주의 필요
- 이러한 방법론의 경우 기업부도위험을 계산할 수 있어도, 부도위험을 낮추기 위한 경영상태 개선전략을 제시하지는 못하는 한계
 - 기업부도예측을 실시할 때에는 예측 목적에 맞는 정보를 제공할 수 있는 적절한 방법론 선정이 요구
 - **각 방법론에 대한 상세한 이해와 활용능력 필요.**

==> XAI 연구 진행 중

분류분석 절차

1. 데이터의 정제 (전처리) : 이상치, 결측치 , 데이터 변환

2. 데이터 분할

: 보통 훈련용 데이터와 시험용 데이터로 분할

- Training data : 모델을 만드는 데이터

- 검증데이터

- Test data : 모델을 적용해 보는 데이터

3. 모델의 훈련

: 훈련용 데이터를 이용하여 모델 생성

4. 타당성 검토

: 훈련용 데이터를 이용해 만들어진 모델을 test 데이터에 적용하여 타당 여부 검토

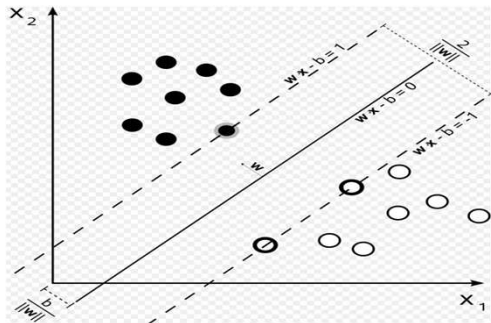
5. 분류모델 적용

: 타당한 모델이면 의사결정에 적용 , 강건성 검토

SVM(Support Vector Machine)

분류(classification)

- Vapnik(1995)에 의해 제안
- 분류 및 회귀 학습 이론으로서 분류 문제에 있어 **두 분류사이의 거리인 마진(margin)을 최대**로 하는 초평면(hyperplane)을 탐색하는 방법
- 기계학습의 분야 중 하나로 패턴 인식, 자료 분석을 위한 지도학습 모델
- 주어진 데이터 집합을 바탕으로 새로운 데이터가 어느 카테고리에 속할지 판단하는 **비확률적 이진선형분류** 모델
→ 만들어진 분류 모델은 데이터가 사상된 공간에서 경계로 표현되는데 SVM 알고리즘은 그 중 가장 큰 폭을 가진 경계를 찾는 알고리즘
- SVM 방법론은 이러한 초평면을 결정한 후, 이를 기준으로 관측치를 **분류**
- p차원 공간에서의 초평면이란 p-1차원의 평면 아핀 부분공간(flat affine subspace)을 의미하며, 선형 커널(linear kernel)을 사용하는 SVM을 수식으로 표시
- **XOR 문제 해소에서 출발**



$$\text{Max } \frac{2}{\|w\|}$$

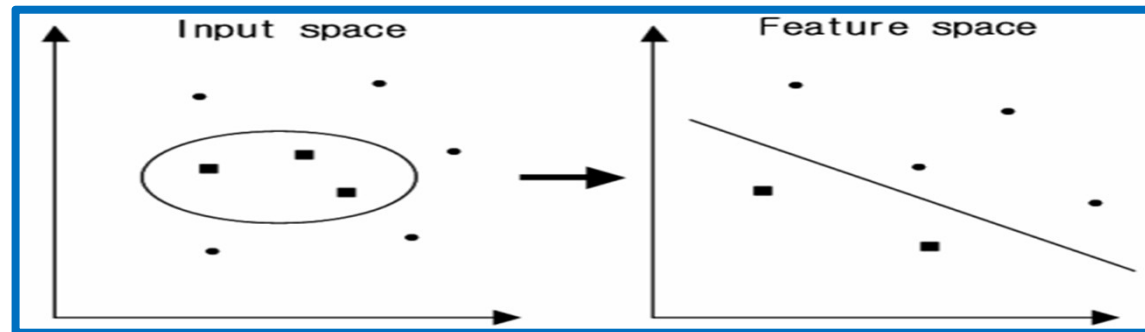
- 데이터를 집합을 분리하는 것을 **초평면 (hyperplane)**
- 초평면의 **마진**: 각 서포트 벡터를 지나는 초평면 사이의 거리
→ 기하학적으로 두 초평면 사이의 거리
- → SVM은 **마진을 최대**로 만드는 알고리즘

https://ko.wikipedia.org/wiki/%EC%84%9C%ED%8F%AC%ED%8A%B8_%EB%B2%A1%ED%84%B0_%EB%A8%B8%EC%8B%A0

SVM(Support Vector Machine)

SVM은 이상평면 방정식의 해를 찾는다는 특징과 커널 함수를 이용하여 주어진 데이터를 다른 dot product 공간으로 표현하는 특징 포함

input space에 존재하는 data들을 feature space 로 옮겨주는 역할을 하는 mapping



→ 커널 함수를 이용 간단히 계산

$$k(x, y) = (\Phi(x) \cdot \Phi(y))$$

대표적인 kernel

linear

$$k(x_1, x_2) = x_1^T x_2$$

Nonlinear

Polynomial

$$K(x, x') = (x \cdot x' + 1)^d$$

Gaussian RBF

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

Exponential RBF

$$K(x, x') = \exp\left(-\frac{\|x - x'\|}{2\sigma^2}\right)$$

Sigmoid

$$K(x, x') = \tanh(kx \cdot x' - \delta)$$

→ 이 커널함수들을 통하여 문제를 선형분리가 가능하도록 유도한 후 이상평면방정식을 도출
→ SVM은 비선형 분류기의 기능을 수행하게 되는 것

SVM(Support Vector Machine)

비선형 SVM(Kernel 함수)

SVM 모형의 구축에 있어서 어떤 **커널함수**를 사용하느냐는 가장 중요한 문제 중의 하나.
input space에 존재하는 data들을 feature space 로 옮겨주는 역할을 하는 mapping

Radial Basis Function (RBF: 원형기준함수) kernel

$$K_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = \exp \left[-\gamma \|\mathbf{x} - \mathbf{x}'\|^2 \right]$$

$$H_i = \exp \left(- \frac{(x_1 - c_{1i})^2 + (x_2 - c_{2i})^2 + \dots + (x_k - c_{ki})^2}{r_i^2} \right)$$

Radial Basis Function (RBF) kernel, Gaussian kernel이라고 불림
감마: 파라미터로 커널의 흠어짐을 세팅한다. **가우시안 분포에서 분산**
에 해당하는 부분으로 흠어짐을 세팅
BF kernel은 **무한 차원**으로의 **projection**이 될 수 있다.

어떠한 임의의 함수에 대한 커널의 형태인 경우

$$K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$$

위에서 파이는 함수로써, 벡터 x를 새로운 벡터 공간으로 사영시키는 기능을 한다.
이 커널 함수는 두 개의 사영된 벡터들 사이에서 내적을 계산한다.
RBF kernel에 대한 파이 함수는 무한 차원 공간으로 사영시킨다.
뉴클리디안 벡터에 대해서, 이 공간은 무한 차원의 뉴클리디안 공간이 된다.

다항 커널 - Polynomial kernel

$$K(x_i, x_j) = (\gamma x_i^T x_j + c)^d$$

Parameter: Cost, Gamma, Coef, Degree
- Degree: Feature space의 차원개수

은닉마디 개수를 k라고 할 때, RBF 신경망에서는 먼저 k개의 중점(center) c_{1i}, c_{2i} 와 반경(radius) r_i 를 군집분석과 같은 추정 방법에 의해서 모수를 추정한 후 은닉층으로부터 출력층으로 전달되는 계수를 추정

(장점)

MLP 신경망에서와 같이 모든 모수를 동시에 추정하는 방법보다는 계산 과정이 간단/시간이 적게 걸린다

(단점)

수렴성의 면에서 보면 MLP 신경망보다 비효율적

SVM(Support Vector Machine)

분류(classification), 회귀(regression), 특이점 판별(outliers detection) → 지도학습 머신러닝 방법
scikit-learn 구분

1) SVC(Support Vector Classification)

Classification 에 사용되는 SVM 모델

2) SVR(Support Vector Regression) `from sklearn.svm import SVR`

Regression 에 사용되는 SVM 모델

3) OneClassSVM

특이점 판별(outlier detection) 사용

선형 구분 여부

- 선형 SVM (linear SVM)
- 비선형 (**RBF 커널 SVM**)

SVM(Support Vector Machine)

SVM : Hyper Parameter

Hyper Parameter	내용
▪ c (default = 1.0)	▪ 정규화 파라미터 ▪ 값이 클수록 정규화 강도가 약 함. ▪ C가 너무 높으면(낮으면) overfitting(underfitting)이 될 가능성 ▪ L2 페널티
▪ kernel	▪ 커널을 결정 ▪ 'linear' , 'poly' , 'rbf', 'sigmoid' 선택
▪ gamma	▪ 'poly' , 'rbf', 'sigmoid kernel에 포함된 하이퍼 파라미터
▪ degree	▪ 'poly 커널에 포함된 하이퍼 파라미터인 d
▪ coef0	▪ poly 커널과 sigmoid kernel 에 포함된 세타
▪ tol	▪ Tolerance for stopping criterion ▪ 학습 iteration이 진행될 때 tol 수치 미만으로 목적 함수가 갱신되는 경우 학습이 종료

SVM(Support Vector Machine)

Tay and Cao(2001) "Application of Support Vector Machines in Financial Time Series Forecasting" by Francis E. H. Tay and Lijuan Cao, 2001

- SVM의 성능에 있어서 커널함수의 상한 C 와 커널 파라미터 γ 가 가장 중요한 역할을 한다고 보고 되었다.
- 따라서 적절한 상한 C 와 커널 파라미터 γ 를 선정하기 위해, 선행연구에서 SVM의 파라미터에 대해 제시된 일반적인 가이드를 따라 일정 범위 내에서 다양한 값을 대입하여 다양한 모형을 생성

→ SVM은 하이퍼파라미터에 따라 성능이 크게 영향

Support Vector Machine(SVM) 모형

- SVM에서는 학습과정이 이루어지기 전에 사용자가 직접 파라미터 값을 결정해야 하며 **파라미터 값에 따라** SVM의 성능은 달라진다.
- 사전에 결정해야 하는 파라미터는 학습과정에서 마진폭과 분류 오류 사이의 타협점(trade-off)을 찾아주는 오류 패널티(penalty) 변수 C 값과 비선형 SVM에 적용되는 커널함수의 파라미터.
- 기업부도예측 모형에 적합한 C 값과 커널함수의 파라미터 값을 결정하고, 이를 바탕으로 RBF 커널함수를 적용한 비선형 SVM의 분류 성능을 비교 분석

γ 의 범위를 0.1~1.0 사이로 오류 패널티 변수 C 값은 10, 20, 40, 60

산출(fitting): 다양한 설정 값 시뮬레이션
모형평가 : Accuracy로 사후적 모형 평가

SVM(Support Vector Machine)

장점

첫째, 명료한 이론적 근거에 기반하므로 결과 해석이 용이하고,
둘째, 실제 응용에 있어 높은 성과를 나타내고,
셋째, 입력변수의 차원에 의존하지 않고 자료의 수에 의존하여 신속하게 학습을 수행할 수 있으며,
넷째, 구조적 위험 최소화 원칙(structural riskminimization)에 기반하므로 과대적합 (overfitting)문제에 견고

기업부실 예측 데이터의 불균형 문제 해결을 위한 앙상블 학습 김명종 (2009)

γ 의 범위를 0.1~1.0 사이로 오류 패널티 변수 C값은 10, 20, 40, 60

산출(fitting): 다양한 설정 값 시뮬레이션
모형평가 : Accuracy로 사후적 모형 평가

Support Vector Machines with Scikit-learn Tutorial

<https://scikit-learn.org/stable/modules/svm.html>

•1.4. Support Vector Machines

- 1.4.1. Classification
- 1.4.2. Regression
- 1.4.3. Density estimation, novelty detection
- 1.4.4. Complexity
- 1.4.5. Tips on Practical Use
- 1.4.6. Kernel functions
- 1.4.7. Mathematical formulation
- 1.4.8. Implementation details

* K-Nearest Neighbors

K-Nearest Neighbors

- 주변의 가장 가까운 K개(1,3,5,...)의 데이터를 보고 데이터가 속할 그룹을 판단하는 알고리즘이 K-NN 알고리즘
- 지도학습
- 거리기반 분류 모델 : 거리 측정은 유클리드거리(Euclidean distance) 사용
 - 상대적으로 거리가 더 짧은 이웃이 더 가까운 이웃으로 취급
 - K-NN 알고리즘은 어떤 새로운 데이터로부터 거리가 가까운 K개의 다른 데이터의 레이블을 참고하여 K개의 데이터 중 가장 빈도 수가 높게 나온 데이터의 레이블로 분류하는 알고리즘
- K 값 지정시 : 홀수로 설정

(장점)

- 단순하여 다른 알고리즘에 비해 구현하기가 쉽다.
- 훈련 속도가 빠르다.

(단점)

모델을 생성하지 않기 때문에 특징(feature)과 클래스 간 관계를 이해하는데 제한적

(주의사항)

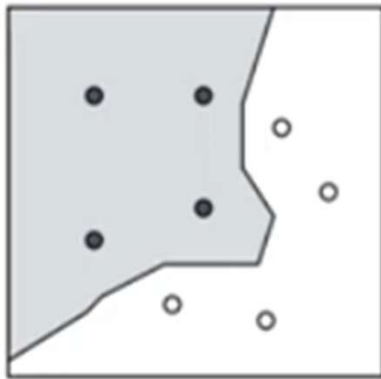
변수 값 범위 재조정 필요 : 분류 모델이 상대적으로 큰 값을 지니는 변수 값은 중요하게 고려하고 상대적으로 값이 작은 변수는 무시하는 경향을 보일 것이기 때문에.....

- 최소-최대 정규화(min-max normalization)
- z-점수 표준화(z-score standardization)

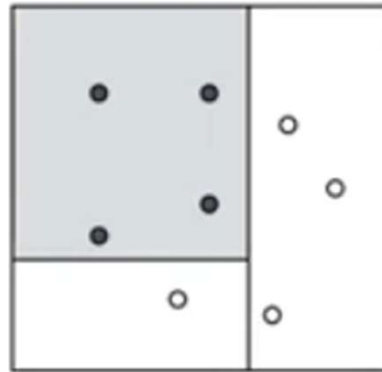
```
from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier()
```

* 쉬어가기 : Discriminant Function

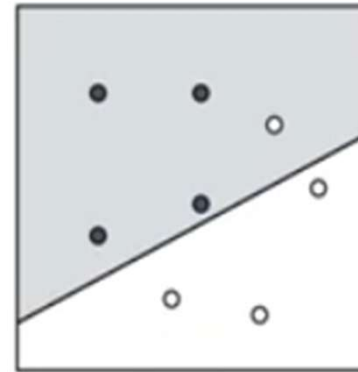
Binary Classification



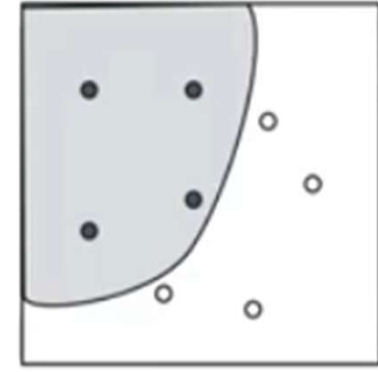
Nearest Neighbor
(KNN)



Decision Tree



Linear function



Nonlinear function

SVM

$$- g(\mathbf{X}) = \mathbf{w}^T \mathbf{X} + b$$

*나이브 베이즈 분류기(Naive Bayes Classifier)

베이즈의 정리(Bayes' theorem)를 이용한 분류 메커니즘

- 베이즈 추정(Bayesian Estimation)에 기반한 통계적 기법
- 지도학습(supervised learning) 일종 : Feature와 Label이 필요
- Feature에 따라 Label을 분류하는데 베이즈 정리를 사용하는 것이 특징
- 타겟변수: 범주형(1,0), 수치형이라면 범주형으로 변환
- 표본을 통해 모수를 추정하기보다는 표본 정보와 사전 정보를 함께 사용하여 모수를 추정하는 것
- 추론 대상의 사전 확률과 추가적인 정보를 기반으로 해당 대상의 사후 확률을 추론하는 통계적 방법을 베이즈 추정

베이즈 정리(Bayes' theorem)

- 사전확률 (prior probability)과 사후확률 (posterior probability) 과의 관계에 대한 설명
- Thomas Bayes는 확률이 상황에 따라 변할 수 있는 것이라고 생각했는데, 이는 기존의 개념과 다른 것으로 추가되는 새로운 증거에 따라 확률을 새로 계산 및 개선
- → '이전의 경험(사전정보)과 현재의 새로운 증거를 토대로 어떤 사건의 확률을 추론하는 알고리즘'.
- 특정사항의 조건부 확률/전체 조건부 확률로 의사결정이론에 기초하여 주어진 주변 확률과 조건부 확률을 이용해서 다른 조건부 확률을 계산하는 것
- 사전 정보를 바탕으로 어떤 사건이 일어날 확률을 토대로 의사결정을 할 때 활용

*나이브 베이즈 분류기(Naive Bayes Classifier)

베이즈 정리(Bayes' theorem)

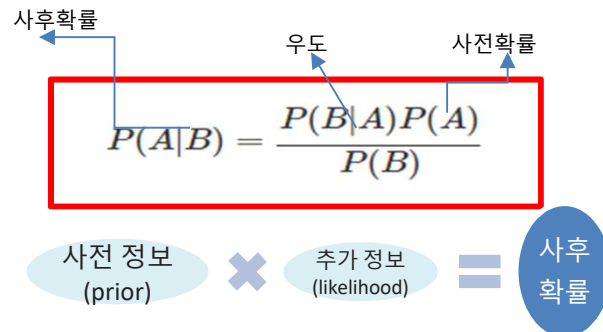
: 사전확률과 사후확률과의 관계에 대한 설명

→ 사건 B가 발생함으로써 사건 A의 확률이 어떻게 변화하는지를 표현한 정리

조건부 확률 (conditional probability)

- $P(A)$: A가 발생할 확률 ,
 - $P(A|B)$: B가 발생한 조건에서 A가 발생할 확률
- 조건부 확률 **Probability of A given B**

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$



- $P(A|B)$: 사후확률(posterior probability). 사건 B가 발생한 후 갱신된 사건 A의 확률
- $P(B|A)$: 우도(가능도, **likelihood probability**) 사건 A가 발생한 경우 사건 B의 확률
- $P(A)$: 사전확률(**prior probability**) 사건 B가 발생하기 전에 가지고 있던 사건 A의 확률 → 주관적확률에서 객관적확률로 발전
- $P(B)$: 정규화 상수(normalizing constant) 또는 증거(evidence). 확률의 크기 조정 → 특정 클래스(A)의 분포에 상관없이 전체 클래스에서 특정 특징(B)이 관측될 확률

A가 발생할 확률: $P(A)$, A가 발생한 조건에서 B가 발생할 확률을 $P(B|A)$

$$P(B|A) = P(B \cap A) / P(A)$$

그림에서 $P(B \cap A)$ 는 (우선 전체 C에서 A가 발생해야 하므로) 'A의 발생 확률인 $P(A)$ '에 'A가 발생한 상태에서 B가 발생할 확률인 $P(B|A)$ '를 곱하면 된다.

즉 $P(B \cap A) = P(B|A) * P(A)$ 이항 정리하면 $P(B|A) = P(B \cap A) / P(A)$

또한 $P(A|B) = P(A \cap B) / P(B)$ 이므로, $P(A \cap B) = P(A|B) * P(B)$, 그리고 $P(B \cap A) = P(A \cap B)$ 이므로, $P(B|A) * P(A) = P(A|B) * P(B)$ 이 되어 결국 **$P(B|A) = P(A|B) * P(B) / P(A)$ 이 된다.**

여기서 $P(A) = P(A|B) * P(B) + P(A|B) * P(B)$ 로 바꾸어 쓸 수 있다.

즉 우리가 알고자 하는 $P(B|A)$ 를 이미 알고 있는 $P(A)$ 와 $P(B)$, 그리고 우도 $P(A|B)$ 를 활용하여 구한다.

$$P(A|B) = \frac{P(A, B)}{P(B)} \rightarrow P(A, B) = P(A|B)P(B)$$

$$P(B|A) = \frac{P(A, B)}{P(A)} \rightarrow P(A, B) = P(B|A)P(A)$$

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

*나이브 베이즈 분류기(Naive Bayes Classifier)

나이브 베이즈 분류기(Naive Bayes Classifier)

Naïve 하다 의미??? : 천진난만은 순진한(naive, naïve)한 상태

나이브 베이즈 분류기에서는 각각의 특징 간에 서로 아무런 상관관계가 없다는 가정,

→ 즉 특징들이 서로 조건부 독립(conditional independent)이라는 가정

$$P(A, B) = P(A \cap B) = P(A|B)P(B) = P(A)P(B)$$

$$P(A|B) = \frac{P(A \cap B)P(A)}{P(B)}$$

나이브 가정하에선 조건부 독립인 확률변수가 가지는 확률들의 곱으로 우도(가능도)가 표현
연관된 확률변수 간에 서로 독립이라는 가정을 함으로써 더 쉽게 베이즈 추론을 할 수 있게 된다.!!!

특징(변수) 1개 경우

여러 특징(변수)을
n개로 확대하면!!!

$$P(A|B) = P(A|B_1 \cap B_2 \cap B_3 \cap \dots \cap B_n) = \frac{P(B_1 \cap B_2 \cap B_3 \cap \dots \cap B_n)P(A)}{P(B)}$$

**독립성 가정으로
+ 가우스분포 가정**

$$P(A|B_1 \cap B_2 \cap B_3 \cap \dots \cap B_n) = \frac{P(B_1|A)P(B_2|A) \dots P(B_n|A)P(A)}{P(B)}$$

→ $P(A|B)$ 를 최대화 시키는 가우스 함수의 평균을 찾는다.

*나이브 베이즈 분류기(Naive Bayes Classifier)

과정

1. Feature와 Label을 파악하는 것

(Label은 결과, Label 결과에 영향을 주는 요소 Feature)

2. Training data set을 통해 classifier 모델 훈련

3. 테스트 단계에서는 classifier 모델의 성능 (performance) 평가

(정밀도(accuracy), 정확성(Precision), 재현율(Recall) 등으로 측정)

1. 기업별 수집된 특성에 따라 부도(1), 정상(0) 분류

2. 가우스 분포의 가정 사용한 생성된 훈련 셋 : 특성에 대한 평균, 분산 값 산출

3. $P(\text{부도}) = P(\text{정상}) = 0.5$ 로 각 클래스의 동등한 값을 할당

4. 부도기업 사후 확률, 정상기업 사후확률 측정

5. 새롭게 주어진 data가 부도(1), 정상(0)의 사후 확률 중 더 높은 범주(부도 또는 정상)로 분류해 줌

*나이브 베이즈 분류기(Naive Bayes Classifier)

- $P(A|B)$: 사건 B를 관측한 후에 그 원인이 되는 A의 확률을 새로 산출한다는 의미 → 사후확률(posterior probability)
- $P(A)$: 사전확률(prior probability) 사건 B가 발생하기 전에 가지고 있던 정보 → 부도기업 확률

- $P(\text{부도기업})=0.1$ 도산 , $P(\text{정상기업})=0.9$
- $P(\text{차입금}|\text{부도기업}) = 0.4$, $p(\text{차입금}|\text{정상기업}) = 0.33$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$P(\text{부도기업}|\text{차입금}) ??$

$P(\text{부도기업}|\text{차입금})$

$$= \frac{p(\text{차입금} | \text{부도기업}) \times p(\text{부도기업})}{p(\text{차입금} | \text{부도기업}) \times p(\text{부도기업}) + p(\text{차입금} | \text{정상기업}) \times p(\text{정상기업})}$$

$$= \frac{0.4 \times 0.1}{0.4 \times 0.1 + 0.033 \times 0.9} = 0.57$$

*나이브 베이즈 분류기(Naive Bayes Classifier)

나이브 베이즈의 장점과 한계점

장점

1. 모형이 단순, 빠르며, 정확한 모델

: 특징간 독립성 가정으로 인해 독립변수의 차원이 늘어날 경우 늘어난 독립변수의 수에 비해 기하급수적으로 많은 데이터를 필요로 하는 차원의 저주(curse of dimensionality) 문제 완화

: 선형 분류기(로지스틱 회귀)보다 훈련 속도가 빠른 편이지만, but 일반화 성능은 뒤떨어지는 것으로 알려짐.

2. 큰 데이터셋에 적합

3. 연속정보보다 이산형 데이터에서 성능이 우수

한계점

단순히 학습에 사용하는 데이터를 이용하여 확률 값을 계산하여 그 크기를 비교하기 때문에 학습에 사용되는 데이터에 따라 분류 성능이 차이가 크게 난다는 특징

1. 예측값은 범주형변수

→ 데이터에 연속형변수가 포함되면 범주형 변수로 변환 필요

2. feature 간의 독립성 확보 (Naïve)

→ X1과 X2라는 feature가 있을 때 X1이 증가하면 X2도 같이 증가하는 경우 X1과 X2는 서로 상관관계가 있다고 말할 수 있고, 이는 X1과 X2가 독립성이 없다는 의미

3. Feature 중요도(가중치)가 동일하다.→ 산술평균

*나이브 베이즈 분류기(Naive Bayes Classifier)

활용

- 부실기업 예측
- 텍스트 분류
- 스팸 메일 필터
- 감성 분석(평가, 정치성향),
- 추천 시스템 등에 활용
- 의학적 질병 진단

sklearn

scikit-learn : sklearn.naive_bayes 모듈은 아래 3가지 클래스 제공

- **GaussianNB**: 정규분포 나이브베이즈 → 연속 data

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

```
from sklearn.naive_bayes import GaussianNB
```

- **BernoulliNB**: 베르누이분포 나이브베이즈

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.BernoulliNB.html

- **MultinomialNB**: 다항분포 나이브베이즈

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

- 딥 러닝을 활용한 재무제표 상 회계부정 탐지기법 연구 2019 학위논문
- 나이브 베이즈 빅데이터 분류기를 이용한 렌터카 교통사고 심각도 예측 2017

* Global vs. Local

"전역(Global) 수준"이란

모델 전체 또는 전체 데이터 관점에서 특성(feature)이 예측값에 미치는 **평균적인(또는 전반적인) 영향**을 보는 방식을 뜻한다.

• **모델 전체**: 특정 인스턴스(샘플) 하나가 아니라, 데이터셋에 포함된 모든 샘플을 고려

• **평균 효과**: 각 특성값 변화에 대한 예측값 변화를 개별 예측값이 아니라 "모든 샘플에 대해 평균"하여 계산

• 모델 전체의 동작 원리나 피쳐(feature)들이 모델 예측에 미치는 전반적인 영향력을 이해하는 데 초점을 맞춘 설명 기법

• 개별 샘플이 아니라 "모델 전체"를 바라본다는 점이 특징

관점	전역(Global)	국지(Local)
설명 대상 scope	전체 데이터셋(모델 전체)에 대한 평균적·전반적 효과	특정 인스턴스(샘플) 하나에 대한 개별적 기여
예시 기법	• Feature importance • Permutation (Feature) Importance • PDP(Partial Dependence Plot)	• SHAP • LIME • ICE(Individual Conditional Expectation)
장점	모델이 전반적으로 어떻게 동작하는지 한눈에 파악 가능	개별 예측 이유를 상세히 이해 가능
단점	특성 간 상호작용이나 개별 편차는 감춰질 수 있음	전체적인 경향성 파악에는 적합하지 않음

예시

•PDP(전역)

나이(age)를 20→30→40... 식으로 바꿔가며, 모든 샘플에 대해 예측값을 계산한 뒤 평균을 냄

•SHAP(국지)

어떤 특정인(예: 나이 45, 소득 5천만원인 샘플)에 대해, 나이가 예측에 얼마나 기여했는지를 개별적으로 계산

* Interpretability 과 Explainability

1. 해석력(Interpretability) 과 설명력(Explainability)

1) 해석력(Interpretability)

- “모델 자체(내부)가 얼마나 직관적으로 이해 가능한가?”를 나타냄.
- 선형 회귀, 의사결정나무처럼 모델 구조를 들여다보는 것만으로도 “왜 이 예측값이 나왔는지” 바로 파악할 수 있어야 함.

▪ 주요 특징

모델 자체(내재적) 해석성

모델의 아키텍처나 수식이 간단하여 “입력 → 출력” 관계를 일목요연하게 살펴볼 수 있음.

예:

선형 회귀(Linear Regression): 가중치(회귀 계수)만 보면 “특성 x_j 가 예측값에 얼마나 비례해 영향을 주는지” 알 수 있음.

의사결정나무(Decision Tree): 트리 구조 상에서 각 노드의 분할 조건을 따라가면 “왜 이 샘플은 A 클래스인가?”를 단계별로 추적할 수 있음.

규칙 기반 모델(Rule-based model): “if-then” 형태의 규칙 집합으로 명시적 해석 가능.

“화이트박스(White-Box)” 모델

설계자가 모델의 수식 구조, 매개변수(Parameters), 분기 조건 등을 **직접 들여다보고 해석**할 수 있는 형태를 말한다.

해석력이 높다는 것은 곧 “왜 이런 예측이 나왔는지 내부적으로 납득 가능한 이유를 바로 설명할 수 있다”는 의미로 연결된다.

2) 설명력(Explainability)

- Black-Box 모델(딥러닝, 앙상블 등)의 예측 결과를 외부적 (external) 으로 해석할 수 있는 정도”를 의미.
- 즉, 모델의 내부 구조를 꼭 그대로 보여주진 않더라도, “특성(feature)이나 데이터 관점에서 ‘왜 이 답이 나왔는지’ 사용자나 의사결정권자에게 이해 가능한 형태로 설명”해 주는 것을 의미한다.
- LIME, SHAP, Integrated Gradients, DeepLIFT 등 사후 해석 기법을 통해 “특성 기여도” 또는 “국소 의사결정 경계”를 도출.

주요 특징

모델 외부(사후) 설명

원래 내부 로직이 복잡하거나 숨겨져 있는 딥러닝·앙상블·부스팅 트리 등 고성능 블랙박스 모델에 대해, 입력 특성이 어떻게 예측 결과에 기여했는지*를 **별도 알고리즘(예: SHAP, LIME, Integrated Gradients 등)을 통해 정량적·시각적으로 제시**해 주는 것. → 따라서 설명력은 “**사후 해석(Post-hoc Interpretation)**”이라고도 부른다.

“블랙박스 모델에 대한 가시화/해석 도구”

내부 파라미터나 네트워크 가중치를 해석하기 어렵더라도, **입력-출력의 관계를 바탕으로 “이 특성이 얼마나 기여했는지”, “어떤 규칙성(pattern)이 작용했는지”** 등을 도출해 줌으로써, 최종 사용자나 비전문가에게 이해를 돕는다.

3) 판단

- 어떤 문제에 대해서는 “해석력 높은 단순 모델”이 더 적합할 수 있고, 다른 문제(고차원·복잡·성능 최우선)에서는 “블랙박스 + 사후 설명력” 방식이 현실적일 수 있다.

* Interpretability 과 Explainability

2. 해석력(Interpretability) 과 설명력(Explainability) 판단

- 어떤 문제에 대해서는 "해석력 높은 단순 모델"이 더 적합할 수 있고, 다른 문제(고차원·복잡·성능 최우선)에서는 "블랙박스 + 사후 설명력" 방식이 현실적일 수 있다.
- 조직·도메인 요구사항, 규제·컴플라이언스, 모델 성능 목표, 이해관계자 이해도 등을 종합적으로 고려하여, "해석력 vs 설명력" 중 어디에 더 중점을 둘지 결정해야 한다.

3. 관점별 비교

1.설계 시점

- 해석력:** 모델 학습 전부터 "어떤 모델을 쓸 때 해석성이 보장되는지"를 염두에 두고 모델을 선택한다.
- 설명력:** 모델 학습 후 "이미 학습된 블랙박스 모델을 어떻게 설명해 줄 것인지"를 고민한다.

2.본질(Internal vs External)

- 해석력(Internal):** 모델 파라미터·구조 자체를 보면서 "입력 → 출력" 과정을 곧바로 추적할 수 있어야 함.
- 설명력(External):** 모델 구조를 그대로 드러내지 않더라도 "입력 특성 기여도"나 "국소적 결정 경계" 등을 도출하여 설명.

3.목표 사용자

1) 해석력:

- 연구자·모델 설계자(데이터 사이언티스트) 입장에서 "왜 이 모델이 유리한가"를 알고 싶을 때.
- 규제 당국·금융권 등 "모델 자체가 해석 가능해야 한다"는 요건이 있을 때.

2) 설명력:

- 비전문가(비즈니스 의사결정권자, 도메인 전문가)에게 "이 예측 결과는 왜 나왔는가"를 설명하고자 할 때.
- 실사용자가 "특정 속성이 예측에 얼마나 영향을 주었는가"를 이해하고 싶을 때.

4.성능-해석성 트레이드오프(Trade-off)

- 해석력 높은 모델:** 종종 복잡한 비선형 관계를 포착하기 어려워 예측 성능(Predictive Power)이 떨어질 수 있음.
- 블랙박스 모델(예: 딥러닝, 앙상블 모델):** 예측 성능은 높지만 해석이 어려움. → 이때 사후 해석 방법(설명력)을 적용하여 타협점을 찾음.

* Interpretability 과 Explainability

4. 구체적인 사례: 해석력 vs 설명력 사례 1: 간단한 신용 대출 승인 모델

- 문제: 은행에서 신용 대출 신청자의 *****대출 승인 여부*****를 예측해야 한다.
- 데이터: 연령, 연봉, 부채 비율, 신용 점수, 기존 대출 회수 등 다수의 피처(features) 존재.

(A) 해석력이 높은 모델을 선택한 경우

모델: 간단한 **의사결정나무(Decision Tree)** 또는 **로지스틱 회귀(Logistic Regression)**

1) 장점(해석력)

- **의사결정나무**: 트리 구조에서 "if 연령 > 35 and 신용 점수 > 700 then 승인" 식으로 규칙이 명확히 드러남. 은행 리스크 팀이 "왜 이 사람이 떨어졌는지"를 나무 경로(분기 기준)를 보면 바로 이해 가능.
- **로지스틱 회귀**: 각 피처에 대한 회귀계수(가중치) 값만 보면, 직관적 해석 가능하다.

2) 단점(성능)

- 복잡한 비선형·교호작용(Interaction) 효과를 포착하기 어려워, 모델 성능(예: AUC)이 너무 낮을 수 있음.
- 실 데이터에 노이즈가 많거나 피처 간 관계가 복잡하면, 미묘한 위험 요인을 놓칠 위험이 있음.

(B) 설명력이 높은 모델을 선택한 경우

모델: XGBoost(Gradient Boosting Decision Tree) 기반 강력한 앙상블 모델

1) 성능

- 일반적으로 트리 기반 앙상블은 시그모이드 함수로만 선형 결정을 하는 로지스틱 회귀보다 훨씬 높은 예측 정확도를 보임.
- 비선형 관계, 상호작용, 희소성 데이터(sparse data)에 대해 강건함.

2) 설명력 확보

TreeSHAP (SHAP의 트리 전용 가속화 버전)을 사용하여, 개별 고객 x^{**} 에 대해 SHAP 값을 계산 → "각 피처가 최종 예측(승인 확률)에 기여한 정도(ϕ_j)"를 구함.

또는 LIME을 사용해 고객 주변 국소 로컬 샘플을 생성한 뒤, "가중 회귀"를 통해 해당 국소 영역에서 피처 기여도 산출.

3) 장점(성능 + 설명력)

실제 성능은 높으면서, 나중에 사후 해석 기법을 통해 "왜 이 예측값이 나왔는지" 설명해 줄 수 있어, 비즈니스 부서나 규제 기관에서도 충분히 납득 가능한 수준의 설명력을 확보함.

4) 단점

: 사후 해석(설명력) → 해석력(속성 파악)보다 약간 덜 직관적

- SHAP 값을 계산하기 전까지는 모델 내부 로직이 복잡해서 선형 회귀처럼 "가중치=해석"이 바로 안 됨.
- 계수 회귀 모델처럼 "한 번에 전체 모델 구조"를 들여다볼 수 있는 건 아니지만, 개별 예측에 대한 기여도는 명확히 수치화 가능.

* Explainable Artificial Intelligence : XAI

■ 설명가능한 AI (Explainable AI, XAI), XML

- AI-머신러닝 모델의 결정 과정을 사람(최종 사용자, 도메인 전문가, 규제 기관 등)이 이해할 수 있도록 "왜 이런 결과가 나왔는가"를 설명해 주는 기술과 방법론을 통칭
- 특히 블랙박스 성격이 강한 딥러닝 모델이나 복잡한 앙상블 모델의 결정 과정을 투명하게 드러내고자 할 때 중요

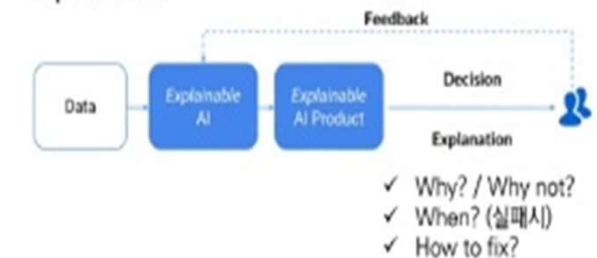
1. 왜 설명가능성이 필요한가?

- 1) **신뢰 구축** : 모델의 예측 근거를 이해해야 사용자나 도메인 전문가가 결과를 신뢰할 수 있다.
- 2) **모델 검증 및 디버깅** : 예측 오류나 편향(bias)이 발생했을 때, 어떤 특성(feature)이 잘못된 결과를 초래했는지 파악해 개선할 수 있다.
- 3) **규제 준수** : 금융, 의료, 법률 등 규제 산업에서는 "왜 이런 결정을 내렸는지"를 설명하도록 요구한다.
- 4) **윤리적·법적 책임** : 예를 들어, 대출 심사에서 거절된 고객에게 이유를 알려줘야 하는 경우(XAI: eXplainable AI)

Black Box AI



Explainable AI



2. 설명가능성의 분류

모델 투명성 (Transparency)	글로벌 투명성	모델 전체 구조와 파라미터가 명확히 이해되는 경우 예: 선형 회귀, 의사결정 나무(Decision Tree)
	지역(local) 투명성	특정 예측 결과에 대해서만 이해 가능 예: 회귀 계수, 가지 분할 기준
설명 대상	Global Explainability	모델 전체 동작 방식 요약
	Local Explainability	개별 샘플(인스턴스) 예측 이유 설명

* Explainable Artificial Intelligence : XAI

3. 주요 기법들

모델 내장형(Intrinsic) 접근	해석 가능한 모델 사용	선형 모델(Linear Models): 가중치 해석 가능
		의사결정 나무(Decision Trees): 분기 규칙으로 경로 추적
		규칙 기반 모델(Rule-based Models)
사후 설명(Post-hoc) 접근	모델 학습 후, 블랙박스 모델에 설명자를 덧씌우는 방법	SHAP (SHapley Additive exPlanations) : 게임 이론의 샤플리 값(Shapley value)을 차용해 각 특성이 예측에 기여한 정도를 계산
		LIME (Local Interpretable Model-agnostic Explanations) 관심 인스턴스 주변에 국소적 데이터를 생성하고, 단순 모델(선형 회귀 등)으로 근사하여 설명
		Partial Dependence Plot (PDP) 특정 특성 값 변화에 따른 예측 결과 평균 변화를 시각화
		Surrogate Model 복잡한 모델을 의사결정 나무나 규칙 모델로 근사하여 전체 또는 부분 설명
		Counterfactual Explanation "이 예측을 다르게 만들려면 어떤 특성을 어떻게 바꿔야 하는가?" 제시

4. 성능평가, 주의사항

- 정밀도 vs. 해석력 Trade-off: 복잡한 모델일수록 해석이 어렵고, 단순 모델일수록 성능이 떨어질 수 있음.
- 설명 품질 지표:**
 - 정밀도(Approximation Accuracy): 설명 모델이 원본 예측을 얼마나 잘 근사하는지
 - 간결성(Sparsity): 설명에 사용된 피처 수가 적을수록 이해하기 쉬움
 - 안정성(Consistency): 유사 입력에 대해 유사한 설명 제공
- 오해 방지: 설명은 근사·추정 결과이므로, "완전한 진실"이 아니라 "이해를 돕는 도구"임을 인지해야 한다.

* Interpretable AI

해석가능한 AI	모델 구조나 내부 동작이 직관적으로 이해될 수 있어, 입력과 출력 간 관계를 사람이 직접 '읽어내는' 것이 가능한 AI.
설명가능한 AI (XAI)	복잡한 블랙박스 모델(예: 딥러닝, 앙상블 등)의 의사결정을 사후적으로 분석·시각화하여, "왜 이렇게 예측했는가"를 사용자에게 설명해 주는 AI.

해석가능한 AI

•모델 단순성

- 선형 회귀, 결정 트리, 규칙 기반 시스템 등 구조가 단순하여 내부 수식이나 규칙을 사람이 바로 이해 가능

•직관적 이해

- 각 피처(feature)가 출력에 미치는 영향을 직접적으로 파악할 수 있음

•제약

- 모델의 복잡도가 낮아 표현력이 제한적 → 고차원·비선형 관계 학습에는 한계

설명가능한 AI (XAI)

•블랙박스 지원

- 복잡한 모델의 예측 결과를 해석하기 위한 다양한 기법 제공 (LIME, SHAP, Grad-CAM 등)

•사후 분석(post-hoc analysis)

- 모델 학습 후에 출력 결과를 설명하는 방식으로, 모델 자체를 단순화하지 않고도 '왜 그런 결론에 이르렀는지' 설명

•유연성

- 복합적인 구조를 유지하면서도 다양한 시각화·정량화 도구를 활용해 설명 가능

* Interpretable AI

예시

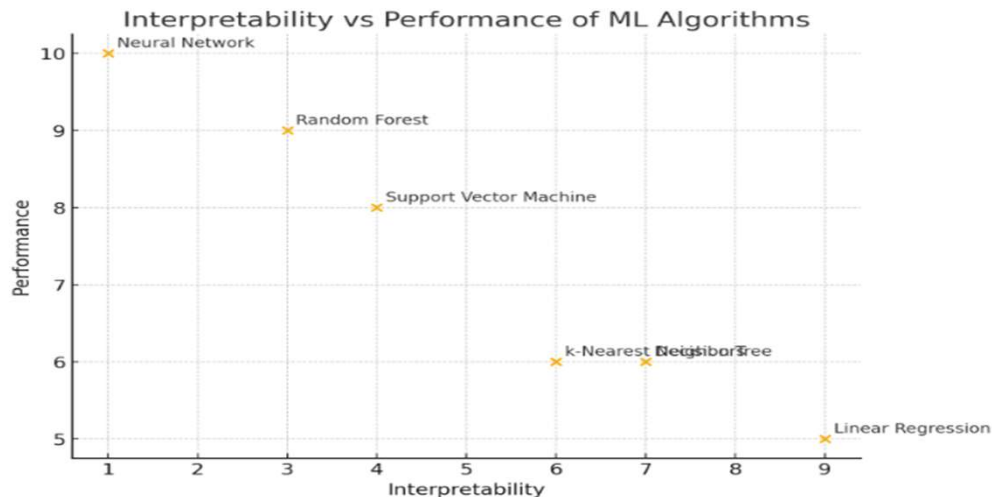
해석가능한 AI 예시

- 결정 트리: 분할 기준과 가지(branch)의 조건이 모두 사람이 읽을 수 있는 규칙 (if-then) 형태
- 선형 회귀: 가중치(coefficients)가 직접적으로 변수의 영향력을 의미

설명가능한 AI 예시

- LIME (Local Interpretable Model-agnostic Explanations): 특정 예측 결과 주변의 입력을 조금씩 변형해 가며 출력 변화를 관찰, 영향도 가중치 산출
- SHAP (SHapley Additive exPlanations): 게임이론 기반으로 피처별 기여도를 공정하게 분배

항목	해석가능한 AI	설명가능한 AI
모델 복잡도	낮음 (단순 모델)	높음 (딥러닝, 앙상블 등 블랙박스 모델)
이해 시점	학습 전·중·후 모두 가능	주로 학습 후 사후 분석 방식
이해 주체	모델 자체의 구조를 사람이 직접 이해	외부 해석 기법을 사용하여 사람이 이해
설명 정확성	내재적 설명 (intrinsic)	근사적 설명 (approximate)
적용 사례	금융 리스크 모델, 간단한 분류·회귀 문제	의료 영상 판독, 자율주행 의사결정, 추천 시스템 등 복잡 영역



* Model-agnostic

Model-agnostic(모델 비종속) 기법이란?

특정한 머신러닝 알고리즘(예: 결정트리, 신경망, SVM 등)에 의존하지 않고, “어떤” 모델이든 설명할 수 있는 범용적인 설명 기법을 말한다.

1. 특징

- 알고리즘 독립성
내부 구조나 파라미터에 접근할 필요 없이, 입력-출력 관계만으로 작동
- 범용 적용성
분류·회귀·클러스터링 등 다양한 문제 유형, 단일 모델 또는 앙상블 모델에 모두 사용 가능
- 외부 설명자(Explainer)로 동작
원본 모델을 “black box”로 간주하고, 그 위에 별도의 설명 모델(혹은 추정 절차)을 얹어 해석

2. 주요 기법 및 예시

기법	개요	장점	단점
LIME	입력 주변에 샘플(perturbed data)을 생성 → 단순 모델(선형회귀·의사결정나무)로 근사 → 지역 설명	쉽고 빠르게 적용 가능	불안정성(샘플링에 따라 결과가 달라짐)
Kernel SHAP	샐플리 값 이론을 일반 모델에 적용하기 위해, 특정 가중치 커널을 이용한 샘플링 기반 근사	공정성(fairness) 속성 보장, 수학적 근거 명확	계산 비용이 높고 느릴 수 있음
Permutation Importance	각 특성을 무작위 섞은 뒤 성능 저하량을 측정 → 중요도 산출	구현이 간단, 전체 모델 기여도 직관적 파악	특성 간 상호작용 무시 가능성
Surrogate Model	원본 블랙박스 모델을 (전역 또는 국소) 의사결정나무 등 해석 가능한 모델로 근사	전체 모델 동작 방식(글로벌) 한 눈에 파악 가능	근사 정확도 한계, 복잡도 조절 필요

* Model-agnostic

Model-agnostic(모델 비종속) 기법이란?

3. 왜 Model-agnostic 기법을 쓸까?

- 다양한 모델에 통일된 워크플로우

팀 내에 여러 가지 모델이 혼재해 있어도, 동일한 설명 도구로 결과를 비교·공유 가능

- 소스 코드·파라미터 비공개 모델 해석

SaaS 형태의 외부 API나 상용 라이브러리처럼 내부 접근이 불가능한 경우에도 사용

- 추가 학습 비용 절감

새로운 모델마다 별도의 설명 기법을 배우지 않아도 됨

Model-Agnostic Interpretation method

Model-Agnostic Interpretation methods

모델-독립적 해석 방법 (Model-Agnostic Interpretation Methods)

- 특정 기계 학습 모델에 종속되지 않고, 다양한 모델에 대해 적용할 수 있는 해석 기법을 말한다.
- 이러한 방법들은 모델의 예측을 이해하고 설명하는 데 사용되며, 어떤 모델이든 적용할 수 있기 때문에 매우 유연하다.

구분	내용
Feature importance	
SHAP (SHapley Additive exPlanations)	. SHAP은 셰이플리 값을 기반으로 한 방법으로, 각 특성이 모델 예측에 기여하는 정도를 공정하게 평가한다. → SHAP 값은 모든 가능한 특성 조합에서 해당 특성을 추가했을 때의 변화량을 평균하여 계산한다.
LIME (Local Interpretable Model-agnostic Explanations)	LIME은 개별 예측을 설명하기 위해 사용되는 방법으로 , 모델이 특정 예측을 내린 이유를 이해할 수 있게 한다. → 개별 예측을 로컬 선형 모델로 근사하여 설명
PDP (Partial Dependence Plot)	특정 특성과 종속변수 간의 관계를 시각화 하여 모델의 예측을 해석하는 방법
ICE (Individual Conditional Expectation)	ICE는 PDP의 확장으로, 개별 데이터 포인트의 예측 변화를 시각화 한다. •작동 원리: 각 데이터 포인트에 대해 특정 특성의 값을 변화시키면서 예측 변화를 관찰한다. •과정: PDP와 달리, 개별 데이터 포인트에 대해 예측 변화를 시각화하여 보다 세밀한 해석이 가능하다.

Model-Agnostic Interpretation method

* Feature importance

트리기반 모델에서 사용→ 중요도를 구분하는 데에 **트리의 분리와 밀접한 관련 정도**

→ 특정 feature가 트리를 분리(feature를 순수도 기준 판단)하는데 얼마나 기여를 했는지에 따라 중요도가 결정되는 것

지표는 절대적?

순수도 → gini 계수 혹은 엔트로피로 계산

'정보 획득량이 가장 높은' 'gini split이 가장 낮은' feature를 선택하여 트리를 분할하고, 중요도에 반영

feature importance는 노드가 분기 할 때의 **정보 획득량이나 지니계수만을 고려하여 중요도를 부여하기 때문에** 과적합에 대해 고려하지 못하기 때문!!!

(단점) **negative(-)을 주는 feature를 알 수 없다**

- XGBoost : Weight, Gain, Cover 3가지 feature importance 제공
- LightGBM : Gain, Split 2가지 feature importance 제공
- → gain, split 모두 중요도 점수가 떨어진다면 제외하는 것을 고려할 수 있다.
→ feature importance는 점수의 영향이 양의 영향인지, 음의 영향인지 알 수 없다는 단점이 있다.

Model-Agnostic Interpretation method

1. Permutation importance

1. 순열 중요도 아이디어

- 모델의 각 특성(feature)이 예측 성능에 얼마나 기여하는지를 평가하는 모델-아그노스틱(model-agnostic) 기법 중 하나
- feature 하나 하나마다 shuffle을 하며 성능 변화를 지켜보고 → 그 feature가 모델링에서 중요한 역할을 하고 있었다면 성능이 크게 떨어질 것이라는 개념으로 출발
- 단일 특성의 값을 무작위로 섞고 그 결과 모델 점수가 어떻게 저하되는지 관찰하는 것을 포함한다.
- → 성능 저하가 크면 그 특성의 중요도가 높고, 저하가 작으면 중요도가 낮다고 판단한다.

2. 계산 절차

1) 기준 성능 측정

검증용 데이터셋 $D = \{(x(i), y(i))\}$ 으로 모델 f 의 성능 지표(예: 정확도, RMSE 등) L_{base} 를 구함.

2) 특성별 순열 수행

각 특성 j 에 대해: 데이터셋 D 에서 j 번째 특성 열 $\{x_j(i)\}$ 만 무작위로 섞은 새 데이터셋 $D_{perm}(j)$ 을 만든다. $D_{perm}(j)$ 로 모델을 평가해 성능 $L_{perm}(j)$ 를 구한다.

3) 중요도 점수(Importance Score) 계산 : $Importance(j) = L_{perm}(j) - L_{base}$

- 회귀의 경우 RMSE가 증가한 양
- 분류의 경우 정확도·AUC 감소한 양 등을 사용할 수 있다.

Model-Agnostic Interpretation method

1. Permutation importance

3. 해석

중요도 점수(Importance Score)의 의미

$$\text{Importance}(j) = L_{\text{perm}}(j) - L_{\text{base}}$$

- 회귀라면 RMSE(또는 MSE)가 "얼마나 증가했는지"(증가량),
- 분류라면 정확도(Accuracy)나 AUC가 "얼마나 감소했는지"(감소량) 를 나타낸다.

해석:

- 점수가 크면 해당 특성을 무작위화했을 때 성능이 많이 나빠진다는 뜻 → 중요도가 높음
- 점수가 작으면 성능 변화가 거의 없다는 뜻 → 중요도가 낮음

순위화 및 시각화

- 바 차트(bar plot) 로 특성별 중요도를 내림차순 정렬해 시각화하면, 어떤 피처가 모델 성능에 가장 큰 영향을 주는지 직관적으로 파악할 수 있다.
- 누적 기여도: 상위 N개 특성만 골라 누적 중요도를 보면 "전체 성능 변화량의 몇 %를 설명하는가"도 알 수 있다.

음수(또는 0 이하) 중요도

- 0 근처: 순열한 뒤 성능 변화가 거의 없다 → 무의미한 특성일 가능성.
- 음수 : 순열 후 성능이 오히려 좋아졌다면
 - 원본 데이터에서 해당 특성이 노이즈(noise) 역할을 하거나,
 - 다른 특성과 강한 상호작용(interaction)을 일으켜 오히려 성능을 방해했을 수 있음. → 이 경우 해당 특성을 제거하고 성능 변화를 재검증해 보는 것이 좋다.

Model-Agnostic Interpretation method

1. Permutation importance

순열 중요도

4. 장점과 단점

장점	단점
• 모델 비종속: 어떤 예측 모델에도 적용 가능	• 상호작용 무시 가능성: 특성 간 강한 상호작용이 있을 때, 단일 순열로 정확히 반영되지 않을 수 있음
• 직관적 해석: 성능 저하량이 곧 중요도로 연결되어 이해하기 쉬움	• 계산 비용: 특성 개수 $\times$ 반복 횟수만큼 모델 평가가 필요해 느릴 수 있음
• 간단한 구현: 추가 학습 없이, 검증 단계에서 바로 적용 가능	• 데이터 분포 파괴: 순열로 인해 현실적이지 않은 조합의 데이터가 생길 수 있음

활용 팁

- **반복 수행:** 순열마다 무작위성이 있으므로, 여러 번 셔플해 평균값을 사용하는 것이 안정적이다.
- **교차검증과 결합:** 교차검증 결과를 특성 중요도로 종합하면 과적합 영향을 줄여준다.
- **상호작용 평가:** 중요한 특성 간에 상호작용이 의심된다면, 두 특성을 동시에 순열해 상호작용 중요도를 분석할 수 있다.

Model-Agnostic Interpretation method

Feature Importance와 Permutation importance 비교

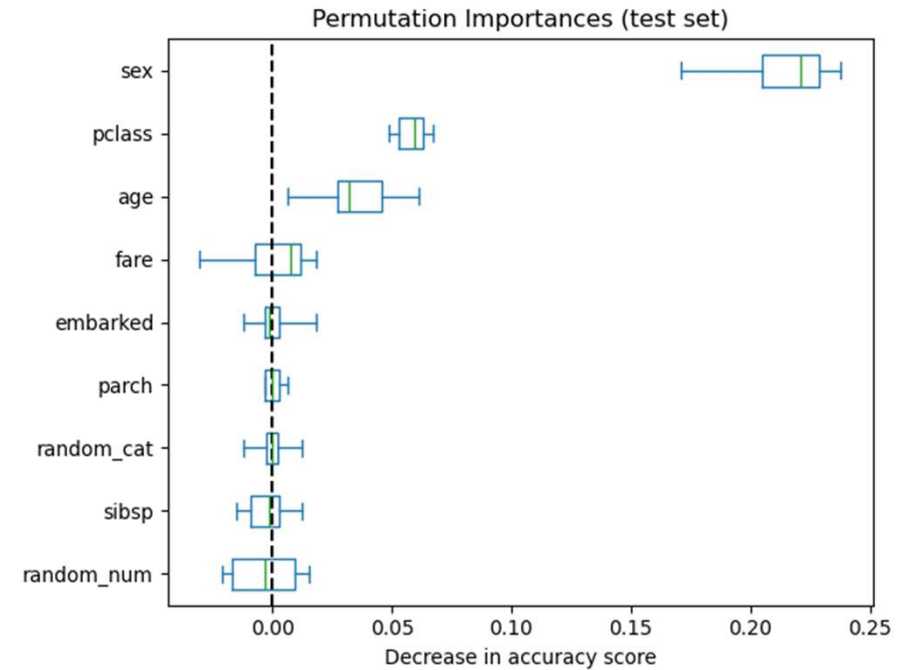
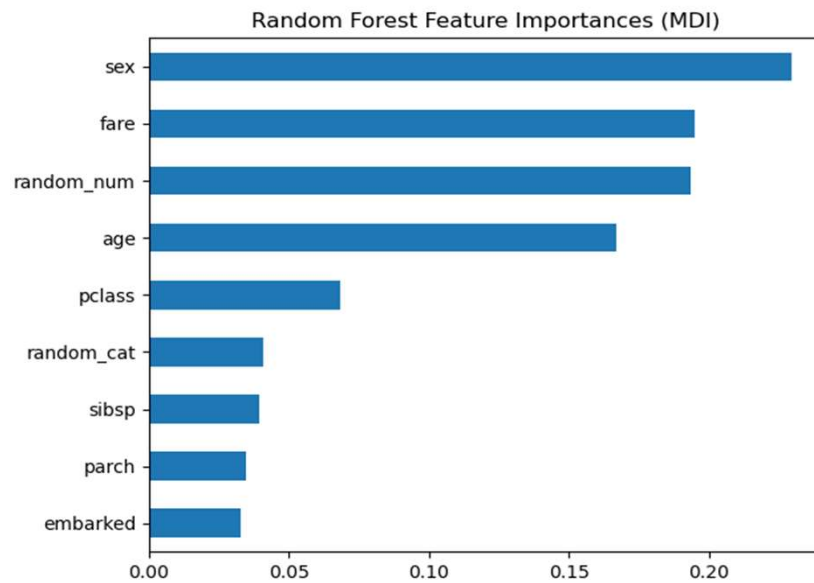
구분	전통적 Feature Importance (트리 기반)	Permutation Importance
정의	- 주로 트리 기반 모델(예: Random Forest, XGBoost, LightGBM 등)에서 사용. - 트리 분할에서 각 피처가 모델의 성능(불순도, 예: Gini, 엔트로피 등)을 얼마나 개선했는지 합산해서 측정.	- 피처 하나의 값을 무작위로 섞어 모델 성능이 얼마나 나빠지는지 측정. - 성능이 많이 떨어질수록 중요한 피처로 간주.
계산 방법	트리 학습 과정 중 노드 분할 시 감소된 불순도(Gini, 분산 등)를 누적 → Tree's Feature Importance from Mean Decrease in Impurity (MDI)	특정 피처 값을 무작위로 섞은 뒤 모델 성능(예: 정확도, RMSE) 하락량 측정
측정 시점	모델 학습 중(학습된 트리 구조 자체에서 바로 얻음)	학습 후(검증 데이터 또는 테스트 데이터에 피처 교란 적용)
해석상 차이	예를들어 A, B, C, D 피처가 "전체 모델 구조에서" 얼마나 자주 크게 분할에 기여했는지 보여줌 (장점) - 계산이 빠르고 간편 (모델 학습 중 자동으로 계산됨) - 트리모델에 최적화 (단점) - 피처 스케일에 민감하지 않지만, 편향(Bias) 있음 → 범주 수가 많거나 연속형 변수에 대해 높은 점수를 주는 경향. - 상관관계가 높은 피처들 간 중요도 분산 문제가 있음 (한 피처의 중요도가 다른 피처로 분산됨).	각 피처를 섞었을 때 모델 성능이 얼마나 떨어지는지 보여줌 (장점) - 모델 불가지론적(Model-agnostic): 어떤 모델이든 적용 가능 (트리, SVM, 딥러닝 등). - 직관적 해석 가능: 피처를 망가뜨렸을 때 성능 저하 → 중요한 피처. (단점) - 계산이 느리다: 피처 수만큼 모델을 여러 번 평가해야 하므로 계산 비용 큼. - 상관관계가 높은 피처가 있으면 해석이 어려움: 하나를 섞어도 다른 피처가 정보를 유지할 수 있음 → 중요도 낮게 나올 수 있음.

언제 무엇을 선택할까?

- 빠른 내부 구조 파악 → 트리 기반 Feature Importance
- 실제 예측 성능 관점 → Permutation Importance
- 피처 간 상호작용 공정성 고려 시 → Permutation Importance (또는 SHAP)

Model-Agnostic Interpretation method

Feature Importance와 Permutation importance 비교

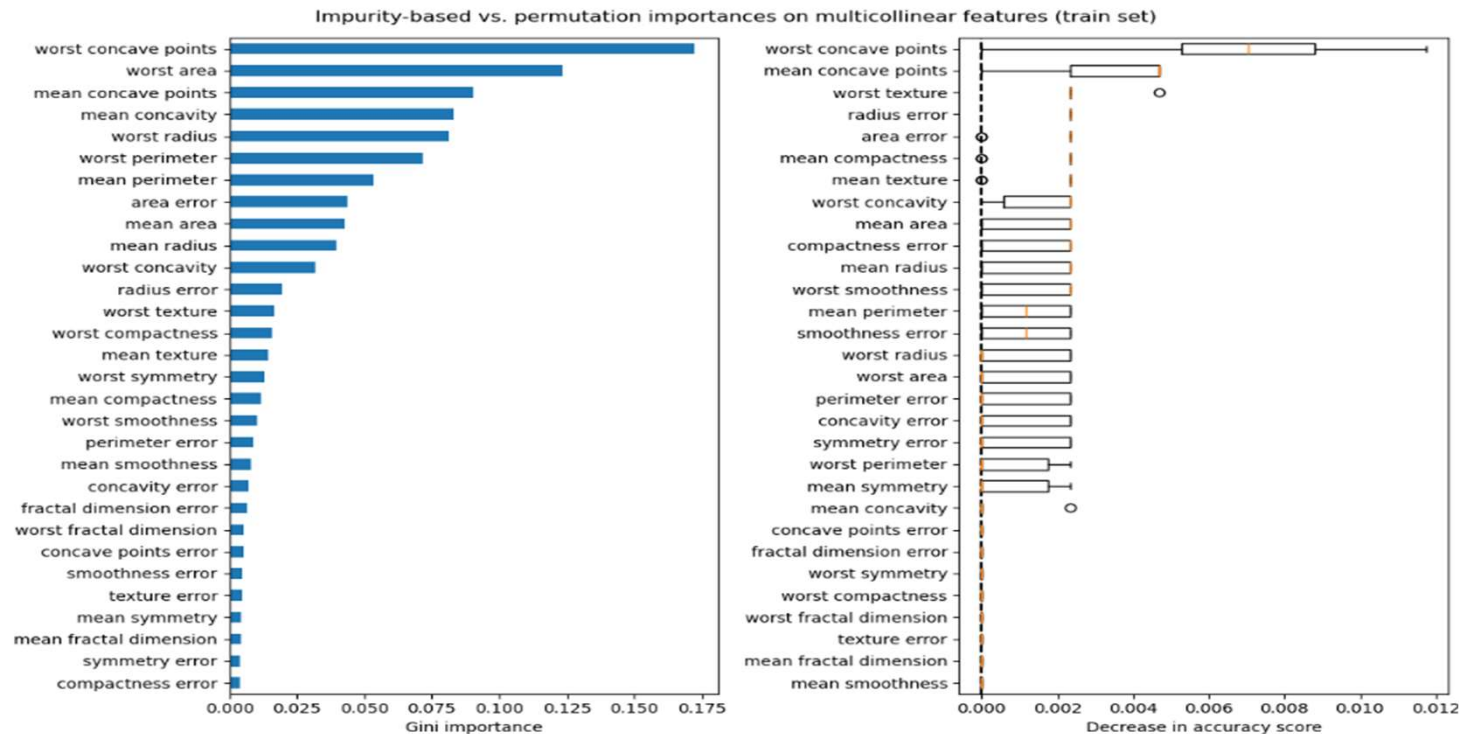


https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance.html#

Model-Agnostic Interpretation method

Permutation Importance with Multicollinear or Correlated Features

https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance_multicollinear.html#sphx-glr-auto-examples-inspection-plot-permutation-importance-multicollinear-py



As a result, **the individual feature importance** may be distributed more evenly among the correlated features. Since the features have large cardinality and the classifier is non-overfitted, we can relatively trust those values.

One way to handle **multicollinear features** is by **performing hierarchical clustering** on **the Spearman rank-order correlations**, picking a threshold, and keeping a single feature from each cluster. First, we plot a heatmap of the correlated features:

Model-Agnostic Interpretation method

*PDP(Partial Dependence Plots)

- **Permutation importance** : 모든 feature들이 타겟에 어떻게 작용하는지 알아볼 수 있는 도구
- **PDP** : 개별 특성이 타겟에 어떻게 작용하는지 알아볼 수 있는 도구 → 특정 특성(feature)이 예측값에 미치는 평균적인 영향을 시각화하여, 모델이 그 특성 값을 어떻게 "이용"하고 있는지 파악
- 작동 원리: 다른 특성 값을 고정하고, 특정 특성의 값을 변화시키면서 모델의 예측 변화를 관찰한다.
- 과정: 특정 특성의 값이 변화함에 따라 종속 변수의 평균 예측 값을 그래프로 시각화한다.

1. 정의

목적: 하나(또는 두 개)의 특성이 예측 결과에 어떤 방향·크기로 영향을 주는지 "전역(Global)" 수준에서 보여줌

수식:

$$PDP_j(x_j) = \mathbb{E}_{X_{-j}}[f(x_j, X_{-j})]$$

- x_j : 관심 대상 특성의 값
- X_{-j} : 나머지 특성들의 분포
- f : 학습된 예측 함수
- $\mathbb{E}_{X_{-j}}$: 나머지 특성들의 실제 분포를 따라 평균

2. 계산 방법

1) 관심 특성 값 범위 설정

보통 최소~최대 혹은 사분위수 범위에서 일정 간격의 값 $\{x_j(1), \dots, x_j(K)\}$ 을 선택

2) 모든 인스턴스에 대해 대체 후 예측

- 각 $x_j(k)$ 에 대해, 전체 데이터 행렬의 j 열 값을 $x_j(k)$ 로 고정
- 나머지 특성은 원본 데이터를 그대로 두고 f 로 예측

3) 평균 취하기

각 $x_j(k)$ 에 대한 예측값들을 평균 → $PDP_j(x_j(k))$

4) 시각화

x 축에 $x_j(k)$, y 축에 $PDP_j(x_j(k))$ 를 놓고 선그래프

Model-Agnostic Interpretation method

*PDP(Partial Dependence Plots)

3. 장.단점

장점	단점
- 직관적: 전역적(전체 데이터 평균) 특성 효과 파악	- 상호작용 무시: 다른 특성과의 상호작용 효과 희석
- 모델 불가지론(agnostic): 트리·신경망·SVM 등 모두 적용	- 희소 구간 불안정: 해당 특성 값이 실제 데이터에 드물면 신뢰도 하락
- 다수의 특성에 대해 한눈에 비교 가능	- 연산 비용: 데이터셋 크기·특성 값 개수에 따라 계산량 급증

4. 언제 사용할까?

- 전역적인 Feature 효과를 알고 싶을 때
- 특정 특성의 변화가 전체 예측에 어떻게 기여하는지 단조성(monotonicity) 판단
- 상호작용이 약한 특성에 대한 간단한 해석

→ 반면, 특성 간의 상호작용이 크거나 인스턴스별(로컬) 해석이 필요하다면 SHAP이나 ICE(Individual Conditional Expectation) plot을 함께 고려하는 것이 좋다.

* Shapley Value

* Shapley Value

• A Value for n-Person Games Lloyd S. Shapley (1953) Contributions to the Theory of Games, Vol. II, Annals of Mathematics Studies 28, edited by H. W. Kuhn and A. W. Tucker, Princeton University Press, pp. 307-317

- 특성값의 S,V 는 가능한 모든 특성값 조합에 대한 결과 값, 가중치 및 합계에 대한 기여도
- S.V는 집합 S에서 플레이어의 가치함수 val을 통해 정의

$$\phi_j(val) = \sum_{S \subseteq \{x_1, \dots, x_p\} \setminus \{x_j\}} \frac{|S|!(p - |S| - 1)!}{p!} (val(S \cup \{x_j\}) - val(S))$$

P : 모든 독립변수의 집합

S : 모델 내에 사용된 특성의 부분집합

x : 설명되는 인스턴스 특성값과 특성의 갯수 p인 벡터

가치함수 val : 특정 상태에서의 반환값들의 기댓값

→ 어떤 상태가 주어질 때 보상의 기댓값 즉 상태에 대한 가치함수

$$v(s) = E[G_t | S_t = s]$$

- 해당 특성(피쳐)이 포함될때와 포함되지 않을때의 차이(변화)를 통해 얻어낸 값 → shapley value는 사용된 변수 유무에 따른 평균적 결과값 변화를 통해 영향도를 추정하는 방식
- gain, split와 달리 양+/음-의 영향도를 알 수 있다
- 양(+) : 종속변수에 긍정적 요인
- 음(-) : 부정적요인 (해석 주의 ; 영향이 크다고 나쁜 것이 아니다.) → 영향이 큰 것이 중요하다.

수학적 이해하기

선형 모델 예측

$$\hat{f}(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

j번째 특성의 기여도 ϕ_j

$$\phi_j(\hat{f}) = \beta_j x_j - E(\beta_j X_j) = \beta_j x_j - \beta_j E(X_j)$$

→ 기여도 ϕ_j = 특성효과 - 평균효과

여러 특성 기여도를 합한 값

$$\begin{aligned} \sum_{j=1}^p \phi_j(\hat{f}) &= \sum_{j=1}^p (\beta_j x_j - E(\beta_j X_j)) \\ &= (\beta_0 + \sum_{j=1}^p \beta_j x_j) - (\beta_0 + \sum_{j=1}^p E(\beta_j X_j)) \\ &= \hat{f}(x) - E(\hat{f}(X)) \end{aligned}$$

→ 특성 기여도는 양수(+) 또는 음수(-)

■ 협력적(협동) 게임이론(Cooperative Game Theory)

- 플레이어들이 연합(coalition)을 형성하여 집단 보상을 어떻게 분배할 것인가를 연구

<https://github.com/freejyb/KDT/tree/main/머신러닝>

* Shapley Value

* Game Theory

- 게임이론은 합리적 의사결정자들이 상호작용하는 상황을 수학적으로 모델링하고 분석하는 학문 분야로, 경제학, 정치학, 사회과학, 생물학, 컴퓨터 과학 등 다양한 분야에 응용된다.
- 크게 비협력적 게임이론(Non-cooperative Game Theory)과 **협력적 게임이론(Cooperative Game Theory)**으로 구분

1. 게임이론의 정의 및 목적

•게임(Game)

- “플레이어(player)”들이 “전략(strategy)”을 선택하고, 그 결과로 “보수(payoff)”를 얻는 상황을 모델로 삼는다.
- 플레이어 각각은 자신의 보수를 극대화하려는 합리적(rational) 존재로 가정한다.

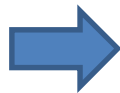
•게임이론(Game Theory)

- 여러 의사결정자(플레이어)가 상호작용(interaction)하는 상황에서, 각자의 전략 선택이 어떻게 결과에 영향을 미치는지를 수학적으로 분석하는 학문
- “상호(inter) 놀이(gameplay) 원리(theory)”라는 말 그대로, 서로의 선택이 얽혀 있을 때(예: 경쟁, 협력, 협상, 전쟁·평화, 거래 등) 어떻게 행동해야 최적의 결과를 얻을 수 있는지를 연구한다.

•목적(Objective)

- **모델링(Modeling)**: 현실의 복잡한 상호작용 상황을 간단하면서도 핵심을 담은 수학적 구조로 표현한다.
- **해석(Analysis)**: 주어진 게임 구조에서 합리적 플레이어들이 어떤 전략을 선택할지를 해(solution)로 도출한다.
- **예측(Prediction) 및 정책 제시(Policy Recommendation)**: 현실에서 사람들이(또는 기업·국가 등) 실제로 어떻게 행동할지 예측하거나, 사회 전체의 효용을 높이기 위해 어떤 메커니즘을 설계해야 하는지를 제안한다.

AI 관점



플레이어(player) → Feature

기여도 → Feature Importance

* Shapley Value

* Game Theory

2.분류

2.1 비협력적 vs 협력적

1.비협력적 게임이론(Non-cooperative Game Theory)

- 각 플레이어가 독립적으로 "어떤 전략을 선택할 것인가"에 대한 문제를 다룬다.
- 플레이어는 "연합(coalition)"을 미리 결성하거나, 결론적으로 이익을 묶어서 분배하는 과정은 모델에 직접 포함되지 않는다.
- 주로 **내쉬 균형(Nash Equilibrium)**, **하위 게임 완전 균형(Subgame Perfect Equilibrium)**, **베이저안 내쉬 균형(Bayesian Nash Equilibrium)** 같은 해법 개념을 사용한다.

2.협력적 게임이론(Cooperative Game Theory)

- **게임 시작 전, 또는 과정 중에 플레이어들이 연합을 맺고, 그 연합이 얻은 이익($v(S)$)을 어떻게 분배할 것인가를 연구한다.**
- 내부적으로는 "특성 함수(characteristic function)"와 "분배 규칙(allocation rule)"을 정의하고, **코어(Core)**, **샤플리 값(Shapley Value)**, **누크레올루스(Nucleolus)** 등 공정성·안정성을 보장하는 해법을 분석한다.
- 앞서 이미 상세히 다뤘듯, 협력적 게임이론은 "어떤 연합이 어떻게 형성되고, 얻은 이익을 어떻게 쪼갤 것인가"에 집중한다.

2.2 제로섬(Zer0-Sum) vs 비제로섬(Non-Zero-Sum)

1.제로섬 게임(Zero-Sum Game)

- 한쪽의 이익이 반드시 다른 쪽의 손실로 직결되는 게임이다.
- 두 플레이어 i, j 가 있을 때 $u_i + u_j = 0$ 이 항상 성립한다.
- 바둑, 체스, 포커(룰에 따라 다르지만) 등이 대표적인 예이며, **미니맥스(Minimax) 이론**이 중요한 도구로 사용된다.

2.비제로섬 게임(Non-Zero-Sum Game)

- 플레이어 전원이 동시에 이익을 볼 수도 있고, 모두 손해를 볼 수도 있으며, 협력적인 결과가 가능한 게임이다.
- 예: Prisoner's Dilemma(협력 시 둘 다 이익 $\rightarrow$ 합이 2), 협력적 비용 분담(Cost-sharing), Cournot·Bertrand 경쟁(기업들 수익이 양 쪽 모두 긍정적일 수 있음).

* Shapley Value 사례로 이해하기

Case/Input	X1	X2	X3	predict
C1	사용안함(X)	X	X	24
C2	사용함(O)	X	X	27
C3	X	O	X	30
C4	X	X	O	30
C5	O	O	X	36
C6	O	X	O	33
C7	X	O	O	29
C8	O	O	O	32

1. X1 shapley (ϕ_1) 계산

1) 변수 1개 사용하는 경우: X1, X2, X3 3가지 → 확률 $\frac{1}{3}$
X1 사용하지 않은 경우 수 : 1가지
 $C2 - C1 = 27 - 24 = 3$
2) 변수 2개 사용하는 경우: X1 X2, X1X3, X2X3 3가지
두 변수 중 한 변수만 사용하지 않은 경우 2가지 (전체는 6가지)
→ 확률 $\frac{1}{6}$
→ (X1를 포함하여 두 변수를 사용한 경우 - X1을 포함하지 않은 1가지 변수만 경우) → 확률 $\frac{1}{6}$
 $C5 - C3 = 6$
(X1를 포함하여 두 변수를 사용한 경우 - X1을 포함하지 않은 1가지 변수만 경우)
 $C6 - C4 = 3$
3) 변수 3개 사용하는 경우: X1, X2, X3 1가지
(세가지 변수 모두 사용한 경우 - X1 변수를 사용하지 않는 경우)
 $C8 - C7 = 3$ → 확률 $\frac{1}{3}$

$$X1 \text{ shapley 계산} = \frac{1}{3} \times 3 + \frac{1}{6} \times 6 + \frac{1}{6} \times 3 + \frac{1}{3} \times 3 = 3.5$$

2. X2 shapley (ϕ_2) 계산

1) X2 사용하지 않은 경우 수 : 1가지
 $C3 - C1 = 30 - 24 = 6$
2) 변수 2개 사용하는 경우: X1 X2, X1X3, X2X3 3가지
두 변수 중 한 변수만 사용하지 않은 경우 각각 2가지 → 6가지 경우
→ (X2를 포함하여 두 변수를 사용한 경우 - X2를 포함하지 않은 1가지 변수만 경우) $C5 - C2 = 9$
(X2를 포함하여 두 변수를 사용한 경우 - X2를 포함하지 않은 1가지 변수만 경우) $C7 - C4 = -1$
3) 변수 1개 사용하는 경우
 $C8 - C6 = -1$

$$X2 \text{ shapley } (\phi_2) \text{ 계산} = \frac{1}{3} \times 6 + \frac{1}{6} \times 9 + \frac{1}{6} \times (-1) + \frac{1}{3} \times (-1) = 3$$

3. X3 shapley (ϕ_3) 계산

동일방법으로
1) X3 사용하지 않은 경우 수 : $C4 - C1 = 6$
2) $C6 - C2 = 6$, $C7 - C3 = -1$
3) $C8 - C5 = -4$

$$X3 \text{ shapley } (\phi_3) \text{ 계산} = \frac{1}{3} \times 6 + \frac{1}{6} \times 6 + \frac{1}{6} \times (-1) + \frac{1}{3} \times (-4) = 1.5$$

$$\sum (\text{shapley value}) X_i = 3.5 + 3 + 1.5 = 8$$

→ C8(32) - C1(24) 값 8과 동일

* SHAP (SHapley Additive exPlanation)

- Shapley Value를 계산하기 위해서는 일부 feature만 있고, 해당 feature는 없는 경우로 계산함.
- Shapley Value는 feature의 기여도를 평균한 것이다. → 해당 feature 를 제외했을때의 나타난 예측값의 차이가 아님.

"A Unified Approach to Interpreting Model Predictions" (NeurIPS 2017) Su-In Lee & Scott M. Lundberg

논문 링크: <https://arxiv.org/abs/1705.07874>

- 복잡한 블랙박스 모델을 해석하기 위한 Additive Feature Attribution Methods의 통합적 틀을 제시하고,
- 그중 샤플리 값(Shapley value)을 이용한 SHAP이 갖는 수리적·공정성(fairness) 특성을 강조한다.
- 왜 이 샘플에 대해 이 특성이 이렇게 기여했는가?라는 질문에 대한 공정(fair)하고 일관성 있는 해답을 제공함으로써, 설명 가능한 AI의 이론적·실용적 토대를 마련한 기념비적 연구

A Unified Approach to Interpreting Model Predictions

Scott M. Lundberg
Paul G. Allen School of Computer Science
University of Washington
Seattle, WA 98105
slund1@cs.washington.edu

Su-In Lee
Paul G. Allen School of Computer Science
Department of Genome Sciences
University of Washington
Seattle, WA 98105
suinlee@cs.washington.edu

* SHAP (SHapley Additive exPlanation)

* Additive Feature Attribution Methods (AFA)

- 머신러닝 모델의 예측을 설명(interpretation)해 주기 위해, 입력 특성(feature)들 각각이 예측 결과에 기여하는 정도를 **더해(additive)** 나누어주는 접근 방식 → **단순한 Explanation Model**을 이용하여 비교적 복잡한 모델을 해석하는 사후 검증(Post-hoc) 기법
- 대표적인 예로는 LIME, SHAP, DeepLIFT, Integrated Gradients 등이 있다.

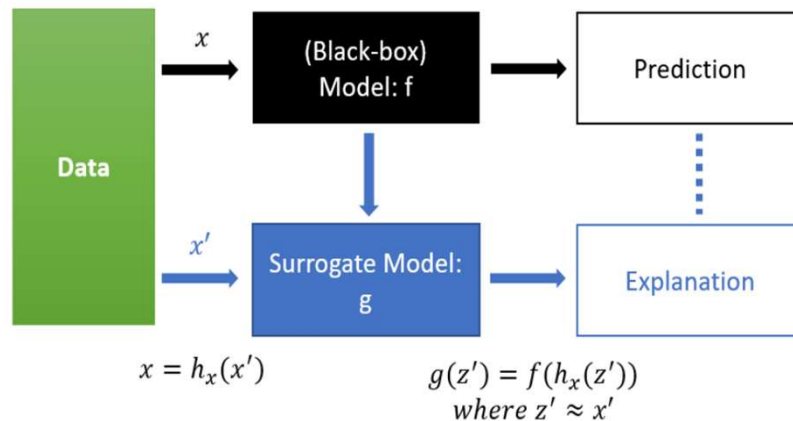
1. 개요 및 배경

설명 가능한 AI(Explainable AI)

- 복잡한 모델(예: 딥러닝, 부스팅 트리, 앙상블 모델 등)은 일반적으로 블랙박스 형태이기 때문에, “왜 이 예측값이 나왔는가”를 정량적으로 파악하기 어렵다.
- AFA 방법들은 모델 내부 구조를 직접 열어보지 않더라도(모델 “흑상자”로 간주), 모델의 입력과 출력 간의 관계를 국소적 혹은 전역적으로 설명해 준다.

Additive Feature Attribution Methods의 핵심 아이디어

- 복잡한 모델 f 의 예측값 $f(x)$ 을, “기준값(baseline)”과 각 특성 j 별 기여도 ϕ_j 를 더하는(additive) 형태로 근사 또는 분해한다.
- 수식적으로는 다음과 같은 설명모델(explanation model) g 를 정의한다.



$$g(\mathbf{x}') = \phi_0 + \sum_{j=1}^d \phi_j x'_j$$

* 선형회귀모델의 가중치를 이용하여 특성의 영향력 및 중요도 계산

- $\mathbf{x}' = (x'_1, x'_2, \dots, x'_d)$ 는 “간단화된 입력(simplified input)” 혹은 특성 존재 여부(feature presence) 벡터로, 각 $x'_j \in \{0, 1\}$ 인 경우가 많다.
- ϕ_0 는 “기준(prediction at baseline)”에 해당하며, 일반적으로 “모든 특성이 꺼져 있는 상태(예: 평균값, 0값 등)”에서 모델 f 가 내놓는 예측값
- ϕ_j 는 특성 j 가 예측에 기여하는 절대적/상대적 크기(기여도, attribution)를 나타내는 스칼라 값
→ 결국, $g(x')$ 는 “간단화된 입력” 환경에서의 선형 모형이지만, ϕ_j 를 잘 산정하면 이 선형 모형이 원래 모델 $f(x)$ 의 값을 근사하도록 만들 수 있다. AFA 방법은 바로 ϕ_j 를 어떻게 계산할 것인가에 중점을 둔다.

<https://www.youtube.com/watch?v=9haIOpIEIGM&t=321s>

* SHAP (SHapley Additive exPlanation)

* Additive Feature Attribution Methods

AFA 방법의 수식적 정의 및 속성

1 수식적 정의

1) 설명모델(Explanation Model)

단순화된 입력 $x' \in \{0,1\}^d$ 와 기준값 ϕ_0 , 각 특성별 기여도 $\{\phi_1, \dots, \phi_d\}$ 를 이용하여:

$$g(\mathbf{x}') = \phi_0 + \sum_{j=1}^d \phi_j x'_j$$

여기서 x' 는 원래의 실 데이터 $x \in \mathbb{R}^d$ 를 이진화(binomial/simplified) 한 것으로,
예: "특성 j 를 포함할 것인가(1) 혹은 제외할 것인가(0)"를 의미하거나, 혹은 "실제값 x_j 이 기준값(baseline)에서 얼마나 떨어져 있는지" 같은 부가 변환을 한 뒤, 다시 선형식 형태로 해석할 수 있도록 만든 것일 수 있다.

2) 목적 함수 (Loss) 및 가중치

- AFA 방법들은 일반적으로 "설명모델 g 와 원본 모델 f 사이의 출력 차이"를 최소화하면서, 동시에 "간단함(simplicity)"을 유지하도록 하는 목적 함수를 정의한다.
- 구체적으로는 다음과 같은 일반 형태의 Loss를 최소화한다.

$$\mathcal{L}(f, g, \pi_x) = \sum_{\mathbf{x}' \in \{0,1\}^d} \pi_x(\mathbf{x}') (f(h_x(\mathbf{x}')) - g(\mathbf{x}'))^2 + \Omega(g)$$

- $h_x(x')$ 는 "간단화된 입력 x' 를 원본 특성 공간으로 역변환(reverse mapping)"하는 함수
→ 예: " $x'_j = 1 \Rightarrow x_j = 1$ → 실제 x_j 가 1인 경우" → 기준값(baseline)인 $x_j = 0$ 형태
- $\pi_x(x')$: x 근처에서 x' 를 샘플링할 확률/가중치"를 나타내며, 로컬(local) 설명을 할지, 글로벌(global) 설명을 할지에 따라 달라진다.
→ 예: LIME은 "원래 x 와 유사한 간단화 샘플 x' 일수록 가중치가 커진다"는 방식을 사용한다.
- SHAP(Shapley-based) 계열은 모든 x' 조합을 특정 확률 분포(주로 조합 수 기반)로 샘플링한다.
- $\Omega(g)$: 설명모델 g 의 복잡도(penalization)를 뜻하며, 보통 해석 가능성을 높이기 위해 "기여도 ϕ_j 가 희소(sparse)하도록" 유도하거나, 특징 개수를 제한한다.

Model-Agnostic Interpretation method

2.SHAP (SHapley Additive exPlanation)

A Unified Approach to Interpreting Model Predictions Scott M. Lundberg, Su-In Lee (2017)
Advances in Neural Information Processing Systems (NIPS),

Attribute Importance 의 통합된 측정방식 (Unified Measyre)

- SHAP은 샐플리 값(Shapley Value)을 기반으로 모델 학습에서 독립변수들과 종속변수의 상관관계를 효과적으로 분석할 수 있는 도구
- SHAP은 학습 모델의 shapley value를 조건부평균 함수를 적용하여 값을 구함.
 - SHAP 그래프 맨 위 feature가 가장 중요
 - 중요한 feature 일수록 SHAP value의 범위가 넓다.

XAI (eXplainable AI)

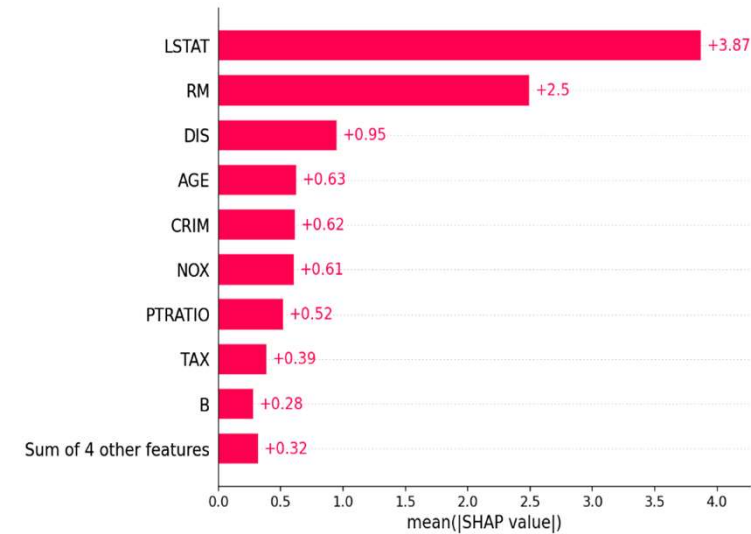
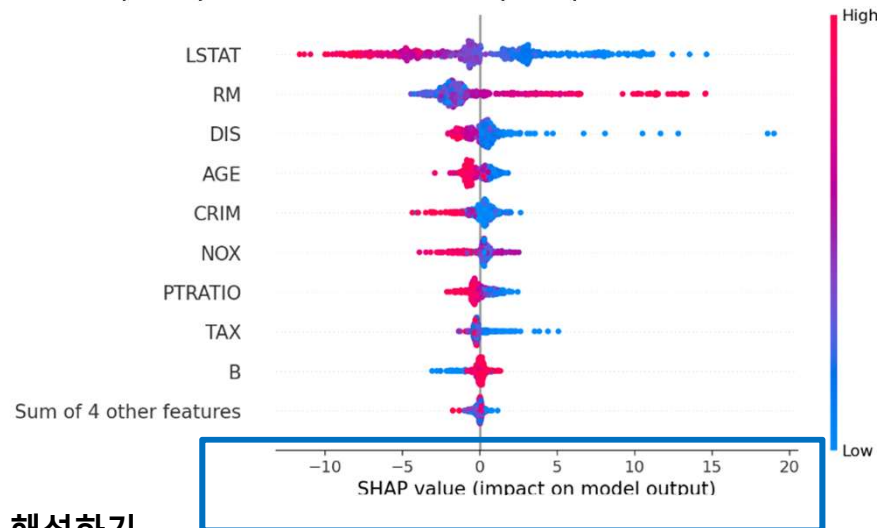
- Local Explanation을 기반으로 하여, 데이터의 전체적인 영역에 대한 해석(Global Surrogate)이 가능
- SHAP values는 어떤 특성의 조건부 조건에서 해당 특성이 모델 예측치의 변화를 가져오는 정도
- Feature Attribution의 3가지 특징(Local Accuracy, Missingness, Consistency)을 만족하는 유니크한 가산적 특성 중요도 측정(additive feature importance measure) 방식을 제공
- SHAP Value는 모델링 시 모든 변수 조합에 대해서 하나의 변수 기여도를 종합적으로 판단

Model-Agnostic Interpretation method

2.SHAP (SHapley Additive explanation, shapley value)

https://www.nature.com/articles/s41551-018-0304-0.epdf?author_access_token=vSPt7ryUfdSCv4qcyeEuCdRgN0jAjWel9jnR3ZoTv0PdqaCSN9qNY_fC0jWklQUd0L2zaj3bbIQEdrTqCczGWv2brU5rTJPxyss1N4yTIHpnSv5_nBVJoUbvejyvvjrGTb2odwWKT2BfvI0ExQKhZw%3D%3D

Code <https://github.com/slundberg/shap>



각 기능에 대한 SHAP 값의 평균 절대값을 사용 표준 막대 그래프

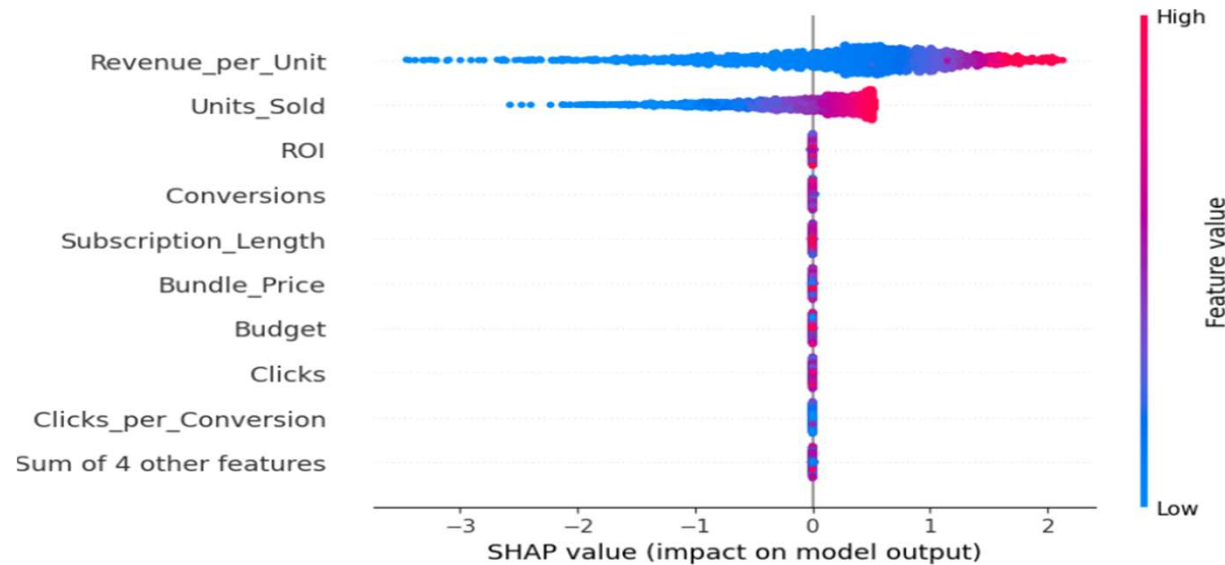
해석하기

- SHAP Value가 음수(-)라는 것은 예측값을 감소시켰다는 것을 의미하며 양수(+)는 예측값을 증가시켰다는 것을 의미한다.
- SHAP Value가 음수인 곳에서 빨간 점들이 많이 분포하는 경우 위 그림에서 피쳐 'LSTAT' 값이 높을 때(빨간색) 예측값을 감소시켰다는 것이며 반대로 SHAP Value 양수인 곳에서 파란점들이 많다는 것은 'LSTAT' 값이 낮을 때(파란색) 예측값을 증가시켰다는 것이 된다.
- 따라서 전체적으로 보았을 때 LSTAT는 예측값에 음(-)의 상관성으로 영향을 미쳤다는 것을 알 수 있다.
- RM은 SHAP Value가 음수인 곳에서 파란점들이 많고 SHAP Value 양수인 곳에서는 빨간 점이 많은 것을 알 수 있다.
- RM은 LSTAT와는 다르게 예측값에 양(+)의 상관성으로 영향을 미쳤다는 것을 알 수 있다.

Model-Agnostic Interpretation method

2.SHAP (SHapley Additive explanation, shapley value)

Kaggle : Marketing and Product Performance Dataset



1. Revenue_per_Unit

- 빨간색 점들이 양수(+)쪽에 몰려 있는데, 이는 Revenue_per_Unit 가 높을수록(빨간색) output 예측치를 높이는 것을 볼 수 있다.
- 반대로, 단가가 낮은 값(파란색)은 매출 예측을 감소시키는 방향으로 작용한다.

2. Units_Sold

판매 수량이 많을수록(빨간색) 모델은 매출을 높게 예측한다.
특히, 수량이 작은 경우(파란색)에는 예측값을 크게 낮추는 역할도 보인다.

3. 기타 피쳐 (ROI, 등)

대부분 SHAP 값이 0 근처에서 밀집되어 있고, 색깔의 경향성도 불분명하다. 이는 모델이 해당 피쳐들의 변화에 대해 거의 반응하지 않았음을 의미한다.

Model-Agnostic Interpretation method

2.SHAP (SHapley Additive exPlanation)

Shapley Value

- 협력 게임 이론에서 "각 참여자(플레이어)가 기여한 공정한 몫"을 수학적으로 정의
- 모델이 아니라, 일반적 가치(v) 분배를 다룸

SHAP

- Shapley Value 개념을 "모델 예측"으로 변환해, 각 특성이 예측에 어느 정도 기여했는지 설명
- 효율화 알고리즘(TreeSHAP 등)과 시각화 도구를 제공해 실제 머신러닝 워크플로우에 통합
- 전역·국지 해석, 상호작용 효과 등 다양한 분석 니즈를 지원

구분	Shapley Value	SHAP
기원·목적	• 협력 게임 이론 (1953년 Shapley 제안)	• 머신러닝 모델의 예측을 설명하기 위해 Shapley Value 아이디어 차용
설명 대상	• 플레이어(참여자) 간 가치 분배	• 특성(feature) 들이 예측값에 기여한 정도 분배
적용 범위	• 이론적(게임의 보상 분배, 공정성 연구 등)	• 트리/신경망/SVM 등 다양한 모델에 적용 가능한 해석 기법
계산 복잡도	• 조합적(특성 수가 늘어나면 계산량 급증)	• TreeSHAP, KernelSHAP 등 효율화 알고리즘 제공
보장되는 속성	• 효율성(Efficiency), 대칭성(Symmetry)	• 효율성, 일관성(Consistency), 대칭성(Symmetry) 등
		• 로컬 정확도(Local Accuracy) 추가 보장
활용 예	• 협력 게임 분석, 경제·네트워크 기여도 측정	• 개별 예측 해석, 전역·국지 해석, 피쳐 중요도 시각화

Model-Agnostic Interpretation method

Feature Importance vs SHAP 비교

구분	Feature Importance	SHAP(Shapley Additive exPlanations)
기본 아이디어	모델 학습 과정에서 얻은 지표(예: Gini 중요도, 분할 기준의 감소량 등)를 통해 각 특성의 상대적 중요도를 측정	게임 이론의 샤플리 값(Shapley value)을 이용하여 각 예측에 기여한 특성별 기여도를 공정하게 분배
설명 범위	글로벌(Global) : 모델 전체 관점에서 특성의 중요도 순위를 제공	로컬(Local) : 특정 인스턴스(샘플) 관점에서 특성 기여도를 제공
이론적 보장	별도의 공정성/일관성 보장 없음	공정성(Fairness), 일관성(Consistency), 로컬 정확도(Local Accuracy) 등의 샤플리 값 속성 보장
해석 용이성	전반적인 중요 순위 파악이 쉬움	인스턴스별 상세 기여 파악 가능
계산 비용	상대적으로 낮음 (특히 트리 모델)	높은 편 (특히 특성 개수가 많을 때)
공정성·일관성(consistent)	보장되지 않음 (특성 간 상호작용 무시) → Split 순서 차이에 따라 결과가 달라짐	샤플리 이론에 따른 공정한 분배 보장
전역 vs. 국지	전역(Global)	국지(Local) + 전역 요약 가능

Model-Agnostic Interpretation method

2.SHAP (SHapley Additive explanation)

- 장점

- ① 모든 학습 데이터에 대한 예측값을 기준으로 시작. (LIME과의 차이점)
- ② 각 특성의 기여도를 명확히 이해할 수 있어, 모델의 해석 가능성을 높인다.
- ③ 모델이 특정 예측을 내린 이유를 직관적으로 설명할 수 있다.

- 단점

- ① KernelSHAP의 경우 모든 순열에 대해 계산을 진행해야 하므로 시간이 많이 소요되어 속도가 느리다.
- ② SHAP value를 원인/결과로 해석할 여지가 있음 (인과관계 아님)

연구사례

- 새플리 값 추정을 이용한 주가 변화 원인 설명 문장 분류 모델 분석(2020)
- 증권 금융상품 거래 고객의 이탈 예측 및 원인 추론(2020)
- 설명 가능한 정기예금 가입 여부 예측을 위한 앙상블 학습 기반 분류 모델들의 비교 분석 2021

Model-Agnostic Interpretation method

* Variant SHAP

- SHAP(Shapley Additive Explanations)는 머신러닝 모델의 예측을 설명하기 위해 각 피처(feature)가 예측값에 기여한 정도를 정량화한 방법
- 기본 SHAP 외에도 **데이터 유형이나 모델 구조에 맞춰** 효율성·정확성을 개선한 여러 변형(variant)들이 제안

1. Kernel SHAP

- 적용 대상: 임의의(black-box) 모델
- 변형 포인트: 원래 Shapley 값을 계산하려면 가능한 모든 피처 조합에 대해 모델 예측을 평가해야 하지만, 이 조합 수는 피처 수에 따라 기하급수적으로 늘어나는 문제점이 있다.
- 변경 내용**
 - 피처 조합 전체 대신 **"가중 표본 추출(weighted sampling)"** 기법을 사용
 - 각 조합에 대해 모델을 호출하는 횟수를 제한하고, **중요도가 높은 조합에 더 많은 샘플링 비중을 부여**
 - ➔ **Linear LIME + Shapley Value** : LIME 은 관측치가 원보데이터에 얼마나 가까운지에 따라 가중치를 매기는데 반해, SHAP는 Shapley Value 측정에서 **x' 가 얻은 가중치에 따라 샘플링된 관측치에 가중치를 부여**

2. Tree SHAP

- 적용 대상: 결정트리 계열 모델(랜덤포레스트, XGBoost, LightGBM 등)
- 변형 포인트: 트리 기반 모델에서는 경로(path) 구조를 활용해 빠르게 Shapley 값을 계산할 수 있음
- 기존 트리 앙상블 모델의 Feature Importance 는 Tree에 의존적임 (inconsistent)
- 트리모델 간에 Feature 가 동일하더라도, 모델에 따라 Split 순서가 다르고 계산되는 Gain, 등의 산출값이 달라짐 → 즉, 같은 데이터로부터 학습된 모델이지만 모델마다 Feature Importance 가 달라짐
- Tree SHAP은 Split 순서와 무관하게 일정한 Feature Importance 가 계산 됨. → consistent 확보됨**
- 변경 내용**
 - 각 노드에서 분할된 피처들의 기여도를 경로별로 합산 : 각 트리마다 조건부 기대값을 구하고 이를 모두 합산하여 최종 값을 산출 함.
 - 동적 프로그래밍(dynamic programming)으로 중복 계산 제거
 - 장점: O(T·L·D) 시간 복잡도(트리 수 T, 최대 리프 수 L, 최대 깊이 D)에 해결 가능

Model-Agnostic Interpretation method

* Variant SHAP

3. Deep SHAP

- 적용 대상: 딥러닝 모델(Keras, TensorFlow, PyTorch)
- 변형 포인트: 딥 뉴럴넷은 레이어가 깊고 비선형성이 강해, 일반적인 Kernel SHAP처럼 블랙박스 호출만으로는 효율적이지 않음
- 변경 내용:
 - "DeepLIFT" 기법과 Shapley 이론을 결합
 - 레이어별 기여도(gradient 기반)와 기준값(baseline)에서의 변화량을 조합해 근사화

4. Gradient SHAP

- 적용 대상: 연속적 입력을 다루는 딥러닝 모델
- 변형 포인트: Deep SHAP의 근사성을 개선하기 위해, 입력과 baseline 사이의 여러 점을 잇는 선형 경로 상에서 gradient를 적분
- 변경 내용:
 - 여러 baseline 샘플을 랜덤으로 섞은 뒤, 미분값을 평균화(Expectation)
 - 기대 기여도를 구하는 Monte Carlo 적분 방식

5. Partition SHAP

- 적용 대상: 계층적 피처나 상호 연관된 그룹이 있는 경우
- 변형 포인트: 피처를 그룹(파티션) 단위로 묶어 조합 수를 줄이고, 그룹별 Shapley 값을 계산
- 변경 내용:
 - 피처들 간의 상호작용을 그룹 구조로 표현(예: 카테고리형 변수의 원-핫 인코딩 피처 묶음)
 - 각 그룹의 기여도를 우선 계산하고, 그룹 내 기여도는 추가 분할

6. Sampling SHAP

- 적용 대상: 고차원 데이터, 계산량을 극단적으로 줄여야 하는 경우
- 변형 포인트: 조합 샘플링 방식을 더욱 단순화
- 변경 내용:
 - 전체 피처 조합 대신, 랜덤하게 선택된 소수 조합만 사용
 - 가중치를 조정해 샘플링 편향을 보정

Model-Agnostic Interpretation method

* Variant SHAP 비교

변형 이름	대상 모델	핵심 변경점	장점	예시 코드
Kernel SHAP	블랙박스 모델	가중 표본 추출을 통한 Shapley 근사	모델 제약 없이 적용 가능, nsamples 조절로 속도/정확도 균형	<code>shap.KernelExplainer(model.predict, bg)</code>
Tree SHAP	결정 트리 계열 (XGBoost, LightGBM 등)	트리 경로별 기여도 합산 + DP	매우 빠른 계산 ($O(T \cdot L \cdot D)$), 정확도 보장	<code>shap.TreeExplainer(gbm)</code>
Deep SHAP	딥러닝 모델 (Keras, PyTorch)	DeepLIFT 기반 레이어별 기여도 + 기준값 변화량	블랙박스 호출 최소화, 딥 네트워크에 최적화	<code>shap.DeepExplainer(model, bg)</code>
Gradient SHAP	연속 입력 딥러닝	입력-baseline 경로 상에서 gradient 적분 (Monte Carlo)	근사 안정성 ↑, 여러 baseline으로 편향 보정	<code>shap.GradientExplainer(model, bg)</code>
Partition SHAP	피처 그룹 구조(계층형·상호 연관)	피처를 그룹 단위로 묶어 Shapley 계산 → 이후 그룹 내 분할	고차원 연관 피처 처리, 계산량 절감	<code>shap.PartitionExplainer(model.predict, data, group_names=groups)</code>
Sampling SHAP	고차원 데이터	랜덤 샘플링된 소수의 조합만 사용, 가중치 보정	극단적 계산 절감, 빠른 근사	<code>shap.SamplingExplainer(model.predict, bg)</code>

Model-Agnostic Interpretation method

3. LIME (Local Interpretable Model-agnostic Explanations)

논문 링크: <https://arxiv.org/abs/1602.04938>

1. MT Ribeiro et al. (2016). "LIME: "Why Should I Trust You? : Explaining the Predictions of Any Classifiers". ACM-KDD
2. Scott Lundberg et al. (2017). "A Unified Approach to Interpreting Model Predictions". NeurIPS.
3. Scott Lundberg et al. (2020). "From local explanations to global understanding with explainable AI for trees". Nature Machine Intelligence

LIME (Local Interpretable Model-agnostic Explanations)은 "Black box" 형태의 복잡한 머신러닝 모델에 대해 **특정 입력**이 어떻게 예측 결과를 낳았는지, 그 근거를 **단순한 국소(로컬) 선형 모델**로 근사하여 설명해 주는 기법

신뢰(Trust)

- 개별 예측값에 대한 신뢰(Trusting a prediction)
- 모델에 대한 신뢰(Trusting a model)

● 개별 예측값에 대한 신뢰(Trusting a prediction)

모델이 Output을 제시했을 때, 이 **Output**이 어떤 **Input**의 영향을 받아 제시되었는지에 대한 설명이다.

● 모델에 대한 설명

- 전체 모델이 어떤 **Feature**들을 중요하게 여기는지에 대한 설명이다.
- 어떤 두 모델 중 하나를 선택하고자 할 때, 전체적으로 더 합리적인 설명들을 제시하는 모델을 선택한다는 것이다.

Model-Agnostic Interpretation method

3. LIME (Local Interpretable Model-agnostic Explanations)

논문 링크: <https://arxiv.org/abs/1602.04938>

1. MT Ribeiro et al. (2016). "LIME: "Why Should I Trust You? : Explaining the Predictions of Any Classifiers". ACM-KDD

설명 모델 (Explainer)들이 갖추어야 할 세 가지 요건 제시

1. Interpretability

- 인간이 이해할 수 있는 방식으로 설명이 제시되어야 한다는 것
- 중요한 변수를 선별하여 설명하는 등의 작업이 필요하다.

2. Local Fidelity

- 모델이 제시하는 설명이 적어도 Locally Faithful 해야 한다는 것
- Global Fidelity 에서는 중요한 Feature 가 개별 Prediction을 예측할 때는 중요하지 않을 수 있다는 사실에서 기인한 것

3. Model-agnostic

- Explainer는 해석의 대상이 되는 모델의 종류와 관계없이 설명을 제시할 수 있어야 한다는 것
- 모델의 복잡성에 구애받지 않고 설명을 제시할 수 있어야한다는 것

Model-Agnostic Interpretation method

3. LIME (Local Interpretable Model-agnostic Explanations)

논문 링크: <https://arxiv.org/abs/1602.04938>

1. 핵심 아이디어

- 1) 모델 불가지론(Model-agnostic)
어떤 모델이든(트리, 신경망, SVM 등) 내부 구조를 몰라도 작동
- 2) 국소(Local) 선형 근사
관심 있는 한 점(샘플) 주변의 작은 영역에서, 복잡 모델 f 를 간단한 선형 모델 g 로 근사
선형 모델 g 의 계수(coef.)가 그 점에서 각 특성의 영향력을 나타냄

2. 작동 방식

- 1) 샘플 주변 데이터 생성
원본 샘플 x 의 각 특성마다 약간씩 '잡음'을 추가해 다수의 이웃(perturbed samples) 생성
- 2) 원 모델 예측값 획득
이 이웃 샘플들을 원본 모델 f 에 통과시켜 예측값을 구함
- 3) 거리 기반 가중치 부여
생성된 이웃 샘플들과 원본 점 x 와의 거리에 따라 가중치 w 할당
가까운 샘플일수록 더 큰 가중치
- 4) 단순 선형 모델 학습
 $\{(x_{\text{perturbed}}, f(x_{\text{perturbed}}))\}$ 에 대해 가중치 w 를 반영해 선형 모델 g 학습
이때 $g(z) = w_0 + \sum_j w_j z_j$ 형태

3. 해석

학습된 선형 계수 w_j 를 통해 "이 특성이 예측을 얼마나 끌어올렸는지(또는 내렸는지)" 이해

Model-Agnostic Interpretation method

3. LIME (Local Interpretable Model-agnostic Explanations)

4. 장. 단점

구분	장점	단점
해석 용이성	• 단순 선형 회귀계수로 바로 해석	• 국소 영역 정의에 민감 (노이즈 크기, 거리 함수 선택 등)
적용성	• 모델 불가지론: 어떤 모델에도 사용 가능	• 인접 샘플 생성 방식이 현실적이지 않을 수 있음
계산 비용	• SHAP KernelSHAP보다 가볍고 빠른 편	• 여러 파라미터 튜닝 필요 (이웃 수, 거리 척도, 복잡도 제한)
안정성	• 실행 속도가 빠르고 간단	• 해석 결과가 불안정: 실행할 때마다 다른 근사 결과 도출 가능

언제 사용하면 좋을까?

- 모델 내부 구조를 알 수 없거나, 복잡 모델을 손쉽게 해석 하고 싶을 때
- 특정 샘플에 대한 국소적 해석이 필요할 때
- 빠른 프로토타이핑 단계에서 직관적 설명이 중요할 때

→ 동일 샘플에 반복 적용 시 결과가 달라질 수 있고, 국소 영역 설정에 따라 민감하므로 SHAP 등의 다른 로컬 설명 기법과 함께 비교 검토하는 것이 좋다.

* Boltzmann Constant

- 오스트리아의 물리학자 루트비히 볼츠만(Ludwig Boltzmann)의 이름을 따서 명명
- 볼츠만 상수 (k_B)는 통계역학과 열역학에서 중요한 물리 상수로, 온도와 에너지를 연결하는 역할을 한다.
- 미시적 세계와 거시적 세계를 연결하는 다리 역할

$$k_B = 1.380649 \times 10^{-23} \text{ J/K}$$

통계역학

1. 미시적 상태의 수 (Ω)와 엔트로피 (S)의 관계

$$S = k_B \ln \Omega$$

- S : 엔트로피, Ω : 가능한 미시 상태의 수
- → 시스템의 무질서를 측정하며, 엔트로피가 높을수록 시스템의 무질서가 증가한다.

열역학:

이상기체의 상태방정식

$$PV = N k_B T$$

P : 압력, V : 부피, N : 분자의 수, T : 절대 온도

→ 기체의 거시적 성질을 미시적 입자의 움직임과 연결한다.

Tree-Based-Model

Decision Tree

→ 특정 Feature 에 대한 질문을 기반으로 데이터를 분리하는 방법

Bagging

→ Bootstrap의 원리로 훈련세트 수를 만든 뒤 → 이를 Aggregate
→ **Random Forest** : 수개의 Decision Tree 가 모여서 생성
→ 수개의 Decision Tree가 예측한 값들 중, **분류모델**은 가장 많이 등장한 값. 즉 다수결 원칙으로 결정하고,
회귀모델은 평균값을 최종 예측 값으로 정하는 것

Boosting

- Bagging은 병렬적으로 여러 개의 독립적인 DT를 만들어 각각의 값을 평균을 구한다.
- Boosting은 직렬적으로 1개의 모델을 계속 업데이트 하여 잔차를 줄여가는 방식으로 모델을 업그레이드 시키는 방법
- **AdaBoost (1997)**
- **Gradient Booting Machine**
- **XGBoost (Kaggle 대회 우승 알고리즘)**
- **LightGBM**
- **CatBoost**

Tree 모델 : 기본 이해

1. 기본 전개

분기 기준(Splitting Rule) 에 의해 node를 분기(분할)해 나가고, 회귀 또는 분류 진행

2. Splitting Rule을 정하는 방법

1) 회귀 : 예측모델

분기된 값들의 잔차 제곱합을 최소화 방향으로 분기 함. (분산 변화를 최대로~)

→ 분기(split)된 각각의 node들로 들어온 값들의 평균을 예측 값으로 정한 것

2) 분류 모델

→ 분기된 값들의 불순도(Impurity)를 최소화 (순수도를 최대화) 해주는 방향으로 분기

3. 장점

- 학습 방법이 단순하고 설명력도 매우 높다.
- 블랙박스 모델이 아니기 때문에 XAI기법 활용하여 Feature importance를 구할 수 있다
 - Tree 모델의 학습과정에서 분기 기준을 만드는 데에 어떤 변수가 가장 많이 사용여부 관점으로 Feature Importance 계산
- ✓ Feature importance (Mean decrease impurity, MDI)
 - : sklearn 트리 기반 분류기에서 디폴트로 사용되는 특성 중요도
- ✓ Drop-Column importance
 - : 독립변수 하나를 제거해보고 학습시켜 보는 방법

의사결정나무 (Decision Tree)

의사결정나무(decision tree) , 나무 모형(tree model) 란?

의사결정 규칙을 나무(Tree) 구조로 나타내어 전체 자료를 몇 개의 소집단으로 분류(classification) 하거나 prediction을 수행하는 분석방법

목표변수에 따른 구분

- 범주형인 경우의 분류나무(classification tree)
- 연속형인 경우의 회귀나무(regression tree)

**CART(Classification And Regression Tree)*

장점

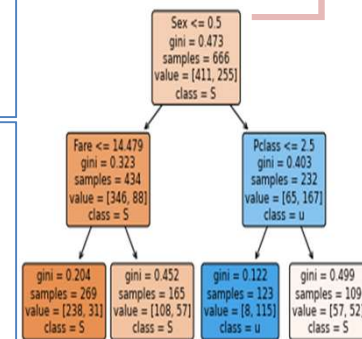
시각화해 의사결정이 이뤄지는 시점과 성과를 한눈에 볼 수 있게 함

- 누구나 이해가 쉽고 설명이 용이
- 계산결과가 의사결정나무에 직접 나타나기 때문에 해석이 간편
- 대용량의 데이터에서도 빠르게 만들 수 있고 데이터의 분류 작업도 신속히 진행 가능
- 비정상적인 잡음 데이터에서도 민감함 없이 분류 가능 → 데이터의 전처리 (정규화, 결측치, 이상치 등) 를 하지 않아도 된다.
- 과대적합을 방지하기 위해서 가지치기 방법이 필요함 (max_depth, max_leaf_nodes 형태의 하이퍼파라미터 설정)

한계점

- 특징 공간을 분할하거나 분기를 생산하는 과정에 **과적합(Overfitting)** 문제가 발생하기 쉽다.
- 예측의 정확도가 다소 떨어진다는 한계
- greedy algorithm (divide and conquer) : 위에서 아래로 내려가며 반복적으로 파티션을 나눈다. 한 번 이뤄진 분리는 다시 되돌릴 수 없다.
- 결정경계(decision boundary)가 축에 평행하다는 한계점 → 축에 평행한 결정경계(decision boundary) 만 갖게 되며 한 속성을 하나씩만 분리할 수 없다는 단점
- 의사결정나무는 결정경계(decision boundary)가 데이터 축에 수직이어서 특정 데이터에만 잘 작동할 가능성이 높다.
- 한 번에 하나의 변수만을 고려하므로 변수간 상호작용을 파악하기가 어렵다.

성별에서 첫번째 분할 발생 (임계치 0.5)
지니계수는 0.473
데이터 숫자 666개 중 411, 255개 분할



트리 구조 예시
(plot_tree 활용)

Overfitting을 해결할 방법은?

→ Random Forest : 데이터에 대해 의사결정나무를 여러 개 만들어 그 결과를 종합해 (분류 ; 다수결 투표/회귀 : 평균화) 예측 성능을 높이는 앙상블 머신러닝기법

의사결정나무 (Decision Tree)

이산형 목표변수 사용되는 분리기준

상위노드에서 가지 분할(분기)을 수행할 때, 분류변수와 분류 기준값의 선택 방법으로 카이제곱 통계량(Chi-square statistic)의 -값, 지니 지수 (Gini index), 엔트로피 지수(entropy index) 등이 사용

→ 분할이 일어날 때, 카이제곱 통계량의 p- 값은 그 값이 작을수록 자식노드 간의 이질성이 크다.

→ 지니 지수나 엔트로피 지수는 그 값이 클수록 자식노드 내의 이질성이 큼을 의미한다.

→ 따라서 이 값들이 가장 작아지는 방향으로 가지분할을 수행하게 된다. (불순도가 낮아지는 방향 분할)

→ 알고리즘 : CHAID(Kass, 1980), CART(Breiman 등, 1984), ID3(Quinlan, 1986), C4.5(Quinlan, 1993), C5.0(Quinlan, 1998) 등

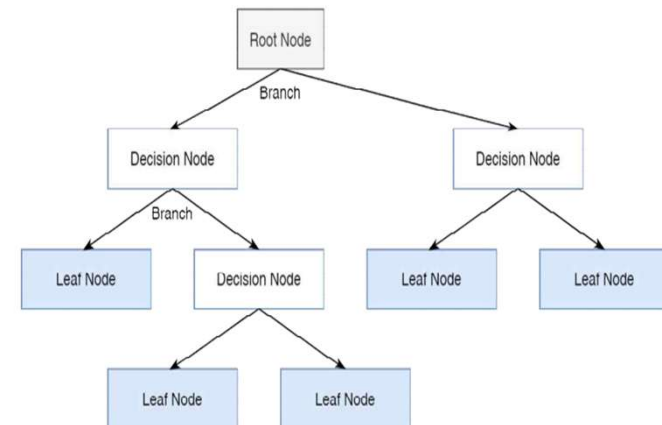
	이산형 목표변수	연속형 목표변수
CHAID(다지분할)	카이제곱 통계량	ANOVA F-통계량
CART(이진분할)	지니지수	분산감소량
C4.5	엔트로피지수	.

활용

- 시장조사, 광고조사, 의학연구, 품질관리 등의 다양한 분야에서 활용
- 고객 타겟팅, 고객들의 신용점수화, 고객행동예측, 고객 세분화

용어 정리

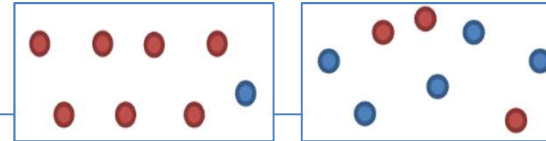
- root node : 분류(또는 예측) 대상이 되는 모든 자료집단을 포함
- parent node : 상위 마디
- child node : 하위 마디
- terminal node : 더 이상 분기되지 않는 마디
- 가지분할(split) : 나무의 가지를 생성하는 과정
- 가지치기(pruning) : 생성된 가지를 잘라내 어 모델을 단순화하는 과정



의사결정나무 (Decision Tree)

의사결정나무 분리기준

불순도(Impurity)



- 해당 범주 안에 서로 다른 데이터가 얼마나 섞여 있는 정도
- 불순도가 최소(혹은 순도가 최대) : 한 범주에 한 종류 데이터만 있는 경우
- 불순도가 최대(혹은 순도가 최소) : 한 범주 안에 서로 다른 두 종류 데이터가 정확히 반반 있는 경우
- 의사결정나무는 불순도를 최소화(혹은 순도를 최대화)하는 방향으로 학습을 진행 (분할점을 설정)

엔트로피(Entropy) : 불순도를 측정하는 지표

: 불순도가 낮아지면 불확실성 감소 (정보획득:Information gain)

→ 엔트로피가 높다는 것은 불순도가 높다는 의미, 엔트로피가 낮다는 것은 불순도가 낮다는 의미

→ 엔트로피가 1이면 불순도가 최대이고 한 범주 안에 서로 다른 데이터가 정확히 반반 있다는 뜻

$$E(Entropy(S)) = -\sum_{k=1}^m p_k \log_2(p_k)$$

정보 이득 (information gain)

- m : 목표변수의 범주의 수
- p_k : S영역에 속하는 레코드 가운데 k 범주(범주1, 범주2)에 속하는 레코드의 비율
- 확률(p)에 log2를 취한다는 값 : 데이터의 정보를 표현하기 위해서 필요한 비트 수(1 또는 0)를 확인할 수 있다는 뜻
- →정보의 기댓값(평균값)
- m이 2인 경우 : $0 \leq E \leq 1$
- 엔트로피 값이 클수록 불확실성이 높다. 엔트로피가 0이면 해당 노드의 모든 데이터가 동일한 클래스로 분류된 것을 의미

$$: GI(Information Gain) = Entropy(before) - \sum_{j=1}^k Entropy(j, after)$$

정보 이득은 노드를 특정 특성에 따라 분할했을 때 엔트로피 감소량 → 현재 노드의 불순도와 자식노드의 불순도 차이

→ 즉, 정보획득량 값이 높을수록 분할의 성능이 좋은 것이고, 낮을수록 분할의 성능이 좋지 못한 것이다.

→ 정보획득량(information gain)이 없을 때까지 (혹은 다른 threshold를 만족할 때까지) 분할 반복

→ 분할(분기)기준 설정 시 현재노드의 불순도에 비해 자식노드의 불순도가 감소되도록 설정

의사결정나무 (Decision Tree)

의사결정나무 분리기준

불순도(Impurity), 엔트로피(Entropy)

볼츠만 상수 (Boltzmann Constant)

$$E(Entropy(S)) = -\sum_{k=1}^m p_k \log_2(p_k)$$

- p : 전체 데이터에서 특정 데이터가 차지하는 비율
- 확률(p)에 $\log_2$ 를 취한다는 값: 데이터의 정보를 표현하기 위해서 필요한 비트 수(1 또는 0)를 확인할 수 있다는 뜻
- →정보의 기댓값(평균값)

엔트로피를 사용해서 불순도를 계산하는 경우 $\log$ 때문에 계산이 조금 느릴 수 있다.

→ **대안**으로 더 간편한 지니지수나 카이제곱스퀘어를 쓰기도 한다.

지니 지수 (Gini index)

$$\text{지니 불순도 } G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- N 이 2인 경우: $0 \leq G \leq 0.5$
- 지니 지수가 0에 가까울수록 그 클래스에 속한 불순도가 낮으므로 좋은 분류

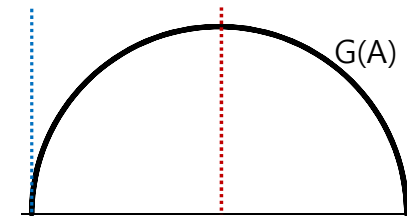
의사결정나무 (Decision Tree)

의사결정나무 분리기준 (불순도, 지니지수)

지니불순도 $G(A) = 1 - \sum_{k=1}^n (p_k)^2$

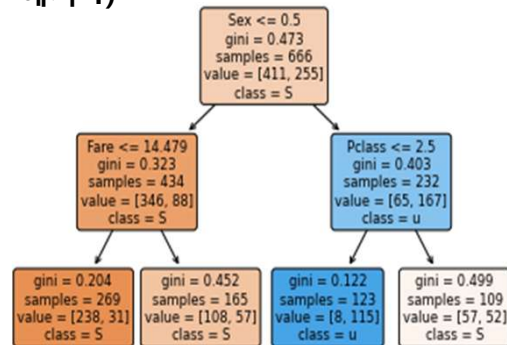
복원추출의 개념이 사용되므로 k 범주에 속하는 p_k 에 제곱이 들어가게 된다.

지니불순도 : 0.5 (최대)
A, B 그룹 데이터 개수가 1:1인 경우



지니불순도 : 0 (최소)
A, B 섞이지 않은 순수한 상태

예시 1)



Sex에 대한 지니 불순도

$$G(\text{sex}) = 1 - \left(\frac{411}{666}\right)^2 - \left(\frac{255}{666}\right)^2 \approx 0.47256$$

예시 2) 5명의 단골과 5명의 이탈고객에 대한 정보 분류 (성별 기준 불순도 감소가 더 크다)

단골	단골	단골	단골	단골	이탈	이탈	이탈	이탈	이탈
남자	남자	남자	남자	남자	남자	여자	여자	여자	여자
20대	30대	20대	30대	30대	30대	20대	20대	30대	20대

뿌리 노드 $G = 1 - \left(\frac{5}{10}\right)^2 - \left(\frac{5}{10}\right)^2 = 0.5$

성별 기준 $G(\text{성별}) = \frac{6}{10}(0.278) + \frac{4}{10}(0) = 0.167$

$$\left\{ \begin{array}{l} G(\text{여}) = 1 - \left(\frac{0}{4}\right)^2 - \left(\frac{4}{4}\right)^2 = 0 \\ G(\text{남}) = 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 = 0.278 \end{array} \right.$$

연령기준 $G(\text{연령}) = \frac{5}{10}(0.48) + \frac{5}{10}(0.48) = 0.48$

$$\left\{ \begin{array}{l} G(20) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48 \\ G(30) = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48 \end{array} \right.$$

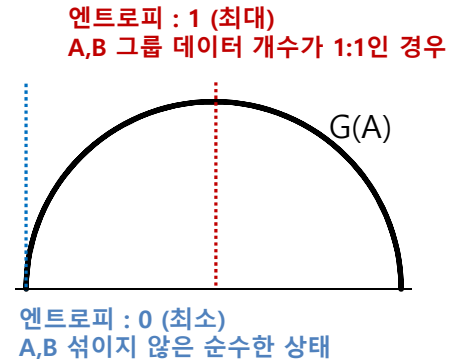
의사결정나무 (Decision Tree)

의사결정나무 분리기준 (엔트로피)

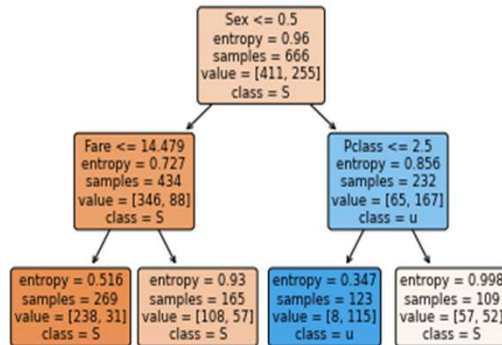
엔트로피 $E(Entropy(S)) = -\sum_{k=1}^m p_k \log_2(p_k)$

p_k = S 영역에 속하는 data 중 k 범주에 속하는 data 비율

정보이득 $GI(Information\ Gain) = Entropy(before) - \sum_{j=1}^k Entropy(j, after)$



예시 1)



Sex에 대한 엔트로피

$$E(\text{sex}) = -\left(\frac{411}{666}\right) \log_2\left(\frac{411}{666}\right) - \left(\frac{255}{666}\right) \log_2\left(\frac{255}{666}\right) \approx 0.96005$$

예시 2) 5명의 단골과 5명의 이탈고객에 대한 정보 분류 (성별 기준 정보이득이 더 크다)

단골	단골	단골	단골	단골	이탈	이탈	이탈	이탈	이탈
남자	남자	남자	남자	남자	남자	여자	여자	여자	여자
20대	30대	20대	30대	30대	30대	20대	20대	30대	20대

뿌리 노드 $E = -\left(\frac{5}{10}\right) \log_2\left(\frac{5}{10}\right) - \left(\frac{5}{10}\right) \log_2\left(\frac{5}{10}\right) = 1$

성별 기준 $E(\text{성별}) = \frac{6}{10}(0.65) + \frac{4}{10}(0) = 0.39$

$$\left\{ \begin{array}{l} E(\text{여}) = -\left(\frac{0}{4}\right) \log_2\left(\frac{0}{4}\right) - \left(\frac{4}{4}\right) \log_2\left(\frac{4}{4}\right) = 0 \\ E(\text{남}) = -\left(\frac{5}{6}\right) \log_2\left(\frac{5}{6}\right) - \left(\frac{1}{6}\right) \log_2\left(\frac{1}{6}\right) = 0.65 \end{array} \right.$$

연령기준 $E(\text{연령}) = \frac{5}{10}(0.971) + \frac{5}{10}(0.971) = 0.971$

$$\left\{ \begin{array}{l} E(20) = -\left(\frac{2}{5}\right) \log_2\left(\frac{2}{5}\right) - \left(\frac{3}{5}\right) \log_2\left(\frac{3}{5}\right) = 0.971 \\ E(30) = -\left(\frac{3}{5}\right) \log_2\left(\frac{3}{5}\right) - \left(\frac{2}{5}\right) \log_2\left(\frac{2}{5}\right) = 0.971 \end{array} \right.$$

의사결정나무 (Decision Tree)

알고리즘 속성 선택

CART (Classification And Regression Test)

전체 데이터 셋에서 시작하여 반복해서 2개의 자식노드를 생성하기 위해 모든 예측변수를 사용하여 데이터셋의 부분집합을 쪼갬으로써 의사결정트리를 생성

ID3 알고리즘

- 정보이득을 통해 속성 선택
- 노드 N이 D파티션의 tuple이라고 가정한 후 정보이득이 가장 큰 속성을 노드 N에 대한 분리속성 (splitting attribute)으로 선정
- 이 경우 이 속성은 결과적으로 만들어지는 파티션 내에서 정보를 분류하는데 소요되는 정보의 양을 최소화하고 이들 파티션 내에서의 불순도 (imprurity) 내지 최소 임의성 (least randomness)을 반영하는 것이 된다.

CHAID (Chai-Square Automatic Interaction Detection)

χ^2 에 기반한 CHAID는 원래 변수들 간의 통계적 관계를 찾는 것이 그 목적이었으나 이를 다시 의사결정트리를 통해 표현될 수 있으므로 이 방법은 분류기법으로도 많이 사용

	이산형 목표변수	연속형 목표변수
CHAID(다지분할)	카이제곱 통계량	ANOVA F-통계량
CART(이진분할)	지니지수	분산감소량
C4.5	엔트로피지수	.

의사결정나무 (Decision Tree)

D.T 분석 과정

`from sklearn.tree import DecisionTreeClassifier`

1. 의사결정트리의 생성 : 분석목적에 따라 적합한 **분리기준과 정지규칙(Stopping Rule)** 지정

1) 분리기준 (Splitting criteria)

→ 여러가지 독립변수 중 하나의 분류변수를 선택하고 그 분류변수에 대한 **분류기준값 (threshold)**을 결정

→ **정보획득량(information gain)**이 큰 분류변수와 **분류기준값** 결정

→ 분류변수와 분류기준값의 선택이 중요 : 노드(집단) 내에서는 동질성이, 노드(집단) 간에는 이질성이 가장 커지도록 선택

→ **정지규칙(Stopping rule)** : 현재 마디가 최종 마디가 되도록 하는 규칙

2) **가지치기(Tree Pruning)** : 분류오류(Classification Error)를 크게 할 가능성이 있거나 부적절한 추론규칙(Induction Rule)을 가지는 가지(Branch)를 제거

2. **타당성 평가**: 이익표(Gains Chart)나 위험도표(Risk Chart)또는 검증용 자료에 의한 교차타당성 분석 등을 이용하여 의사결정나무 평가

3. 해석 및 예측 모형 설정

→ 반복적으로 수행하여 다양한 의사결정나무 결과, 비교 검토

의사결정트리 귀납 알고리즘 기본 전략

- Tree는 훈련 샘플들을 나타내는 단일 node로 시작
- 샘플들이 모두 같은 class라면 node는 잎이 되고 해당 class로 label을 부여
- 그렇지 않다면 정보이득의 정도를 기준으로 각 class로 가장 분리가 잘되는 속성을 선택하기 위해 heuristic한 방법을 사용한다. 이 속성은 그 node에서 "검사" 또는 "의사결정" 속성이 된다.
- 가지(branch)는 검사속성이 갖는 값에 대해 각각 생성되고 샘플들은 적절하게 분할된다.
- 분할된 각 샘플에 대한 의사결정트리를 형성하기 위해 동일한 과정이 재귀적으로 수행

의사결정나무 (Decision Tree)

*가지치기 (Tree Pruning)

- 최대트리로 형성된 결정트리의 특정 노드 밑의 하부 트리를 제거하여 일반화 성능을 높이는 것을 의미
→ 분할(분기)가 너무 많으면 Training data에 과적합(overfitting)되는 문제점이 발생
- terminal node가 너무 많으면 새로운 데이터에 대한 예측 성능인 일반화(generalization) 능력이 저하되는 문제점.

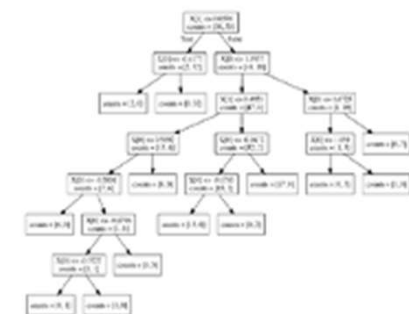
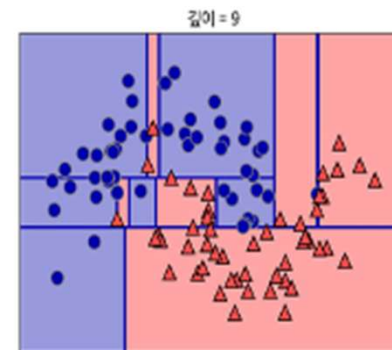
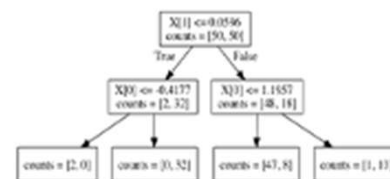
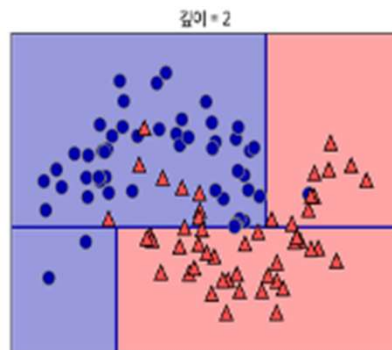
(1) 사전 가지치기 (Prepruning)

트리가 생성되는 초기에 수행되는 것으로서 가지치기가 정지되면 그 노드는 잎이 된다. 트리를 생성할 때 정보이득, 통계적 유의성, χ^2 등을 사용하여 분기의 적합성을 평가할 수 있다.

(2) 사후 가지치기 (postpruning)

모두 완성된 트리에서 가지를 제거해 나가는 것이다.

- CART 알고리즘 : 비용 복잡도 (cost complexity)에 의거한 가지치기 방법
 - C4.5 알고리즘 : pessimistic 가지치기 알고리즘을 이용
 - MDL (Minimum Description Length, 최소 서술 길이) 원칙을 기준으로 하는 방법
- 사전 가지치기와 사후 가지치기는 혼용 (interleave)하여 사용



의사결정나무 (Decision Tree)

의사결정나무의 Hyper-Parameter

```
def __init__(
    self,
    *,
    criterion="gini",
    splitter="best",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    min_weight_fraction_leaf=0.0,
    max_features=None,
    random_state=None,
    max_leaf_nodes=None,
    min_impurity_decrease=0.0,
    class_weight=None,
    ccp_alpha=0.0,
```

1. **Criterion** : 불순도 계산함수 불순도 척도. 'gini', 'entropy'
2. **splitter** : 각 가지의 분할 전략. 최적분할과 최적 랜덤분할. 'best', 'random'
3. **max_depth**: 트리 깊이의 최대값. 기본값은 모든 leaf가 pure해질때까지 혹은, 분기된 가지 속 샘플수가 설정한 최소 샘플수(min_samples_split)보다 적게 될때까지 분기한다. 'int값 입력'
4. **min_samples_split**: 분기 할 가지 내 샘플의 최소 개수. 최소 개수보다 node 내 샘플 수가 적으면 leaf가 pure하지 않더라도 분기를 멈춘다. float 입력 시, 전체 샘플 개수 대비 float 비율만큼의 개수로 최소 샘플수가 설정된다. 'int 혹은 float값 입력', default=2
5. **min_samples_leaf**: leaf 가지에 있어야 하는 최소 샘플 수. 왼쪽 혹은 오른쪽 branch 에 각각 설정치만큼 훈련 샘플이 있어야 분기가 된다. 'int 혹은 float값 입력', default=2
6. **min_weight_fraction_leaf**: 모든 샘플의 가중치의 합계의 최소 가중치의 비율, 입력되지 않으면 모든 샘플의 가중치는 동일하다. 'float 입력', default=0.0
7. **max_features**: 최적을 찾기 위해 고려하는 feature의 개수. 'int or float값', 'auto', 'sqrt', 'log2',
(일반적으로 전체 Feature 개수의 제곱근만큼의 값의 성능이 가장 좋은편이라 경험적으로 알려져 있음) default=None.
8. **random_state**: 랜덤 제어. 'int 입력', default=None
9. **max_leaf_nodes**: 최대 가지 수. default=None
10. **min_impurity_decrease**: 최소 불순도 감소 값 (혹은 정보이득률) 이하인 경우에는 분기가 일어나지 않는다. 'float', default=0.0
11. **class_weight**: class별로 가중치를 둔다. 'dict', 'list of dict', 'balanced', default=None, dict의 예시 - {class_label: weight} or [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}] ', default=None
12. **ccp_alpha**: 최소 비용-복잡성 가지치기에 사용되는 복잡도 매개변수. ccp_alpha보다 작은 비용-복잡도를 가진 하위 트리 중에 비용-복잡도가 가장 큰 하위 트리가 선택된다. default=0.0

의사결정나무 (Decision Tree)

의사결정나무의 Hyper-Parameter

Grid Search CV를 통한 Overfitting 방지

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# 분석 모형 설정
model = DecisionTreeClassifier()
# 하이퍼 파라미터 dict 형태로 설정
params = {'max_depth': [1,2,3], 'min_samples_split':[2,3]}

# 그리드서치 시행
grid_clf = GridSearchCV(estimator = model,
                        param_grid=params,
                        cv=3,
                        refit=True,
                        n_jobs=-1)

grid_clf.fit(X_train,y_train)

# 최선의 결과 출력
print('최적 하이퍼 파라미터 :{0}, 최적 평균 정확도 :{1:.3f}'
      .format(grid_clf.best_params_, grid_clf.best_score_))
```

의사결정나무의 Overfitting 방지 기법 (깊이 제한, 노드분류 최소 데이터 개수)

Max_depth는 node의 깊이를 의미하며, **min_sample_split**은 노드 분류 최소 데이터 개수이다.

CV (Cross Validation) : 교차검정을 의미하며 기본 값은 5로 설정되어 있다. 해당 코드는 데이터 셋을 3개로 분할하여 train,validation을 총 3번 진행하게 된다.

refit : 최적의 파라미터를 찾고 입력된 개체에 해당 하이퍼파라미터를 넣어 재학습하는 효과를 준다.

n_jobs : sklearn에는 CPU 병렬처리를 지원한다. 보통 Grid서치를 진행할 때 시간이 많이 소요되는 점이 있는데, 이를 코어 개수 만큼 병렬로 수행하면 빠른 결과 도출이 가능해진다.

참고)

```
# GridSearchCV의 결과 값 보기
pd.DataFrame(grid_clf.cv_results_)
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_max_depth	param_min_samples_split	params	split0_test_score	split1_test_score	split2_test_score	mean_test_score	std_test_score	rank_test_score
0	0.002932	0.000331	0.002380	0.000646	1	2	{'max_depth': 1, 'min_samples_split': 2}	0.765766	0.761261	0.783784	0.770270	0.009731	3
1	0.003870	0.000999	0.002124	0.000454	1	3	{'max_depth': 1, 'min_samples_split': 3}	0.765766	0.761261	0.783784	0.770270	0.009731	3
2	0.013310	0.015236	0.002040	0.000435	2	2	{'max_depth': 2, 'min_samples_split': 2}	0.765766	0.779279	0.743243	0.762763	0.014864	5
3	0.003933	0.000688	0.013519	0.011940	2	3	{'max_depth': 2, 'min_samples_split': 3}	0.765766	0.779279	0.743243	0.762763	0.014864	5
4	0.004273	0.000741	0.003381	0.001267	3	2	{'max_depth': 3, 'min_samples_split': 2}	0.774775	0.765766	0.774775	0.771772	0.004247	1
5	0.003437	0.000421	0.005747	0.001250	3	3	{'max_depth': 3, 'min_samples_split': 3}	0.774775	0.765766	0.774775	0.771772	0.004247	1

의사결정나무 (Decision Tree)

트리 구조

```

샘플 코드)
import pandas as pd
from sklearn import tree

# 타이타닉 train 데이터 불러오기 (생략)
X = train.drop('Survived', axis=1)
y = train['Survived']

# feature, target 분리 (훈련, 테스트셋도 분리)
X_train, X_test, y_train, y_test = train_test_split(X,y, random_state=1)

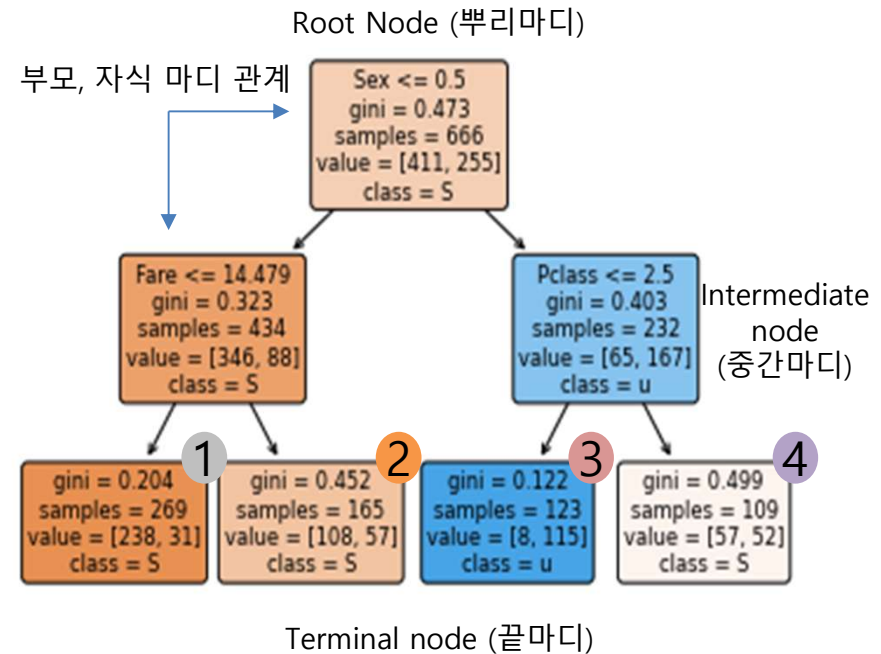
# 의사결정나무 모델 형성 및 y 분류 시행 (사전 가지치기 시행)
model = tree.DecisionTreeClassifier(criterion = 'gini', max_depth=2,
max_leaf_nodes=4)
model.fit(X_train, y_train)

# 트리 시각화
tree.plot_tree(model,
                feature_names=X_train.columns,
                class_names='Survived',
                filled = True,
                rounded=True,
                impurity=False,
                proportion=False)
    
```

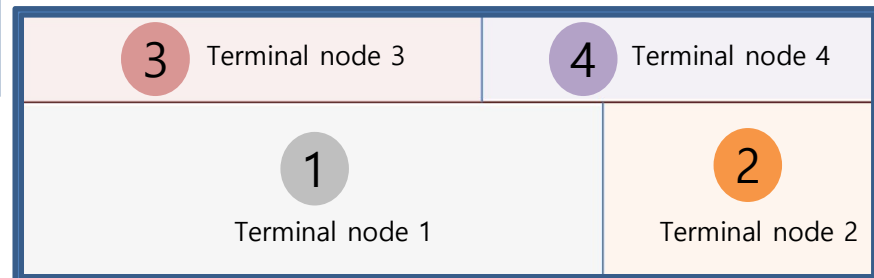
Default 옵션 : gini (entropy로 설정 변경 가능)

색 채우는 옵션
둥근 모서리 옵션
불순도 측정 옵션
Sample 숫자 -> 비율 전환 옵션

Root Node data 분할
By Sex feature



Intermediate node data 분할
By Pclass feature



Intermediate node data 분할
By Fare feature

의사결정나무 회귀(Decision Tree Regression)

예시 (캘리포니아 집값 회귀분석)

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

data = fetch_california_housing()
x = pd.DataFrame(data['data'], columns=data['feature_names'])
y = pd.DataFrame(data['target'], columns=['target'])

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state=2022)

model = DecisionTreeRegressor(criterion="squared_error",
                              splitter="best",
                              max_depth=2,
                              min_samples_split=30,
                              min_samples_leaf=1,
                              min_weight_fraction_leaf=0.0,
                              max_features=None,
                              random_state=2022,
                              max_leaf_nodes=4,
                              min_impurity_decrease=0.0,
                              ccp_alpha=0.0)

model.fit(X_train, y_train)

y_predict = model.predict(X_test)
```



https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing

- 1. criterion:** 불순도 평가 척도 'squared_error', 'friedman_mse', 'absolute_error', 'poisson'
- 2. splitter:** 각 가지의 분할 전략. 최적분할과 최적 랜덤분할. 'best', 'random'

이하 나머지는 분류분석과 동일한 파라미터 적용

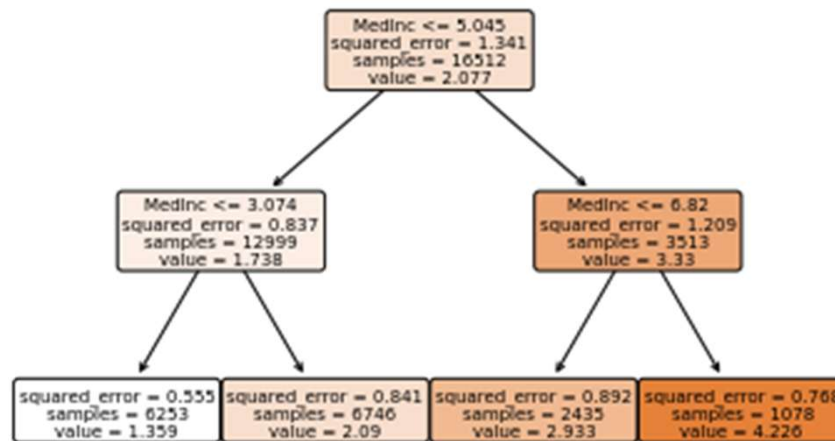
평가 척도 (참고)

'squared_error': 평균 제곱 오차(MSE), 변수선택 기준으로서의 분산 감소와 동일하다. 각 끝 가지의 평균을 사용한 L2 loss를 최소화
 'friedman_mse': 잠재적 분할을 위한 Friedman의 improvement score를 포함하는 MSE.
 'absolute_error': 오차의 절대값의 평균, 각 끝 가지의 중앙값을 사용한 L1 loss를 최소화
 'poisson': 분기를 찾기 위해 포아송 편차 감소를 사용.

의사결정나무 회귀(Decision Tree Regression)

회귀 결과 (캘리포니아 집값 회귀분석)

```
# tree 시각화
plot_tree(model, feature_names=X_train.columns, class_names='target', filled = True, rounded=True)
```



```
# 모형의 결정계수 출력
print(model.score(X_train, y_train))
print(model.score(X_test, y_test))
```

0.4511628747501255

0.4295379977717926

```
# 글자로 출력해주는 tree 구조
print(export_text(model, feature_names=X_train.columns.to_list()))
```

```

|--- MedInc <= 5.05
|   |--- MedInc <= 3.07
|   |   |--- AveRooms <= 4.10
|   |   |   |--- value: [1.68]
|   |   |   |--- AveRooms > 4.10
|   |   |   |--- value: [1.20]
|   |   |--- MedInc > 3.07
|   |   |--- AveOccup <= 2.37
|   |   |   |--- value: [2.81]
|   |   |   |--- AveOccup > 2.37
|   |   |   |--- value: [1.88]
|   |--- MedInc > 5.05
|   |   |--- MedInc <= 6.82
|   |   |   |--- AveOccup <= 2.59
|   |   |   |   |--- value: [3.58]
|   |   |   |   |--- AveOccup > 2.59
|   |   |   |   |--- value: [2.67]
|   |   |--- MedInc > 6.82
|   |   |--- MedInc <= 7.82
|   |   |   |--- value: [3.73]
|   |   |   |--- MedInc > 7.82
|   |   |   |--- value: [4.58]
  
```


분류 모델 선택

분류기법은 기법별로 장단점을 가지므로 비교평가가 용이하지 않으며 어떤 단일 방법이 무조건 모든 데이터집합에 대하여 다른 방식보다 우수하다고 하기는 어렵다.

(평가 기준)

- 예측 정확도
- 강건성 (robustness)
- 확장 적용성
- 해석력 (interpretability)

의사결정나무의 예측력을 높이기 위하여 Bagging, Random Forests, Boosting 등의 방법 사용.

✓ Bagging은 표본의 수를 높이면 예측의 분산이 줄어든다는 점에 서 착안한 방법

Bootstrap Aggregating 방식

무작위로 추출한 표본으로 총 N번의 의사결정나무를 실행하고, N번의 결과 중 다수결원칙에 의해 가장 많이 나온 집단으로 분류

✓ Random Forest

→ 표본뿐만 아니라 독립변수들도 무작위(Random)로 추출하는 방식으로, 실행과정은 bagging과 동일하다.

✓ Boosting

→ 한번 시행한 모형의 결과를 참조하여 순차적으로 N번 개선해 나가는 방법

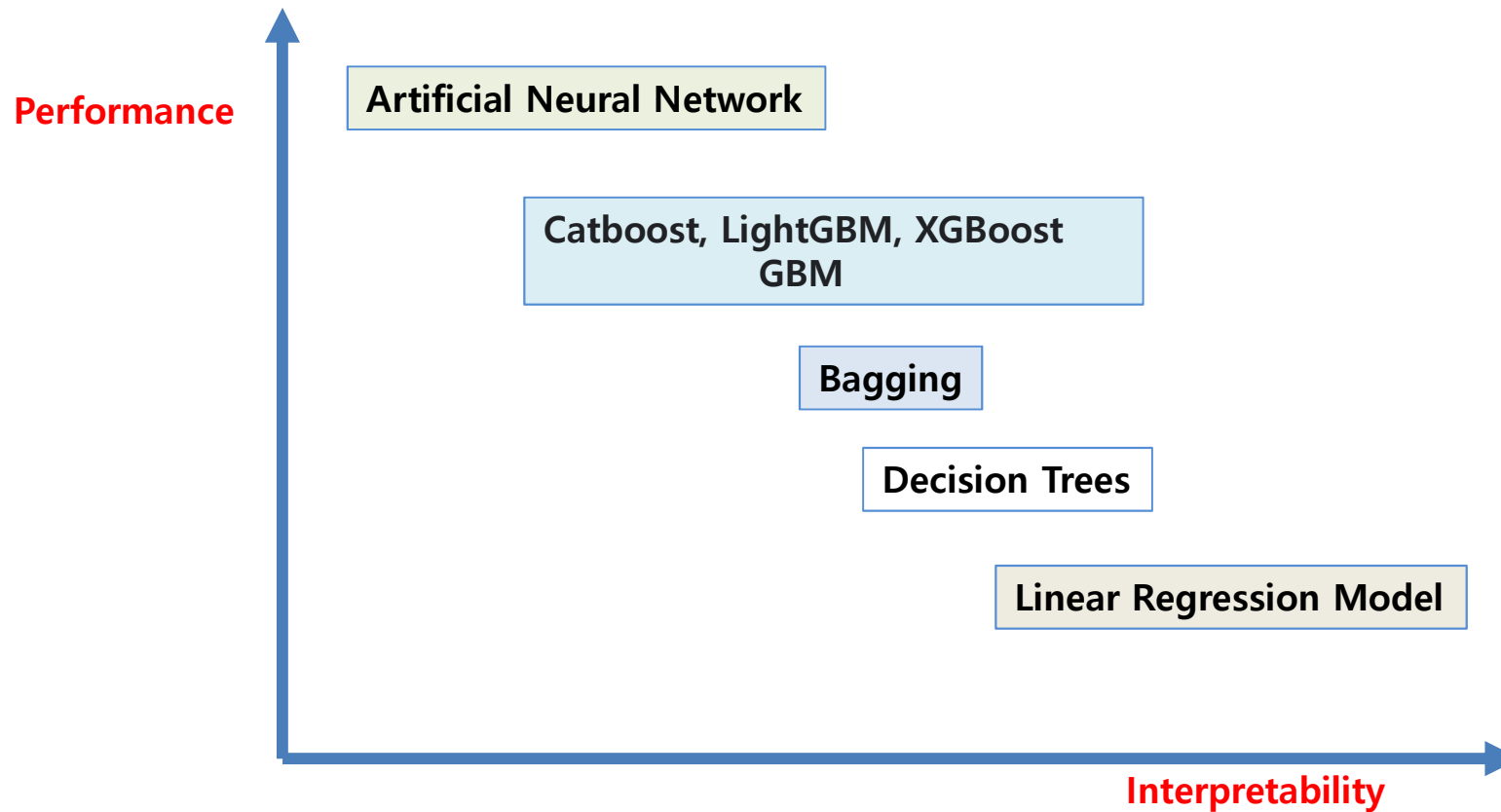
<https://scikit-learn.org/stable/modules/tree.html>

• 1.10. Decision Trees

- 1.10.1. Classification
- 1.10.2. Regression
- 1.10.3. Multi-output problems
- 1.10.4. Complexity
- 1.10.5. Tips on practical use
- 1.10.6. Tree algorithms: ID3, C4.5, C5.0 and CART
- 1.10.7. Mathematical formulation
- 1.10.8. Minimal Cost-Complexity Pruning

* 쉬어가기

Performance 와 Interpretability 관계



앙상블 (Ensemble) 모형

- 주어진 Data 에 대하여 여러 개의 분류 모형에 의한 결과를 종합하여 하나의 최종값으로 분류 → 정확도를 높이는 방법
- 적절한 표본추출법으로 데이터에서 여러 개의 훈련용 데이터 집합을 만들어 각각의 데이터 집합에서 하나의 분류기를 만들어 앙상블 하는 방법
- 1996년 Breiman 제안한 **Bagging**이 효시

- Bagging
- Boosting
- Stacking (meta modeling)

(앙상블 모형 사용)

- 분산(variance)을 감소시킨다:
 - 다수결/ 평균을 취함으로써 편의(bias)를 제거해준다.
- 한 개 모형으로부터의 단일 의견보다 여러 모형의 의견을 결합하면 변동이 작아진다.
- Overfitting 의 가능성 감소

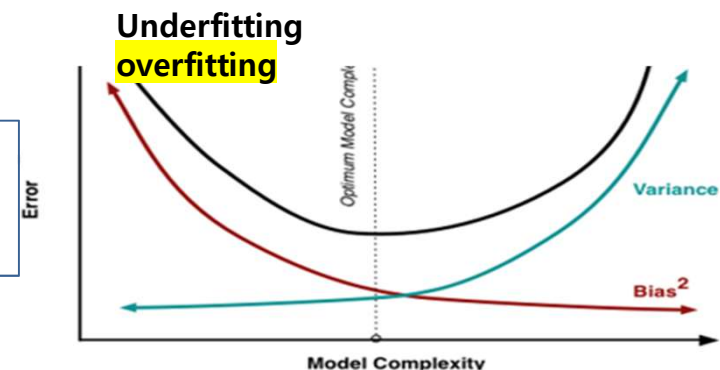
발생 가능한 학습 오류

- 높은 bias → Underfitting
- 높은 Variance → Overfitting

Bias와 Variance 관계 : Trade-off

수학 전개 참조

https://ko.m.wikipedia.org/wiki/%ED%8E%B8%ED%96%A5-%EB%B6%84%EC%82%B0_%ED%8A%B8%EB%A0%88%EC%9D%B4%EB%93%9C%EC%98%A4%ED%94%84#



앙상블 (Ensemble) 모형

Decision Tree

하나의 깊이가 깊은 결정트리를 만드는 것

많은 Feature들을 기반으로 Target을 예측하는 경우 → overfitting 문제 발생

Bagging (Bootstrap + Aggregating)

여러 모델을 병렬로 연결 → 각각의 결과값 고려
Bias 낮은 모델을 이용 분산을 줄인다.

Random Forest (Bagging + Random)

<https://www.pluralsight.com/guides/ensemble-methods:-bagging-versus-boosting>

Bagging, Boosting

복잡해지면서 설명력은 조금 줄어들지만 예측력은 크게 올라가는 방식

한 명의 천재보다 10명의 보통사람이 문제를 더 잘 푸는 원리 !!!

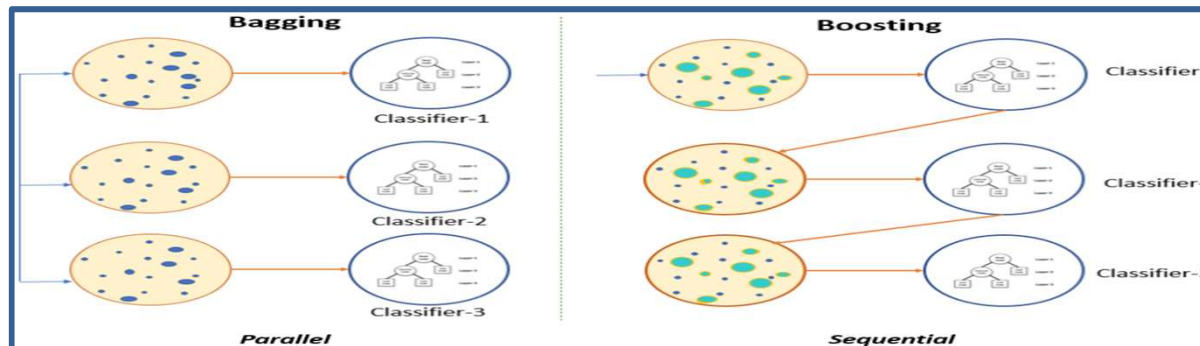
Boosting Bootstrap + 가중치 부여 (잔차) → Additive model

- 가중치를 활용하여 Weak Classifier를 순차적(직렬)으로 이용 → Strong Classifier로 만드는 방법
- 분산이 낮은 모델들을 합쳐서 Bias를 줄인다.

하나의 데이터 소스에서 **랜덤하게 데이터들을 만들어 여러 개의 학습된 '결정 트리' 모델을 만들고 이를 종합하는 방식**

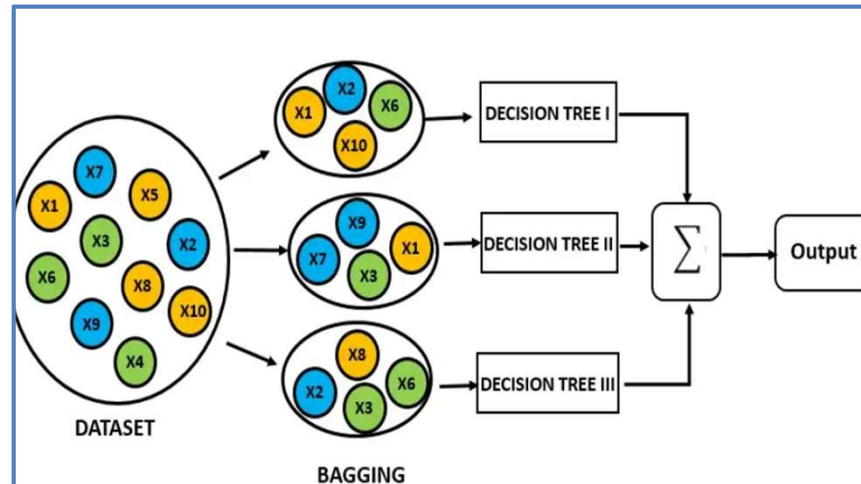
Random Forest : 많은 Decision Tree가 모여서 생성

- → Decision Tree마다 하나의 예측 값 → 다수결, 평균값으로 최종 예측값



Bagging

Bagging (Bootstrap + Aggregating)



1. Bootstrap Sample : 독립적이며 동일하게 분포되어 있으므로(i.i.d) → 이 과정을 거치면서 분산이 적은 모델을 얻을 수 있게 된 데이터 셋

→ Bagging 은 분산을 감소시키는 중점

2. Aggregation

각각의 모델을 학습시켜 결과물을 집계(Aggregation)

• Bootstrap

: 학습에 사용할 값들을 여러 번 무작위 복원 추출하여 추출된 값들의 통계량을 계산하는 것 (**중복 허용**)
(참고) **Pasting** : 중복을 허용하지 않고 데이터셋을 나누는 방식

- Leo Breiman (1996). "BIAS, VARIANCE, AND ARCING CLASSIFIERS"
- Leo Breiman (1999) ARCING CLASSIFIERS

1단계 : 대상 데이터로부터 여러 개의 Bootstrap 자료 생성 (동일 크기의 표본을 무작위로 복원 샘플링 N개를 함.)

→ N 개에 Training set에서 임의로 N개의 데이터를 중복을 허용하여 선택하여 Tree를 만든다.!!!!

→ 2단계 : 선택한 N개의 데이터 샘플에서 데이터 특성 (feature)의 개수 S 개에서 값을 중복하지 않게 $\sqrt{S}$ 개 선택

→ 3단계 : 의사결정나무를 학습하고 생성

→ 4단계 : 1~3단계를 K번 반복

→ 마지막단계 : 각 Bootstrap 자료에 대한 예측모형 구축
예측모형을 결합하여 최종모형 구축

--표본들을 동일한 모델에 학습 후 학습된 모델의 예측변수들을 집계(**Aggregating**) 하여 그 결과로 모델 생성

Bagging

Bagging 수학적 이해

- 원본 훈련 데이터셋 D 가 주어졌을 때
- 크기가 N 인 부트스트랩 샘플 D_i 를 B 번 생성한다..
- 각 부트스트랩 샘플 D_i 에 대해, 학습 알고리즘 A 를 사용하여 모델 M_i 를 독립적으로 학습시킨다.
- 최종 모델(M_{final})의 예측은 생성된 모든 모델의 예측을 결합하여 얻는다.

회귀 문제의 경우,
최종 예측은 모든 모델의 예측값의 평균으로 계산

$$M_{final}(x) = \frac{1}{B} \sum_{i=1}^B M_i(x)$$

분류 문제의 경우,
최종 예측은 모든 모델의 예측에 대한 다수결(가장 많이 예측된 클래스)로 결정
각 클래스 c 에 대한 모델 M_i 의 예측을 $vote(M_i(x), c)$ 로 나타낼 때,

$$M_{final}(x) = \arg \max_c \sum_{i=1}^B vote(M_i(x), c)$$

$vote(M_i(x), c)$ 는 모델 M_i 가 클래스 c 를 예측한 경우 1, 그렇지 않은 경우 0의 값을 가진다.

Bagging

Bagging (Bootstrap + Aggregating)

Bagging Features

- 트리를 형성할 때 데이터셋에만 변화를 주는 것이 아닌 feature를 선택하는데 있어서도 변화를 준다.
- Feature를 선택할 때도 기존에 존재하는 feature의 부분집합을 활용
- 일반적으로 후보 feature가 N개 인 경우, 임의로 선택하는 feature 는 $\sqrt{N}$ 을 활용
- → 만약 36개의 feature 가 있는 경우 임의로 6개의 feature 를 선택하고 그 중에서 information gain 이 높은 feature 를 선택하여 데이터를 분류하게 됨.

최종 모형 결정

Bagging은 각 샘플에서 나타난 결과를 일종의 중간값으로 맞추어 줌으로써 → Overfitting 해소

1. 범주형자료(Categorical Data) : 투표 방식 (Voting)으로 집계 (mode, 최빈값)

- 1) Hard Voting : 개수로 Class 구별
- 2) Soft Voting : 확률로 Class 구별
- 3) Weighted Voting: 모델의 Score에 비례하여 결과값에 가중치 부여(hard나 soft에 옵션으로 적용)

2. 연속형변수(Continuous Data) : 평균 (Average)으로 집계

각 결정트리 모델이 예측한 값에 평균을 취해 최종 Bagging Model의 예측값을 결정한다는 것

Bagging

Bagging (Bootstrap + Aggregating)

- 훈련자료를 얻은 모집단의 분포를 모르기 때문에 train data를 모집단으로 전제
- 이것의 평균예측모형을 구한 것이 배깅의 예측모형
- 배깅은 주어진 예측모형의 평균예측모형을 구하고, 분산을 줄여줌으로써 예측력을 높이는 것!!

(Bagging 효과)

- 배깅은 예측모형의 Bias에는 영향을 미치지 않고 분산을 크게 감소 시킨다.

(이유 : 수개의 Tree가 예측한 값의 분포의 평균 즉 표본 평균의 분포는 분산값이 작은 정규 분포를 보이기 때문)

(한계점)

자료의 작은 변화에 예측모형이 크게 변하는 경우 학습방법이 불안정

- (예시: 의사결정나무에서 첫번째 노드의 분리변수가 바뀌면 자식노드에 포함되는 자료가 완전히 변경되어 최종DT가 변경

예측력 저하

모형 해석이 어려움

- 알고리즘의 안정성과 정확성을 향상 개선

- 가지치기가 불필요

Random-Forest

Random-Forest

- 분류, 회귀 분석 등에 사용되는 일종의 앙상블(Ensemble) 학습 방법
- **Leo Breiman 논문 (2001) Random Forests**
- : 랜덤 노드 최적화(randomized node optimization, RNO)와 배깅(bootstrap aggregating, bagging)을 결합한 방법과 CART(classification and regression tree)를 사용해 상관관계가 없는 트리들로 포레스트를 구성하는 방법을 제시
- → 의사결정나무의 분산이 크다는 특징을 감안하여 붓스트랩(배깅)에 입력변수들에 대한 무작위 추출(랜덤)을 추가하여 약한 학습기들을 생성한 후 이를 선형결합하여 최종학습기를 만드는 기법
- 1) 원 자료로부터 **붓스트랩 샘플을 추출하고**(다수 데이터 셋 형성) → 각 붓스트랩 샘플에 대해 트리를 형성해 나가는 과정(배깅과 유사) : **Forest**
- 2) **피쳐 선택** : 각 노드마다 모든 예측변수 안에서 최적의 분기(split)를 선택하는 방법 대신 예측변수들을 임의로 추출하고 (**Random**), 추출된 변수 내에서 최적의 분할을 만들어 나가는 방법 사용 (설명변수의 임의 추출을 통해 트리의 다양성 확보 → 모형 간 상관관계를 줄여보자!!!)
- 결정 트리의 훈련 알고리즘은, 데이터셋의 '불순도'를 가장 크게 낮추는 방향으로 하나의 특성(변수)과 그의 임계값을 순차적으로 찾아가면서 트리 구조를 만들어 간다.
- 이 트리를 가지고, 향후 신규 데이터에 대해서 트리 형태 판단을 통해 분류(예측)를 하게 된다.
- Random : 각각의 Decision Tree 를 만드는데 있어 사용하는 Feature를 무작위로 선정한다는 의미
- 전체 Feature 중 랜덤으로 일부 Feature만 선택해 하나의 DT를 만들고, 또 전체 Feature 중 random으로 일부 Feature를 선택해 또 다른 DT를 만들며, 여러 개의 Decision Tree를 만드는 방식으로 구성.

scikit-learn : RandomForestClassifier

Random-Forest

예측방법

1. 분류(classification)의 경우

: Decision Tree 들이 내린 예측 값들 중 다수결(majority votes) 원칙으로 최종 예측값 결정

2. 회귀(regression)의 경우

: 평균을 취하는 방법을 사용하여 최종 예측 값으로 정하는 것

(장점)

- Classification 및 Regression 문제 모두 가능
- **앙상블 기법을 통해서 overfitting 문제를 감소** : 랜덤 샘플링 → 일부 사건에 overfitting되지 않은 범용적인 판단 모델, 피처의 개수를 랜덤하게 줄인 데이터를 각각의 결정 트리 모델에 사용하는 기법도 적용
- Missing value가 있거나 Outlier가 포함되어 있어도 작동에 문제가 없을 만큼 유연성이 높다.
- 모델의 노이즈를 심화시키는 Overfitting 문제를 회피하여, 모델 정확도를 향상시킴
- **예측력 향상**
- Classification 모델에서 상대적으로 중요한 변수를 선정 및 Ranking 가능 → **Feature importance** 계산

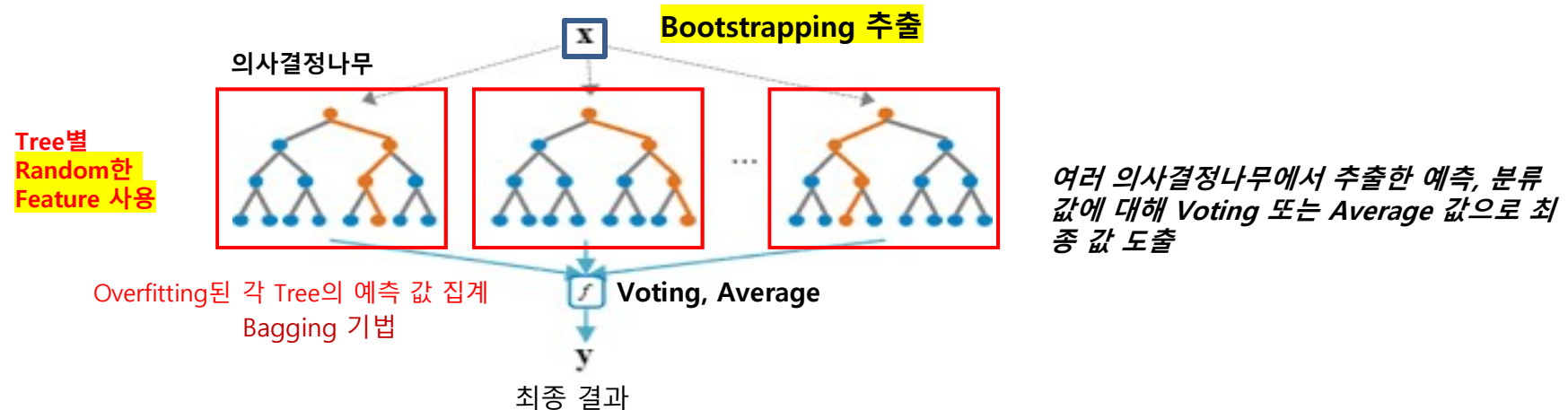
(단점)

- 이론적 설명이나 최종 결과에 대한 해석이 어렵다 → 여러 트리 모델을 겹쳐서 사용하므로 전체적인 구조를 직관적으로 이해하기 힘든 'Black Box' 알고리즘

Random-Forest

CART (Classification & Regression Trees) 특징

1. 설명변수 또는 예측 인자의 비선형성(nonlinearity)과 상호작용(interactions)을 최대한 활용하여 반응 변수(종속변수)에 대한 영향을 판단하는 기법
2. 설명변수를 중요도 기준에 따라 줄기(branch)를 만들어 나가며, 마지막 노드(node: 마디)에서 반응변수에 대해 판단
3. CART는 반응변수가 이항변수, 다항변수, 그리고 연속형인 경우에도 사용
4. 예측인자(설명변수) 역시 연속형과 범주형 변수의 구분 없이 선택 가능
5. CART의 경우, 하위 노드가 많아 질수록 예측오차의 편의(bias)는 줄어들지만 분산은 증가하는 문제가 있다. 반면, 랜덤 포리스트에서는 동일하게 분포된 결정트리를 반복적으로 생성함으로써 예측오차의 분산을 줄일 수 있다.



Random-Forest

Random Forest의 Hyper-Parameter

```
def __init__(
    self,
    n_estimators=100,
    *,
    criterion="gini",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    min_weight_fraction_leaf=0.0,
    max_features="auto",
    max_leaf_nodes=None,
    min_impurity_decrease=0.0,
    bootstrap=True,
    oob_score=False,
    n_jobs=None,
    random_state=None,
    verbose=0,
    warm_start=False,
    class_weight=None,
    ccp_alpha=0.0,
    max_samples=None,
```

- **n_estimators** : 생성되는 의사결정나무의 개수 → 기본값은 100개
- **max_depth**: 트리의 최대 높이, 너무 크면 과적합될 수 있음
- **max_leaf_nodes**: 리프노드의 최대 개수
- **min_samples_split**: 노드 분할을 하기 위한 최소한의 샘플 데이터수, 과적합 제어
- **min_samples_leaf**: 리프 노드가 되기 위한 최소한의 샘플 데이터수, 과적합 제어
- **max_features**: 최적의 분할을 위해 고려할 최대 피쳐 개수.

▪ **Bootstrap** : 복원추출법을 사용하는지 여부를 표시함 (만약 False인 경우, 모든 데이터셋이 각 나무에 적용되게 됨)

▪ **Oob_score** : Out_of_bag, oob는 복원추출법으로 추출되지 않는 데이터를 의미 → 이 데이터를 모델에 대입하여 검증을 진행하는지 여부를 나타낸다. 애초에 train, validation, test 데이터로 분할하는 것과는 별개의 방식으로 train-test split을 수행하기에 데이터 수가 적은 경우 사용하게 된다. 기본값은 False로 수행하지 않는다.

▪ **N_jobs** : CPU 사용 개수를 의미한다. (병렬처리로 연산을 더 빠르게 할 수 있도록 -1로 설정가능)

▪ **Verbose** : 모형 적합과정에서 나오는 결과를 출력하는지 여부로 0은 미출력, 1은 상세한 출력, 2는 요약정보를 출력

▪ **Warm_start** : 기본값인 False는 모델을 fitting하는 과정에서 이전의 가중치 (weight, coef)를 초기화하고 다시 수행한다는 의미이다. 만약 True로 수행하는 경우, 가중치를 초기화하지 않고 계속 데이터를 fitting하며 업데이트 진행

▪ **Max_samples** : Bootstrap이 True인 경우, 각 의사결정나무마다 데이터를 부트스트래핑으로 가져오게 되는데, 이때 기본값인 None인 경우는 Sample을 가져오는 숫자가 Train의 데이터 크기와 동일한 양만큼 가져오게 된다. (10000개의 데이터라면 10000번 복원추출 진행) 만약, int로 입력하게 되면 int 값 만큼 복원추출을 진행하고 만약, float 0.3정도로 입력하면 Train 데이터 크기에 0.3배 만큼 복원추출을 진행하게 된다

Random-Forest

Random Forest의 Hyper-Parameter

Grid Search CV (예시)

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# 파라미터 정의
params = {'criterion': ['entropy', 'gini'],
          'max_depth': [3,5],
          'max_features': ['auto', 'sqrt'],
          'min_samples_leaf': [4, 6, 8],
          'min_samples_split': [5, 7,10],
          'n_estimators': [20]}

model = RandomForestClassifier()
grid = GridSearchCV(estimator = model, param_grid = params,
cv = 4, n_jobs = -1)
grid.fit(X_train,y_train)
```

의사결정나무와 거의 동일한 파라미터로 구성되어 있으나,
의사결정나무의 개수인 n_estimators가 추가되어 있음

참고)
GridSearchCV의 결과 값 보기
pd.DataFrame(grid.cv_results_)

최적 해의 파라미터 : {'criterion': 'entropy', 'max_depth': 3, 'max_features': 'auto', 'min_samples_leaf': 6, 'min_samples_split': 7, 'n_estimators': 20}, 최적 평균 정확도 : 0.967

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_depth	param_max_features	param_min_samples_leaf	param_min_samples_split	param_n_estimators	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	mean_test_score	std_test_score	rank_test_score
0	0.074863	0.019589	0.017143	0.018052	entropy	3	auto	4	5	20	{'criterion': 'entropy', 'max_depth': 3, 'max_...	1.0	0.966667	0.866667	1.0	0.958333	0.054645	14
1	0.086788	0.018355	0.010082	0.003823	entropy	3	auto	4	7	20	{'criterion': 'entropy', 'max_depth': 3, 'max_...	1.0	0.966667	0.866667	1.0	0.958333	0.054645	14
2	0.074751	0.011786	0.010623	0.007879	entropy	3	auto	4	10	20	{'criterion': 'entropy', 'max_depth': 3, 'max_...	1.0	0.966667	0.800000	1.0	0.941667	0.082916	72
3	0.081592	0.013184	0.012701	0.005495	entropy	3	auto	6	5	20	{'criterion': 'entropy', 'max_depth': 3, 'max_...	1.0	0.966667	0.866667	1.0	0.958333	0.054645	14
4	0.084248	0.006752	0.014158	0.007280	entropy	3	auto	6	7	20	{'criterion': 'entropy', 'max_depth': 3, 'max_...	1.0	0.966667	0.900000	1.0	0.966667	0.040825	1

* 참조 : RF이용 주가 단기 예측 4

KOSPI 단기 예측 (향후 3개월 예측)

- 피처(독립변수) : 매크로 지표(환율, 유가, 수출증가율...) , 시장 투자 지표 (VIX, VKOSPI , put call ratio.....)
→ 단기 예측 측면에서 일별수익률 사용
- 레이블(종속변수) : '상승/보합/하락'

Data set

- 과거10년~ 현재:일별 시계열 자료
- 피처(feature): 매크로 지표(환율, 유가...) , 시장 투자 지표 일별 data
- 레이블(label): t일의 사후 KOSPI 방향성. 상승/보합/하락으로
- KOSPI의 사후(T+1영업일~ T+ 60영업일 수익률) 수익률 → 5% 이상이면 '상승', 5%~0% 사이면 '보합', 0% 미만이면 '하락' 으로 인코딩

* 참조 : RF이용 주가 단기 예측 5

KOSPI 단기 예측 (향후 3개월 예측)

모델 학습 프로세스

1. Raw data 가져오기 : 일별
2. features, label 전체데이터 생성
3. train, test data split
4. 최적 Hyperparameter, Parameter : 모델 튜닝
5. 학습 K-fold cross validation : 모델 성능 확인
6. 최종 모델 학습
7. 최종 성능평가 : test data 확인
8. confusion matrix 확인
9. 변수 중요도 체크모델 사용여부 결정



예측 프로세스

1. 모델링 로드
2. new daily raw data 가져오기
3. new daily raw data 전체 학습

* Tree 모델 : 학습 속도를 높이기 위한 데이터 세 가지 방법

- 의사결정 나무는 단순하고 해석이 용이
- 대규모 데이터셋을 처리할 때 시간이 많이 소요될 수 있다.
- 학습 속도를 높이기 위한 대표적인 방법 세 가지는 다음과 같다:

1. 히스토그램 기반 그라디언트 부스팅 (Histogram-based Gradient Boosting)		▪ 데이터의 연속형 변수를 구간화하여 히스토그램을 생성한 뒤, 각 구간을 기반으로 학습하는 방법 ▪ 계산량을 줄이고 학습 속도를 높일 수 있다.
2. 샘플링 방법 (Sampling Techniques)	부트스트랩 샘플링 (bootstrap sampling)	▪ 학습에 사용할 값들을 여러 번 무작위 복원 추출하여 추출된 값들의 통계량을 계산하는 것
	GOSS(Gradient-based One-Side Sampling)	▪ 그라디언트 기반의 한쪽 샘플링 방법 → 중요도가 높은 샘플에 더 많은 비중을 두어 학습을 진행하는 방식 ▪ 계산량을 줄이고 학습 속도를 높일 수 있다.
3. EFB(Exclusive Feature Bundle)		▪ 상호 배타적인 특징 번들을 사용하여 다수의 특징을 하나의 번들로 묶는 방식 ▪ 고차원 데이터의 특징 수를 줄여 계산 복잡도를 낮추고 모델 학습 속도를 향상시킨다.

* 특징 선택 및 축소 (Feature Selection and Reduction)

불필요한 특징을 제거하거나, 상관관계가 높은 특징들을 축소하면 학습 속도를 크게 높일 수 있다.

PCA(Principal Component Analysis) 같은 차원 축소 방법도 유용하다.

* 학습 속도를 높이기 위한 데이터 세 가지 방법

구분	장점	단점
Histogram-based	▪ 데이터 압축: 데이터를 구간으로 나누어 빈도를 계산함으로써 데이터의 크기를 줄일 수 있다. ▪ 빠른 계산: 데이터의 구간별 빈도를 사용하므로 계산이 단순하고 빠르다. ▪ 분포 파악 용이: 데이터의 전체적인 분포를 쉽게 파악할 수 있다.	▪ 정확도 저하: 데이터가 구간별로 나누어지면서 세부 정보가 손실될 수 있다. ▪ 구간 설정의 어려움: 적절한 구간 수와 경계를 설정하는 것이 어렵다. ▪ 특성 변환 필요: 원래 데이터를 구간화된 데이터로 변환하는 추가 작업이 필요하다.
GOSS	▪ 효율적인 샘플링: 중요한 샘플에 더 많은 비중을 두어 학습 효율성을 높인다. ▪ 계산량 감소: 전체 데이터셋을 사용하는 대신 일부 샘플만 사용하므로 계산량이 줄어든다. ▪ 성능 유지: 중요한 샘플을 유지함으로써 모델의 성능을 크게 떨어뜨리지 않는다.	▪ 복잡한 구현: 샘플의 중요도를 계산하고 샘플링하는 과정이 복잡할 수 있다. ▪ 불균형 데이터 처리의 어려움: 데이터의 불균형 문제가 있을 경우 샘플링 결과가 왜곡될 수 있다. ▪ 추가 매개변수 필요: 샘플링 비율 등 추가 매개변수를 조정해야 한다.
▪ EFB	▪ 차원 축소: 상호 배타적인 특징들을 하나의 번들로 묶어 특징 수를 줄인다. ▪ 효율적인 계산: 차원이 감소하여 계산 복잡도가 낮아지고 학습 속도가 빨라진다. ▪ 메모리 절약: 사용되는 메모리 양을 줄일 수 있다.	▪ 특징 선택의 어려움: 어떤 특징들을 번들로 묶을지 선택하는 과정이 까다로울 수 있다. ▪ 정보 손실 가능성: 번들로 묶으면서 일부 중요한 정보가 손실될 수 있다. ▪ 복잡한 전처리: 번들링 과정이 추가적인 전처리 작업을 요구한다.

*

* 학습 속도를 높이기 위한 데이터 세 가지 방법

조기 종료(Early Stopping) 콜백 설정

- 머신러닝 모델의 학습 과정에서 과적합을 방지하고 최적의 성능을 가진 모델을 선택하기 위해 사용
 - 조기 종료는 검증 데이터(validation set)에서 일정 횟수 이상 성능이 향상되지 않으면 학습을 중지하는 방법이다.
- 이는 학습 시간이 절약되고 모델이 불필요하게 복잡해지는 것을 막아준다.

조기 종료는 특히 *LightGBM, XGBoost*와 같은 부스팅 라이브러리에서 많이 사용

<https://github.com/freejyb/KDT/blob/main/> 머신러닝
TREE 모델속도개선

Boosting

Boosting

- 편향과 지도 학습의 분산을 줄이기 위한 앙상블 메타 알고리즘
- 여러 개의 약한 학습기(weak learner)를 **순차적으로** 학습-예측하면서 오답 예측한 데이터에 높은 가중치를 부여해 오류를 개선해 나가며 정답에 대해서는 낮은 가중치를 부여하여 오답에 집중할 수 있게 되는 것으로 정확도가 높은 학습 방식 모형
- 이전에 학습한 모델을 update시켜 **잔차를 줄여가는 방식으로 모델을 여러 번 업그레이드 시키는 방법이기 때문에 편향치를 감소**
- 가법모형(Additive model)의 하나로 비모수 회귀모형

- 1997 Freund & Schapire → **AdaBoost (Adaptive Boost)**
- A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*

Bagging과 차이점

- **Bagging** : 병렬적으로 여러 개의 독립적인 DT를 만들어 최종 결과 값을 예측 , 분산을 줄이자!!!
- **Boosting** : 직렬적으로 1개의 모델을 계속 업데이트 시켜가는 것이 차이점→ 즉 Boosting 은 모델 간 연결고리가 생겨 팀워크가 이루어진다. 편향을 줄이자!!!

(과정)

각 데이터 샘플에 동일한 가중치를 할당→ 부스팅에서는 붓스트랩 표본을 추출하여 분류기를 만든 후→ 그 분류결과를 이용하여 각 데이터가 추출될 확률을 조정한 후 (잔차를 보고 가중치 부여) → 다음 붓스트랩 표본을 추출하는 과정을 반복

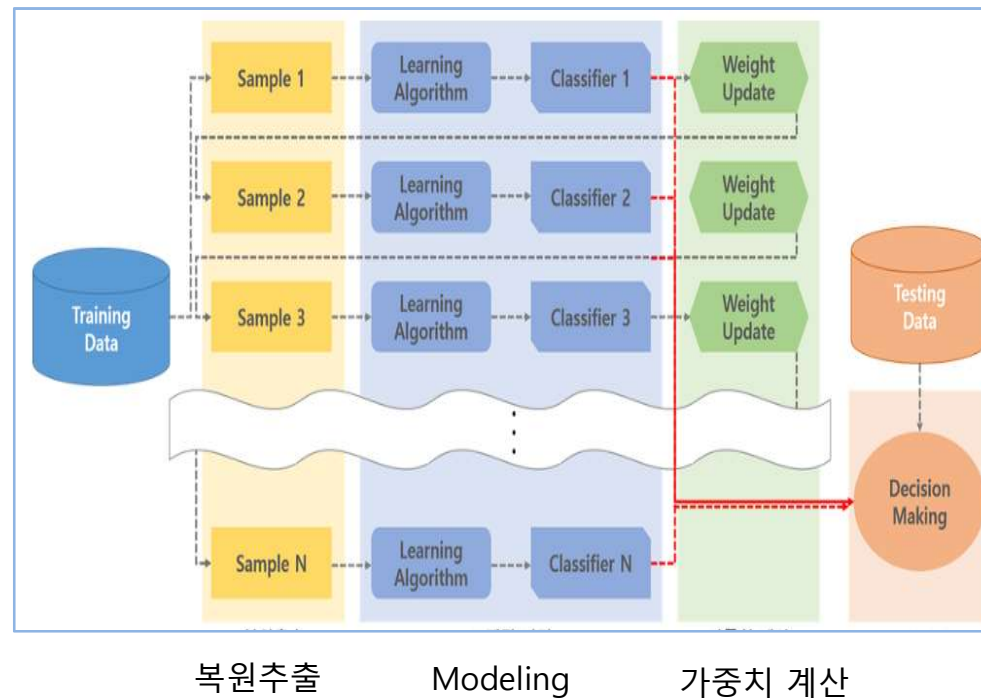
→ 학습이 끝나면 나온 결과에 따라 가중치가 재분배된다.

- **가법모형(Additive Model) : 종속변수를 각 개별 설명변수들만의 비모수적 함수의 합으로 표현하는 방법**

Boosting

Boosting 출발

- 모델은 overfitting되면 안된다!!!
- 모델이 학습 데이터에 overfitting 되는 것을 막기 위해 **약한 모델을 여러 개 결합시켜 그 결과를 종합한다!!!**
- 부스팅은 여기에 sequential이 추가→ 연속적인 weak learner, 바로 직전 weak learner의 error를 반영한 현재 weak learner를 잡겠다는 아이디어
- → GBM에서 loss를 계속 줄이는 방향으로 weak learner를 강 분류기로 만들겠다는 개념으로 확장



1. training data에서 random sampling을 하고 1번 weak learner로 학습
2. 그 결과로 생긴 에러를 반영해 그 다음 데이터 샘플링과 2번 weak learner 학습 반복
3. 이 과정을 N번 하면 iteration N번 돌린 boosting modeling이 되는 것

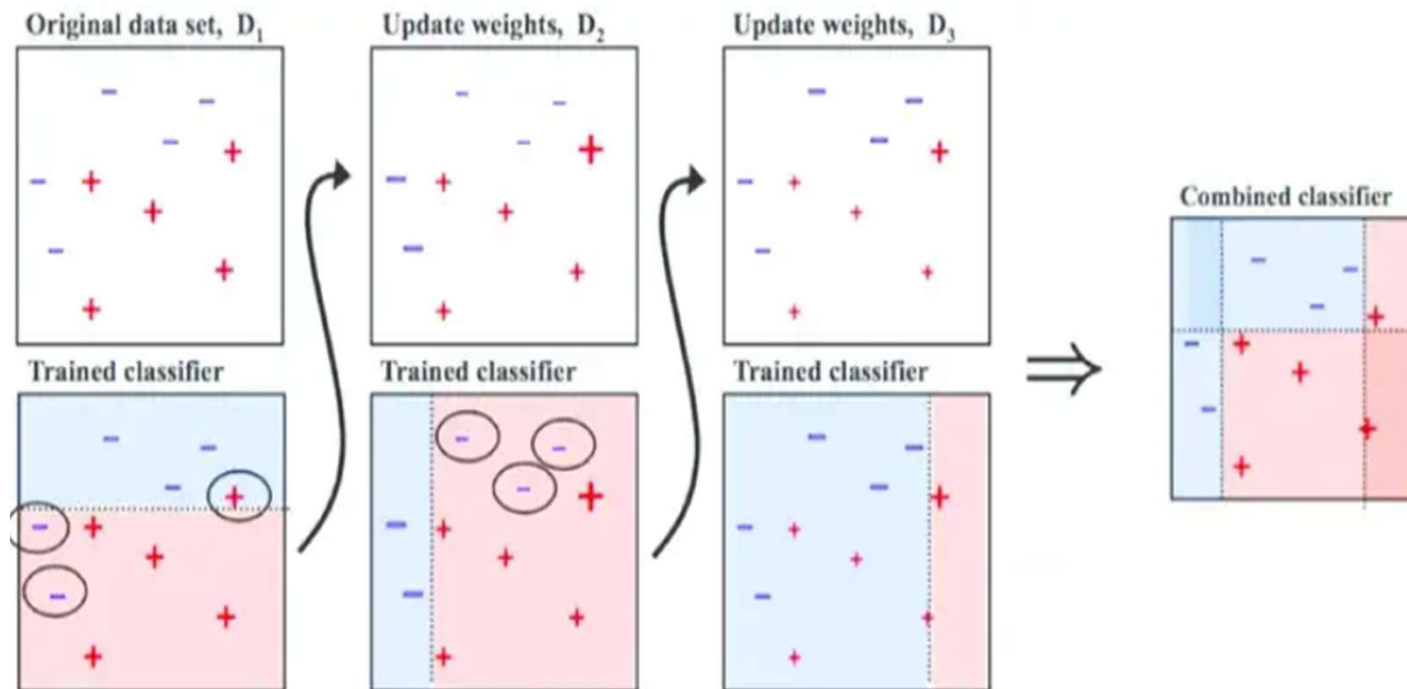
Sample1, 에서 sample2 구성 : 가중치 잘못 분류된 데이터를 다음 바구니 구성에 넣어버리는 것을 '가중치를 준다' 라고 표현한다. 즉, 잘못 분류된 데이터에 가중치를 높이고 잘 분류된 데이터의 가중치를 낮춤으로써 다음 바구니를 구성하는 것

Boosting

<https://medium.com/swlh/boosting-and-bagging-explained-with-examples-5353a36eb78d>

Boosting algorithms is the family of algorithms that combine weak learners into a strong learner.

Working of boosting



Boosting 수학적 표현

부스팅에서는 학습 데이터셋 $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_n,y_n)\}$ 이 주어지며,
여기서 x_i : 특성 벡터, y_i : 해당 벡터의 레이블,
목표는 손실 함수(loss function) $L(y,f(x))$ 를 최소화하는 모델 $f(x)$ 를 찾는 것 !!!!!

부스팅 알고리즘은 T 번의 반복을 통해 T 개의 약한 학습기 $h_t(x)$ 를 생성하고,
최종 모델은 이러한 약한 학습기들의 가중합으로 표현된다.

$$f(x) = \sum_{t=1}^T \alpha_t h_t(x) \quad \alpha_t: t\text{번째 약한 학습기의 가중치}$$

Boosting

Boosting 장. 단점

(장점)

bagging에 비해서 Boosting이 error 가 적어 성능이 좋다.

→ 잘못 분류된 값들을 가중치 반영해서 모델을 반복해서 만들어 오류가 적어지는 것!!

(단점)

- 속도가 느리고 overfitting 될 가능성 존재

→ 모델을 만들고 test 하고 다시 만들고 test 하는 과정을 반복하다 보니 속도가 느려진다.

→ 잘못 분류된 값들에 가중치를 부여하며 모델을 만들어 가다보면 train set 에 최적화 → test set 적용시 예측률 감소로 과적합 문제가 발생

- 이상치에 영향

Boosting

Boosting algorithm

- **AdaBoost** (Adaptive Boost) **1997**
- **Gradient Booting Machine (GBM)**
- **eXtreme Gradient Boost (XGBoost)**
- **LightGBM**
- **CatBoost**
- **AUCBoost**

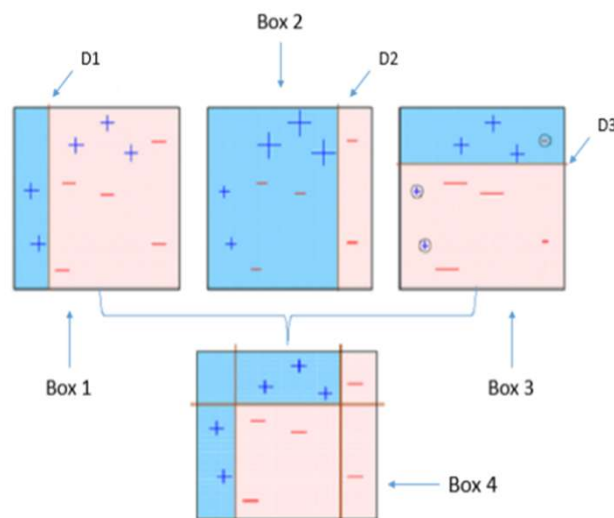
Boosting : 1. AdaBoost

AdaBoost(Adaptive Boost) 1997

- 1997 Freund & Schapire → **AdaBoost (Adaptive Boost)**
- A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*

weak learner의 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 대표적인 알고리즘

- 약한 학습기를 순차적으로 학습 → 개별 학습기에 가중치를 부여 → 모두 결합
→ 개별 약한 학습기보다 높은 정확도의 예측 결과를 만들
속도나 성능적인 측면에서 decision tree를 약한 학습기로 사용함



과정 이해

- Step 1)** 전체 data에서 random sampling
첫 번째 약한 학습기가 첫 번째 분류기준(D1)으로 + 와 - 를 분류
- Step 2)** 잘못 분류된 데이터에 대해 가중치를 부여(두 번째 그림에서 커진 + 표시)
- Step 3)** 두 번째 약한 학습기가 두 번째 분류기준(D2)으로 +와 - 를 다시 분류
- Step 4)** 잘못 분류된 데이터에 대해 가중치를 부여(세 번째 그림에서 커진 - 표시)
- Step 5)** 세 번째 약한 학습기가 세 번째 분류기준으로(D3) +와 -를 다시 분류해서 오류 데이터를 찾음
- Step 6)** 마지막으로 분류기들을 결합하여 최종 예측 수행

- **약한 학습기(weak learner: 랜덤하게 예측하는 것보다 약간 좋은 예측력을 지닌 모형. 약한예측모형)**
- **강한 학습기 : 예측력이 최적에 가까운 예측모형**

Boosting : 1. AdaBoost

AdaBoost(Adaptive Boost) 1997

n개의 학습 표본과 K개의 기저 분류자로 구성된

앙상블 $C = \{C_1, C_2, \dots, C_K\}$ 을 가정하면

k번째 기저분류자의 오류율 ε_k 다음과 같이 **단순평균**으로 계산된다

$$e_k = \frac{1}{n} \sum_{i=1}^n L(C_k(x_i), y_i)$$

$$L(C_k(x_i), y_i) = \begin{cases} 1 & C_k(x_i) \neq y_i \\ 0 & C_k(x_i) = y_i \end{cases}$$

x_i : i번째 관측치의 예측변수 벡터

y_i : i번째 관측치의 범주

$\blacksquare \rightarrow \textcircled{\bullet}$: 예측변수 벡터 x_i 에 대한 k 번째 분류기의 분류 결과

k+1 번째 분류기에서 i 번째 관측치에 부여되는 가중치

: $w_{k+1}(i) = w_k(i) \exp(\alpha_k L(C_k(x_i), y_i))$ 조정되어 오분류된 관측치에 더 높은 가중치가 부여된다

α_k : 분류기의 중요도 또는 정확도의 개념으로 해석

$$\alpha_k = \ln\left(\frac{1-\varepsilon_k}{\varepsilon_k}\right)$$

K+1번째 분류기의 학습표본을 구성할 때 가중치가 높은 오분류 관측치가 많이 포함되기 때문에 부스팅 알고리즘은 오분류 관측치에 초점을 맞춘 학습을 진행할 수 있게 된다.

앙상블 학습은 $\varepsilon_k < 0.5$ 일 때 학습을 중단하며, i번째 관측치의 최종결과를 아래와 같이 앙상블의 결과치의 가중평균으로 계산하여 산출

$$C(x_i) = \text{sign}\left(\sum_{k=1}^K \alpha_k C_k(x_i)\right)$$

Boosting : 1. AdaBoost

AdaBoost(Adaptive Boost) 1997

부스팅 알고리즘은 단순 평균 개념에 기초한 주요 파라미터로 인하여 다음과 같은 문제

첫째, 학습의 조기중단 문제이다.

분류기의 오류율 e_k 는 단순평균 오류율로 전체 표본 대비 오분류 표본 비율로 계산된다.

불균형 데이터의 경우 다수 범주의 낮은 오류율로 인하여 단순평균 오류율은 빠르게 중단조건 ($e_k < 0.5$)에 수렴하게 되고 충분한 학습이 이루어지기도 전에 학습이 중단될 수도 있다.

둘째, 분류기의 성과를 나타내는 α_k 역시 단순평균 정확도에 기초한 개념이다.

데이터 불균형 하에 서 단순평균 정확도는 성과지표로 유효하지 않기 때문에 다수 범주와 소수 범주를 동시에 고려한 가중평균 정확도 개념으로 대체할 필요가 있다.

Boosting : 1. AdaBoost

AdaBoost : Hyper Parameter

Hyper Parameter	내용
▪ base_estimators	▪ 학습에 사용하는 알고리즘 - Default = None → DecisionTreeClassifier(max_depth=1)가 적용
▪ n_estimators ▪ (Default = 50)	▪ 생성할 weak learner의 갯수를 지정 ▪ n_estimators를 늘린다면 생성하는 weak learner의 수는 늘어남 ▪ 이 여러 학습기들의 decision boundary가 많아지면서 모델이 복잡해짐
▪ learning_rate ▪ (Default = 1.0)	▪ 학습을 진행할 때마다 적용하는 학습률 ▪ (0~1) ▪ Weak learner가 순차적으로 오류 값을 보정해 나갈 때 적용하는 계수 ▪ learning_rate을 줄인다면 가중치 갱신의 변동폭이 감소해서, 여러 학습기들의 decision boundary 차이가 줄어들

→ 만약, *n_estimators*를 늘리고, *learning_rate*을 줄인다면 서로 효과가 *trade-off* 관계

* Boosting

GM-Boost (Geometric Mean-based Boosting)

- Kubat et al.(1997)이 제안
- AdaBoost 알고리즘에 기하평균 개념을 도입한 새로운 부스팅 알고리즘
- 데이터 불균형이 심각한 상황에서도 높은 예측력을 확보할 수 있으며 데이터 불균형 정도에 관계없이 견고한 학습능력을 제공한다는 장점

1. 가중치 초기화: $w_k(x_i) = 1/n, i = 1, 2, \dots, n$
2. 중단조건($e_k < 0.5$)을 만족할 때까지 분류자 생성 반복($C_k, k = 1, 2, \dots, K$)
 - a) 가중치 기준에 의거하여 추출된 학습 데이터의 분류자 학습: $C_k(x_i) \in \{-1, 1\}$
 - b) 기하평균 오류율 산출

$$e_k = \sqrt{e_k^+ \cdot e_k^-}$$

$$e_k^+ = \frac{1}{n^+} \sum_{i=1}^{n^+} L(C_k(x_i), y_i) \text{ and } e_k^- = \frac{1}{n^-} \sum_{i=1}^{n^-} L(C_k(x_i), y_i)$$
 - c) 기하평균 정확도 산출

$$\alpha_k = \sqrt{\alpha_k^+ \cdot \alpha_k^-} \text{ 단, } \alpha_k^+ = \ln\left(\frac{1-e_k^+}{e_k^+}\right) \text{ and } \alpha_k^- = \ln\left(\frac{1-e_k^-}{e_k^-}\right)$$
 - d) 가중치 조정: $w_{k+1}(i) = w_k(i) \exp(\alpha_k L(C_k(x_i), y_i))$
3. 최종 결과 생성: $C(x_i) = \text{sign}(\sum_{k=1}^K \alpha_k C_k(x_i))$

기업부실 예측 데이터의 불균형 문제 해결을 위한 앙상블 학습 김명종(2009)

Boosting : 2. Gradient Boost Machine(GBM)

GBM = Boosting + Gradient Descent

<https://machinelearningmastery.com/gentle-introduction-gradient-boosting-algorithm-machine-learning/>

- Freund(1995) Freund and Schapire(1997), Friedman(2001) 등에 의해 제시
- 트리 모형과 경사하강법(Gradient Descent) 이용 가중치 업데이트에 따른 추가적인 학습을 결합으로 오류값을 최소화하는 방식으로 학습
- 무작위로 샘플링된 데이터에 관해 DT 모델을 구성하고, 경사 하강법(Gradient Descent)을 통해 잔차를 점차 줄여 모델을 재구성

AdaBoost 한계점 → **weight가 낮은 데이터 주위에 높은 weight를 가진 데이터가 있으면 의도치 않게 잘못 분류가 되면서 성능 저하**

GBM 작동과정

1. 초기 모델 생성

처음에 단순한 모델(대부분 평균 값으로 예측하는 모델)로 시작하여 초기 예측값을 설정한다. 이 초기 모델이 첫 번째 약한 학습기 역할을 한다.

2. 잔차 계산

각 데이터 포인트에 대해 예측값과 실제 값의 차이인 잔차(residual)를 계산한다. 이 잔차는 다음 학습 단계에서 줄여야 할 오류를 나타낸다.

3. 새로운 모델 학습

GBM은 각 단계에서 새로운 약한 학습기를 추가하여 기존 모델의 잔차를 예측한다. 보통 결정 트리를 사용하며, 각 트리가 이전 모델의 잔차를 줄이는 방향으로 학습한다.

4. 모델 업데이트

새롭게 학습된 모델을 이전 모델에 합산하여 업데이트한다. 이 과정은 보통 학습률(learning rate) 하이퍼파라미터를 곱하여 새로운 학습기가 예측에 너무 큰 영향을 미치지 않도록 조정한다.

5. 반복

여러 약한 학습기를 순차적으로 추가하면서 위 과정을 반복한다. 각 단계에서는 이전 단계의 예측 오류를 줄이기 위한 새로운 모델이 학습한다.

6. 최종 예측

모든 약한 학습기의 예측값을 합산하여 최종 예측값을 만든다. 이때 각 학습기의 예측에 가중치가 부여될 수 있다. 159

Boosting : 2. Gradient Boost Machine(GBM)

GBM = Boosting + Gradient Descent

GBM 방식

- Mean Squared Error를 쓰는 경우 : 현재 모델의 MSE의 미분값인 residual을 target으로 두고 다음 모델을 학습함으로써 이전 모델에서 오류를 다음 모델에서 줄일 수 있도록 한다. (뒷면 참조)
- 트리 모델을 순차적으로 추가하되 종속변수가 이전 트리 모델이 설명하지 못한 잔차항이라는 특징
- 파라미터에 대한 목적함수의 최소화(잔차항의 최소화) : 경사 벡터를 계산한 후 음(-)의 부호를 붙이면 그 방향이 결국 잔차항을 최소화하는 방향이 된다.
 - 모수에 대한 학습은 음(-)의 경사 방향으로 어떤 크기(또는 비율)만큼 모수(학습률)를 변화시킴으로써 이루어진다.
 - ➔ Sequential 한 weak learner들을 잔차(residual)를 줄이는 방향으로 결합하여 object function과의 loss를 줄여나가는 방식 → 결합한 분류기를 강한 분류기 (strong learner)로

Boosting : 2. Gradient Boost Machine(GBM)

GBM 방식

Loss function= MSE로 정의

$$Loss = MSE = L(y_i, f(x_i)) = L(y_i, \bar{y}_i) = \sum (y_i - \bar{y}_i)^2$$

where, y_i = target value, $\bar{y}_i$ = predction

$$j(y_i, f(x_i)) = \frac{1}{2} (y_i - f(x_i))^2$$

Residual (잔차) = 실제값 - 예측값

RSS(Residual Sum of Square) = 손실 함수(loss function)

Loss function을 미분해 보면 negative gradient는 residual

Loss function을 사용해 negative gradient를 유도하고 새로운 model의 target으로 fitting하는 방식으로 최종 모델 → Gradient Boost

$$\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} = \frac{\partial [\frac{1}{2} (y_i - f(x_i))^2]}{\partial f(x_i)} = f(x_i) - y_i$$

negative gradient = residual

- residual(잔차) = 실제값 - 예측값
- negative gradient = residual

$$y_i - f(x_i)$$

다음 모델을 만들 때, negative gradient를 이용해 만들기 때문에 gradient boosting

Boosting : 2. Gradient Boost Machine(GBM)

장.단점

(장점)

GBM은 예측 성능이 높다.

→ Tabular format 데이터에 대한 예측에서 머신러닝 알고리즘 중에서도 가장 예측 성능이 높다고 알려진 **알고리즘**

***XGBoost, LightGBM, CatBoost* : Gradient Boosting Algorithm을 구현한 패키지들**

(단점)

1. 긴 학습 시간 : GBM은 트리 기반 모델을 순차적으로 학습해나가는 방식이기 때문에 많은 데이터와 복잡한 모델을 다룰 때 학습 시간이 길어질 수 있다

2. 과적합(Overfitting) 위험

- GBM은 매우 강력한 예측 모델이지만, 과적합의 위험이 크다.
- 특히 금융 데이터와 같은 복잡한 데이터에 적용할 때, 너무 많은 트리를 학습하거나 깊은 트리를 사용할 경우 과적합이 발생할 수 있다.

3. 하이퍼파라미터 튜닝의 복잡성

GBM 모델은 많은 하이퍼파라미터를 가지고 있으며, 이들 하이퍼파라미터를 최적화하는 것이 쉽지 않다.

트리의 수, 학습률(learning rate), 각 트리의 깊이 등 여러 매개변수를 조정해야 최적의 성능을 낼 수 있다.

이 과정이 까다롭고 시간이 많이 걸리며, 경험이 부족한 경우에는 과적합이나 성능 저하의 원인이 될 수 있다.

4. 해석 가능성 부족

- GBM은 트리 앙상블을 사용하는 복잡한 모델이기 때문에, 예측 결과의 해석이 어렵다.
- 금융 분야에서는 모델의 결정 근거를 설명할 수 있어야 하는 경우가 많다.
- 그러나 GBM은 "블랙박스" 모델로 여겨져서, 각 변수나 예측이 결과에 미치는 영향을 명확하게 설명하기가 어렵다.

5. 메모리 및 자원 소모

- GBM은 많은 트리와 노드를 생성하므로, 메모리와 계산 자원을 많이 소모
- 특히 데이터셋이 크거나 특징이 많은 경우에는 고성능의 하드웨어가 필요

6. 데이터 전처리 필요성

Boosting : 2. Gradient Boost Machine(GBM)

금융시장 활용

1. 신용 위험 평가

GBM은 고객의 신용 기록, 거래 내역, 소득 수준 등 다양한 변수를 고려하여 대출 상환 가능성을 예측하고, 이를 통해 대출 승인 여부와 이자율을 결정.

2. 거래 사기 탐지

고객의 거래 패턴과 과거 사기 데이터를 학습하여, 이상 징후를 조기에 발견하고 사기를 예방하는 데 기여
GBM은 특히 불균형 데이터에서도 강력한 성능을 발휘하기 때문에 사기 탐지에 적합

3. 주가 및 자산 가격 예측

과거 가격 데이터와 매크로 경제 지표, 기업 재무 정보 등을 학습하여 미래의 주가와 자산 가격 변동을 예측

4. 고객 이탈 예측

금융 서비스 이용 고객이 서비스를 중단할 가능성을 예측하는 데도 GBM이 활용
고객의 이용 패턴, 거래 내역, 피드백 등을 학습하여 이탈 가능성이 높은 고객을 식별하고, 이탈을 방지할 수 있는 맞춤형 마케팅 전략을 수립할 수 있다.

5. 고객 세분화 및 맞춤형 상품 추천

고객의 나이, 소득, 금융 상품 이용 이력을 바탕으로 적절한 예금, 대출 상품 등을 추천하여 고객 만족도를 높이고, 크로스셀링 및 업셀링을 유도할 수 있다.

***GBM은 XGBoost, LightGBM 등과 같이 Gradient Boosting의 여러 변형 모델들로도 적용될 수 있다.
금융 분야에서 중요한 모델로 자리 잡고 있으며,
특히 데이터 전처리와 하이퍼파라미터 튜닝을 통해 성능을 극대화할 수 있다.***

Boosting : 2. Gradient Boost Machine(GBM)

GBM Hyper Parameter from sklearn.ensemble import GradientBoostingClassifier

1) tree에 관한 Hyper Parameter

Hyper Parameter	내용
max_depth [default = 3]	tree의 최대 깊이 - 깊이가 깊어지면 overfitting 문제 발생하므로 제어 필요
max_features [Default = 'none']→ 모든 피쳐 사용	- 최적의 분할을 위해 고려할 최대 feature 개수 - int형으로 지정 → 피쳐 갯수 / float형으로 지정 → 비중 - sqrt 또는 auto : 전체 피쳐 중 $\sqrt{(\text{피쳐개수})}$ 만큼 선정
min_samples_split [default = 2]	- node를 분할하기 위한 최소한의 샘플 데이터 수 → overfitting 제어 용도 - 작게 설정할 수록 분할 노드가 많아져 overfitting 가능성 증가
min_samples_leaf [default = 1]	leaf node가 되기 위해 필요한 최소한의 샘플 데이터 수 - overfitting 제어 용도 - 불균형 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있으므로 작게 설정 필요
max_leaf_nodes [default = None] → 제한없음	- leaf node의 최대 개수

Boosting : 2. Gradient Boost Machine(GBM)

GBM Hyper Parameter from sklearn.ensemble import GradientBoostingClassifier

2) Boosting에 관한 하이퍼파라미터

Hyper Parameter	내용
Loss [default = deviance]	경사하강법 (Gradient Descent)에서 사용하는 cost function을 지정
learning_rate [default = 0.1]	- GBM이 학습을 진행할 때마다 적용하는 학습률 : 순차적으로 오류 값을 보정해가는 데 적용하는 계수 : 0~1사이의 값 → 너무 작은 값 : 꼼꼼하지만 시간이 많이 걸린다. → 너무 큰 값 : 최소 오류 값을 찾지 못할 수 있지만 빠르다.
n_estimators [default = 100]	- weak learner의 개수(생성할 트리의 개수) → 역시 많을수록 성능 향상되지만, 시간이 많이 걸린다.
subsample [default = 1]	weak learner가 학습에 사용하는 데이터의 샘플링 비율 (0~1)

Boosting : 3. XGBoost (eXtreme Gradient Boost)

XGBoost = GBM + regularization term + 다양한 Loss function 지원

1. 개요

<https://xgboost.readthedocs.io/en/stable/>

Chen and Guestrin(2016)

- AdaBoost , Gradient Boost 방법은 성능은 좋지만 학습/연산 시간에서 느리다.
→ XGBoost 시간적인 요소를 개선 (XGBoost는 병렬 처리를 구현-각 트리가 이전 트리 이후에만 구축될 수 있다는 것)
- **GBM에 기반**하고 있지만, Gradient Boost **모형을 간소화(GBM의 오랜 학습시간 및 과적합 문제) 와 실행 속도 면에서 크게 개선한 기법**
- **높은 유연성** : 사용자 정의 최적화 목표 및 평가 기준을 정의 가능
- Approximate 알고리즘을 기반으로 데이터 정렬 및 분할을 통해 병렬처리를 수행하여 학습 속도를 향상
- **정규화(Regularization, LASSO 모형 이용)를 적용해 모델 구조를 간소화하여 Overfitting 문제를 해결**
- 트리 모형에서 부모 노드를 자식 노드로 분리하는 기준을 탐색하는 과정에 수치 근사법을 적용함으로써 실행 속도의 개선이 이루어졌고 기존 트리모형이 가지는 다차원 탐색 문제 완화
- XGBoost는 지정된 **max_depth**까지 분할 한 다음 트리를 뒤로 가지치기 시작하고 긍정적인 이득이 없는 분할을 제거
- **내장 교차 검증** : 부스팅 프로세스의 각 반복에서 교차 검증을 실행

<https://xgboost.readthedocs.io/en/latest/parameter.html>

Boosting : 3. XGBoost (eXtreme Gradient Boost)

2. 학습과정

XGBoost = GBM + regularization term + 다양한 Loss function 지원

1. 초기 모델 생성

초기 모델은 모든 데이터 포인트에 대해 일정한 예측 값을 가지는 단순한 모델로 시작한다. 이 초기 모델은 첫 번째 학습기의 기반이 된다.

2. 잔차(residuals) 계산

각 데이터 포인트에 대해 현재 모델의 예측 값과 실제 값 간의 오차, 즉 잔차를 계산한다. 이 잔차는 새로운 학습기에서 줄여야 할 오류를 의미한다.

3. 새로운 트리 학습

XGBoost는 각 단계에서 결정 트리(약한 학습기)를 추가하여, 현재 모델이 줄여야 할 잔차를 예측하도록 학습한다.

각 노드의 분할 기준은 손실 함수를 최소화하는 방향으로 결정된다.

XGBoost는 이를 위해 second-order (이차 미분)까지 고려한 Taylor Series를 활용한다.

4. 정규화와 학습률 적용

각 트리의 예측값에 학습률(learning rate)을 곱하여 업데이트한다. 학습률은 새로운 트리의 예측이 전체 모델에 과도한 영향을 미치지 않도록 조정한다.

또한, XGBoost는 모델이 복잡해지는 것을 방지하기 위해 L1 및 L2 정규화를 통해 과적합을 제어한다.

5. 중요한 최적화 기법

- Column Subsampling: 각 트리를 학습할 때 일부 열만을 무작위로 선택하여 사용한다. 이를 통해 과적합을 방지하고 학습 속도를 개선한다.
- Tree Pruning (Max Depth): XGBoost는 트리의 깊이를 제한하고, F-Score 기반으로 가지치기를 수행하여 과적합을 방지한다.
- 빠른 계산 (Histogram-based Algorithm): XGBoost는 연속형 변수를 히스토그램 방식으로 변환하여 계산 효율을 높인다.
- 병렬 처리: XGBoost는 각 트리를 병렬로 학습할 수 있도록 설계되어 있어, 대규모 데이터셋에서 빠른 학습이 가능하다.

6. 반복

모델이 충분히 학습될 때까지, 위 과정을 반복하여 새로운 트리를 추가합니다. 이때, 조기 종료 조건을 설정하여 모델이 더 이상 개선되지 않을 경우 학습을 중단할 수도 있습니다.

7. 최종 예측 생성

- 모든 트리의 예측값을 합산하여 최종 예측값을 생성한다.
- 각 트리의 예측에 학습률이 적용되어, 모델이 더 신중하게 업데이트되도록 한다.

Boosting : 3. XGBoost (eXtreme Gradient Boost)

* XGBoost 의 모델과 목적함수

$$\text{XGBoost loss function} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

$$\text{obj} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

K : 의사결정나무의 개수

f_k : k 번째 의사결정나무

ℓ : 손실함수 → 일반적으로 Mean squared error, 로지스틱 손실함수를 사용

Ω : 정규항을 의미

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

리프의 개수(T)와 리프 스코어의 L2 Norm으로 구성되어 있어 의사결정나무의 복잡도를 나타낸다.

3. XGBoost 장점

- (1) 예측 성능이 높다.
- (2) GBM 과적합 개선 : Overfitting Regularization → **XGBoost는 'regularized boosting' 기법**
- (3) Tree pruning(트리 가지치기) : 긍정 이득이 없는 분할을 가지치기해서 분할 수를 줄임
- (4) 자체 내장된 교차 검증
 - 반복 수행시마다 내부적으로 교차검증을 수행해 최적화된 반복 수행횟수를 가질 수 있음
 - 지정된 반복횟수가 아니라 교차검증을 통해 평가 데이터세트의 평가 값이 최적화되면 반복을 중간에 멈출 수 있는 기능이 있음
- (5) 결측값을 내부적으로 자체 처리

Boosting : 3. XGBoost (eXtreme Gradient Boost)

4. XGBoost의 Hyper Parameter

4.1 General Parameters

일반적으로 실행 시 Thread 의 개수나 silent 모드 등의 선택을 위한 파라미터,

→ default 값을 바꾸는 일은 거의 없으나, Booster 에 대한 고민

Hyper Parameter	설명
Booster [default = 'gbtree']	-기본이 되는 Function ▪ gbtree(Gradient Boosting Tree) tree based model 또는 ▪ gblinear: Gradient Boosting Linear Function중 선택 ▪ dart: Dropouts meet Multiple Additive Regression Trees
Silent [default = 1]	- 출력 메시지를 나타내고 싶지 않을 경우 1로 설정
nthread	- CPU 실행 스레드 개수 조정 - Default는 전체 다 사용하는 것 - 멀티코어/스레드 CPU 시스템에서 일부CPU만 사용할 때 변경
num_feature	사용하는 Feature의 개수 자동적으로 데이터의 최대 Feature 갯수로 정의
validate_parameters	설정한 Input Parameter들이 실제로 사용되는 지에 대한 검증
verbosity	Message Printing의 단계 얼마나 자세하게 말해줄지에 대한 파라미터

Boosting : 3. XGBoost (eXtreme Gradient Boost)

4.2 주요 Booster Paramter : Overfitting 방지, 트리 최적화, 부스팅, regularization 관련 파라미터

파라미터 명 (파이썬 래퍼)	파라미터 명 (사이킷런 래퍼)	내용
Eta [default=0.3]	learning rate	Shrinkage라는 개념을 사용하여 Overfitting을 방지 - 범위: 0 ~ 1
num_boost_round	n_estimators	- 생성할 weak learner의 수
min_child_weight [default=1]	min_child_weight (1)	- GBM의 min_samples_leaf와 유사 - Split을 했는데 Child의 Node들의 instance weight가 min_child_weight를 넘기지 못하면 Split하지 않음. - 과적합 조절 용도 - 범위: 0 ~ ∞
Gamma [default=0]	min_split_loss (0)	- 리프노드의 추가분할을 결정할 최소손실 감소값 → 해당값보다 손실이 크게 감소할 때만 분리 진행 - 값이 클수록 과적합 감소효과 - 범위: 0 ~ ∞
max_depth [default=6]	max_depth (3)	- 트리 기반 알고리즘의 max_depth와 동일 - 0을 지정하면 깊이의 제한이 없음 - 너무 크면 과적합(통상 3~10정도 적용) - 범위: 0 ~ ∞
sub_sample [default=1]	subsample (1)	- GBM의 subsample과 동일 - 훈련 데이터에서 Sample Data를 뽑는 비율 만약 0.5으로 설정한다면, 매 tree를 구성할 때마다 데이터에서 0.5만큼 sampling_method 에 따라 추출한 후 훈련 - 일반적으로 0.5~1 사이의 값을 사용 - 범위: 0 ~ 1
colsample_bytree [default=1]	colsample_bytree (1)	- GBM의 max_features와 유사 - 트리 생성에 필요한 피처의 샘플링에 사용 - 피처가 많을 때 과적합 조절에 사용 - 범위: 0 ~ 1
lambda [default=1]	reg_lambda (1)	- L2 Regularization 적용 값 - 피처 개수가 많을 때 적용을 검토 - 클수록 과적합 감소 효과
alpha [default=0]	reg_alpha (0)	- L1 Regularization 적용 값 - 피처 개수가 많을 때 적용을 검토 - 클수록 과적합 감소 효과
scale_pos_weight [default=1]	scale_pos_weight (1)	- 불균형 데이터셋의 균형을 유지

3. XGBoost(eXtreme Gradient Boost)

4.3 학습 태스크 파라미터

학습 수행 시의 객체함수, 평가를 위한 지표 등을 설정하는 파라미터

파라미터 명	내용
objective	- 'reg:linear' : 회귀 - binary:logistic : 이진분류 - multi:softmax : 다중분류, 클래스 반환 - multi:softprob : 다중분류, 확률반환
eval_metric	- 검증에 사용되는 함수정의 - 회귀 분석인 경우 → 'rmse'를, 클래스 분류 문제인 경우 'error' - RMSE : Root Mean Squared Error - mae : mean absolute error - logloss : Negative log-likelihood - error : binary classification error rate - AUC: Area Under Curve

3. XGBoost(eXtreme Gradient Boost)

Overfitting 제어

- eta 값 (**learning rate**)을 낮춘다.(0.01 ~ 0.1) → eta 값을 낮추면 num_boost_round(n_estimator)를 반대로 높여주어야 함.
- max_depth 값을 낮춘다.
- min_child_weight 값을 높인다.
- gamma 값을 높인다.
- subsample과 colsample_bytree를 낮춘다.

3. XGBoost(eXtreme Gradient Boost)

금융 활용 사례

1. 신용 점수 예측

- XGBoost는 대출 신청자의 신용 점수를 예측하는 데 많이 사용
- 고객의 신용 기록, 금융 거래 패턴, 사회적 데이터 등을 기반으로 대출 상환 가능성을 평가해 신용 점수 산출
- 이 모델을 통해 금융기관은 리스크 관리와 대출 승인 여부를 보다 효율적으로 결정할 수 있다.

2. 거래 사기 탐지

- 시장에서 XGBoost는 주가, 채권 가격, 외환 등 자산의 미래 가격을 예측하는 데 사용됩니다. XGBoost 모델은 역 XGBoost는 금융 거래에서 사기성 활동을 탐지하는 데 탁월한 성능을 보인다.
- 금융기관은 거래 데이터, 사용자의 행동 패턴, 장비 정보 등을 XGBoost에 입력하여 실시간으로 이상 거래를 감지하고 사기 방지 조치 활용

3. 주식 및 자산 가격 예측

- 금융 사적 가격 데이터, 거시 경제 지표, 시장 변동성 등의 데이터를 학습하여 단기 또는 장기적 자산 가격 추세를 예측하는 데 효과적

4. 고객 이탈(유지) 예측

- 금융 기업은 고객 이탈을 방지하기 위해 고객의 행동 데이터를 기반으로 이탈 가능성을 예측
- XGBoost를 통해 고객 이탈 가능성이 높은 고객을 식별하고 이를 바탕으로 고객 유지 전략을 수립할 수 있다.

5. 상품 추천 시스템

- XGBoost는 고객의 거래 기록, 인구 통계 데이터 등을 활용해 개인화된 금융 상품을 추천하는 데도 활용
- 고객의 선호와 행동 패턴을 반영한 추천 시스템을 구축해 크로스셀링 및 업셀링 효과를 높이는 데 기여할 수 있다.

3. XGBoost(eXtreme Gradient Boost)

한계점

1. 해석 가능성의 한계

- XGBoost는 복잡한 의사결정 트리를 기반으로 한 모델이기 때문에, 결과의 해석이 어려울 수 있다.
- 금융 분야에서는 모델의 예측 근거를 명확하게 설명해야 할 때가 많은데, XGBoost는 특정 변수의 기여도는 파악할 수 있어도 각 예측의 이유를 상세히 설명하기 어렵다.

2. 고성능 하드웨어 필요

- XGBoost는 대규모 데이터와 고차원의 특징을 다룰 때 매우 강력하지만, 트리 앙상블 기법 특성상 메모리와 계산 자원이 많이 필요
- 특히 데이터가 클수록 학습 시간이 길어지고 고성능 하드웨어가 필요하여 비용이 증가할 수 있다.

3. 하이퍼파라미터 튜닝의 복잡성

- XGBoost는 다양한 하이퍼파라미터가 존재하고, 최적의 성능을 위해 이를 정교하게 조정해야 한다.
- 하이퍼파라미터 튜닝에 실패할 경우 과적합(overfitting)이나 성능 저하가 발생할 수 있다. 이 과정은 많은 시간과 경험을 요구하며, 최적의 성능을 끌어내기 위해 자동화된 튜닝 방법이나 고도의 경험이 필요하다.

4. 과적합(Overfitting) 문제

- XGBoost는 데이터의 작은 변화에 민감하여 과적합이 발생할 가능성이 있다.
- 특히 소규모 데이터셋이나 노이즈가 많은 데이터셋에서는 성능이 저하될 수 있다.
- 금융 데이터는 노이즈가 많고 불안정한 경우가 많아 모델이 불필요한 패턴을 학습하는 경우가 발생할 수 있다.

5. 데이터 사전 처리의 중요성

- XGBoost는 입력 데이터의 품질에 크게 의존한다.
- 결측치 처리, 이상치 제거, 특징 엔지니어링 등을 사전에 제대로 수행하지 않으면 모델의 성능이 저하될 수 있다.
- 또한 XGBoost는 수치형 데이터에 더 잘 작동하므로, 범주형 데이터는 별도의 인코딩이 필요하다.

6. 동적인 데이터에 대한 적응성 부족

- 금융 데이터는 매우 동적인 특성이 있어, 시간이 지나면서 데이터의 분포가 변화하거나 새로운 패턴이 나타날 수 있다.
- 하지만 XGBoost는 정적 데이터에 최적화된 모델이므로 실시간 데이터 또는 주기적으로 변화하는 데이터에서 성능이 저하될 수 있으며, 주기적인 모델 재학습이 필요하다.

Boosting : 4. LGBM (LightGBM)

Boosting 알고리즘 문제점

- 부스팅 계열의 대부분 computational cost는 각 단계에서 weak learner인 best tree를 찾는데 쓰인다. 따라서 백만 개의 데이터를 XGBoost로 iteration=1000을 학습시킨 경우, 각 단계에서 tree를 fitting 시키기 위해 백만개 데이터를 전부 scan 해야 한다.
- 해당 과정을 1000번 반복하니 computational cost가 너무 많이 들고 시간이 오래 걸린다.

LightGBM : 학습 속도와 정확도 개선해 보자!!!

- Microsoft에서 개발한 머신 러닝
- Gradient Boosting Tree 기반 학습 알고리즘

(속도 개선)

다른 알고리즘의 높은 cost 문제 : 기존의 Gradient boosting 모델은 사전 정렬 방식(Pre-sort-based)을 사용하여 최적 분할점을 찾아내 의사결정나무를 학습한다. 이는 간단하지만 최적화에 어려운 점이 있으며 학습속도와 메모리 사용에 불리하다

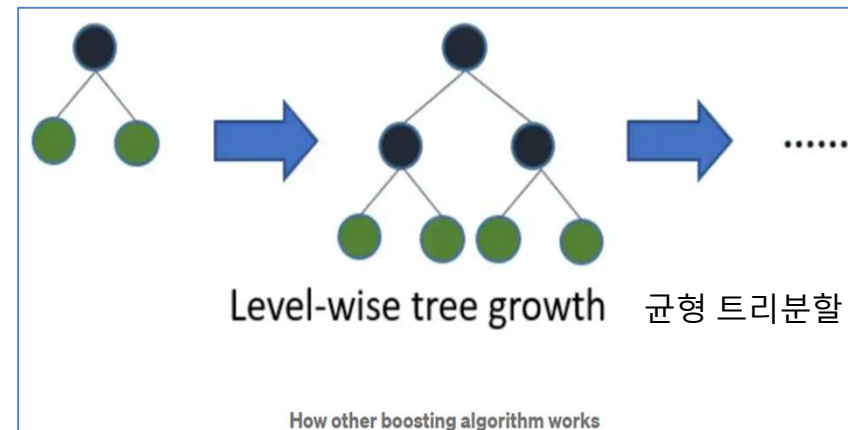
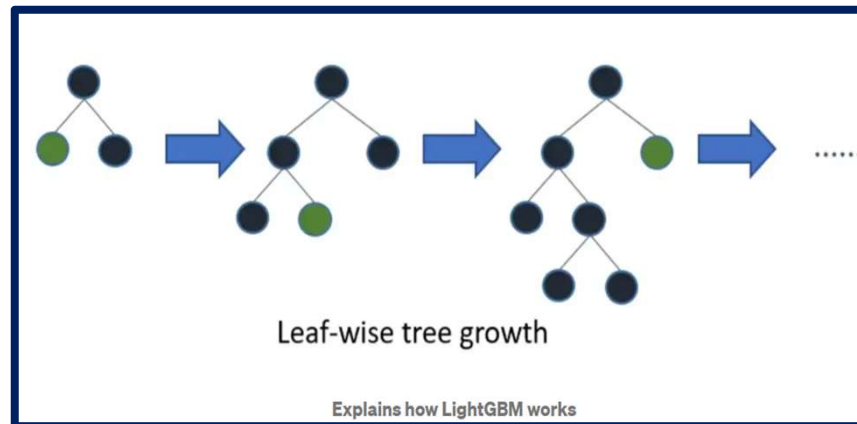
- 학습속도를 높이기 위해서 히스토그램 방식(Histogram-based), GOSS(Gradient-based One-Side Sampling), EFB(Exclusive Feature Bundle) 등의 방법을 사용→ 속도가 빠르다. (Light)
- 고도로 최적화된 히스토그램 기반 결정 트리 학습 알고리즘을 구현하여 효율성과 메모리 소비 모두에서 큰 이점

(분할방식 변경)

- LightGBM은 XGBoost의 병렬화, 정규화 등의 장점은 계승하면서 단점을 보완하고자 하였다. → LightGBM의 가장 큰 특징으로 Leaf wise 방법을 사용하는 것이다.: Tree 확장 방식 개선 : Tree가 수직적으로 확장 , Leaf-wise (리프 중심 트리 분할 방식)

Boosting : 4. LGBM (LightGBM)

- Light GBM : Tree가 **수직적**으로 확장 , **Leaf-wise** (리프 중심 트리 분할 방식)
 → 트리의 균형을 맞추지 않고, 최대 손실 값(max delta loss)을 가지는 leaf node를 지속적으로 분할하면서 트리의 깊이가 깊어지고 비대칭적인 규칙 트리가 생성
 → 깊이가 깊어지기는 하지만 최대 손실 값을 가지는 리프 노드를 지속적으로 분할해 생성된 규칙 트리는 학습을 반복하면 결국 균형 트리 분할 방식보다 **예측 오류 손실을 최소화**
- 다른 알고리즘 : Tree가 **수평적**으로 확장 , **Level-wise** 방식의 모델 생성 (**균형 트리 분할 방법**)
 → 동일한 leaf를 확장할 때, leaf-wise 알고리즘은 level-wise 알고리즘보다 더 많은 loss를 감소 가능



<https://medium.com/@pushkarmandot/https-medium-com-pushkarmandot-what-is-lightgbm-how-to-implement-it-how-to-fine-tune-the-parameters-60347819b7fc>

Boosting : 4. LGBM (LightGBM)

LightGBM 학습 과정

1. 초기 모델 생성

LightGBM도 다른 Gradient Boosting 알고리즘과 마찬가지로 초기 예측 모델을 생성합니다. 보통은 평균값을 예측하는 단순한 모델로 시작합니다.

2. 잔차(residuals) 계산

현재 모델이 각 데이터 포인트에 대해 예측한 값과 실제 값의 차이인 잔차를 계산합니다. 이 잔차는 다음 학습에서 줄여야 할 오차입니다.

3. 새로운 트리 학습 (Leaf-wise Growth)

LightGBM의 가장 큰 특징 중 하나는 Leaf-wise 트리 성장 방식이다. 기존의 Depth-wise 방식과는 달리, 가장 큰 손실 감소를 가져오는 리프(leaf)를 먼저 확장한다.

Leaf-wise 방식은 Depth-wise 방식에 비해 과적합이 될 가능성이 있으므로, max_depth 파라미터를 통해 깊이를 제한하는 것이 일반적이다.

4. Histogram-based Algorithm 사용

연속형 변수는 히스토그램으로 변환하여 계산 효율을 높인다. 이를 통해 메모리 사용량을 줄이고, 빠르게 최적의 분할을 찾을 수 있다.

히스토그램 기반의 방법은 연속형 값을 일정한 구간으로 나눠서 처리하므로, 계산 속도가 빨라지고 메모리 사용이 효율적이다.

5. Feature Bundling

LightGBM은 희소한 데이터나 높은 차원의 데이터에서 Feature Bundling 기법을 사용하여 비슷한 패턴을 가진 피쳐들을 하나로 묶어준다. 이를 통해 피쳐 개수를 줄이고 연산량을 줄일 수 있다.

6. Gradient-based One-Side Sampling (GOSS)

LightGBM은 Gradient-based One-Side Sampling (GOSS) 방식을 사용하여, 중요한 정보가 포함된 데이터만 샘플링하여 학습한다. 이는 데이터의 전체 크기를 줄여서 계산 속도를 높이는 방법으로, gradient가 큰 데이터 포인트를 더 많이 샘플링하여 학습에 활용한다.

7. 정규화와 학습률 적용

각 트리의 예측값에 학습률을 곱하여 모델을 업데이트한다.

8. 반복

위 과정을 반복하여 각 트리가 잔차를 줄이는 방향으로 순차적으로 학습한다. 충분히 많은 트리가 학습되거나 조기 종료 조건이 만족되면 학습을 중단한다.

9. 최종 예측 생성

Boosting : 4. LGBM (LightGBM)

Why Light GBM is gaining extreme popularity?

- 1) 속도가 빠르다.→ 'Light'
- 2) 대용량 데이터를 처리 가능→ 실행하는 데 더 적은 메모리를 사용
- 3) 결과의 정확성에 중점을 두기 때문에
- 4) LGBM은 또한 GPU 학습을 지원

Can we use Light GBM everywhere?

- 작은 데이터 세트에 LGBM을 사용하는 것은 권장하지 않음
 - Light GBM은 과적합에 민감하며 작은 데이터를 쉽게 과적합 할 수 있다.
- ROW 수에 대한 임계값은 없지만 (ㄱㅇㅈ 으로) ROW가 10,000개 이상인 데이터에만 사용하는 것이 좋다.

<https://medium.com/@pushkarmandot/https-medium-com-pushkarmandot-what-is-lightgbm-how-to-implement-it-how-to-fine-tune-the-parameters-60347819b7fc>

Microsoft LightGBM Documentation

* Parameter Tuning: LightGBM

모델 정확도 향상

Training Set과 Validation Set을 통해 각 모델마다 ROC AUC 및 PR AUC가 안정적으로 유지될 수 있도록 진행

- max_depth : Tree 최대 깊이, max_depth 값을 설정 가능
- num_leaves : 전체 트리의 리프 수, Tree 모델의 복잡성을 컨트롤하는 주요 파라미터.
- Min_data_in_leaf : 큰 값으로 세팅하는 것은 Tree가 너무 깊게 확장되는 것을 막을 수 있지만 under-fitting 언더피팅이 발생할 수도 있다.
- **더 빠른 속도를 위하여**
- bagging_fraction과 bagging_freq 을 설정하여 bagging 을 적용
- feature_fraction 설정 : feature sub-sampling
- **num_boost_round**: 부스팅 반복 횟수, 일반적으로 100 이상
- 작은 max_bin 값을 사용
- save_binary 값을 통해 다가오는 학습에서 데이터 로딩 속도 단축
- parallel learning 병렬 학습 적용

What is LightGBM, How to implement it? How to fine tune the parameters?

- **더 나은 정확도를 위해**
- 큰 max_bin 값을 사용 : 특성 값이 있는 최대 빈 수
- 작은 learning_rate 값을 큰 num_iterations 값과 함께 사용
- → 0.1, 0.001, 0.003
- 큰 num_leaves 값을 사용 → 과적합 유발 가능성
- **과적합을 해결하기 위해 :**
- 작은 max_bin 값을 사용
- 작은 num_leaves 값을 사용
- min_data_in_leaf : leaf 가 가질 수 있는 최소값. 기본값 20
- min_sum_hessian_in_leaf 파라미터를 사용
- bagging_fraction : 각 반복에 사용할 데이터 비율을 지정
- bagging_freq 을 사용하여 bagging 적용
- feature_fraction을 세팅하여 feature sub-sampling
- lambda_l1, lambda_l2 & min_gain_to_split 파라미터를 이용해 regularization (정규화) 를 적용

* Parameter Tuning: LightGBM

LightGBM(LGBM) 금융시장 활용

1. 신용 리스크 평가 및 신용 점수 예측
2. 사기 탐지(Fraud Detection)
3. 주가 및 자산 가격 예측
4. 고객 세분화 및 맞춤형 상품 추천
5. 고객 이탈 예측
6. 시장 리스크 분석 및 시나리오 시뮬레이션

LGBM은 다양한 거시 경제 변수와 금융 시장 데이터를 입력으로 받아 시장 리스크를 계산하는 데 적합하며, 여러 시나리오에 대한 시뮬레이션을 통해 리스크 관리에 기여할 수 있다.

7. 포트폴리오 최적화

주가 변동성, 상관 관계, 수익률 등의 데이터를 바탕으로 리스크가 낮으면서도 수익률이 높은 자산 구성을 찾아내는 데 기여 → LGBM의 빠른 학습 속도 덕분에 대규모 금융 데이터를 효율적으로 다루며, 포트폴리오 관리와 리밸런싱에도 도움을 준다.

8. 대출 연체 가능성 예측

대출자의 소득, 채무 이력, 거래 패턴 등을 분석하여 연체 가능성이 높은 대출을 사전에 식별함으로써 금융 기관의 리스크를 줄일 수 있다.

LightGBM(LGBM) 한계점

1. 과적합(Overfitting) 문제
2. 해석 가능성 부족
3. 하이퍼파라미터 튜닝의 복잡성
4. 범주형 데이터 처리의 한계
5. 데이터 전처리에 민감
6. 메모리 사용량 증가
7. 시간 변화에 대한 민감성 부족

LGBM은 기본적으로 정적 데이터에 맞춰져 있으며, 시계열 데이터처럼 시간에 따라 변하는 데이터 패턴을 잘 다루지 못할 수 있다.

8. 불균형 데이터에서의 성능 제한

5.CatBoost : 범주형(Categorical Boosting)

CatBoost is based on gradient boosting

- Yandex에서 개발한 gradient boosting 라이브러리로, 고유한 몇 가지 기능과 최적화를 통해 gradient boosting 모델의 성능을 높이고 학습 속도를 개선하는 데 중점을 둔 라이브러리
- **범주형(Categorical) 변수를 처리하는 데 유용한 알고리즘**

CatBoost의 주요 작동 원리

1. Gradient Boosting의 기본 개념

- 각 단계에서 이전 모델의 오차(잔차)를 줄이기 위해 새로운 모델이 추가.
- CatBoost도 이러한 일반적인 boosting 과정을 따르며, 각 트리가 이전 트리의 에러를 줄이기 위해 학습한다.

2. Categorical Feature Handling

- 평균 타깃 인코딩 (Mean Target Encoding): 범주형 변수를 처리하기 위해 각 범주에 대해 타깃 값의 평균을 사용한다.
- Ordered Target Statistics: CatBoost는 각 데이터를 랜덤하게 섞은 후, 순서대로 타깃 평균을 계산한다. 이 방식은 학습 중에 데이터 누수를 방지하고, 새로운 데이터에 대해 안정적인 예측을 제공한다.

3. Ordered Boosting

- CatBoost는 전통적인 gradient boosting의 트리 구축 순서 대신 Ordered Boosting이라는 방식으로 오버피팅을 방지한다.
- 기존 방식의 gradient boosting은 모델이 이전 단계의 오차를 참조하며 업데이트하기 때문에 학습 데이터에 과적합될 가능성이 있다.
- Ordered Boosting은 학습 시 데이터를 순차적으로 샘플링하며, 새로운 트리를 학습할 때 이전 모델의 예측 결과를 사용할 때, 학습에 사용되지 않은 데이터의 잔차를 기반으로 계산한다.

4. 성능 최적화

- 대칭 트리: CatBoost는 트리 구조를 대칭으로 유지하여, 트리 구조의 일관성과 계산 효율성을 높인다. 이로 인해 학습 속도가 빨라지고, 예측 시 일관성이 증가한다.
- CPU 및 GPU 병렬 처리: CatBoost는 CPU와 GPU를 모두 활용하여 빠른 속도로 학습이 가능하도록 최적화되었다.
- L2 정규화 및 다른 파라미터 튜닝: 과적합을 방지하기 위해 다양한 정규화 기법을 적용할 수 있다.

5.CatBoost : 범주형(Categorical Boosting)

CatBoost의 작동과정

1. 데이터 전처리 및 인코딩 단계

CatBoost는 범주형 데이터를 Ordered Target Statistics로 처리하므로, 각 범주형 변수에 대해 학습 데이터를 무작위로 섞고, 특정 데이터 포인트보다 앞에 있는 데이터만 참조하여 평균을 계산한다.
→데이터 누수를 방지한다.

2. Gradient Boosting 순차적 학습 과정

- CatBoost는 기본 gradient boosting과 유사하게, 초기 모델을 만들고 순차적으로 약한 학습기를 추가한다.
- 각 학습기마다 잔차(residuals)를 계산하고, 이를 줄이는 방향으로 새로운 트리를 학습한다.
- 일반적인 Gradient Boosting과의 차이점은 Ordered Boosting을 사용한다는 점이다. 이 과정에서는 순차적으로 데이터를 추가하면서 모델이 이전 단계의 오차를 참고하여 과적합을 방지한다.

3. 대칭 트리 구조

- CatBoost는 대칭 트리를 사용하여 각 레벨에서 동일한 조건으로 분기되도록 한다. 이를 통해 학습 속도와 예측 속도가 개선된다..
- 이러한 트리 구조는 시각화에서 대칭적으로 나타내며, 트리의 각 분기점에서는 새로운 노드로 잔차를 줄이는 분할 조건을 추가한다.

4. 최종 예측 합산

모든 트리가 학습되면 각 트리의 예측값을 합산하여 최종 예측값을 생성한다. 이는 일반적인 Boosting 방법과 동일하며, 여러 트리의 결과를 가중 평균 또는 합산하여 최종 예측을 만든다.

5. CatBoost

CatBoost 의 특징

1. Level-wise 방식의 모델 생성 Tree : Tree가 수평적으로 확장

2. Ordered Boosting

Catboost 는 기존의 부스팅 과정과 전체적인 양상은 비슷하되, 조금 다르다.

기존의 부스팅 모델이 일괄적으로 모든 훈련 데이터를 대상으로 잔차 계산을 하는 것에 반하여,

Catboost 는 일부만 가지고 잔차 계산, 이걸로 모델을 만들고, 후에 데이터의 잔차는 이 모델로 예측한 값을 사용

→ 순서에 따라 모델을 만들고 예측하는 방식

3. Random Permutation

Ordered Boosting 을 할 때, 매번 같은 순서대로 잔차를 예측하는 모델을 만들 가능성 있으므로 Catboost 는 이점을 감안해서 데이터를 shuffling 하여 뽑아낸다. → 이때 그 중 일부만 가져오게 할 수 있다.

4 . Categorical Feature Combinations

Catboost는 information gain 이 동일한 두 feature 를 하나의 feature 로 →결과적으로, 데이터 전처리에 있어 **feature selection 부담 감소**

→ 두 범주형 변수를 합쳐주는 것인데, label값을 기준으로 split을 할 때 두 범주형 변수가 서로 대체 가능하다면 이 둘을 합쳐준다

5. Optimized Parameter tuning

Catboost 는 기본 파라미터가 기본적으로 최적화가 내부적으로 잘 되어있어서 파라미터 튜닝 필요성이 없음.

5. CatBoost

CatBoost 금융시장 활용

CatBoost는 범주형 데이터 처리가 용이하고 과적합 방지를 위한 다양한 기능이 내장되어 있어 금융 분야에서의 활용도가 높다.

1. 신용 리스크 평가 및 신용 점수 예측
2. 사기 탐지(Fraud Detection)
3. 고객 세분화 및 맞춤형 금융 상품 추천
4. 주식 및 자산 가격 예측
5. 고객 이탈 예측
6. 대출 연체 가능성 예측
7. 시장 리스크 분석 및 시나리오 시뮬레이션

8. 포트폴리오 최적화 및 자산 배분

포트폴리오를 최적화하고 자산을 배분하는 데 CatBoost를 사용할 수 있다. CatBoost는 투자 포트폴리오의 다양한 자산 클래스 간의 상관 관계와 변동성을 학습하여, 리스크를 최소화하면서 수익을 극대화하는 최적의 자산 배분 전략을 수립하는 데 기여한다.

9. 신규 고객 평가 및 신용 한도 설정

신용 정보가 부족한 신규 고객의 경우, 유사한 고객 그룹의 데이터를 기반으로 신용 점수를 예측하고 적정 신용 한도를 설정할 수 있다.

범주형 데이터 처리에 강한 CatBoost 덕분에, 기존 고객과의 비교를 통해 정확한 신용 평가가 가능하다.

5. CatBoost

CatBoost 한계점

1. 높은 계산 비용

대규모 데이터셋을 학습할 때 학습 시간이 오래 걸리고, 많은 메모리를 필요로 할 수 있어 리소스가 제한된 환경에서는 사용이 어려울 수 있다.

2. 복잡한 하이퍼파라미터 튜닝

CatBoost의 자동 튜닝 기능이 있지만, 복잡한 문제에서는 추가적인 튜닝이 필요할 수 있다.

3. 해석 가능성 부족

4. 범주형 데이터의 특성 처리 한계

매우 고차원의 범주형 데이터(즉, 유니크한 값이 많은 경우)에서는 성능 저하가 발생할 수 있다. 이런 경우 별도의 데이터 전처리가 필요할 수 있다.

5. 불균형 데이터 처리의 한계

이 경우 클래스 가중치 조정이나 오버샘플링/언더샘플링 같은 기법을 사용해야 한다.

6. 데이터 전처리 필요성

CatBoost는 결측값을 기본적으로 처리할 수 있지만, 이상치나 데이터 스케일링과 같은 전처리는 여전히 작업이 필요하다.

7. 모델 업데이트가 어려움

새로운 데이터를 반영하려면 전체 데이터를 다시 학습해야 하는 경우가 많아, 실시간 데이터가 계속 유입되는 환경에서는 불편할 수 있다. 이를 해결하려면 주기적으로 모델을 재학습해야 하며, 이는 계산 비용을 증가시킨다.

8. 다양한 데이터 타입에 대한 제한

CatBoost는 수치형 및 범주형 데이터에 대해 강점을 보이지만, 시계열 데이터나 복잡한 텍스트 데이터를 처리할 때는 한계가 있다.

시계열 데이터의 경우, 시간 종속성을 반영한 추가 전처리나 특성 공학이 필요하며, 자연어 처리와 같은 비정형 데이터에서는 별도의 피처 엔지니어링과 전처리 과정이 요구된다.

9. 고성능 모델과의 비교 시 성능 격차

CatBoost는 다른 Gradient Boosting 모델(XGBoost, LightGBM 등)과 비교해 학습 속도와 정확도 면에서 강점을 보이지만, 딥러닝과 같은 최신 모델에 비해서는 성능이 떨어질 수 있다. 특히 대규모 데이터셋에서 복잡한 패턴을 학습하는 경우, 신경망 기반 모델이 더 나은 성능을 보일 수 있다.

6. AUCBoost

Area Under Curve-based Boosting 기법

- AUCBoost는 부스팅(Boosting) 기법을 이용해 **ROC 곡선 아래 면적(AUC; Area Under the ROC Curve)** 을 직접 최적화하려는 알고리즘
- 특히, **이진분류 문제에서 모델의 성능을 평가하는 데 사용되는** AUC (Area Under the ROC Curve) 지표를 최대화하는 것을 목표로 한다.
- ROC (Receiver Operating Characteristic) 곡선 아래의 면적인 AUC는 모델이 양성 클래스와 음성 클래스를 얼마나 잘 구분하는지를 나타내는 지표로, 값이 1에 가까울수록 모델의 성능이 뛰어난 것으로 평가

AUCBoost의 원리

- AUCBoost는 부스팅 기법의 일종 : 여러 약한 학습기(Weak Learner)를 순차적으로 학습시키면서 각 학습기의 예측을 결합하여 최종적으로 강한 학습기(Strong Learner)를 생성
- **AUC 최적화**
: AUCBoost는 각 단계에서 AUC를 최대화하는 방향으로 약한 학습기를 조정한다. → 이는 전통적인 부스팅 방법과 다른 점으로, 일반적인 **오류율 최소화**가 아닌 **AUC 최적화에 초점을 맞춘다.**
- **가중치 조정**
: AUCBoost는 잘못 분류된 사례뿐만 아니라, 분류 결정 경계에 가까운 사례들에 대해서도 가중치를 조정한다.
→ 이는 모델이 더 정확하게 양성 클래스와 음성 클래스를 구분할 수 있도록 돕는다.
- **순차적 학습**
: 각 단계에서 학습된 약한 학습기는 전체 모델의 성능을 개선하는 데 기여한다.
: AUCBoost는 이전 학습기들의 성능을 바탕으로 다음 학습기가 더 중요한 사례에 집중할 수 있도록 한다.

6. AUCBoost

AUCBoost의 핵심 아이디어

1. 쌍(pairwise) 기반 손실 함수

•AUC는 샘플 쌍에 대한 순위 정확도로 정의되므로, AUCBoost는 양성-음성 샘플 쌍 (x_i^+, x_j^-) 에 대한 손실을 최소화한다.

•Hinge 손실을 쓰면: $\ell(f(x_i^+), f(x_j^-)) = \max(0, 1 - (f(x_i^+) - f(x_j^-)))$

2. 부스팅 프레임워크

기존 부스팅처럼 단계별로 약분류기(weak learner)를 추가하며, 각 단계에서 현재 모델이 잘못 순위를 매긴(즉 손실이 큰) 샘플 쌍에 더 큰 가중치를 준다.

가중치 업데이트: $w_{ij}^{(t+1)} \propto w_{ij}^{(t)} \exp(\alpha_t \partial \ell_{ij}^{(t)})$

여기서 $\partial \ell_{ij}^{(t)}$: 현재 예측 차이에 대한 손실의 기울기(gradient)

3. 최종 예측

•TTT단계의 약분류기 $h_t(x)$ 와 가중치 α_t 를 합산하여 최종 스코어를 계산 $F(x) = \sum_{t=1}^T \alpha_t h_t(x)$

AUCBoost의 장.단점

구분	장점	단점
성능 최적화	AUC를 직접 최적화하므로 불균형 데이터에서도 높은 순위 성능	샘플 쌍 개수가 $O(n^2)$ 로 늘어나 메모리·계산 부담
구현 복잡도	부스팅 프레임워크를 재활용 가능	가중치 업데이트·쌍 관리 로직이 상대적으로 복잡

*Hinge Loss

1. 정의 및 수식

Hinge Loss는 주로 서포트 벡터 머신(SVM)과 같은 대칭(Margin-based) 분류기에서 사용하는 손실 함수로, **분류의 Margin을 최대화**하도록 설계되어 있다.

이진 분류에서 레이블 $y \in \{+1, -1\}$ 이고, 모델의 예측 점수를 $f(x)$ 할 때, 힙지 손실 (Hinge Loss)은 다음과 같다:

$$\ell_{\text{hinge}}(f(x), y) = \max(0, 1 - y f(x))$$

- $yf(x)$: 올바른 방향으로 얼마나 'margin'를 두고 분류했는지를 나타내는 값
- $1 - y f(x) > 0$ 인 경우에만 손실이 발생하며, 그 값이 곧 손실의 크기
- $\max(0, \cdot)$ 로 음수인 경우(Margin이 충분한 경우) 손실을 0으로 처리

2. 해석

- $yf(x) \geq 1$ 이면 손실이 0 : "충분히 올바른 분류"
- $0 < yf(x) < 1$ 이면 선형으로 손실 발생 : "분류는 맞췄지만 — Margin이 부족"
- $yf(x) \leq 0$ 이면 손실이 $1 - yf(x)$ 만큼 크게 발생 → "잘못 분류"

*Hinge Loss

3. 장·단점

구 분	장점	단점
강건성	큰 잘못 분류한 샘플이 과도한 기울기를 발생시키지 않음	손실 구간이 선형이어서 부드럽지 않아 최적화 시 불연속점 존재
Margin 확보	마진을 최대화하여 결정 경계의 일반화 성능 향상	Margin 을 위해 '슬랙변수(slack variable) 을 추가해야 하므로 문제 복잡도 상승
계산 효율	간단한 기울기(상수) 계산	0-1 손실에 비해 다소 복잡하지만, 로지스틱 손실보다 단순

4. 최적화 시 기울기

$$\frac{\partial \ell_{\text{hinge}}}{\partial f(x)} = \begin{cases} -y, & 1 - y f(x) > 0 \\ 0, & 1 - y f(x) \leq 0 \end{cases}$$

- Margin 부족(손실 발생) 구간에서는 상수 $-y$
- Margin 확보 구간에서는 0

힌지 손실은 “맞게 분류될 뿐 아니라, 충분한 margin 까지 확보”하도록 유도하는 손실 함수이다. SVM을 비롯한 마진 기반 학습에서 결정 경계의 일반화 성능을 높이는 데 핵심적인 역할을 한다.

* AUROC기반의 부도예측 앙상블 모형

AUROC기반의 부도예측 앙상블 모형, 2021. 윤 우 섭*** • 김 명 중**

- **AdaBoost 알고리즘**은 오분류된 표본에 강화된 학습기회를 제공하는 장점이 있지만, **산술평균**에 기초하여 기저 분류자의 결합 가중치를 결정 한다는 점에서 범주 불균형 문제의 해결에 한계점
- AUCBoost 알고리즘
 - 범주 불균형의 성과지표로 활용되는 **AUROC를 활용하여 AdaBoost 앙상블 학습을 수정한 부스팅 기법.**
 - (학습목표) 이차계획법을 활용하여 최종 분류자의 AUROC를 최대화하는 것

Data set

- 1:1 : AUCBoost 우수 (10% 유의수준)
- 2:1 : AUCBoost 우수 (1% 유의수준)
- 4:1 : AUCBoost 우수 (1% 유의수준)
- 10:1 : AUCBoost 우수 (1% 유의수준)
- 20:1 : AUCBoost 우수 (1% 유의수준)
- (이하 논문 참조)

금융 범주 불균형 최적화를 위한 AUC 부스팅 학습 2023

벤치마킹 모형으로 부스팅 학습 기법인 AdaBoost, GBM, XGBoost 알고리즘을 채택하였으며 30회 교차 검증을 통해 실험을 진행하여 그 결과는 다음과 같다.

첫째, 다수 범주에 초점을 맞추어 불균형 문제에서 소수 범주의 민감도를 고려하지 못하는 기존의 알고리즘과 달리 AUCBoost는 다수 범주의 특이도와 소수 범주의 민감도를 동시에 고려하는 균형 잡힌 학습을 통해 범주 불균형 문제에 유의한 성과를 보여줌을 확인하였다.

둘째, 기존 부스팅 모형과 비교하여 AUCBoost는 AUC 및 GM 등의 성과지표 에서도 향상된 성과를 보여주고 있으며, 부스팅 학습의 성과지표에 관한 직접적인 최적화 적용 가능성을 시사한다.

* AUROC기반의 부도예측 앙상블 모형

AUC 최적화 학습 알고리즘의 기업 부실 예측 연구 2023 김명종 ,안재현

AUCBoost는 다변량 정규분포 가정을 도입하여 성과지표에 대한 직접적인 최적화가 가능하도록 수정한 알고리즘으로써 한국, 폴란드, 러시아 기업 부실 데이터에 대해 성과 개선 효과를 확인했다.

데이터	입력 변수의 수	소수 범주	다수 범주	불균형 비율
한국 기업 부도	7	500	10,000	1:20
폴란드 기업 부도	64	410	5,500	1:13
러시아 기업 부도	55	456	2,001	1:4

특이도(SPE), 민감도(SEN), 정확도(ACC), AUC,GM의 평균값을 성과지표로 산출

Panel A. 한국 기업 부도 예측 (IR 1:20)

Performance Metrics	Training Set				Test Set			
	AdaBoost	GBM	XGB	AUCBoost	AdaBoost	GBM	XGB	AUCBoost
SPE	0.99	0.99	0.99	0.80	0.99	0.99	0.99	0.78
SEN	0.05	0.07	0.07	0.77	0.04	0.05	0.06	0.75
ACC	0.95	0.96	0.96	0.80	0.95	0.95	0.95	0.78
AUC	0.52	0.53	0.53	0.78	0.52	0.53	0.53	0.76
GM	0.22	0.26	0.26	0.78	0.18	0.21	0.22	0.76

Panel B. 폴란드 기업 부도 예측 (IR 1:13)

Performance Metrics	Training Set				Test Set			
	AdaBoost	GBM	XGB	AUCBoost	AdaBoost	GBM	XGB	AUCBoost
SPE	0.99	0.99	0.99	0.89	0.99	0.99	0.99	0.84
SEN	0.48	0.32	0.44	0.88	0.42	0.29	0.39	0.82
ACC	0.96	0.95	0.96	0.89	0.95	0.95	0.95	0.83
AUC	0.73	0.66	0.72	0.89	0.70	0.64	0.69	0.83
GM	0.69	0.56	0.66	0.89	0.64	0.53	0.62	0.83

Panel C. 러시아 기업 부도 예측 (IR 1:4)

Performance Metrics	Training Set				Test Set			
	AdaBoost	GBM	XGB	AUCBoost	AdaBoost	GBM	XGB	AUCBoost
SPE	0.96	0.99	0.98	0.81	0.95	0.98	0.97	0.75
SEN	0.49	0.33	0.46	0.81	0.43	0.30	0.40	0.73
ACC	0.88	0.86	0.88	0.81	0.85	0.85	0.86	0.75
AUC	0.73	0.66	0.72	0.81	0.69	0.64	0.68	0.74
GM	0.69	0.57	0.67	0.81	0.63	0.54	0.62	0.74

* Boosting 방법 비교

알고리즘	목적 함수	기본 손실 함수 / 평가 지표	샘플 가중치 업데이트	주요 장점	주요 단점
AUCBoost	AUC 최적화	Pairwise 순위 손실 (Hinge Loss, 로지스틱 등)	양성-음성 쌍별 가중치 재조정	불균형 데이터·순위 성능에 최적화	쌍 연산 비용·메모리 과다
AdaBoost	분류 정확도 개선	지수 손실 $\exp(-yF(x))$	잘못 분류된 샘플에 지수 가중치 부여	간단·이해하기 쉬움, 잡음에 대해 어느 정도 강건	이상치(outlier)에 민감, 순위 성능 직접 최적화 못함
GBM	임의 미분 가능 손실	로지스틱, 제곱 오차, 최소 절대 편차 등	잔차(residual)에 기반한 샘플 가중치	다양한 손실 함수 지원, 회귀·분류 모두 활용 가능	하이퍼파라미터 조정 복잡, 느린 학습 속도
XGBoost	GBM + 정규화	GBM 손실 + L1/L2 규제	2차 테일러 전개 기반 최적화	과적합 방지 위한 규제, 병렬·분산 처리, 속도·성능 우수	파라미터 많아 튜닝 복잡, 순위 목적 직접 미반영
LightGBM	GBM 계열	GBM 손실 함수 그대로	Gradient 기반, leaf-wise 성장	대규모 데이터셋에 빠름, 메모리 절약	과적합 위험(leaf-wise), 작은 데이터에서는 오버헤드 클 수 있음
CatBoost	GBM + 범주형 특성 처리	GBM 손실 + 순위 손실 옵션 제공 가능	Ordered boosting, 그룹별 처리	범주형 변수 자동 처리, 순위 학습 모드 지원	내부 로직 복잡, 학습 속도는 XGBoost·LightGBM보다 느릴 수 있음

* Boosting 방법 비교

언제 어떤 부스팅을 쓸까?

- 순위(Order) 성능이 최우선이고 양성/음성 불균형이 크다면 → **AUCBoost**
- 빠른 프로토타이핑 및 이해 용이성이 중요하다면 → **AdaBoost**
- 다양한 손실 함수, 특히 회귀도 필요하다면 → **GBM**
- 대용량 데이터에 속도·성능 균형을 원한다면 → **XGBoost** 또는 **LightGBM**
- 범주형 변수가 많고, 자동 처리 및 순위 학습이 필요하다면 → **CatBoost**

7. Stacking Ensemble → meta modeling

Stacking Algorithm

- 1) 복수의 개별 알고리즘의 예측 결과값(y)이 다시 **최종모델(Meta Model)**의 학습data set으로 만들어 사용
→ 최종 모델에서는 y 예측값과 실제값이 독립변수와 종속변수로 작용
- 2) 별도 머신러닝 알고리즘으로 최종 학습을 수행 후
- 3) test data를 기반으로 최종 예측을 수행하는 방식
meta modeling : 개별 모델의 예측된 dataset을 기반으로 학습하고 예측하는 방식

메타 모델 데이터 이해 사례 : 개별 알고리즘 (3개)
: m개의 row, n개의 column (feature)을 가진 dataset
→ 각 개별 모델을 학습을 시킨 뒤 예측을 수행하면 각각 m개의 row를 가진 1개의 label 값이 도출
→ 각 개별 모델에서 도출된 label들을 stacking하면 m개의 row와 3개의 column을 가진 새로운 dataset으로 만들어 최종 모델에 적용하여 예측을 수행하는 것이 스택킹 앙상블 모델

최종 모델(meta modeling)의 (train, test)데이터 모습

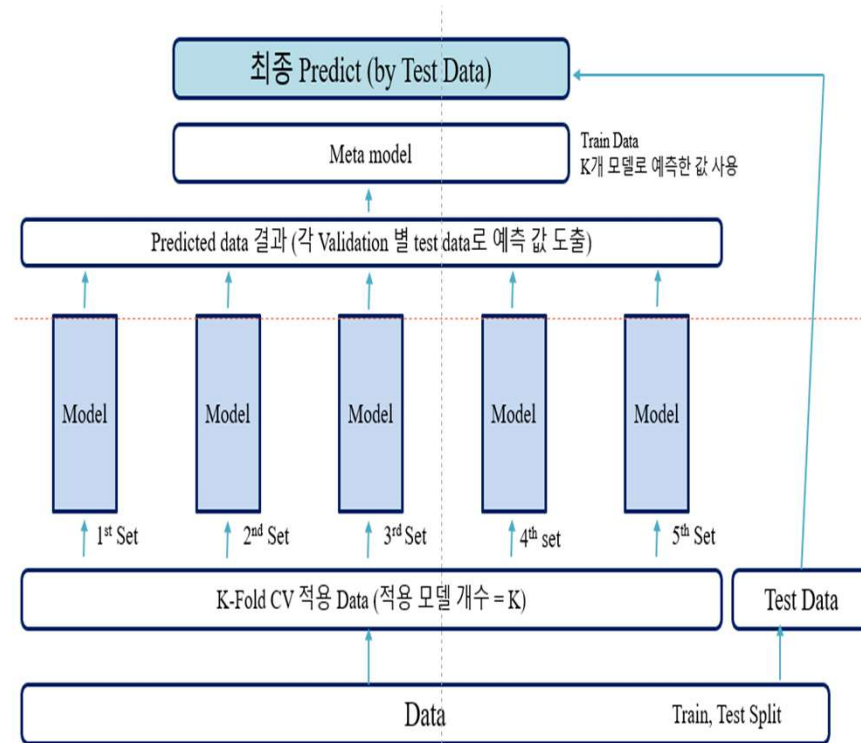
최종 모델(meta modeling)의 x-train역할

최종 모델(meta modeling)의 y-train역할

	1 모델링 예측값	2 모델링 예측값	3 모델링 예측값	
train	y- train 예측값			y- train 실제값
test	y- test 예측값			y- test 실제값

y 실제값이 이정도 일 때, 1모델링에서 이정도 값, 2모델링에서 이정도 값, 3모델링에서 이정도 값을 예측한다는 것을 학습시키는 것이 **Stacking Algorithm**

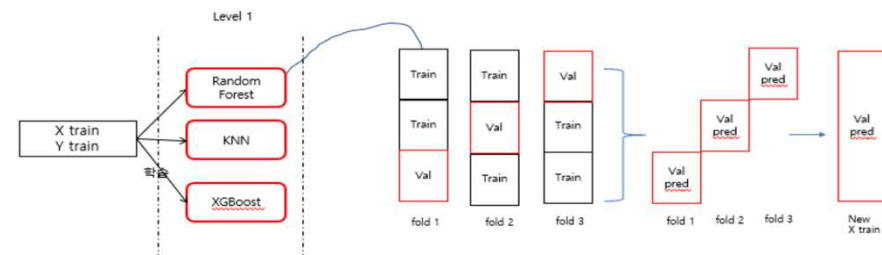
7.Stacking Ensemble → meta modeling



- > 원본 데이터의 train, test가 존재
- > 원본 training data를 3개의 머신러닝 모델이 학습
- > 각 모델마다 X_test를 넣어서 예측 후 predict를 뽑아냄(3개의 predict된 값)
- > 3개의 predict를 다시 학습 데이터로 사용
- > 최종 model을 하나 선정해 학습
- > 최종 평가

Test Data로 뽑은 Predicted data가 Input Data로 들어가기 때문에 Overfitting의 가능성이 존재
Cross-Validation 기법을 사용하여 과적합 가능성을 낮춰주는 형태로 적용하는 것이 일반적

Cross Validation을 사용할 경우, 모델 개수만큼 Fold를 나눠서 과적합을 방지한다.

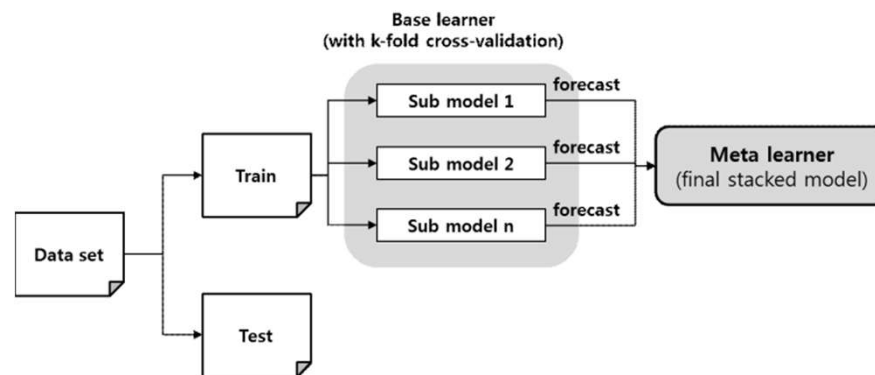


** 머신러닝 기반 기업부도위험 예측모델 검증 및 정책적 제언: 스택킹 앙상블 모델을 통한 개선을 중심으로 2020 엄하늘, 김재성, 최상욱

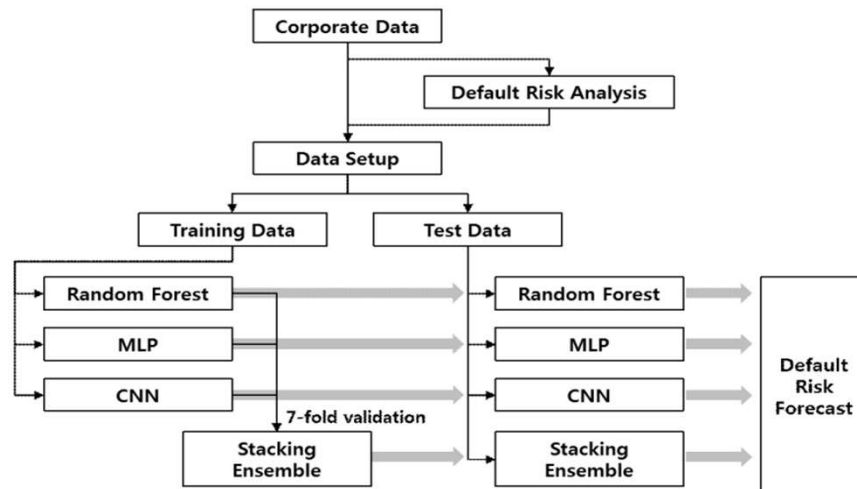
7. Stacking Ensemble

Stacking Ensemble Model

** 머신러닝 기반 기업부도위험 예측모델 검증 및 정책적 제언: 스택킹 앙상블 모델을 통한 개선을 중심으로 2020 엄하늘, 김재성, 최상욱



Division	Model 1	Model 2	Model 3	Model 4
	Random Forest (RF)	MLP	CNN	Stacking Ensemble
Main logic	Decision tree	Neural network	Neural network	Combining predictions using MLP
Input data for training	Training set	Training set	Training set	Base learner (RF, MLP, CNN) prediction results using segmented training set (7-fold)
Input data for testing	Test set	Test set	Test set	Base learner prediction results in test set
Training data dimension	(7382, 160)	(7382, 160)	(7382, 160)	(7382, 4)
Test data dimension	(3163, 160)	(3163, 160)	(3163, 160)	(3163, 4)



Division	Model	Hyper parameter	Value
Sub Model	Random forest	Number of trees	1000
		Loss function	MSE
		Random state	1
	MLP	Number of nodes	100
		Number of hidden layers	2
		Epoch	300
		Loss function	MSE
		Optimizer	Adam
	CNN	Dimension	1
		Filter size	2
		Kerner size	2
		Pool size	2
		Epoch	300
		Loss function	MSE
Stacking Ensemble Model	MLP	Optimizer	Adam
		Number of nodes	20
		Number of hidden layers	2
		Epoch	300

* DART: Dropouts meet Multiple Additive Regression Trees

DART 배경

- CART (Classification And Regression Trees): 자체적으로 Gradient Boosting Tree의 역할을 하는 것이 아니라, Decision Tree의 일종 → 분류를 진행한 뒤 분류 노드들에 대한 Node Value를 설정하여 Tree를 구성
- **MART** (Multiple Additive Regression Trees): GBRT(Gradient Boosting Regression Trees)와 동치
 - MART(Friedman, 2001, 2002)는 Boosting된 Regression Tree의 Ensemble모델로 다양한 task에 대해 높은 예측 정확도를 제공
 - iteration에서 tree가 추가될 때 몇 개의 instance의 prediction에 영향을 주고, 나머지 instance 예측에 무시하는 over-specialization 문제 발생
 - 이는 training data에 없는 데이터에 대해 모델의 성능이 부정적인 영향을 미치며, 초기에 추가된 training data에 모델이 overfitting
 - 이러한 over-specialization 문제를 완화하고 해결하기 위해 등장
 - DART는 MART에 Dropout을 적용한 것 → DNN(Deep Neural Network) Training에서 제안된 dropouts을 사용하여 문제를 해결 : dropout (Hinton et al., 2012)

MART와 Boosting Regression Tree문제점

- Ensemble이 만들어진 개별 예측자(predictor)의 경우 보다 더 나은 정확도를 달성하려면?
- predictor가 정확해야 하며, 서로 상관 관계(uncorrelated)가 없어야 한다.
 - predictor가 존재할 수 있는 특정 feature 또는 instance에 대한 sensitivity를 줄임으로써, 모델의 정확성을 높여주는데, 이때 MART나, 부스팅은 반복적으로 predictor를 합쳐가면서 진행된다.
 - Boosting 알고리즘은 현재 모델을 개선하는데 중점을 둔 predictor를 추가하며, iteration 사이의 training 문제를 수정함으로써 달성 → 추가된 predictor가 ensemble 모델과 다를지라도, 새로운 predictor는 일반적으로 문제의 작은 subset 초점을 맞추므로, 원래의 문제에서 측정할 때 강력한 예측력을 갖지 못한다. → 이 경우 특정 instance에 overfitting하는 위험 증가

Ensemble Learning 비교

앙상블 모형은 단일 분류기보다 더 robust !!

구분	Bagging (bootstrap + aggregate)	Boosting	Stacking
목적	Variance 감소	Bias 감소	Performance 향상
algorithm	Random forest	▪ Adaboost ▪ GBM ▪ XGboost ▪ Light GBM	학습 구조가 있음
학습 프로세스	모델별 독립적인 병렬 모델 (majority vote/ average)	연속적(직렬) 앙상블 모델 → 이전 모델의 오류 고려	서로 다른 독립적 모델
Sampling	Random sampling	Random sampling with 오차 가중치 **	
특징	▪ Overfitting 방지 가능 ▪ 학습 데이터의 다양화 ▪ 분류기의 다양화	• Outlier에 취약 • 학습 데이터의 다양화	▪ 모델 간의 보완이 가능 ▪ 연산량이 많음 ▪ 예측 성능 향상

** 이전 단계에서 오분류된 데이터의 가중치를 증가시키고, 정확하게 분류 된 데이터의 가중치는 감소 → 오분류 데이터에 더 집중하는 프로세스를 가진다는 것

Boosting 비교 / 하이퍼파라미터

algorithm	특징	비고
AdaBoost	다수결로 정답분류, 오답에 가중치 부여	1997년 시초 Tree-Based-Model
GBM (Gradient Boost Machine)	Cost function 의 Gradient 로 오답 가중치 부여	2001 Tree-Based-Model
XGBoost	GBM 성능 향상 , GBM + 규제화 n_estimators [default = 100] ▪ Booster Method : default = 'gbtree' (<i>gradient boosting tree</i>) ▪ learning_rate : default = 0.3 ▪ Maximum Depth : 3~12(16) → max_depth : default = 6 ▪ Maximum Leaves의 파라미터명 max_leaves default = 255 ▪ Positive BinaryScaling : scale_pos_weight → default = 1 ▪ Tree Building Method : 정확한 학습을 위해서는 Exact method('exact'), 학습속도를 높이기 위해서는 Approximate method('approx', 'hist', 'gpu_hist')를 이용	2014 Tree-Based-Model
LightGBM	대용량 학습 가능 Approximates Split를 통한 성능 향상 ▪ Booster Method : default = 'gbtree' (<i>gradient boosting tree</i>) ▪ learning_rate : default = 0.1 ▪ max_depth : default = -1 (무한 깊이) ▪ Maximum Leaves의 파라미터명 num_leaves default = 31 Positive BinaryScaling : scale_pos_weight → default = 1	2016 Tree-Based-Model
TabNet	딥러닝 성능 향상 - XAI	2020

* Overfitting

Overfitting 방지

- regularization
- Dropout * (별도 정리)
- Domain Adaptation

영역(Domain)이 약간 달라졌을 때, 다르지만 관련 있는 새로운 영역에 기존 영역의 정보를 적응(Adaptation)시켜서 사용하고자 하는 목적을 가지고 있는 연구 분야

Domain Shift 문제 : 학습 데이터 (Source)와 테스트 데이터 (Target) 의 Distribution의 차이 → Domain shift가 심할수록 test data의 정확도 하락

Traditional Domain Adaptation

1) Metric Learning (Adapting Visual Category Models to New Domains , Metric Learning: Asymmetric Transformations)

: 거리 기반으로 Domain adaptation을 진행

:가장 기본적인 생각으로 Source domain과 Target domain이 같은 클래스면 가깝게 만들고 다른 클래스면 멀게 만드는 것

2) Subspace Representation

: Source와 Target은 다른 subspace에 놓여져 있고 이 둘의 subspace를 alignment하면 문제를 해결할 수 있다고 생각

3) Matching Distribution (MMD, Sample Reweighting / Selection, Transform Learning (TCA, DIP) , Hellinger Distance, SIE)

Domain Shift로 인한 Distribution 차이를 직접적으로 해결하는데 초점

*Overfitting 생각해보기

1. Model averaging

예측 결과에 대한 편차를 줄여줌으로써 overfitting을 감소시키는 효과가 있다
(한계점)

sub-model의 정확도와 무관하게 최종 prediction에 각 sub-model이 동등하게 사용
→ regression문제 평균값, classification 문제 최빈값

2. weighted averaging model

각 sub-model의 결과값을 결합하여 최종 output을 내지만, Model averaging 처럼 각 sub-model의 기여도가 동등한 것이 아니라 weight 값에 따라 다르게 부여된다.

3. Stacked generalization (stacking)

weighted averaging model을 조금 더 일반화해서 각 sub-model의 예측결과를 결합하는 과정을 linear weighted sum으로 대체하는 방법

sub-model의 각 예측결과는 linear regression 모델의 train data로 사용될 수 있다.

Stacking procedure
2 단계

1단계: Level 0 : training dataset을 이용하여 sub-model 예측 결과를 생성
2단계 Level 1 : level 0의 output 결과가 level 1의 input 값이 된다. level 0의 output을 training data로 사용하여 meta learner 모델을 생성
위 단계 과정을 거치며 stacked generalization (Meta learner)은 level 0에서 사용한 training data와 다른 dataset을 사용하므로 overfitting 방지하면서 bias 감소

Stacked generalization 과정에서 level 0 모델들은 되도록 다양한 예측 결과를 Meta learner에서 input 값으로 활용할 수 있도록 각기 다른 알고리즘을 사용하는 것이 보편적

Machine learning models and bankruptcy prediction

Flavio Barbozaa,* , Herbert Kimura b , Edward Altman

Bankruptcy prediction is associated with credit risk, which has been thrust into the spotlight due to the recent financial crisis.

Machine learning models have been very successful in finance applications, and many studies examine their use in bankruptcy prediction.

The Altman and Ohlson models are still relevant, due not only to their predictive power but also to their simple, practical, and consistent frameworks.

Few studies can improve on their results concerning forecasting accuracy or the simplicity of the models

Regarding accuracy ratios, our results show that the traditional models (MDA, LR, and ANN) have lower predictive capacity (between 52% and 77%) than the machine learning models (71% to 87%), corroborating Wang et al. (2012), Breiman (2001), Kim and Upneja (2014) and Chen et al. (2010). New studies can adapt these machine learning techniques for other credit risk studies, such as general default events not limited to bankruptcy.

However, machine models are not perfect. The SVM-Lin results show that it is difficult to address non-separable datasets, which produces more misclassifications.

SVM-RBF, a non-linear kernel, reduced error rates but had weaker performance than other machine learning models. Moreover, SVM models took significantly more computational processing time than others did; however, this was not an important issue because no model took more than one minute to run. Although bagging, boosting, and RF incorporate similar procedures, RF generally produced better accuracy and error rates. Output variability is a critical problem typically found in ANNs, whereas the machine learning models produced stable solutions.

Machine learning models and bankruptcy prediction

Flavio Barbozaa,* , Herbert Kimura b , Edward Altman

We now highlight some strengths of the study.

First, we argue that significant predictive accuracy was achieved by using machine learning models under restrictive conditions such as existence of high correlated variables, outliers, and missing values.

Since we use raw data, without making any transformation or adjustments to the variables, results suggest that machine learning techniques can easily be applied and generate substantial classification accuracy, when compared to traditional mechanisms such as linear discriminant analysis, logistic regression, and artificial neural networks.

Another differential element of the study is the use of metrics that reflect growth or change in some variables, which are not usually incorporated into predictive models of bankruptcy.

Following arguments from Carton and Hofer (2006), we argue that failure of firms is likely to follow from difficulties over time and not just in the year prior to bankruptcy. Instead of using the time series or survival analysis approaches, our procedure easily incorporates information about short-term evolution of a variable in a machine learning model. Results show that, by adding variables that depict a dynamic yet simple behaviour of firms, machine learning techniques may improve predictive accuracy. We also highlight that our using of an extensive database of defaults is not common in papers related to corporate bankruptcy. Many studies that investigate default are restricted to a specific credit product of a specific financial institution. Since we use broad training and testing samples, the results can be useful in analysing bankruptcy of the overall North American corporate credit market. The number of observations also reduces problems of overfitting.

* 머신러닝 : 1. 지도학습

algorithm(활용)	장점	단점	활용
Linear Regression (회귀)	- 모델이 간단 - 해석(설명력)이 쉽다. - 중요 feature를 쉽게 구분 가능하다.	- 최신 알고리즘에 비해 예측력 저하 - 독립변수와 예측변수의 선형 관계를 가정 한계 - Feature 스케일링이 사전에 고려 (표준화/비표준화 계수 구분)	- 독립변수와 종속변수 간 선형 관계에 있는 데이터 - 가정 성립 여부 검증
Logistic Regression (분류)	- 모델이 간단 - 중요 feature를 쉽게 구분 가능하다.	선형 관계가 아닌 데이터에 대한 예측력이 저하	- 독립변수와 종속변수 간 선형 관계에 있는 데이터 - 이진분류에 주로 활용
Naive Bayes (분류)	- 비교적 간단하여 - 처리속도가 빠르다.	Feature 간 상호 독립적이라는 가정에 한계	- 독립변수들 간에 독립적이고 중요도가 비슷할 때 유용 - 범주형 변수가 많을 때 적합
Support Vector Machine(회귀/분류)	선형/비선형 분류	Hyperparameter 에 따라 성능이 크게 영향	
K Nearest Neighbors (회귀/분류)	- 모델이 간단하여 구현이 쉽다. - 훈련 속도가 빠르다. - 가정이 필요 없다.	Feature와 클래스 간 관계를 이해하는데 제한적 - Outliers data가 포함시 취약	- Outliers가 적은 데이터의 경우 - 다진분류에 활용 가능
Decision Tree (회귀/분류)	- Non-parametric Model - Outliers data에 영향을 받지 않는다. - 시각화에 우수 - 직관적으로 이해	- Overfitting 문제 - Bagging이나 boosting 방법에 비하여 상대적으로 예측력 저하	
Random Forest (회귀/분류)	- 다양한데이터에 상관없이 잘 적용 - Outliers data에 영향을 받지 않는다 - 전처리가 필요 없다.	- 처리시간이 길게 소요 - 해석력 저하	하이퍼파라미터 튜닝
XGBoost (회귀/분류)	- 예측력 우수 데이터가 많을수록 우수한 성능	해석력 저하	하이퍼파라미터 튜닝
LightGBM	XGBoost보다도 빠르고 높은 예측력. feature 중요도를 확인 가능	해석력 저하	하이퍼파라미터 튜닝

* 머신러닝 : 2. 비지도학습

algorithm(활용)	장점	단점	활용
K Means Clustering	- 구현이 간단 - 해석이 용이	- 최적의 K값을 분석가가 직접 선택해야 하는 어려움 - 거리기반으로 feature의 스케일 영향 - Feature 스케일링이 사전에 필요	
Principal Component Analysis (PCA)	- 차원 축소 - 변수간 공선성 문제 해결 - Overfitting 문제 해결 수단 - 시각화에 유용	차원이 축소에 따라 정보 손실	

<https://scikit-learn.org/stable/modules/ensemble.html>

• 1.11. Ensemble methods

- 1.11.1. Bagging meta-estimator
- 1.11.2. Forests of randomized trees
- 1.11.3. AdaBoost
- 1.11.4. Gradient Tree Boosting
- 1.11.5. Histogram-Based Gradient Boosting
- 1.11.6. Voting Classifier
- 1.11.7. Voting Regressor
- 1.11.8. Stacked generalization

인공신경망 (Artificial Neural Networks)

- **Deep learning** : deep neural network (DNN)
- **deep** : feed-forward network에서 가운데 hidden layer가 수개

- 1958 Rosenblatt : Perceptrons
- 1980 Neocognitron (Fukushima, 1980)
- 1982 Hopfield network, SOM (Kohonen, 1982), Neural PCA (Oja, 1982)
- 1985 Boltzmann machines (Ackley et al., 1985)
- 1986 Multilayer perceptrons and backpropagation (Rumelhart et al., 1986) 1988 RBF networks (Broomhead&Lowe, 1988)
- 1989 Autoencoders (Baldi&Hornik, 1989), Convolutional network (LeCun, 1989) 1992 Sigmoid belief network (Neal, 1992)
- 1993 Sparse coding (Field, 1993)

- xor 문제 : linear classifier
- learning → overfitting 문제
- overfitting 문제 해소 → regularization method : dropout, ReLU
- computation power
- deep learning 활용 : computer vision , language model, NLP, machine translation

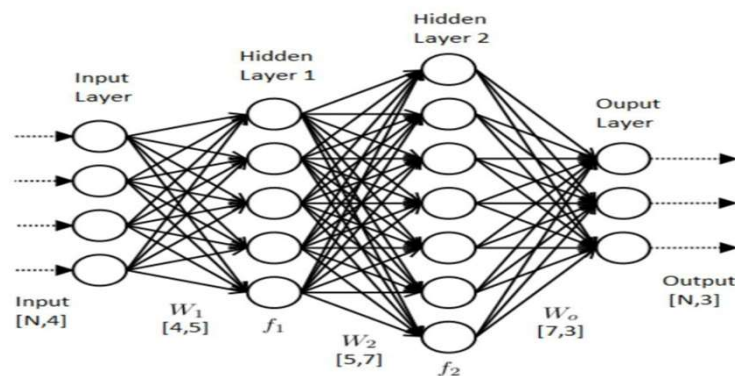
인공신경망 (Artificial Neural Networks)

Deep learning : deep neural network (DNN)

- Bengio : Greedy layer-wise training of deep networks (NIPS 2006)
- Hinton : A fast learning algorithm for deep belief nets (NIPS 2007)
- Bengio, Yoshua. "Learning deep architectures for AI." Foundations and trends® in Machine Learning 2.1 (2009): 1-127.
- Hinton, Geoffrey. "A practical guide to training restricted Boltzmann machines." Momentum 9.1 (2010):
- Fischer, Asja, and Christian Igel. "An introduction to restricted Boltzmann machines." Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications. Springer Berlin Heidelberg, 2012.

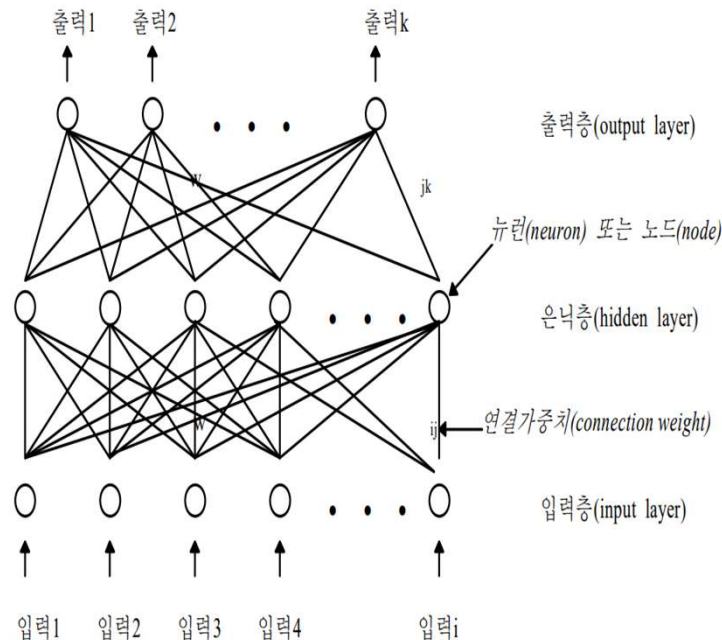
인공신경망 (Artificial Neural Networks)

- 1980년대 후반부터 인공신경망 기법들이 부도예측 연구
- 인공신경망 기법을 적용한 부도예측 모형에서는 주로 기업의 재무정보 등을 입력변수로, 기업의 부도여부를 출력변수로 설정하여 학습을 통해 이들의 관계를 추출하며 예측 변수들간에 잠재적으로 숨겨진 상관관계를 탐색한다.
- 부도예측 분야에서 인공신경망을 적용하는 가장 중요한 연구 중의 하나는 축적된 데이터를 이용하여, 독립변수와 종속변수 간의 결합관계를 추출해 냄으로써 패턴인식, 분류, 예측 등의 기능을 수행하는 모형을 구축하는 것
- 최근 인공신경망과 기존 통계기법의 성과를 비교, 분석하는 연구에서 나아가 다양한 방법론을 혼합적으로, 적용하여 인공신경망의 학습은 입력변수의 선정, 아키텍처, 데이터의 특성 등에 따라서 달라지고 예측치에도 차이가 있게 된다.
- 인공신경망 기법이 부도예측과 관련하여 기존의 통계적 기법을 활용한 연구보다 더욱 향상된 예측 모형이 될 수 있음을 제시



* Multi Layer Neural Networks

MLP(Multilayer Perceptron)



Activation function 왜 필요한가???

Non-linearity

- **입력층 (input layer)** : 각 입력변수에 대응되는 마디들로 구성되어 있다. 명목형(nominal) 변수에 대해서는 각 수준에 대응하는 입력마디를 가지게 되는데, 이는 통계적 선형모형에서 가변수(dummy variable)를 사용하는 것과 같다.
- **은닉층 (hidden layer)** : 여러 개의 은닉마디로 구성. 각 은닉마디는 입력층으로부터 전달되는 변수값들의 선형결합(linear combination)을 비선형함수(nonlinear function)로 처리하여 출력층 또는 다른 은닉층에 전달
- **출력층 (output layer)** : 목표변수(target)에 대응하는 마디들을 갖는다.
여러 개의 목표변수 또는 세 개 이상의 수준을 가지는 명목형 목표변수가 있을 경우에는 여러 개의 출력마디들이 존재

- **순전파(feedforward)**
입력층 → 은닉층 → 출력층
순전파 과정에서 거중치 부여에 따른 오차 발생으로 이를 수정하기 위한 역전파를 통한 가중치 update
- **역전파 알고리즘(Backpropagation)**
가중치 수정이 출력층 → 은닉층 → 입력층
다층 신경망 학습이 가능해졌고 이전에 XOR문제도 해결
- **알고리즘 최적화**
최적의 가중치를 찾아 나가는 과정

ANN , DNN(Deep Neural Network)

ANN (artificial neural network)

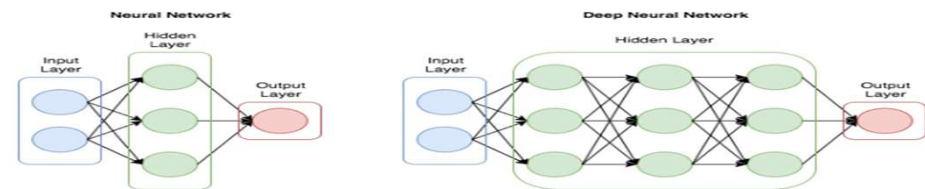
- 기계학습과 인지과학에서 생물학의 **신경망**(동물의 중추신경계중 특히 뇌)에서 영감을 얻은 통계학적 학습 알고리즘
→ Deep Learning 용어로 대체
- ANN기법의 여러 문제가 해결되면서 모델 내 은닉층을 많이 늘려서 학습의 결과를 향상시키는 방법이 등장
- DNN은 은닉층을 2개이상 지닌 학습 방법 → 다층 퍼셉트론(MLP: Multiple Layers Perceptron)
- DNN(Deep Neural Network): 입력층(input layer)과 출력층(output layer) 사이에 여러 개의 은닉층(hidden layer)들로 이뤄진 인공**신경망**(Artificial Neural Network, ANN)
- 컴퓨터가 스스로 분류레이블을 만들어 내고 공간을 왜곡하고 데이터를 구분짓는 과정을 반복하여 최적의 분류선을 도출
- 많은 데이터와 반복학습, 사전학습과 오류역전파 기법을 통해 현재 널리 사용
- → DNN을 응용한 알고리즘이 바로 RNN, CNN인 것

RNN (recurrent neural network)은 학습에 사용되는 데이터를 각각 독립적으로 분석하는 것이 아니라 순차적으로 처리. 분석하는 방법으로, 시계열 자료의 예측이나 텍스트(text) 자료의 문맥 인식 등에 사용

CNN (convolutional neural network)은 학습에 사용되는 정보를 여러 영역으로 분할한 뒤, 제한된 정보를 사용하여 관계성이 높은 영역끼리 분석하는 방법으로 패턴을 찾아 이미지 인식(image recognition) 분야에서 특화된 알고리즘

• DNN(Deep Neural Network)

$$Y_k = \sum_{j=1}^m f \left(\sum_{i=1}^n v_i w_{ij} + b_j \right) w_{jk} + b_k$$



NN vs DNN

CNN(Convolution Neural Network)

CNN(Convolution Neural Network)

- (활용) 딥러닝에서 주로 이미지나 영상 데이터를 처리 사용
- 일반 DNN은 기본적으로 1차원 형태의 데이터를 사용 → (예를 들면 1028x1028같은 2차원 형태의)이미지가 입력값이 되는 경우, 이것을 flatten시켜서 한줄 데이터로 만들어야 하는데 이 과정에서 이미지의 공간적/지역적 정보 (spatial/topological information)가 손실 발생 + 추상화과정 없이 바로 연산과정으로 넘어가 버리기 때문에 학습시간과 능률의 효율성이 저하
- → 이러한 문제점에서부터 고안한 해결책이 CNN
- Convolution이라는 전처리 작업이 들어가는 Neural Network 모델
- CNN은 이미지를 raw input 그대로 받음으로써 공간적/지역적 정보를 유지한 채 특성(feature)들의 계층을 빌드업 → **CNN의 중요 포인트는 이미지 전체보다는 부분을 보는 것, 그리고 이미지의 한 픽셀과 주변 픽셀들의 연관성을 살리는 것** (새의 경우 : 전체가 아니라 부리로 구분 판단)
- (목적) 우리의 입력값 이미지의 모든 영역에 같은 필터를 반복 적용해 패턴을 찾아 처리하는 것

- CNN은 데이터의 특징을 추출하여 특징들의 패턴을 파악하는 구조
- **CNN 알고리즘 : Convolution과정과 Pooling과정을 통해 진행**
- Convolution Layer와 Pooling Layer를 복합적으로 구성하여 알고리즘

CNN은 입력층과 필터의 합성곱(Convolution)을 이용하는 방법론으로 하나의 입력계층과 출력계층, 하나 이상의 합성곱 계층(Convolution layer)과 풀링 계층(Pooling layer)로 구성
입력층을 통해 입력하고자 하는 데이터를 입력하고 합성곱 계층을 통해 필터링되어 적절한 특징을 추출 .

CNN : Convolution의 작동 원리

1. Feature mapping

- 2차원 이미지는 픽셀 단위로 구성
- **input image + filter (Kernel) → 결과값(feature mapping)**
- 전체 이미지 matrix (input image) 에 필터 matrix를 조금씩 움직이며 반복 → matrix와 matrix간의 Inner Product라는 것을 사용

2. Zero Padding

- **feature mapping 과정에서 정보 손실**→ 이런 문제점을 해결하기 위해 Padding이라는 방법을 사용
- 0으로 구성된 테두리를 이미지 가장자리에 감싸 준다.

3. Stride

- 필터를 얼마만큼 움직여 주는가에 대한 것
- stride값이 1일 경우 필터를 한칸씩 움직여 준다
- stride-1이 기본값이고 stride는 1보다 큰 값이 될 수도 있다.

4. The Order-3 Tensor

- 입력값이 N차원인 경우도 존재
- 3차원 이미지 : order-3 Tensor, 2차원 이미지 : order-2 Tensor

5. Pooling (Layer)

한 개의 이미지에서 수개의 이미지 결과값이 도출되어버리면 값이 너무 많아졌다는 것이 문제점

→ 이를 해결하기 위하여 고안된 방법이 Pooling이라는 과정

→ *Pooling은 각 결과값(feature map)의 dimensionality를 축소해 주는 것을 목적* : correlation이 낮은 부분을 삭제하여 각 결과값을 크기(dimension)을 줄이는 과정

1) **Max pooling** : matrix에서 가장 큰 값(max)으로

2) **Average pooling**: matrix에서 평균값(average)를 가져와 결과값의 크기를 반으로 줄여준다. --> next page 참조.

CNN : Convolution의 작동 원리

Max pooling

35	15	20	90
5	50	40	10
10	15	5	10
25	30	40	25



2x2 pool size

50	90
30	40

Average pooling

35	15	20	90
5	50	40	10
10	15	5	30
25	30	40	25

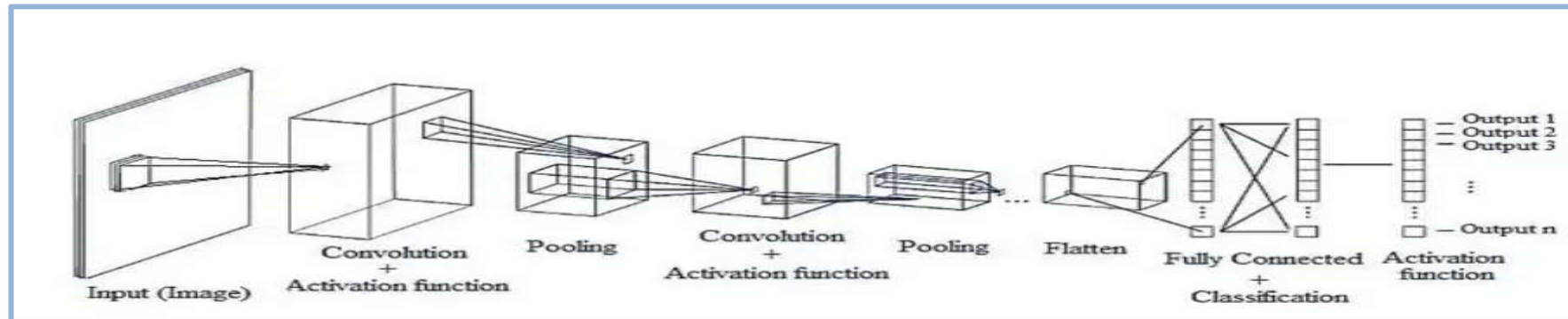


2x2 pool size

25	40
20	25

CNN(Convolution Neural Network)

CNN : 네트워크 구조



1. 첫번째 Convolutional Layer

- CNN는 Convolutional Layer과 Pooling Layer들을 활성화 함수 앞뒤에 배치
- A Convolutional Layer = convolution + activation function
- 활성화함수(Activation function) : 선형함수(linear function)인 convolution에 비선형성(nonlinearity)를 추가하기 위해 사용하는 것

2. 첫번째 Pooling Layer

3. 두번째 Convolutional Layer

- → 텐서 convolution을 적용

4. 두번째 Pooling Layer

5. Flatten(Vectorization) → 텐서를 일차 형태의 데이터로 쭉 펼쳐주는것 !!

→ 각 세로줄을 일렬로 쭉 세워 두는 것 → 벡터(vector)형태로 전환 됨.

6. Fully-Connected Layers(Dense Layers) : 하나 혹은 하나 이상의 Fully-Connected Layer를 적용시키고 마지막에 Softmax activation function을 적용 → 최종 결과물 출력

CNN(Convolution Neural Network)

매개변수(Parameter)와 Hyper-매개변수(Hyper-parameter)

CNN 모델의 학습 가능한 매개변수(trainable parameters)

(사례) 5x5크기의 필터

- 1) 첫 convolution과정에서 5x5크기의 필터 10개를 사용하였으므로 **250개**의 매개변수가 존재
 - 2) 두번째 convolution 과정에서 5x5x10크기의 텐서 스무개를 사용하였으므로 **5,000개**의 매개변수가 추가
 - 3) 두개의 Fully-connected layer들에서 각각 매개변수 **32,000개, 1000개**가 추가
- 1) + 2) + 3) 총 **38,250 이상**의 학습 가능한 매개변수

Hyper-parameters

- **Convolutional layers** : 필터의 갯수, 필터의 크기, stride값, zero-padding의 유무
- **Pooling layers** : Pooling방식 선택(MaxPooling or AvgPooling), Pool의 크기, Pool stride 값(overlapping)
- Fully-connected layers: 넓이(width)
- **Activation function** : ReLU(주로 사용), SoftMax(multi class classification), Sigmoid(binary classification)
- **Loss function** : Cross-entropy for classification, L1 or L2 for regression
- **optimizer** : SGD(Stochastic gradient descent), SGD with momentum, Adaptive Gradient (AdaGrad), Adam, RMSprop
- Random initialization: Gaussian or uniform, Scaling

* 쉬어가기

활성화 함수(Activation Function)

인공 신경망의 각 뉴런에서 입력 신호(input)의 가중합을 계산한 후 그 결과를 변환하는 함수

체인룰 (Chain Rule)

- 학습 과정에서는 가중치(weight) 및 편향(bias)에 대한 손실 함수의 기울기(gradient)를 계산하는데, 이 때 체인룰 (Chain Rule)이 중요한 역할
- 체인룰은 합성 함수의 도함수를 계산하는 규칙으로, 합성 함수는 여러 함수의 조합으로 구성 됨.
→ 미분값을 계산할 때 체인룰을 적용하여 각 함수의 도함수를 곱해주어야 한다.
- 인공 신경망에서의 체인룰은 주로 역전파(backpropagation) 알고리즘에서 사용됩니다. 역전파는 네트워크의 출력과 실제 값 사이의 오차를 최소화하기 위해 가중치와 편향을 조절하는 과정
- 활성화 함수의 역할 중 하나는 네트워크의 출력을 생성하는 데 사용되는 함수로, 이 함수의 미분값이 역전파 과정에서 필요

(사례) 시그모이드 활성화 함수 $\sigma(x) = \frac{1}{1 + e^{-x}}$

시그모이드 함수의 도함수 $\rightarrow \sigma'(x) = \sigma(x)(1 - \sigma(x))$

체인룰을 사용하여 뉴런 출력에 대한 손실 함수의 기울기를 계산→

$$\frac{\partial \text{손실}}{\partial \text{가중치}} = \frac{\partial \text{손실}}{\partial \text{출력}} \times \frac{\partial \text{출력}}{\partial \text{입력}} \times \frac{\partial \text{입력}}{\partial \text{가중치}}$$

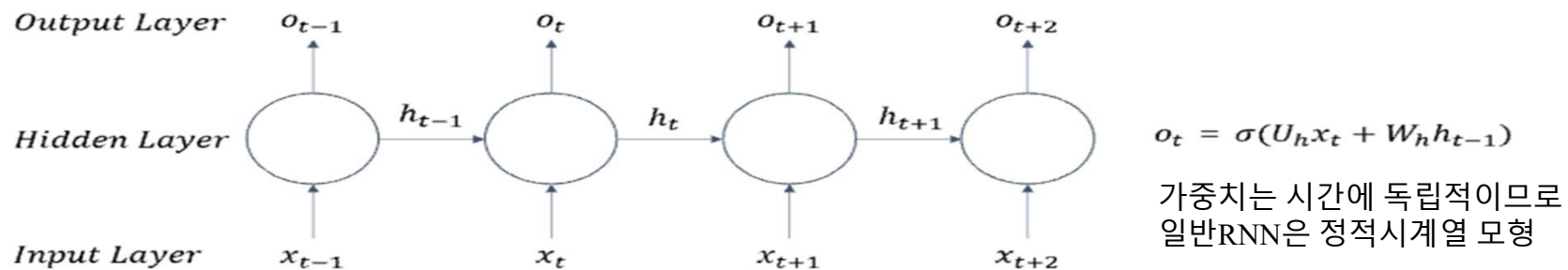
체인룰을 사용하여 역전파 알고리즘이 각 층의 가중치와 편향을 조절하며 네트워크를 학습

RNN(Recurrent Neural Network)

RNN(Recurrent Neural Network)

- **시계열 알고리즘: 시간이 변함에 따라 변화를 반영가능한 시계열모형**
- RNN : 인공신경망 알고리즘 중 **이전 기록의 데이터들까지도** 다시 현시점의 데이터 학습에 이용하는 알고리즘
→ 이전 기록의 Hidden Layer에 남아있는 기록을 현시점의 데이터와 함께 Forward-propagation화하여 Back-Propagation을 통해서 학습하는 알고리즘
- RNN 알고리즘은 반복적이고 **순차적인 데이터(Sequential data)학습에 특화된 인공신경망**
- 내부의 순환구조가 들어있다는 특징
- 순환구조를 이용하여 과거의 학습을 Weight를 통해 현재 학습에 반영
- 기존의 지속적이고 반복적이며 순차적인 데이터학습의 한계를 해결하는 알고리즘
- 현재의 학습과 과거의 학습의 연결을 가능하게 하고 시간에 종속된다는 특징
- (문제점) 시점간 간격이 멀어질수록 BackPropagation Through Time방법을 이용함에 있어 gradient값들이 0에 점점 가까워져Vanishing Gradient현상이 발생하게 되는 단점 존재

RNN Model Diagram



LSTM(Long Short Term Memory)

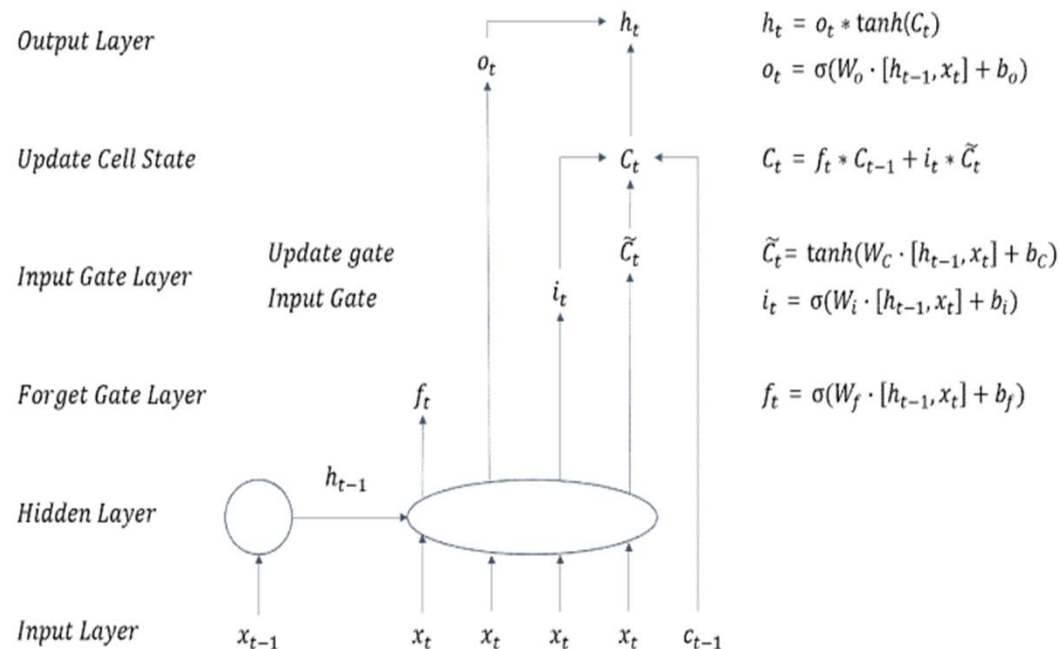
LSTM(Long Short Term Memory)

Cell State라 불리는 Layer층을 하나 더 넣어서 Weight함수를 기억할 것인지 아닌 지(추가 혹은 삭제하는 방식을 통해) 결정.

즉, Cell State 기능을 기반으로 RNN에서 발생했던 Gradient Vanishing 문제를 해결한 모형

LSTM 유닛은 여러 개의 게이트가 붙어있는 셀로 이루어져 있으며 셀의 정보들을 저장, 불러오기, 유지 하는 기능이 있다. 이러한 기능 때문에 RNN에서 보였던 기울기가 사라지는 문제를 해결할 수 있으며 오래전 시점의 정보도 더 잘 기억 할 수 있게 한다.

LSTM Model Diagram



LSTM셀 구조

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hv}v_t + W_{hm}\tilde{m}_{t-1})$$

$$i_t^g = \text{sigmoid}(W_{igh}h_t + W_{igv}v_t + W_{igm}\tilde{m}_{t-1})$$

$$i_t = \tanh(W_{ih}h_t + W_{iv}v_t + W_{im}\tilde{m}_{t-1})$$

$$o_t = \text{sigmoid}(W_{oh} + W_{ov}v_t + W_{om}\tilde{m}_{t-1})$$

$$f_t = \text{sigmoid}(b_f + W_{fh}h_t + W_{fv}v_t + W_{fm}\tilde{m}_{t-1})$$

$$m_t = m_{t-1} \odot f_t + i_t \odot i_t^g$$

$$\tilde{m}_t = m_t \odot o_t$$

$$z_t = g(W_{yh}h_t + W_{ym}\tilde{m}_t)$$

stateful Long Short Term Memory (stateful LSTM)

- 각 배치의 학습이 진행될 때 은닉상태 벡터(hidden state vector)와 셀 상태 벡터(cell state vector)를 초기화 하는 기법
- 하나의 시퀀스로 데이터를 해석하는 기본 LSTM 모델 보다 데이터의 배치 단위로 시퀀스를 기억하여 학습하는 stateful LSTM 기법의 성능이 우수하게 나타났다고 판단

* RBF(radial basis function, 원형기준함수)신경망

RBF 신경망에서 추정해야 할 모수의 수는 MLP에서와 동일하나 그 추정방법이 차이가 있음.

은닉마디의 개수를 k 라고 할 때, 일반적으로 RBF 신경망에서는 먼저 k 개의 중점(center) 와 반경(radius) 를 군집분석(cluster analysis)과 같은 자율추정방법(unsupervised method)에 의해서 추정→

다음에 은닉층으로부터 출력층으로 전달되는 계수를 지도추정방법(supervised method)에 의해서 추정

→ 은닉층에서의 모수를 먼저 추정→ 출력층에서의 모수를 추정 : 이러한 2단계 추정방법 : hybrid method

(장점)

MLP 신경망에서와 같이 모든 모수를 동시에 추정하는 방법보다 계산과정이 간단하고 시간이 작게 걸린다

(단점)

수렴성의 측면에서 볼 때 MLP 신경망에 비해서 비효율적

즉, RBF 신경망은 전체 최소값(global minimum)에 수렴하지 않을 가능성이 많은데, 특히 중점을 잘못 잡거나 은닉마디의 수가 충분하지 않은 경우에 이런 현상 발생

Elliptical Basis Function ; EBF(타원형기준함수)신경망

EBF 신경망은 2개의 은닉층으로 구성.

첫번째 은닉층: 입력층의 차원을 감소시키며 → RBF 신경망이 가지는 문제점을 보완

입력공간을 회전시키는 역할로 입력변수의 단순 선형결합으로 표현

$$z_j = b_j + w_{1j}X_1 + w_{2j}X_2 + \cdots + w_{pj}X_p$$

두 번째 은닉층(RBF)

$$H_i = \exp\left(-\frac{(z_1 - c_{1i})^2 + (z_2 - c_{2i})^2 + \cdots + (z_d - c_{di})^2}{r_i^2}\right)$$

* 확률 신경망 (Probabilistic Neural Network ; PNN)

PNN 신경망은 전방향 구조를 가지며 역전파(Back propagation)알고리즘과 유사한 supervised 학습 알고리즘을 가진다. 이 신경망은 입력된 데이터의 가중치를 Network에서 일단 통과 시키므로 매우 빠른 특징을 가지고 있다.

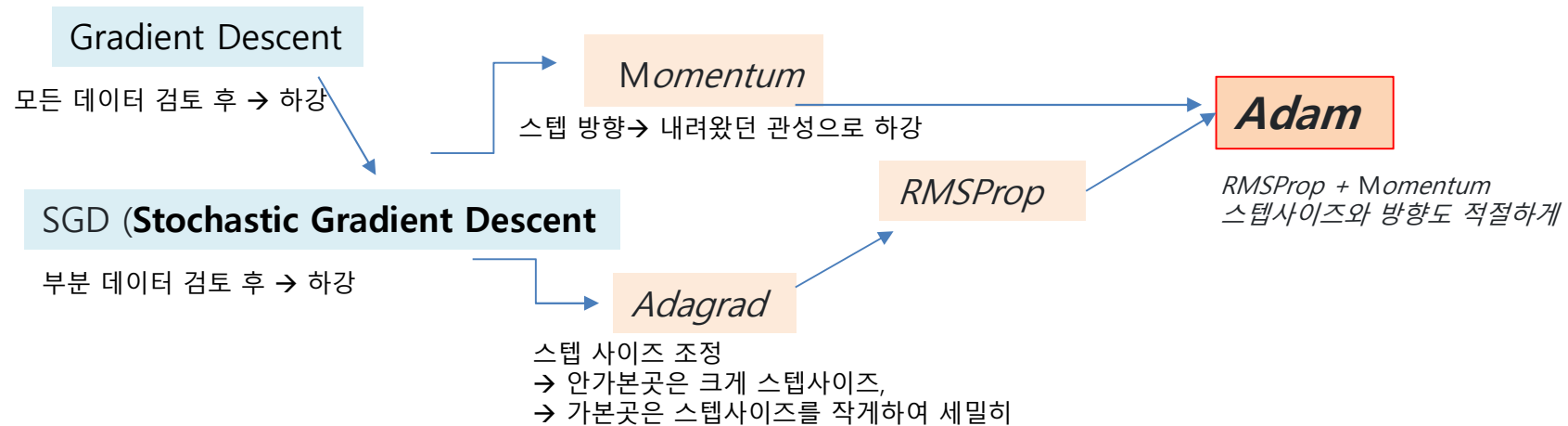
* 쉬어가기 : Optimizer

딥러닝

층이 깊어질수록 모듈과 함수에 따른 하이퍼파라미터(hyper-parameter) 도 비례하여 많아지기에 이 하이퍼파라미터를 결정하여 모델이 정확하게 결과를 얻는게 핵심

Loss Function을 최적화 → 최적화 알고리즘 = Optimizer

- Gradient Descent
- **SGD (Stochastic Gradient Descent)**
- *Momentum*
- *Adagrad* : **Adaptive gradient**
- *RMSProp* : Root Mean Square Propagation
- *Adam* : Adaptive Moment Estimation



* 쉬어가기 : Optimizer

Gradient Descent, 경사하강법

비용 함수(Loss function) 를 최소화하기 위해 매개변수를 반복적으로 조정하는 과정 사용
 → 현 가중치에서의 기울기 (gradient)를 구해서 Loss를 줄여가는 방향으로 업데이트 해 나가는 방법
 → 경사 하강법의 목표는 주어진 함수의 최소값을 찾는 것
 가중치 (weight)의 업데이트 = 현지점의 기울기 (gradient) x 에러를 낮추는 방향 (decent) x learning rate (한 스텝 량)

기본 원리

경사 하강법은 함수의 기울기(Gradient)를 사용하여 최소값을 찾는다.
 기울기는 함수가 가장 빠르게 증가하는 방향을 나타내므로, 반대 방향으로 매개변수를 조정하면 함수의 최소값을 찾을 수 있다.

수학적 표현

$$\bullet \theta = \theta - \eta \nabla_{\theta} J(\theta)$$

- η : 학습률 → 매개변수가 업데이트되는 정도를 결정
- θ : 최적화하려는 매개변수
- $\nabla_{\theta} J(\theta)$: 비용함수 J 의 매개변수 θ 에 대한 gradient

* 쉬어가기 : Optimizer

Gradient Descent, 경사하강법 Stochastic Gradient Descent

1.정의: Gradient Descent는 전체 훈련 데이터 세트를 사용하여 (full batch) 비용함수의 gradient 를 계산

→ 이 gradient는 모델 파라미터를 업데이트하는 데 사용 → 전부

2. 작동 방식: 각 반복에서 전체 데이터 세트에 대한 비용함수의 gradient를 계산하고, 이를 사용하여 모델 파라미터를 업데이트

3. 변형

1) 배치 경사 하강법(Batch Gradient Descent): 전체 데이터셋에 대해 한 번에 gradient를 계산 → GPU가 한번에 처리하는 데이터의 묶음인 배치(batch)를 하나로 !!!

→ 전체 데이터를 통해 학습시키기 때문에, 가장 업데이트 횟수가 적다. (1 Epoch 당 1회 업데이트)

전체 데이터를 모두 한 번에 처리하기 때문에, 메모리가 가장 많이 필요

2) 확률적 경사 하강법(Stochastic Gradient Descent, SGD):

- 전체 데이터 중 단 하나의 데이터를 이용하여 경사 하강법을 1회 진행(배치 크기= 1)하는 방법
전체 학습 데이터 중 랜덤하게 선택된 하나의 데이터로 학습을 하기 때문에 확률적 이라 부른다.

- 배치 경사 하강법에 비해 적은 데이터로 학습할 수 있고, 속도가 빠른 장점이 있다. 무엇보다 큰 특징은 수렴에 Shooting이 발생한다는 점이다.

- SGD는 신경망을 학습할 때마다 가중치를 갱신한다. → 학습할 때마다 가중치를 갱신하면서 학습할 때마다 신경망의 성능이 변하면서 정답에 가까워지기 때문에

학습 데이터 전체에 대해 보편적으로 좋은 값을 내는 방향으로 수렴한다. 다만, 최저점에 안착하기는 어렵다.

또한, Shooting은 최적화가 지역 최저점(Local Minima)에 빠질 확률을 줄여준다.

→ Global Minimum에 수렴하기 어렵다.

노이즈가 심하다. (Shooting이 너무 심하다.)

3) 미니배치 경사 하강법(Mini-batch Stochastic Gradient Descent):

전체 데이터를 batch_size개씩 나눠 배치로 학습(배치 크기를 사용자가 지정)시키는 것.

→ 전체 데이터가 1000개인 데이터를 학습시킬 때, batch_size가 100이라면, 전체를 100개씩 총 10 묶음의 배치로 나누어 1 Epoch당 10번 경사하강법을 진행

- 데이터셋의 작은 부분집합(미니배치)을 사용하여 gradient 를 계산 → 이는 배치와 확률적 경사 하강법의 장점을 결합

* 쉬어가기 : Optimizer

배치(Batch)

hidden layer의 노드 수가 많으면 많을수록 계산이 복잡

→ 경사하강법 1회에 사용되는 데이터 묶음

→ 학습 속도에 대한 고민

데이터를 1개씩 입력받아 수십만번의 연산을 진행하는 것보다 한번에 큰 묶음으로 데이터로 입력받아(Batch) n번의 연산을 진행하는 것이 더 빠르다.

데이터를 하나 하나씩 신경망에 넣어 학습을 시킨 경우 소요되는 시간이 매우 짧다는 것이다.

--> but 데이터를 하나씩 입력 받으면 계산 속도가 사실상 늦어진다.
(GPU의 병렬처리에 있다)

--> 데이터를 한개씩 넣어 인공지능을 학습시키면 사실상 GPU의 병렬처리를 사용하지 않으니 그만큼 자원 낭비

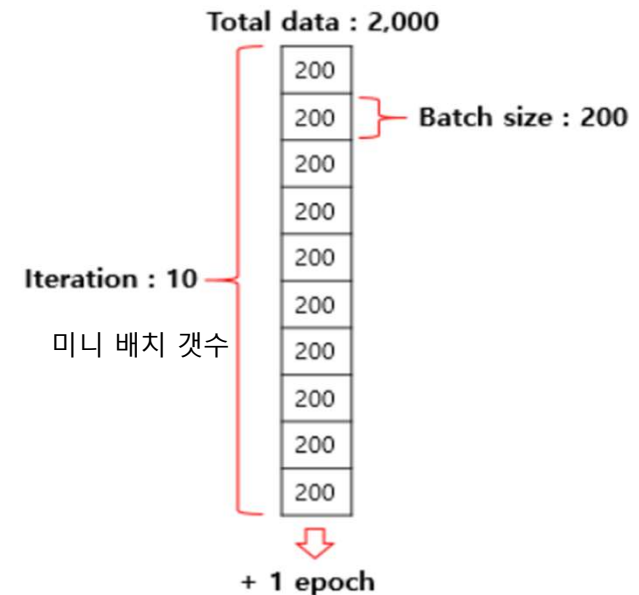
--> Loss Function에서 최적의 파라미터를 설정하는데 상당히 시간 소요
(개선 : minibatch)

전체 데이터를 입력하는 경우는 한번에 여러개의 데이터에 대해서 신경망을 한 번 학습시키는데 소요되는 시간이 매우 길어 이런 문제점과 장점을 적절히 섞은 것이 미니 배치 !!!

미니 배치(Mini-Batch)

SGD(Stochastic Gradient Descent) 와 배치를 섞은 것으로 전체 데이터를 N등분하여 각각의 학습 데이터를 배치 방식으로 학습시킨다.

최대한 신경망을 한 번 학습시키는데 걸리는 시간을 줄이면서 전체 데이터를 반영할 수 있게 된다.



minibatch



- 배치 경사 하강법 (BGD) → 배치 크기 = 전체 학습 데이터
- 확률적 경사 하강법 (SGD) → 배치 크기 = 1
- 미니 배치 확률적 경사 하강법 (MSGD) → 배치 크기 = 사용자가 지정

* 쉬어가기 : Optimizer

모멘텀(Momentum Gradient Descent)

- 경사 하강법 (Gradient Descent)에 **속도와 방향성을 추가하여** 보다 빠르고 안정적인 수렴을 도모
- 물리학에서의 관성의 개념을 차용한 것으로, 이전 단계의 업데이트가 현재 단계의 업데이트에 영향을 미치게 된다.

일반적으로 금융 및 투자 분야에서 사용되며, 특정 자산의 가격이 상승 또는 하락 추세를 지속하는 경향 이는 주식, 채권, 상품 등 다양한 금융 자산에 적용 → 모멘텀 전략은 최근 성과가 좋은 자산을 매수하고, 성과가 나쁜 자산을 매도하는 전략

모멘텀의 기본 원리

- 매개변수의 업데이트에 이전 업데이트를 '모멘텀'으로서 고려
- 공이 언덕을 굴러 내려가는 것과 비슷한 원리로, 속도(모멘텀)를 이용하여 최적점을 찾아가는 과정에서 gradient의 진동을 줄이고 수렴 속도를 빠르게 한다.

(핵심 아이디어)

- 이전 단계의 gradient가 현재 단계의 업데이트에 일정 부분 기여한다는 것 !!! (아래 수식 참조)
- 이를 통해 알고리즘은 더 안정적이고 빠르게 최적점에 도달할 수 있다.
- 이전 gradient가 어떤 방향으로 강하게 움직였다면, 모멘텀은 그 방향으로의 이동을 가속화한다.
- 이 방법은 특히 딥러닝과 같은 복잡한 최적화 문제에서 매우 효과적

모멘텀의 수학적 표현

$$\begin{aligned} \bullet v_t &= \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta) \\ \bullet \theta &= \theta - v_t \end{aligned}$$

- v_t : 시간 t에서의 **모멘텀(속도)**
- γ : **모멘텀계수** (통상 0.9정도 설정) → 이전단계의 모멘텀이 얼마나 큰 영향을 미칠지 결정
- η : 학습률
- θ : 업데이트할 매개변수
- $\nabla_{\theta} J(\theta)$: 비용함수 J의 매개변수 θ 에 대한 gradient

* 쉬어가기 : Optimizer

SGD Momentum, (SGD : Stochastic Gradient Descent)

모멘텀(Momentum)

(장점)

1. 속도와 안정성 향상

: 모멘텀은 경사 하강법의 수렴 속도를 가속화하고, 최적화 과정을 안정화시킨다.

2. 지역 최소값(Local Minima)과 안장점 (Saddle Points) 극복

: 모멘텀은 매개변수가 지역 최소값이나 안장점에 갇히는 것을 방지하는 데 도움을 준다.

3. 방향성 유지

: 연속적인 업데이트가 비슷한 방향으로 이루어질 경우, 모멘텀은 그 방향으로 더 빠르게 이동하도록 한다.

(주의점)

- 모멘텀은 매개변수 업데이트에 관성을 부여하기 때문에, 최적점 근처에서는 너무 멀리 지나칠 수 있다. 이를 조절하기 위해 적절한 학습률과 모멘텀 계수의 선택이 중요
- 모멘텀은 딥러닝 최적화에서 널리 사용되며, 특히 복잡하고 깊은 네트워크에서 그 효과가 두드러진다.

* 쉬어가기 : Optimizer

Adagrad : Adaptive gradient

(핵심 아이디어)

- 매개변수의 각 차원에 대해 학습률을 독립적으로 조정하는 것 !!
- 이는 자주 업데이트되는 매개변수의 학습률을 감소시키고, 드물게 업데이트되는 매개변수의 학습률을 증가시킨다.
- 각 매개변수에 대해 개별적으로 학습률을 조정함으로써, 특히 희소한 데이터에서 좋은 성능을 보인다.
- AdaGrad는 특히 대규모 기계 학습 문제에 적합하며, 특히 자연어 처리(NLP)와 같은 분야에서 많이 사용

AdaGrad 수학적 표현

파라미터 업데이트 규칙

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}$$

$\theta_{t,i}$: 시간 t에서의 i번째 매개변수
 η : 전역 학습률
 G_t : 시간 t까지의 모든 gradient의 제곱합의 대각행렬
 ϵ : 0으로 나누는 것을 방지하기 위한 작은 상수
 $g_{t,i}$: 시간 t에서 i번째 매개변수에 대한 gradient

(장점) 각 매개변수에 대해 맞춤형 학습률을 제공한다는 것

(단점) 학습 과정이 진행됨에 따라 학습률이 지나치게 감소할 수 있으며, 이는 장기적으로 학습이 멈추는 원인이 될 수 있다.

이러한 단점을 극복하기 위해 AdaGrad의 변형들이 개발되었으며, 이에는 RMSprop과 Adam

* 쉬어가기 : Optimizer

RMSProp

- *AdaGrad*에서 학습이 안 되는 문제를 해결하기 위해 hyperparameter β 추가
- 변화량이 더 클수록 학습률이 작아져서 조기 종료되는 문제를 해결하기 위해 학습률 크기를 비율로 조정할 수 있도록 제안된 방법.
- **RMSProp**은 최근 경로의 곡면 변화량을 측정하기 위해 **지수가중이동평균**을 사용

장점

- 미분값이 큰 곳에서는 업데이트 할 때 큰 값으로 나눠주기 때문에 기존 학습률 보다 작은 값으로 업데이트 된다.
→ 따라서 진동을 줄이는데 도움
- 반면 미분값이 작은 곳에서는 업데이트시 작은 값으로 나눠주기 때문에 기존 학습률 보다 큰 값으로 업데이트 된다..
→ 이는 조기 종료를 막으면서도 더 빠르게 수렴하는 효과
→ 따라서 최근 경로의 gradient는 많이 반영되고 오래된 경로의 gradient는 작게 반영
- gradient 제공에 곱해지는 가중치가 지수승으로 변화하기 때문에 지수가중이동평균이라 부른다.
- 매번 새로운 gradient 제공의 비율을 반영하여 평균을 업데이트 하는 방식

* 쉬어가기 : Optimizer

Adam Optimizer : Adaptive Moment Estimation

모멘텀(momentum) + RMSprop 알고리즘

- 모멘텀 : 이전 gradient 의 지수적 감소 평균을 사용하여 방향성을 유지
- RMSprop : gradient의 제곱의 지수적 감소 평균을 사용하여 학습률을 조정

Adam Optimizer의 작동 원리

- 모델의 가중치를 무작위로 초기화.
- 학습 데이터에서 mini batch를 무작위로 선택
- 선택한 mini batch를 사용하여 손실 함수를 계산
- 손실 함수의 기울기를 계산
- 기울기를 사용하여 가중치를 업데이트
- 위 단계를 반복하여 전체 학습 데이터에 대해 가중치를 업데이트

Adam Optimizer의 가중치 업데이트 수식

$$\begin{aligned}
 & \bullet m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \\
 & \bullet v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\
 & \bullet \hat{m}_t = \frac{m_t}{1 - \beta_1^t} \\
 & \bullet \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\
 & \bullet \theta_{t+1} = \theta_t - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}
 \end{aligned}$$

- m_t : 1차 모멘트 추정(이동평균)
- v_t : 이동 평균 제곱(2차 모멘트 추정) → 스케일된 gradient 추정치
- g_t : 시간 t에서의 gradient
- β_1, β_2 : 모멘텀 하이퍼파라미터
- θ : 업데이트 할 매개변수
- α : learning_rate(학습률)
- ϵ (epsilon) : 숫자 안정성을 위한 작은 값

* 쉬어가기 : Optimizer

Adam Optimizer : Adaptive Moment Estimation

Adam Optimizer의 주요 Hyperparameter

Hyperparameter	내용
learning rate	- 가중치 업데이트의 크기를 조절하는 파라미터 - 너무 작으면 학습이 느리고, 너무 크면 발산
beta1 , beta2	- 이동 평균 계산에 사용되는 모멘텀 하이퍼파라미터 - 일반적으로 0.9와 0.999로 설정
epsilon	숫자 안정성을 위한 작은 값

Adam Optimizer의 장점과 한계

장점	한계점
- 빠른 학습 속도와 좋은 최적화 성능을 제공 - 다양한 하이퍼파라미터 값에 대해 상대적으로 민감하지 않는다. - 로컬 최적점에 빠지지 않고 전역 최적점에 가까이 수렴한다.	- 메모리 사용량이 크다 - 하이퍼파라미터 튜닝 필요

* 쉬어가기 : 최적화 문제

최적화 문제에서 발생할 수 있는 여러 가지 현상들은 문제의 복잡성과 해결 방법의 선택에 큰 영향을 미친다.

구분	내용
지역 최소(Local Minima)	- 지역 최소값: 함수의 특정 영역에서 최소값 - 주변 영역에 비해 낮은 값을 가지지만, 전체 영역에서는 최소값이 아닐 수 있는 점 - 특히 복잡한 비선형 문제나 고차원 문제에서 지역 최소값에 갇히는 것은 큰 문제가 될 수 있다.
글로벌 최소(Global Minima)	- 글로벌 최소값: 함수 전체 영역에서 가장 낮은 값 - 최적화 문제의 전체 영역에서 가장 낮은 값을 가지는 점 - 많은 최적화 문제에서 글로벌 최소값을 찾는 것이 목표이다.
안장점 (Saddle Points)	- 함수의 특정 지점에서 해당 지점의 주변에 비해 함수 값이 어떤 방향에서는 더 크고, 다른 방향에서는 더 작은 특징을 가진 점 → 안장점은 최대점도 아니고 최소점도 아닌 중간적인 성질을 가진다. - 안장점에서는 함수의 gradient (기울기 벡터) = 0 - 방향에 따른 함수 값의 변화: 안장점에서는 어떤 방향으로 이동하면 함수 값이 증가하고, 다른 방향으로 이동하면 감소 → 이는 마치 안장의 모양과 유사 - 안장점은 최적화 과정에서 알고리즘이 이러한 점에서 멈추거나 느려질 수 있기 때문에 주의가 필요하다. - 고차원 최적화 문제에서는 안장점이 지역 최소값보다 훨씬 더 흔하게 발생하며, 최적화 과정에서 큰 장애물이 될 수 있다.
과적합 (Overfitting)	- 모델이 학습 데이터에 지나치게 잘 맞춰져서 새로운 데이터에 대한 일반화 능력이 떨어지는 현상 - 이는 최적화 과정에서 너무 많은 변수를 고려하거나, 모델이 너무 복잡할 때 발생
차원의 저주 (Curse of Dimensionality)	- 고차원 공간에서 데이터를 분석하고 이해하는 데 있어 발생하는 여러 문제들을 총칭 - 고차원에서는 데이터가 희소해지고, 이로 인해 최적화가 더 어려워질 수 있다.
수렴 문제 (Convergence Issues)	- 최적화 알고리즘이 최적점에 도달하지 못하고 멈추거나, 너무 느리게 진행되는 문제 → 학습률 설정, 알고리즘의 선택, 비효율적인 gradient 계산 등 다양한 원인에 의해 발생할 수 있다.

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020
<https://github.com/dreamquark-ai/tabnet>

Background

*tabular data 분석은 주로 Tree-based 모델 사용(Xgboost나 lightGBM..)
 비정형 데이터 (이미지나 텍스트, 오디오) : 주로 딥러닝이 활용
 → 왜 정형 데이터에서는 아직 Tree-based 모델 방법론들이 우세한 걸까요?*

- Decision Tree-based 장점 + DNN(gradient boosting) 기법들의 장점을 이용한 모델
- 딥러닝에도 Tabular data에 특화된 모델
- 기존 딥러닝 모델들은 이미지, 텍스트, 음성 등 다양한 비정형 데이터 영역에서 매우 우수한 성능
- 정형 데이터의 경우 : 주로 Tree기반의 앙상블 모델들이 주로 사용

1. 정형 데이터에 대해서는 기존의 decision tree-based gradient boosting과 같은 모델에 비해 신경망 모델은 성능이 떨어진다. → 두 구조의 장점을 모두 갖는 TabNet을 제안
2. feature selection 리소스 경감
3. interpretability (설명 가능)가 가능한 신경망 모델
 : 순차적인 어텐션(Sequential Attention)을 사용하여 각 의사 결정 단계에서 추론할 feature를 선택하여 학습 능력이 가장 두드러진 기능에 사용 → 설명력을 높이고 학습효과 향상

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

Tree- based 앙상블 모델들이 Deep Learning 모델보다 우수한 이유

- (1) their representation power for decision manifolds with approximately hyperplane boundaries that are commonly observed for tabular data.
 - (2) decision tree-based approaches are easy to develop and fast to train
 - (3) They are highly-interpretable in their basic form (e.g. by tracking decision nodes and edges) and various interpretability techniques have been shown to be effective for their ensemble form.
- CNN or MLP may not be the best fit for tabular data decision manifolds due to being vastly overparametrized – the lack of appropriate inductive bias often causes them to fail to find robust solutions for tabular decision manifolds.

정형데이터 : manifolds with approximately hyperplane boundaries

- 이미지(사진) : 각 픽셀들(Feature)이 강한 Correlation을 가지는 특성
- 텍스트나 음성 : 각 Feature가 Sequential하게 이어지는 특성 존재

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

Tabular data에 딥러닝 모델을 사용하는 것은 가치 있다.

(1) 훈련 데이터가 매우 많아지면, 계산 비용은 증가하지만, 성능은 개선

(2) Multi-modal Learning

Tabular 데이터와 이미지(텍스트) 등 다른 데이터 타입을 학습에 함께 사용 가능

(3) Tree 기반에서 필수적인 Feature Engineering과 같은 단계를 크게 요구하지 않음.

Feature Engineering 고민

(4) Streaming 데이터로부터의 학습이 용이

Tree 기반의 모델들은 데이터의 분기를 통해 Global한 통계적 정보를 이용해야 하므로 스트리밍 학습은 어렵다는 큰 단점이 존재 → 딥러닝 모델은 그런 학습에 유연

(5) 딥러닝 End-to-End모델 : Domain adaptation, Generative modeling, Semi-supervised learning과 같은 가치있는 Application이 가능하다는 장점

스트리밍 데이터

- 수천 개의 데이터 원본에서 연속적으로 생성되는 데이터
- 모바일이나 웹 애플리케이션을 사용하는 고객이 생성하는 로그 파일, 전자 상거래 구매, 게임 내 플레이어 활동, 소셜 네트워크의 정보, 주식 거래소, 지리공간 서비스등 다양한 데이터가 포함

end-to-end learning

전처리 → 변수 선택 → 모델링의 과정이 한번에 이루어지는 것

파이프라인 네트워크란 전체 네트워크를 이루는 부분적인 네트워크

end-to-end trainable neural network

- 모델의 모든 매개변수가 하나의 손실함수에 대해 동시에 훈련되는 경로가 가능한 네트워크

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

기존 연구 (한계점)

- 1) 인공 신경망의 Block을, 자동적인 의사결정나무로 표현하면 중복으로 인해 비효율적인 학습이 됨.
- 2) Soft (neural) decision tree는 미분가능한 결정함수를 사용하였으나 Feature selection 기능을 상실하게 됨.
- 3) Feature들의 결합과 모델의 복잡도를 줄이는 방법이 제안→ 성능 향상이 제한적, 해석 가능성을 고려하지 않음.

본 연구 Contribution (저자 주장)

- (1) **Gradient descent-based optimization**으로 학습하기 때문에, **별도의 전처리가 없이, raw 데이터를 입력으로 사용할 수 있고, Gradient-descent 기반 최적화를 사용하여 End-to-End learning을 가능하게 하였음.**
- (2) **성능과 해석력을 향상시키기 위하여**, TabNet은 Sequential attention mechanism을 사용하여 각 의사결정에서 어떤 feature를 사용할지를 선택→ Feature selection은 instance-wise하게 입력 각각마다 다르게 수행됨.
- (3) 해석력의 관점에서 입력 Feature의 중요도와 Feature들이 어떻게 결합되었는지를 시각화한 local한 해석력과, 학습된 모델에서 각 입력 Feature들이 얼마나 자주 결합되었는지의 Global한 해석력을 제시함.
- (4) 여러 데이터셋에서 기존의 정형 분류, 회귀 모델들보다 성능의 우수성을 가짐

Self-supervised learning = Unsupervised pre-training + Supervised fine-tuning

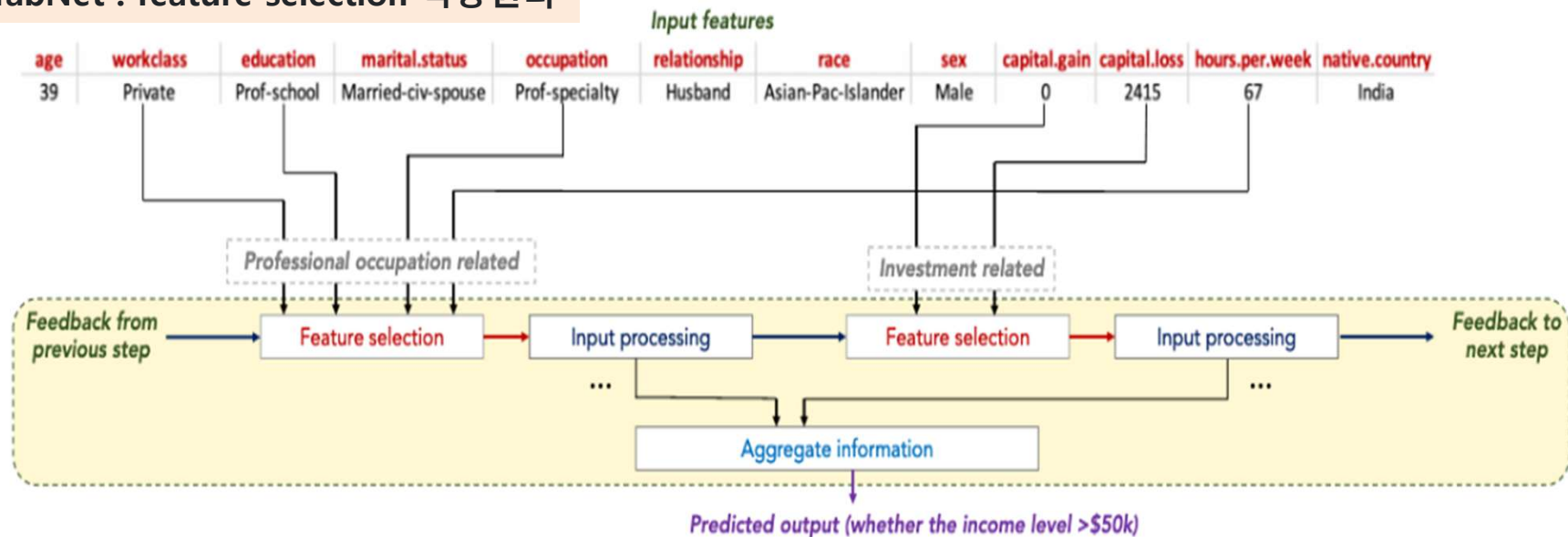
- (5) 데이터 전처리 과정 생략

- Feature importance로 Global explainability를 확인 가능, Mask값으로 Local explainability 확인
→ 각 instance별 step별 feature 영향도 확인 가능한 장점
- 전처리를 해야 모델 성능 개선 효과

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

TabNet : feature selection 작동원리



TabNet의 주요 컨셉

input feature에 대해 훈련 가능한 마스크(Trainable Mask)로 Sparse Feature selection을 수행하는 것

Sparse Feature selection 통해서 학습 효율 증대 + 설명력 증대

Feature selection mask 학습하도록 변수를 지정 , 로 Sparse selection이 가능하도록 활성화함수 사용(Sparsemax)

TabNet의 Feature selection은 특정 feature들만 선택하는 것이 아니라, 마치 Linear Regression처럼 각 feature에 가중치를 부여하는 것 설명 가능한(explainable) 모델이 되는 것

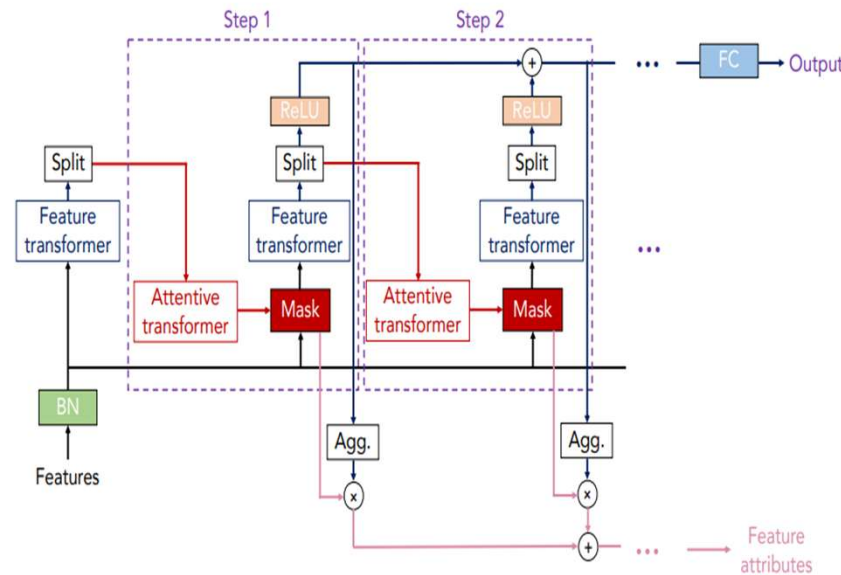
TabNet

TabNet 구조 : AutoEncoder 구조

Encoder-Decoder- 결측값들을 예측할 수 있는 Autoencoder 구조

OVERALL ARCHITECTURE

Encoder



BatchNorm

BatchNorm레이어 : Tabular 데이터는 데이터 변환 (Min, St) 등Normalization을 BatchNorm 레이어를 사용하여 대체

Decision Step 구성, Step 내의 두가지 블록 존재

- **Feature transformer** 블록 : 임베딩을 수행(=인코딩 수행)
- **Attentive transformer** 블록 : trainable Mask를 생성

입력부분과 Step1~N으로 구분

1. 입력부분

Tabular 데이터 = Numerical + Categorical Feature (원-핫 인코딩)

각 단계마다 Feature transformer와 Attentive transformer, Feature masking으로 구성

2. Split block : Feature transformer로부터 나온 representation을 두 개로 나누어, 하나는 ReLU를 통과 후 최종 output 보내주고, 나머지는 다음 Attentive transformer → 그 후, Feature를 selection하는 Mask block은 각 Step에서 Feature가 작동하는 것에 대한 Insight를 제공할 수 있고, Agg(egate) Block을 통해 궁극적으로는 어떤 Feature가 중요한지 판단.

•Mask의 3가지 용도

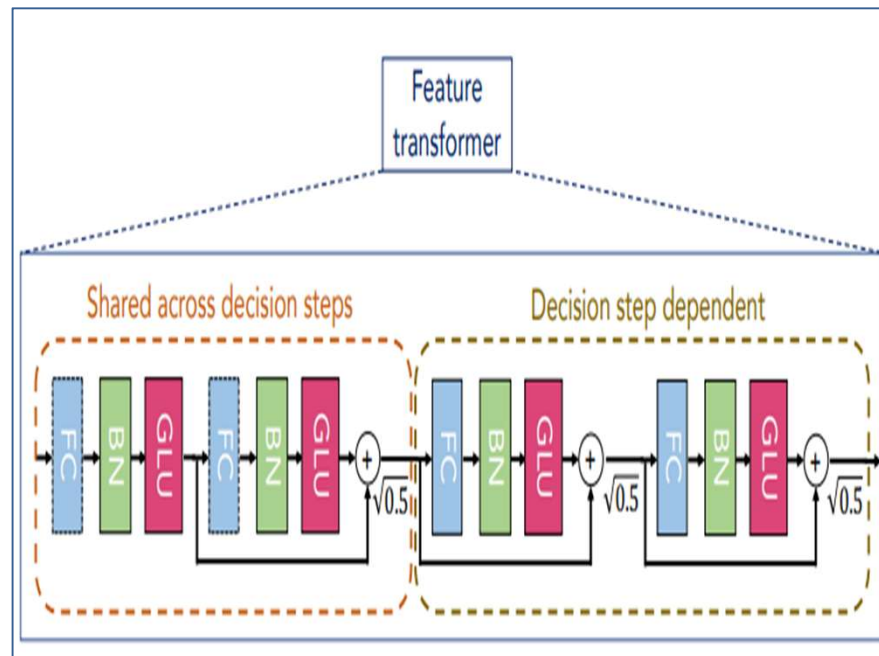
- 1) Feature Importance 계산
- 2) 이전 Step의 Feature에 곱하여 Masked Feature(다음 step에서 input feature) 생성
- 3) 다음 Step의 Mask에서 적용할 Prior scale term 계산

•Masked Feature는 다음 Step의 Input feature가 되며, 이전 step에서 사용되었던 Mask의 정보를 피드백하기에 Feature의 재사용 빈도를 제어할 수 있다.

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

Decoder

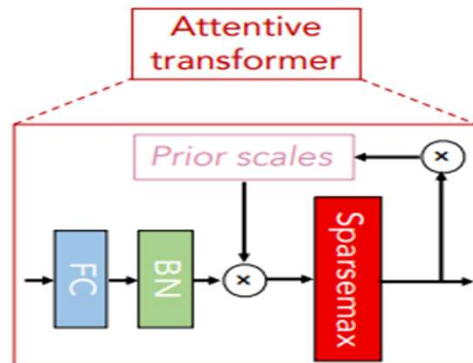


- FC-BN-GLU (Gated Linear Unit) 를 4번 반복 구조
- 앞 2개의 GLU 블록들을 묶어서 shared 블록
- 마지막 2개의 GLU 블록들을 묶어서 decision block
- → 각 GLU 블록 간에는 Skip Connection을 적용하여 vanishing gradient 현상 개선
- 앞 2개의 Block은 모든 decision step에서 공유되고
- 뒤 2개의 Block은 해당 decision step에서만 사용
- GLU는 아래와 같이 어떤 Linear Mapping을 통해 나온 결과물을 정확히 반(A, B)으로 나누어 A는 Residual connection, B는 Sigmoid function을 거친 후 Element-wise로 계산한 것

$$v([A \ B]) = A \otimes \sigma(B)$$

GLU Notation

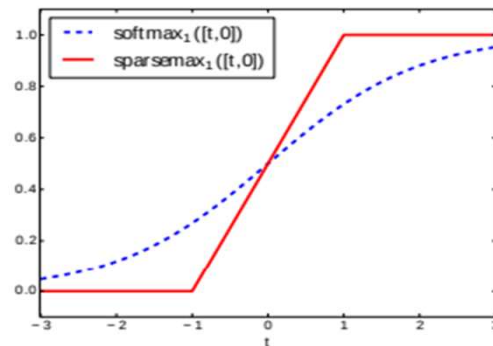
TabNet



$$M[i] = \text{sparsemax}(P[i-1] \cdot h_i(a[i-1])).$$

$$P[i] = \prod_{j=1}^i (\gamma - M[j]),$$

Sparsemax activation 함수



Softmax의 sparse한 버전 : Sparse한 데이터셋에 적용했을 때, 좋은 성능을 보인 normalization 기법

- Prior scale은 이전 decision step들에서 각 feature가 얼마나 많이 사용되었는지를 집계 (Agg block)한 정보
- 이전 단계들의 중요한 Feature를 selection하기 위해 Sparse Mask를 학습
- Masking을 통해 decision step 과정에서 학습에 큰 영향을 미치지않는 변수들은 영향력을 축소해야 함 →. 그러한 Mask를 구하기 위해 Attentive transformer를 사용

- **M[i]** : i번째 Mask , Sparsemax라는 normalization 수행
→ 이는 각 decision step에서 가장 두드러진 Feature를 선택할 수 있는 기법
- **Sparsemax** : Softmax에 sparsity(희소성)를 강화한 활성화 함수로 미분이 가능
→ Sparse한 데이터셋에 적용했을 때, 좋은 성능을 보인 normalization 기법
- **P[i]** 이전 decision step로부터 처리된 Feature(a[i-1])과 Mask들의 영향을 고려하여 새로운 Mask를 만들겠다는 것!! → (감마-이전 마스크)들의 곱으로 표현
- 감마 : relaxation parameter로 감마가 1인 경우 Feature가 하나의 decision step에서만 사용되도록 강제하고, 감마가 커질수록 여러 decision step에서 사용되도록 하는 Hyper-parameter

TabNet

TabNet: Attentive Interpretable Tabular Learning : Sercan O. Arık, Tomas Pfister "Google Cloud AI 2020

<i>Model</i>	<i>Test accuracy (%)</i>
XGBoost*	89.34*
LightGBM*	89.28*
CatBoost*	85.14*
AutoML Tables (2 node hours)**	94.56**
AutoML Tables (5 node hours)**	94.95**
AutoML Tables (10 node hours)**	96.67**
AutoML Tables (30 node hours)**	96.93**
TabNet	96.99

<i>Model</i>	<i>Test accuracy (%)</i>
Decision tree*	50.0*
Multi layer perceptron*	50.0*
Deep neural decision tree*	65.1*
TabNet	99.3
Rule-based	100.0

CONCLUSIONS

We propose TabNet, a novel deep learning architecture for tabular learning

pip install pytorch_tabnet

```
from pytorch_tabnet.tab_model import
TabNetRegressor
```

CTGAN 및 TabNet 기법을 활용한 불균형 정형 데이터 이진분류 모델링 개발 : 2022 공학학위

텍스트 마이닝 (Text Mining)

텍스트 마이닝

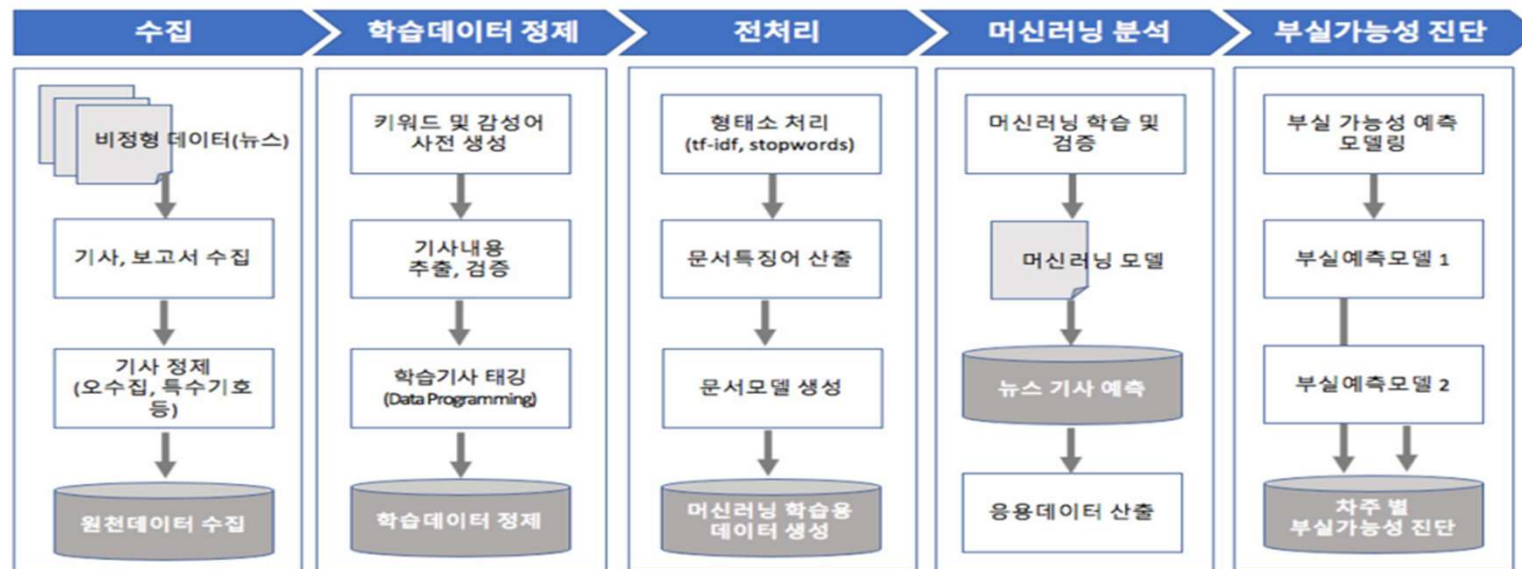
비/반정형 텍스트 데이터에서 자연어 처리 기술에 기반하여 유용한 정보를 추출, 가공하는 것을 목적으로 하는 기술

(가) 텍스트 마이닝 주요 알고리즘 TF-IDF(Term Frequency – Inverse Document Frequency)

여러 문서로 이루어진 문서군이 있을 때 어떤 단어가 특정 문서 내에서 얼마나 중요한 것인지를 나타내는 통계적 수치

(나) TF-IDF의 활용 특정한 단어가 문서 내에서 얼마나 자주 등장하는지를 나타내는 값을 통계적 수치로 계산한 것으로, 값이 높을수록 문서에서 중요하다고 판단

활용 내용 순위 결정 문서의 핵심어를 추출하거나, 검색 엔진에서 검색 결과의 순위 결정 유사도 문서들 사이에서 비슷한 정보를 도출



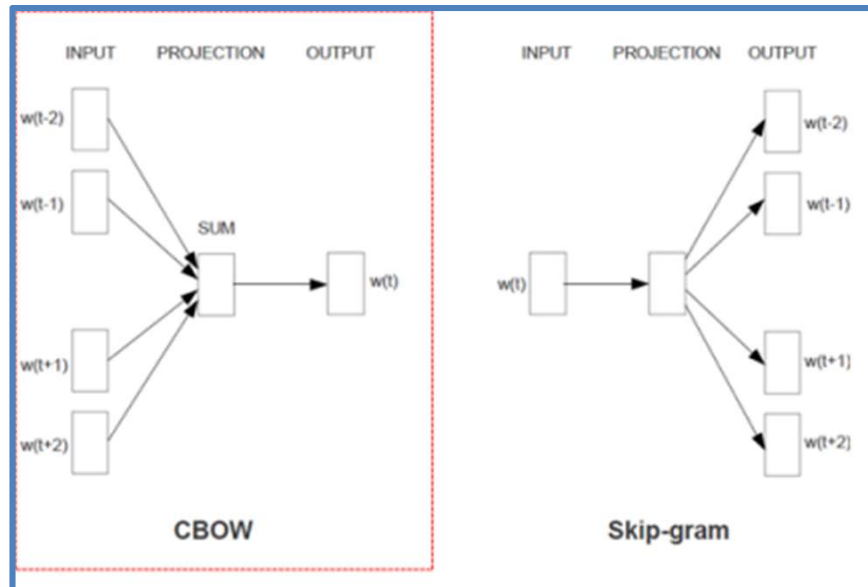
실제 사례 기반 비정형 데이터를 활용한 기업의 부실징후 예측에 관한 효용성 연구 2018

부도 연관어 : 자연어처리 관련

'Word2vec'

단어들 간의 연관된 규칙을 찾아서 각 단어의 관계를 계량적으로 산출하는 방법론으로서, 각 단어 간의 앞 뒤 관계를 보고 근접도를 벡터의 형태로 계산하는 알고리즘

'Word2vec'은 사전적으로 학습시키는 단계를 수행하지 않으므로 '비지도 학습' 기반의 인공지능(머신러닝)의 일종
단어간의 관계에 대한 정확한 벡터를 산출하기 위해서는 분석 대상이 되는 대규모의 텍스트 데이터 문서(corpus) 데이터베이스를 필요



CBOW

여러 단어로부터 한 단어를 추정하는 방법으로서, 주로 주변 단어로부터 목적이 되는 한 개의 단어를 찾는 과정에 활용-CBOW 는 상대적으로 작은 Data-set 일 때도 효과적으로 동작하고 추정 속도도 빠른 것으로 알려져 있다.

Skip-gram

한 개의 단어로 연관되는 여러 단어를 예측할 경우 활용한다. 예를 들어, 어떠한 단어가 현재 나타났을 때 향후 어떤 단어가 나타날 지를 추정하는 것을 목적으로 하는 경우 사용

부도 연관어 : 자연어처리 관련

'부도 관련 기사 비율'을 측정
기간 별로 전체 기사 중 부도와 관련된 기사의 비중을 산출하고, 이 비율이 높게 나타날 경우 이를 사전적인 '부도'의 징후로 판단하여 부도 예측에 활용하는 것이다

$$\text{부도 기사 비율}_{it} = \frac{\text{부도 관련 기사 수}}{\text{총 정상 기사 수}}, \quad i = \text{기업}, t = \text{분석 기간(월간, 부도발생 기준 직전 각 12개월)}$$

부도 유사 단어(1): '부도' 단어와 'Word2vec' 유사도 상위 20 개 단어

부도 유사 단어(2): '부도' 와 '상장폐지' 단어와 동시 'Word2vec' 유사도 상위 20 개 단어

부도 유사 단어(3): '회생', '공시', '자금', '횡령', '증자', '채권단', '워크아웃'

(논문참조)

- 빅데이터와 인공지능 기법을 이용한 기업 부도예측 연구 :최정원* 오세경** 장재원
- 최정원·한호선·이미영·안준모(2015): 부도가 발생한 기업의 뉴스 텍스트 데이터를 텍스트마이닝 기법으로 분석하여 기업 부도예측의 가능성을 시도

자연어처리 관련

ETRI, 최첨단 한국어 언어모델 '코버트(KorBERT)'

- 한국전자통신연구원(ETRI)은 최첨단 한국어 언어모델 '코버트(KorBERT)'를 2019년 공개
- 구글의 언어표현 방법을 기반으로 더 많은 한국어 데이터를 넣어 만든 언어모델과 교착어 특성까지 반영해 만든 언어모델.
- 전처리 과정에서 형태소를 분석한 언어모델
- 한국어에 최적화된 학습 파라미터
- 방대한 데이터 기반 등

<https://ksp.etri.re.kr/ksp/>

경제.금융 : 텍스트 마이닝

빅데이터를 활용한 경제데이터 분석 사례 및 방법론 연구

- 2019. 10.

- 금융통화위원회 회의록 분석

금통위 의사록 분석을 통해 파악할 수 있는 한가지 유용한 정보는 금리인상과 인하에 대한 예측으로 소위 매파와 비둘기파의 의견을 구분하고 우세를 비교하는 것임(한국은행, 2017)

- 하나금융경영연구소(2019): 공공 데이터를 활용해 서울 직장인의 통근 방법 및 소요 시간의 변화 등 전반적인 출퇴근 관련 트렌드를 분석하고 시사점을 제시

금융시계열 데이터에 대한 교차검증(CV)

Cross Validation

- **in-sample testing** : training data set 이용하여 종속변수값을 얼마나 잘 예측하였는지를 나타내는 성능
- **out-of-sample testing / cross validation**
→ 학습에 사용하지 않은 표본 데이터 집합의 종속변수 값을 얼마나 잘 예측 하는가를 성능 평가하는 것

train_test_split 교차검증을 통해 비교적 **일반화**(generalization)된 모델을 선택

교차검증 목적

- : 보유하고 있는 **데이터의 크기가 작아서** 검증 데이터 세트를 분리해내기가 여의치 않으면 교차 유효성 기법을 사용
- : 머신러닝 알고리즘의 일반화 오차를 알아내어 과적합(overfitting)을 막는 것
- : 데이터 일반구조를 알아내어 새로운 특징을 보고 미래를 예측할 수 있게 된다.

고정적인 training set으로 모델을 만드는 경우 overfitting(과적합) 가능성!!!!

Holdout Cross Validation

- 특정비율로 1회 분할
- 일반적으로 사용하는 데이터셋을 train과 validation으로 나누어 사용하는 것
- 보통 데이터셋을 8(7) : 2(3)로 나누어 많이 사용

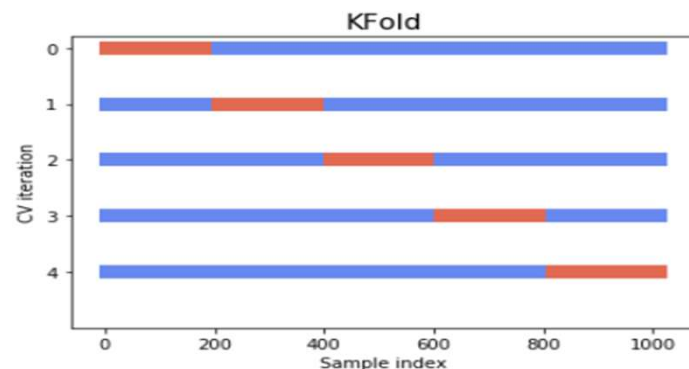
금융시계열 데이터에 대한 교차검증(CV)

K-fold cross validation 과정

1. 전체 데이터셋을 Training Set과 Test Set 분할
2. Training Set를 Training Set + Validation Set으로 사용하기 위해 k개 fold로 구분
→ **K-Fold**는 모든 샘플을 동일한 크기 folds라고 하는 샘플 그룹
3. 전체 데이터를 K개로 등분한 (K -fold) 후에 (K-1)개의 fold를 train data로 분할하고 나머지 1개를 test data로 사용, 이 과정을 반복함으로써 train 및 test data를 교차 변경하는 방법론 → data 별도 구분없이 k번 test하는 결과
4. 총 k 개의 성능 결과가 나오며, **이 k개의 평균을 해당 학습 모델 성능으로 평가**

(목적) 반복적인 중첩 평가 과정을 통하여 1회 데이터 세트 분할을 통한 모델 성능 평가보다 **안정적인 모델 성능** 평가 결과 기대

→ 교차검증 수행을 통하여 최적의 모델링 방법을 선택한 뒤에는 해당 모델링 방법을 이용하여 훈련 데이터 세트를 대상으로 모델링 과정을 다시 수행 반복. 이때 별도의 평가 데이터 세트로 결과를 평가하고 평가 결과가 기준치에 부합하거나 목적에 적합하다고 판단될 경우 분석 모델링 과정을 종료 → 최종 분석결과 제출



The best way to grasp the intuition behind blocked and time series splits is by visualizing them.

<https://hub.packtpub.com/cross-validation-strategies-for-time-series-forecasting-tutorial/>

금융시계열 데이터에 대한 교차검증(CV)

Cross Validation

https://scikit-learn.org/stable/modules/cross_validation.html

장점

1. test set에 모든 데이터가 최소 한번씩 들어가기 때문에 모델이 오버피팅을 방지하게 되어 일반화 됨
2. 분할을 한 번 했을 때보다 데이터를 더 효과적으로 사용해 더 정확한 모델을 만들 수 있음

단점

모델 훈련 및 평가 소요시간 증가 (반복 학습 횟수 증가)

K-fold cross validation 실패하는 가능성!!!

- 관측값이 IID 프로세스에서 추출되었다고 가정할 수 없다.
- 테스트 집합이 모델 개발 과정 프로세스에 여러 번 사용되었다. → 선택의 편향(Bias) 초래

금융시계열 데이터에 대한 교차검증(CV)

Cross Validation iterators

https://scikit-learn.org/stable/modules/cross_validation.html

데이터 세트의 구조나 특성은 각 상황에 따라 다양(분포 동일여부, 시계열 데이터, 데이터 그룹화) 각 상황에 맞는 반복자(iterators)를 사용하여야 한다.

1. data들이 독립적이고 동일한 분포 경우 (i.i.d)

1) K-Fold : 모든 샘플을 동일한 크기(가능한 경우)로 folds라고 하는 샘플 그룹

2) Repeated K-Fold

K-Fold를 여러 번 반복하는 것
각 반복마다 서로 다른 분할 발생

3) Leave One Out (LOO)

각 학습 세트는 하나의 샘플을 제외한 모든 샘플을 추출하여 생성
제외된 sample이 test set → data 수가 적은 경우
데이터의 낭비를 막기 위해서 사용

4) Leave P Out (LPO)

Leave P Out은 전체 데이터 세트에서 p개의 샘플을 제거하여 가능한 모든 훈련/검증 데이터 세트를 생성

5) Shuffle & Split

- 사용자가 정의한 수의 독립적인 훈련/테스트 data set 분할 생성
- 샘플을 먼저 섞은 다음 훈련/검증 데이터 세트를 나눈다. random_state 값을 정해두고 결과를 재현하도록 제어 가능

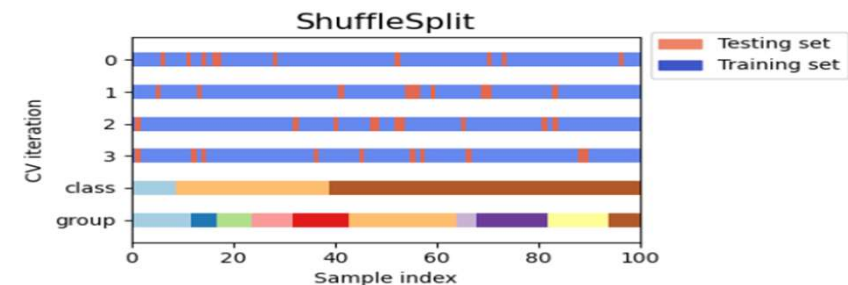
```
from sklearn.model_selection import KFold
```

```
from sklearn.model_selection import RepeatedKFold
```

```
from sklearn.model_selection import LeaveOneOut
```

```
from sklearn.model_selection import LeavePOut
```

```
from sklearn.model_selection import ShuffleSplit
```



금융시계열 데이터에 대한 교차검증(CV)

Cross Validation iterators

https://scikit-learn.org/stable/modules/cross_validation.html

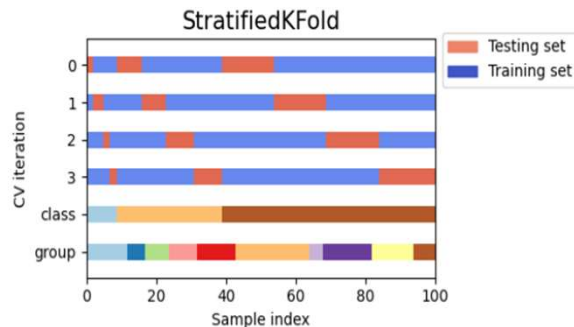
2. data의 분포가 다른 경우

1) Stratified K-Fold : 층화 추출 교차검증

`from sklearn.model_selection import StratifiedKFold, KFold`

층화된 folds를 반환하는 기존 K-Fold의 변형된 방식

각 집합에는 전체 집합과 거의 동일하게 클래스의 표본 비율이 포함된다 → 데이터 불균형(Data Imbalance)의 경우 사용.

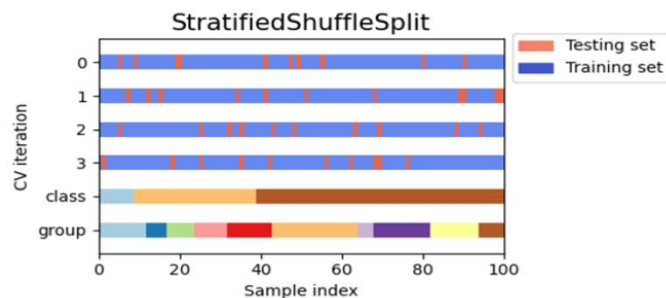


2) Stratified Shuffle & Split: 임의분할

`from sklearn.model_selection import StratifiedShuffleSplit`

층화된 분할을 반환하는 Shuffle & Split의 변형

전체 집합에서와 같이 각 대상 클래스에 대해 동일한 비율을 보존하여 분할을 생성



금융시계열 데이터에 대한 교차검증(CV)

Cross Validation iterators

https://scikit-learn.org/stable/modules/cross_validation.html

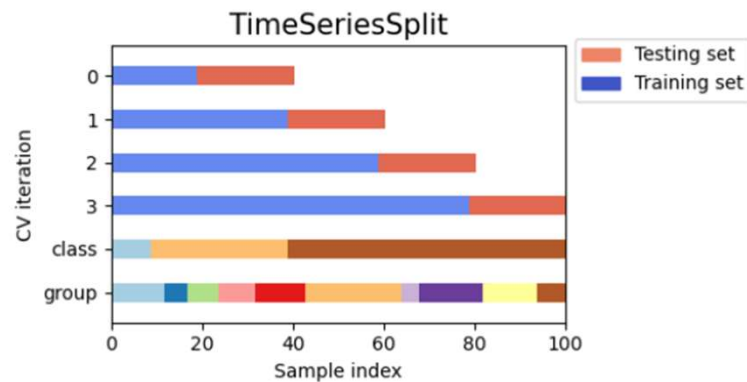
3. data가 시계열 데이터의 경우 분포

Time Series Split

첫 번째 fold는 훈련 data set, 두 번째 fold는 test set로 split

연속적 훈련 데이터 세트는 그 이전의 훈련 및 검증 데이터를 포함한 상위 집합

→ 해당 검증 방법은 고정 시간 간격으로 관측된 시계열 데이터 샘플을 C.V할 때 사용



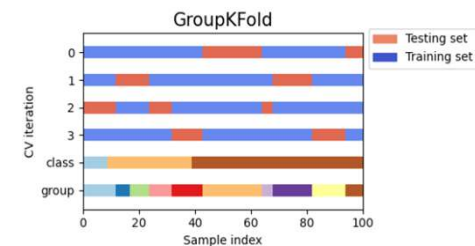
```
from sklearn.model_selection import TimeSeriesSplit
```

4. 데이터가 그룹화되어 있는 경우

1) Group K-Fold

동일한 그룹이 훈련 및 검증 데이터 셋에 동시에 포함되지 않도록 하는 방법

```
from sklearn.model_selection import GroupKFold
```



금융시계열 데이터에 대한 교차검증(CV)

Cross Validation iterators

https://scikit-learn.org/stable/modules/cross_validation.html

4. 데이터가 그룹화되어 있는 경우

2) Leave One Group Out

Leave One Out (LOO)에서는 하나의 데이터를 검증 데이터로 남겨두고 나머지 데이터로 학습 데이터를 구성하는데 반해, **Leave One Group Out** 하나의 그룹을 남겨두고 나머지 그룹으로 학습 데이터를 구성

```
from sklearn.model_selection import LeaveOneGroupOut
```

3) Leave P Groups Out

Leave One Group Out와 유사하며 P개의 그룹을 남겨두고 (removes samples related to P groups for each training/test set) 나머지 그룹을 훈련 데이터로 구성 ($P > 1$)

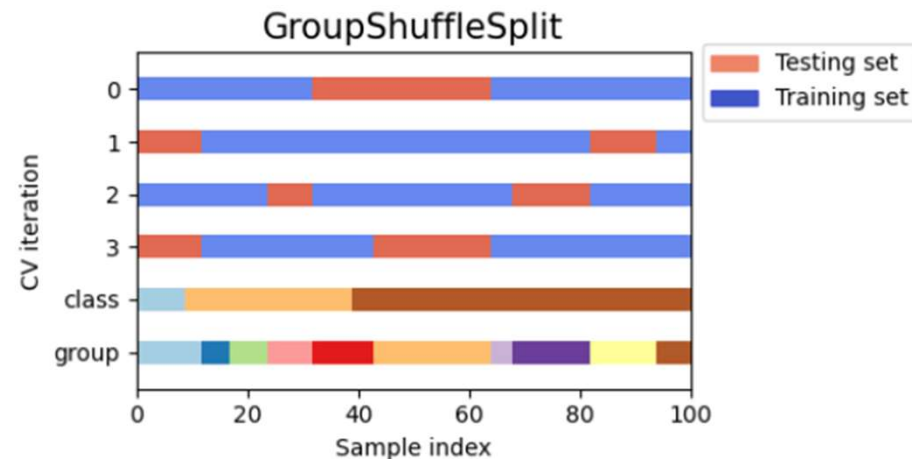
```
from sklearn.model_selection import LeavePGroupsOut
```

4) Group Shuffle Split

Shuffle & Split과 **Leave P Groups Out** 의 조합

```
from sklearn.model_selection import GroupShuffleSplit
```

각 분할에 대해 그룹의 하위 집합이 보류되는 랜덤화된 분할 시퀀스를 생성 (generates a sequence of randomized partitions)



* StratifiedKFold 파라미터

```
from sklearn.model_selection import StratifiedKFold skfolds =
StratifiedKFold(n_splits=3, random_state=42, shuffle=True)
```

파라미터	내용
n_splits	▪ K-fold 교차검증을 몇 번 수행할 것인가를 지정하는 것 ▪ 기본값= 5 ▪ 주로 5 또는 10으로 많이 설정
random syate	작업을 여러번 해도 폴드가 바뀌지 않아 결과값을 유지할 수 있다. 반드시 필요한 것은 아니지만 똑같은 작업을 여러번 하는 경우가 많기 때문에 보통은 아무 숫자로나 설정한다.
shuffle	▪ 디폴트값 = False ▪ True로 설정을 하면 fold를 나누기 전에 데이터셋을 무작위로 섞게 된

금융시계열 데이터에 대한 교차검증 방법

•Bengio, Yoshua, and Yves Grandvalet. 2004. "No Unbiased Estimator of the Variance of K-Fold Cross-Validation." J. Mach. Learn. Res. 5 (December). JMLR.org: 1089–1105.

k -겹 교차검증의 k 값이 바뀔 때 추정값의 Bias와 variance 영향을 미치는지 분석 비교

→ k -fold 교차검증방법이 분산을 모든 분포에서 통용되는 편향없이 추정하는 보편적인 방법은 없다. (Bengio and Grandvalet, 2004)

→ 우리는 분산과 편향 사이에 적절한 절충점으로 보이는 최선의 지점을 찾는 것에 관심이 있다.

편향-분산 트레이드 오프(Bias - variance trade off)

지도학습 알고리즘이 트레이닝 셋의 범위를 넘어 지나치게 일반화 하는 것을 예방하기 위해 두 종류의 오차(편향, 분산)를 최소화 할 때 겪는 문제

- 편향은 학습 알고리즘에서 잘못된 가정을 했을 때 발생하는 오차
- → 높은 편향값 → 과소적합(under fitting) 문제 발생
- 분산은 트레이닝 셋에 내재된 작은 변동(fluctuation) 때문에 발생하는 오차
- → 높은 분산값 → 큰 노이즈까지 모델링에 포함 → 과적합(over fitting) 문제를 발생

모델을 선택할 때(model selection), 트레이닝 데이터의 규칙을 정확하게 포착하는 것 뿐만이 아니라, 보이지 않는 범위에 대해서 일반화(generalization)까지 하는 것이 이상적 → 둘을 동시에 완전히 성취하는 것은 사실상 불가능

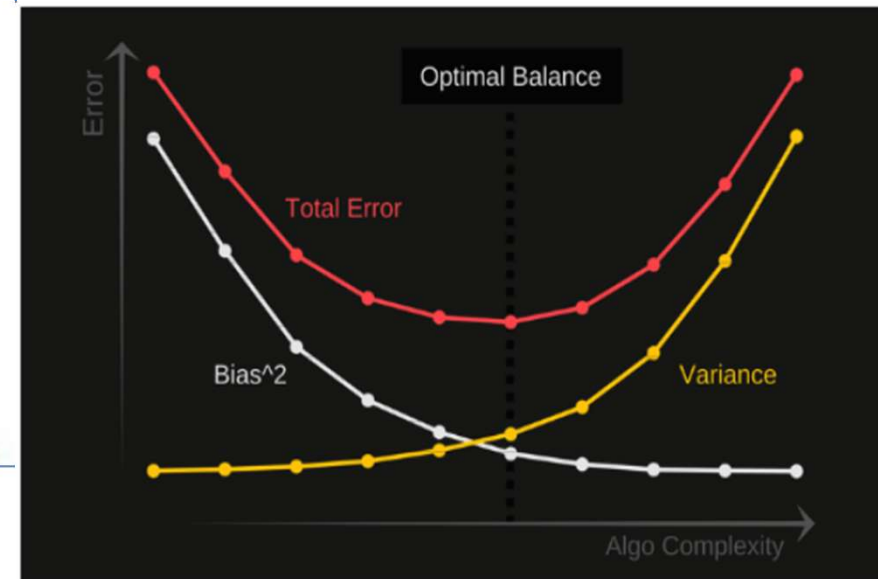
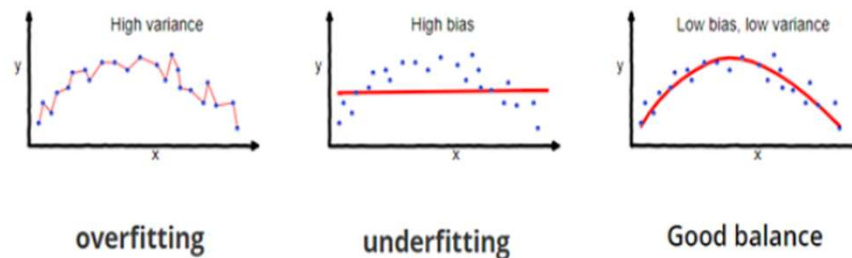
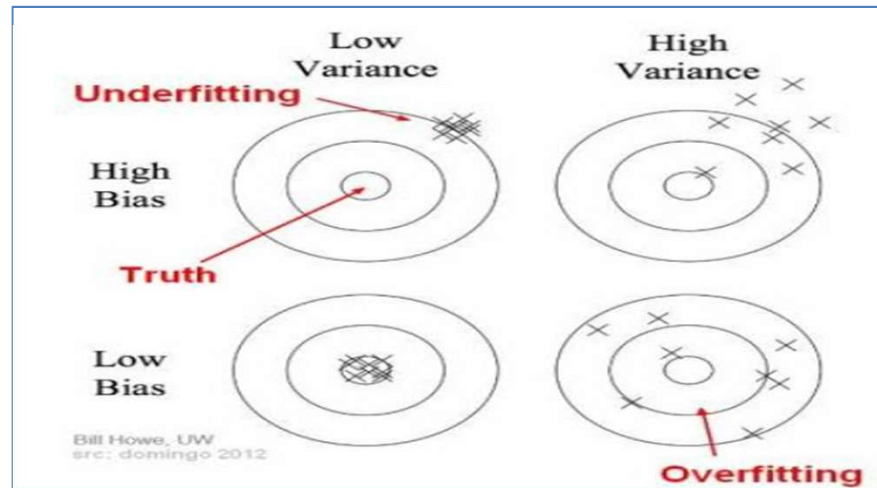
→ 고분산 학습 알고리즘은 트레이닝 셋을 잘 표현하기는 하지만, 지나치게 큰 노이즈나 아예 부적절한 트레이닝 데이터까지 과적합(overfitting)할 위험이 있다.

→ 반대로 고편향 학습 알고리즘은 과적합(overfitting) 문제가 거의 없는 단순한 모델을 제시하지만 트레이닝 데이터로부터 중요한 규칙성을 제대로 포착하지 못하는 과소적합(underfitting) 문제가 발생한다.

금융시계열 데이터에 대한 교차검증 방법

- High bias → Underfitting
- High variance → Overfitting

$$\text{error}(X) = \text{noise}(X) + \text{bias}(X) + \text{variance}(X)$$



An optimal balance of bias and variance would never overfit or underfit the model.

Therefore understanding bias and variance is critical for understanding the behavior of prediction models.

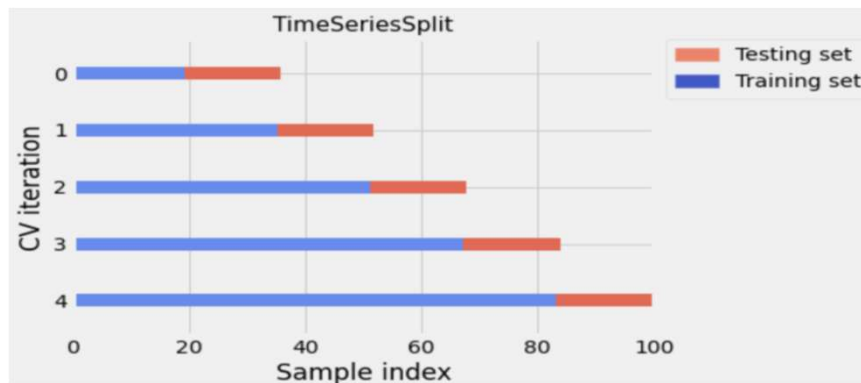
<https://www.quora.com/What-is-an-intuitive-explanation-for-bias-variance-tradeoff>

금융시계열 데이터에 대한 교차검증 방법(대안방법)

walk -forward cross-validation

Training data는 검증용보다 항상 앞선 시간으로 할당되었다는 것

→ 모델링을 통해 미래의 데이터를 예측하려는 것이기 때문에 과거 시간을 훈련용으로 쓰는 것



검증세트가 항상 training set 앞서 있다는 조건으로

각 반복에서 훈련 세트를 두 겹으로 나누는 것

첫 번째 반복에서는 후보 모델을 1월부터 3월까지의 종가
로 학습하고 4월의 데이터를 검증하고,

다음 반복에서는 1월부터 4월까지의 데이터를 학습하고
5월의 데이터를 검증하는 식으로 끝까지 계속 진행

plotting with a simple array data 시계열이라 가정

TimeSeriesSplit이라는 sklearn 라이브러리 함수를 사용 입력 변수는 몇 번의 반복으로 교차검증을 할 것인지 정하는 것이다. n_split으로 표기하고 5를 주었다. 이것은 5개의 훈련용+검증용 데이터셋을 만든다는 것 의미

```
from sklearn.model_selection import TimeSeriesSplit
```

```
from matplotlib.patches import Patch
```

```
import matplotlib.pyplot as plt
```

```
XX = np.arange(100)
```

```
n_split = 5
```

```
tscv = TimeSeriesSplit(n_splits=n_split)
```

```
plot_cv_indices(tscv, XX, n_splits=n_split)
```

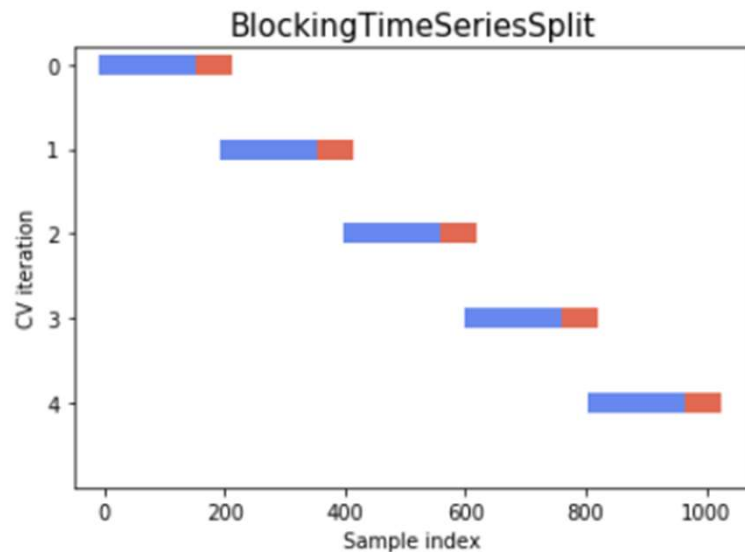
<https://hub.packtpub.com/cross-validation-strategies-for-time-series-forecasting-tutorial/>

금융시계열 데이터에 대한 교차검증 방법(대안방법)

Blocking walk forward cross-validation

훈련용과 검증용 데이터의 크기를 모든 교차검증 횟수에서 고정시키는 것.

→지원하는 라이브러리가 없기 때문에 generator 클래스를 정의 (코드는 아래 링크 참고)



The first is between the training and validation folds in order to prevent the model from observing lag values which are used twice, once as a regressor and another as a response. The second is between the folds used at each iteration in order to prevent the model from memorizing patterns from an iteration to the next.

<https://hub.packtpub.com/cross-validation-strategies-for-time-series-forecasting-tutorial/>

금융시계열 데이터에 대한 교차검증 방법(대안방법)



Pt 3 11 면 소개

<https://www.orie.cornell.edu/faculty-directory/marcos-lopez-de-prado>
<https://www.truepositive.com/>

금융 데이터들 i.i.d가 아니면 문제 발생

•Purge

•테스팅 셋에 포함된 라벨의 시간과 겹치는 라벨을 갖고 있는 Train set의 모든 observation을 purge하는 것

•Embargo

Test Set 뒤에 바로 뒤에 Train Set이 올 경우, Serial Correlation으로 인해 Test Set의 Information이 Train set에 Leaking

Advances in Financial Machine Learning

파이썬 코드 소개되어 있음.

<https://github.com/boyboi86/AFML>

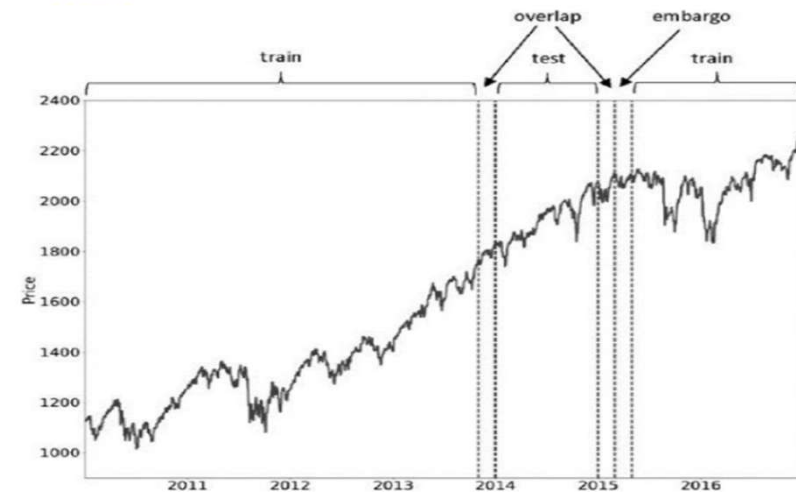
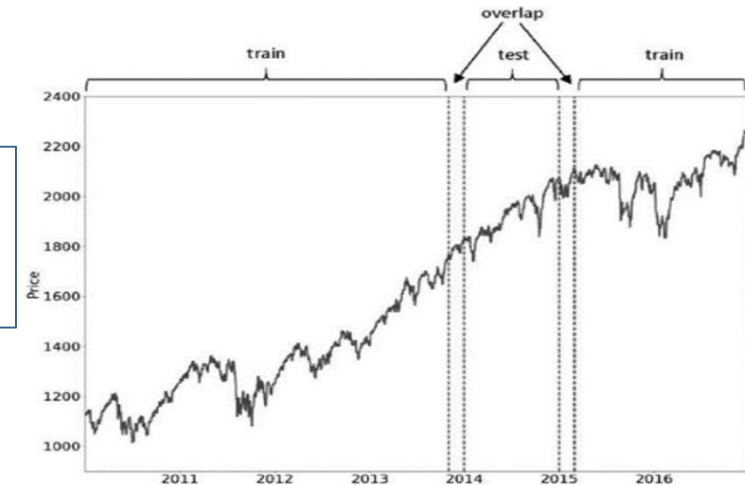


FIGURE 7.3 Embargo of post-test train observations

(Classification Model) 빅데이터 모델 평가 검증하기

Confusion Matrix

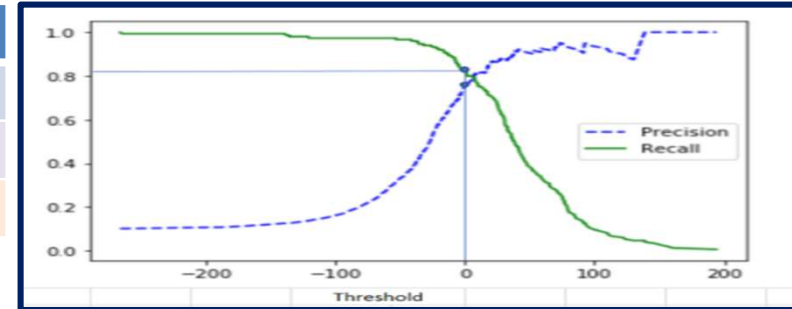
		실제 값	
		Positive	Negative
예측 값	참	True Positive (TP)	False Positive (FP)
	거짓	False Negative (FN)	True Negative (TN)

- True Positive(TP) : 실제 True인 정답을 True라고 예측 (정답)
- False Positive(FP) : 실제 False인 정답을 True라고 예측 (오답)
→ 1종오류
- False Negative(FN) : 실제 True인 정답을 False라고 예측 (오답)
→ 2종오류
- True Negative(TN) : 실제 False인 정답을 False라고 예측 (정답)

구분	계산	의미
정확도 (Accuracy)	$(TP+TN) / (TP+TN+FP+FN)$	실제 분류 범주를 정확하게 예측한 비율 (전체 예측에서 참 긍정(TP)과 참 부정(TN)이 차지하는 비율) (1-오류율)
재현율(Recall) = 민감도 (Sensitivity)	$(TP) / (TP+FN)$	실제로 '긍정(positive)'인 범주 중에서 '긍정'으로 올바르게 예측 (True Positive)한 비율 → 참 긍정비율
정밀도 (Precision)	$(TP) / (TP+FP)$	긍정(positive)'으로 예측한 비율 중에서 실제로 '긍정'(True Positive)인 비율
특이도 (Specificity)	$(TN) / (TN+FP)$	실제로 '부정(negative)'인 범주 중에서 '부정'으로 올바르게 예측 (True Negative)한 비율 →
Fall-out (1-특이도)	$(FP) / (TN + FP)$	실제 false data인데 True라고 잘못 예측(분류)한 것 → 거짓긍정비율
F1 score	$\frac{2X(\text{정밀도} \times \text{재현율})}{\text{정밀도} + \text{재현율}}$	정밀도와 재현율의 조화평균(Harmonic mean) → 정밀도와 재현율 중 하나가 매우 낮아서 0에 가까운 값일 때 지표에 잘 반영되도록 하기 위하여 산술평균 대신에 조화평균을 사용

(Classification Model) 빅데이터 모델 평가 검증하기

		실제 값	
		Positive	Negative
예측 값	참	True Positive (TP)	False Positive (FP)
	거짓	False Negative (FN)	True Negative (TN)

**Accuracy : 직관적인 평가지표**

→ 고려해야할 점이 domain의 bias(편향)
 예시) 두 집단의 데이터의 불균형이 심할 때 (domain 불균형)
 정상기업 1,000개, 부도기업 10개 인 경우 정상기업 예측력은 높
 지만 부도기업 예측력은 낮아진다.→ 보완할 지표 필요

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

- Precision은 모델(예측)의 입장에서 정답을 맞춘 경우 : $\left(\frac{TP}{TP+FP}\right)$
- Recall 은 실제 정답의 입장에서 True 라고 맞춘 경우 : $\left(\frac{TP}{TP+FN}\right)$

위의 경우에는 확실하지 않은 경우 예측을 하지 않고 보류하여 FP의 경우를 줄여, Precision을 편법으로 끌어 올리는 경
 우 (제2종오류와 제 1종오류 trade - off)

→ Precision 과 Recall 의 trade-off 관계

Precision과 Recall의 Trade-off를 결정하는 수치는 Threshold(임계치 :Positive 예측값을 결정하는 확률의 기준)
 보통 Binary classification의 경우 확률이 0.5를 넘어가면 Positive로, 0.5 이하면 Negative로 예측하는데, 이때 사용하는
 값이 Threshold(0.5)

→ Threshold를 낮추면 Positive로 더 많이 예측하니 **Precision 감소 + Recall 증가**

→ Threshold를 높이면, Positive로 예측하는 값이 더 적어져서 **Precision 증가 + Recall 감소**

→ 위 두 값을 조화평균한 값이 필요 !!!!!

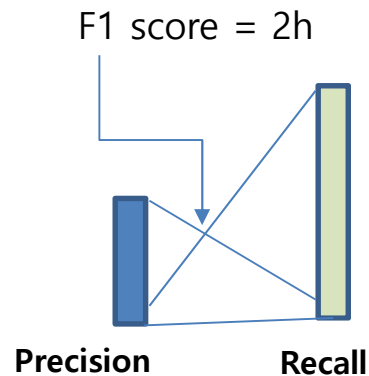
(Classification Model) 빅데이터 모델 평가 검증하기**Balance data** : 부도기업 100, 정상기업 100**imbalanced data** : 부도기업 50, 정상기업 1,000

		실제 값	
		부도기업	정상기업
예측 값	부도기업	90 (TP)	5 (FP)
	정상기업	10 (FN)	95 (TN)

		실제 값	
		부도기업	정상기업
예측 값	부도기업	10 (TP)	50 (FP)
	정상기업	40 (FN)	950 (TN)

- Accuracy = $\frac{90+9}{90+10+5+95} = \frac{185}{200} = 0.925$
- Precision = $\frac{90}{90+5} = 0.947$
- Recall = $\frac{90}{90+10} = 0.9$
- F1 score = $\frac{2(0.947 \times 0.9)}{0.947+0.9} = \frac{1.7046}{1.847} = 0.923$

- Accuracy = $\frac{10+9}{10+40+50+95} = \frac{960}{1,050} = 0.914$
- Precision = $\frac{10}{10+50} = 0.167$
- Recall = $\frac{10}{50} = 0.2$
- F1 score = $\frac{2(0.167 \times 0.2)}{0.167+0.2} = \frac{0.0668}{0.367} = 0.182$



- Precision = 0.2
- Recall = 0.4
- F1 score = $\frac{2(0.2 \times 0.4)}{0.2+0.4} = 0.267$
- 좌측 교차점 $0.267/2 = 0.1335$

부도기업에 대한 분류 성능저하 !!!

큰 비중이 끼치는 bias가 감소한다고 볼 수 있음

(Classification Model) 빅데이터 모델 평가 검증하기**Balance data** : 부도기업 100, 정상기업 100

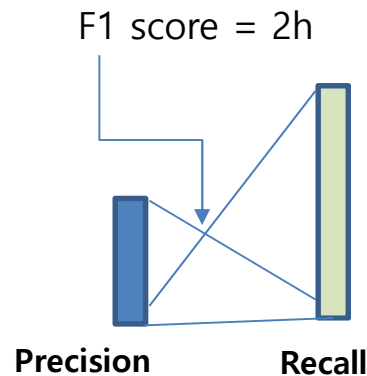
		실제 값	
		부도기업	정상기업
예측 값	부도기업	90 (TP)	5 (FP)
	정상기업	10 (FN)	95 (TN)

imbalanced data : 부도기업 50, 정상기업 1,000

		실제 값	
		부도기업	정상기업
예측 값	부도기업	45 (TP)	50 (FP)
	정상기업	5 (FN)	950 (TN)

- Accuracy = $\frac{90+9}{90+10+5+95} = \frac{185}{200} = 0.925$
- Precision = $\frac{90}{90+5} = 0.947$
- Recall = $\frac{90}{100} = 0.9$
- F1 score = $\frac{2(0.947 \times 0.9)}{0.947+0.9} = \frac{1.7046}{1.847} = 0.923$

- Accuracy = $\frac{45+950}{10+40+50+950} = \frac{995}{1,050} = 0.947$
- Precision = $\frac{45}{45+50} = 0.474$
- Recall = $\frac{45}{50} = 0.9$
- F1 score = $\frac{2(0.474 \times 0.9)}{0.474+0.9} = \frac{0.8532}{1.374} = 0.621$

imbalanced data 경우 : F1 score 성과 평가

- Precision = 0.2
- Recall = 0.4
- F1 score = $\frac{2(0.2 \times 0.4)}{0.2+0.4} = 0.267$
- 좌측 교차점 $0.267/2 = 0.1335$

(Classification Model) 빅데이터 모델 평가 검증하기

F - Measure https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

Van Rijsbergen, C.J., Information Retrieval, 2nd Edition, London: Butterworths, 1979.

Van Rijsbergen의 F-measure는 정보 검색 및 분류 문제에서 성능 평가를 위해 사용되는 중요한 지표(1979)

$$\text{F-score} = \frac{1}{\alpha \frac{1}{\text{Precision}} + (1-\alpha) \frac{1}{\text{Recall}}} = \frac{(1+\beta^2)(\text{Precision} * \text{Recall})}{\beta^2 (\text{Precision} + \text{Recall})}$$

→ 위 식에서 베타가 1인 경우 : F1 score

→ Recall에 더 비중을 두고자 하는 경우 Beta값을 1.0보다 크게 입력하여 계산

→ Beta값이 1.0보다 작으면 Precision에 비중을 두고 계산하는 연구에서 적용

- 산술평균 = $\frac{a+b}{2}$, 기하평균 = $\sqrt{ab}$
 - 조화평균 = $\frac{2ab}{a+b}$ → 조화평균 = $\frac{\text{기하평균}^2}{\text{산술평균}}$
 - **산술평균 ≥ 기하평균 ≥ 조화평균**
- $\frac{a+b}{2} \geq \sqrt{ab} \geq \frac{2ab}{a+b}$

F1 score : 조화평균 → Precision, Recall 중요성 동일 관점 (베타 = 1)

기하평균(G) 한 경우 : $G = \sqrt{\text{Precision} \times \text{Recall}}$

$$\text{G-mean} = \sqrt{\text{Specificity} \times \text{Recall}}$$

$$\text{G-mean} = \sqrt{(1-\alpha) \times (1-\beta)}$$

$$\text{G-mean} = \sqrt{\frac{TN}{TN+FP} \times \frac{TP}{TP+FN}}$$

G-mean

Kubat와 Matwin (1997) 제안

(계산) **Specificity와 Recall의 기하평균(Geometric mean)으로 계산**

→ 따라서 다수집단(정상기업)이 정확하게 분류되었지만 소수집단(부도기업)에 대한 예측력이 낮다면 이들의 기하평균값인 G-mean 값은 낮게 된다. 즉, **소수집단의 정분류율이 좋을수록** G-mean 값도 높아지는 경향 → 제1종 오류와 제2종 오류 중 성능이 나쁜쪽에 더 가중치를 둬

(Classification Model) 빅데이터 모델 평가 검증하기

F - Measure

$$\text{F-score} = \frac{(1+\beta^2)(\text{Precision} * \text{Recall})}{\beta^2 (\text{Precision} + \text{Recall})}$$

→ 위 식에서 베타가 1인 경우 : F1 score

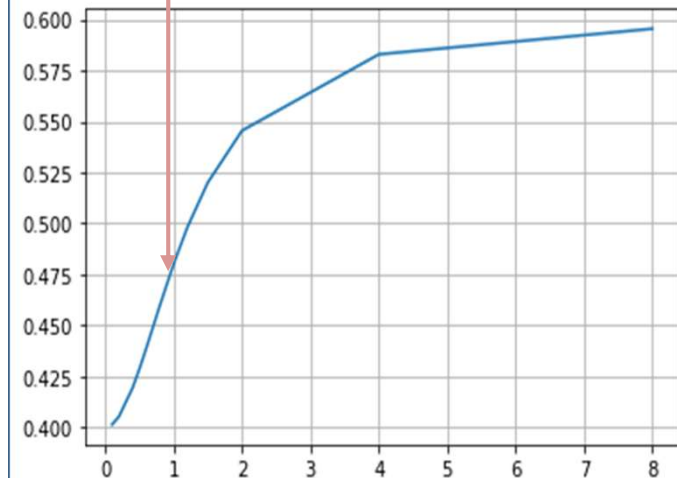
→ Recall에 더 비중을 두고자 하는 경우 : Beta값을 1.0보다 크게 입력하여 계산

→ Precision에 비중을 두고 하는 경우 : Beta값이 1.0보다 작은 값 대입

```
def f1_score(precision, recall):
    return 2.0 / (1/precision + 1/recall)

def fbeta_score(precision, recall, beta=1.0):
    return (1.0 + (beta ** 2)) * (precision * recall) / (((beta ** 2) *
precision) + recall )
import matplotlib.pyplot as plt
precision = 0.4
recall    = 0.6
betas     = [0.1, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0, 1.2, 1.5, 2.0, 4.0, 8.0] # 베타= 1
f1 score
results   = []
for beta in betas:
    results.append(fbeta_score(precision, recall, beta))

plt.plot(betas, results)
plt.grid()
```



(Classification Model) : Multilabel classification**F - Measure****1. Macro-F1 score (macro avg)**

다중 클래스 분류 문제에서 모델의 성능을 평가하는 데 사용되는 지표

- 각 클래스별로 계산된 F1 Score의 평균을 구하여 모델의 전체 성능을 평가
- 클래스별/레이블별 F1-score의 평균으로 정의
- 라벨별 f1-score를 산술평균

Macro F1 Score 계산 과정

각 클래스에 대해 Precision과 Recall을 계산 → 각 클래스에 대해 F1 Score를 계산 → 모든 클래스에 대한 F1 Score의 평균

→ 0과 1사이의 값, 1에 가까울수록 우수

→ Macro-F1의 경우 모든 class의 값에 동등한 중요성 (동일 가중)을 부여

2. weighted F1 score 각 범주 (class)가 불균형 상태의 경우

라벨별 f1-score를 샘플 수(support)의 비중에 따라 가중평균한 값

3. micro average 전체 TP, FN, FP를 평균한 것

→ 다중 분류에서 정확도(acc)

(Classification Model) : Multilabel classification

Macro-F1 score (macro avg)

weighted F1 score

micro average

confusion matrix

True Positives (TP) and False Positives (FP), False Negatives (FN)

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \quad \text{f1-score}$$

	TP	FP	FN	Precision	Recall	F1score
사과	1	2	1	$0.33 (= \frac{1}{1+2})$	$0.5 (= \frac{1}{1+1})$	$0.4 (= \frac{2(0.33)(0.5)}{0.33+0.5})$
귤	1	2	2	0.33	0.33	$0.33 (= \frac{2(0.33)(0.33)}{0.33+0.33})$
배	3	1	2	0.75	0.6	$0.67 (= \frac{2(0.75)(0.6)}{0.75+0.6})$
계(15)	5	5	5			

		실제 값			
		사과	귤	배	계
(모델) 예측 값	사과	1	1	1	3
	귤	1	1	1	3
	배	0	1	3	4
	계	2	3	5	10

1. macro average

사과, 귤, 배 Label에 대한 f1-score 를 평균한 값

$$\text{macro avg} = \frac{0.4+0.33+0.67}{3} = 0.47$$

2. weighted average

사과, 귤, 배 Label의 개수에 비례하게 가중치를 부여하여 F1-score를 평균한 값

$$\text{weighted average} = 0.4(\frac{2}{10}) + 0.33(\frac{3}{10}) + 0.67(\frac{5}{10}) = 0.51$$

3. micro average

전체 샘플에 대해 종합적으로 f1-score 계산

$$\begin{aligned} \text{precision} &= \frac{5}{5+5} = 0.5 \\ \text{recall} &= \frac{5}{5+5} = 0.5 \\ \text{Micro} &= 0.5 \end{aligned}$$

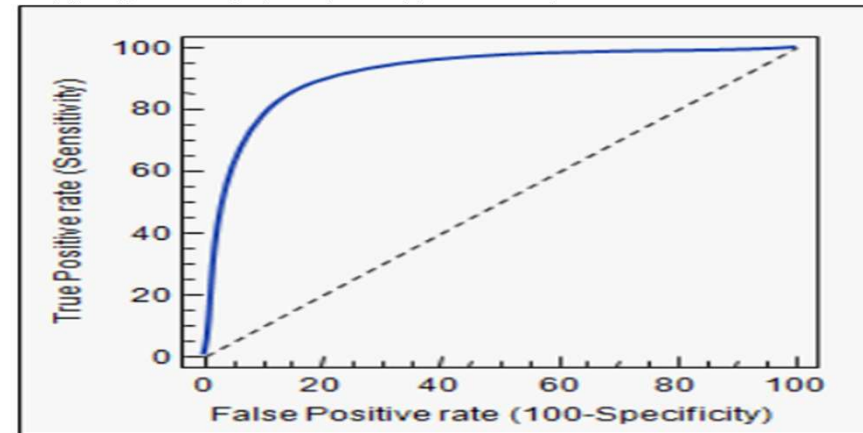
f1-score (micro) 는 accuracy 값과 동일 !!**어떤 성과지표를 기준으로 볼 것인가?**

- macro average : 모든 Label이 유사한 중요도를 가지는 경우
- weighted average : 샘플이 많은 Label에 중요도를 더 두고 싶은 경우
- micro average : Label에 상관없이 전체적인 성능을 평가하고 싶은 경우

(Classification Model) 빅데이터 모델 평가 검증하기

ROC(Receiver Operating Characteristic) Curve

- Threshold 이 달라짐에 따라 분류모델의 판단결과의 성능을 보여주는 그래프
- X축에 (1-특이도) y축 민감도 간 관계를 표현한 그래프
- → 거짓 긍정비율(FP Rate)과 참 긍정비율(TP Rate)간의 관계
- → 이 둘을 동시에 높이는 것은 어려운 일이다. 따라서 이 두 값을 어떠한 기준을 가지고 연속적으로 변경해가면서 측정한 후, 이들 간의 관계를 한눈에 볼 수 있게 한 것



FPR(False Positive Rate, 1 – 특이도): X축

특이도 (Specificity)	$(TN) / (TN+FP)$	실제로 '부정(negative)'인 범주 중에서 '부정'으로 올바르게 예측 (True Negative)한 비율 →
Fall-out (1-특이도)	$(FP) / (TN + FP)$	실제 false data인데 True라고 잘못 예측(분류)한 것 → 거짓긍정비율

TPR(True Positive Rate, 민감도): Y축

민감도 (Sensitivity) 재현율(Recall)	$(TP) / (TP+FN)$	실제로 '긍정(positive)'인 범주 중에서 '긍정'으로 올바르게 예측 (True Positive)한 비율 → 참 긍정비율,
----------------------------------	------------------	---

(Classification Model) 빅데이터 모델 평가 검증하기

AUC(Area Under Curve)

- (정의) ROC 곡선을 통한 정량적인 성능분석 결과 값 $\rightarrow$ ROC 곡선 아래 면적
- 쌍(pairwise) 해석 : AUC는 양성(Positive) 샘플과 음성(Negative) 샘플을 임의로 한 쌍 뽑았을 때, 모델이 양성에 더 높은 점수를 줄 확률 $\rightarrow AUC = P(f(x+) > f(x-))$, $f(x)$: 예측 점수(score), $x+$: 양성, $x-$: 음성 샘플을 의미
- (수치 해석)
 - 0.5일 때는 무작위 분류기와 동일
 - 1.0에 가까울수록 완벽한 분류기

(의의)

불균형 데이터셋에서도 모델의 판별 능력을 비교적 편향 없이 평가할 수 있어, 클래스 비율이 크게 다른 문제에서 유용

쌍(pairwise) 해석"

AUC를 양성(positive) 샘플과 음성(negative) 샘플을 한 쌍으로 묶어 생각하는 관점이다.

1. 샘플 쌍 구성

1. 데이터 전체에서 임의로 한 개의 양성 샘플 $x+$ 과 한 개의 음성 샘플 $x-$ 을 뽑는다.

2. 모델 점수 비교

1. 모델이 각 샘플에 대해 매기는 점수 $f(x)$ 를 비교한다.
2. 만약 $f(x+) > f(x-)$ 이면 "올바른 순위"로 판단해 1점을 부여한다.
3. $f(x+) < f(x-)$ 이면 0점,
4. 같으면 0.5점을 줄 수도 있다.

3. 확률 해석

$$AUC = P(f(x+) > f(x-)) = \frac{\text{올바른 순위 쌍의 수}}{\text{전체 양성-음성 쌍의 수}}$$

$\rightarrow$ 즉 "랜덤으로 뽑은 양성·음성 한 쌍에서, 모델이 양성을 더 높게 평가할 확률"이 AUC이다.

$\rightarrow$ 모든 쌍에 대해 이 과정을 반복한 뒤 평균을 내면 AUC가 된다.

$\rightarrow$ 이렇게 "샘플 간 순위(순위)" 성능에 집중하기 때문에, 불균형 데이터셋에서도 모델이 얼마나 잘 "양성보다 음성을 낮게" 점수화하는지 객관적으로 평가할 수 있다.

(Classification Model) 빅데이터 모델 평가 검증하기

AUROC

- 모형에 의한 변별력 부분인 ROC곡선 아랫부분의 면적을 적분하여 구한 값
- 변별력이 우수한 모형은 부도위험이 높은 것으로 평가된 낮은 점수에서 정상기업들의 비율이 매우 낮고
- 부도기업들의 비율은 매우 높게 나타나야 하므로 모형의 변별력이 우수할수록 AUROC는 큰 값을 가질 것

$$AUROC = \sum_i \left[\frac{1}{2} Good_i \times Bad_i + (1 - cum\ Good_i) \times Bad_i \right]$$

(감염 진단 사례)

ROC 곡선의 면적(area under the ROC curve, AUC)은 0-1의 범위를 갖고 면적이 넓을수록 진단검사의 판별능력(discriminatory ability) 즉 정확도(accuracy)가 높다는 것을 의미한다.

AUC = 1.0은 민감도가 1이고 거짓긍정률(가양성률)이 0이므로 완벽한 검사(perfect test)를 의미

AUC = 0은 모든 검사 대상자에 대하여 환자군을 정상군으로, 정상군을 환자군으로 완벽하게 잘못 분류하는 상황

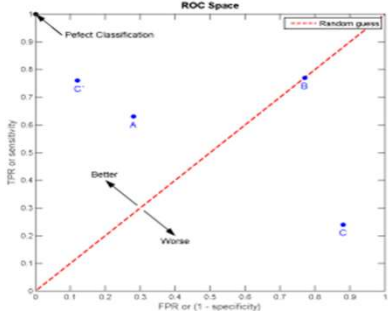
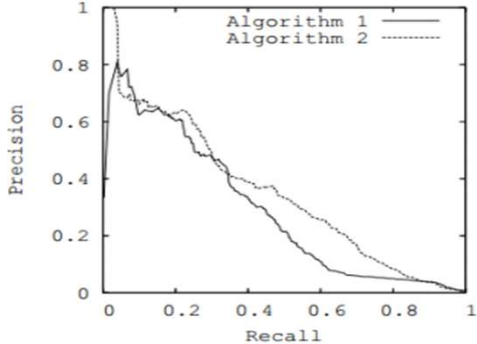
→ 실질적인 의미에서 AUC의 하한값은 AUC = 0.5인 경우

(일반적인 해석)

- $0.5 < AUC \leq 0.7$ 일 때 낮은 정확도(less),
- $0.7 < AUC \leq 0.9$ 일 때 중등도(moderate),
- $0.9 < AUC < 1.0$ 일 때 높은(high) 정확도

- AR값 : 0~ 1사이
- AUROC값 : 0.5~ 1사이 값

ROC AUC , PR AUC

ROC AUC	 Area under curve that FPR, TPR are expressed as X axis, Y axis.	$\text{X axis: } FPR = \frac{FP}{FP + TN}$ $\text{Y axis: } TPR = \frac{TP}{TP + FN}$
PR AUC	 Area under curve that Recall, Precision are expressed as X axis, Y axis.	X axis: Recall Y axis: Precision

자료 : 딥러닝 시계열 알고리즘 적용한 기업부도예측모형 유용성 검증* 차성재,강정석

PR AUC

average precision

recall의 변화에 따른 precision들을 다 모아서 평균을 낸다는 뜻,
recall에 따른 평균이기 때문

(Classification Model)CAP (cumulative accuracy profile) curve

CAP Curve(cumulative accuracy profile) 곡선

-Gini Curve, Power Curve, Lorenz Curve

부도여부에 대한 사전적 판정기준을 가정하지 않고 각 상태의 발생빈도를 파악하는 데에는 발생가능한 모든 판정기준 가운데 임의의 판정기준 하에서 발생하는 각 상태의 빈도를 나타내는 점들을 연결한 곡선을 이용한 분석이 이용

X축: 전체기업 중 부도로 예측된 기업이 차지하는 비중

Y축: 실제 부도기업 중 부도로 예측된 기업이 차지하는 비중



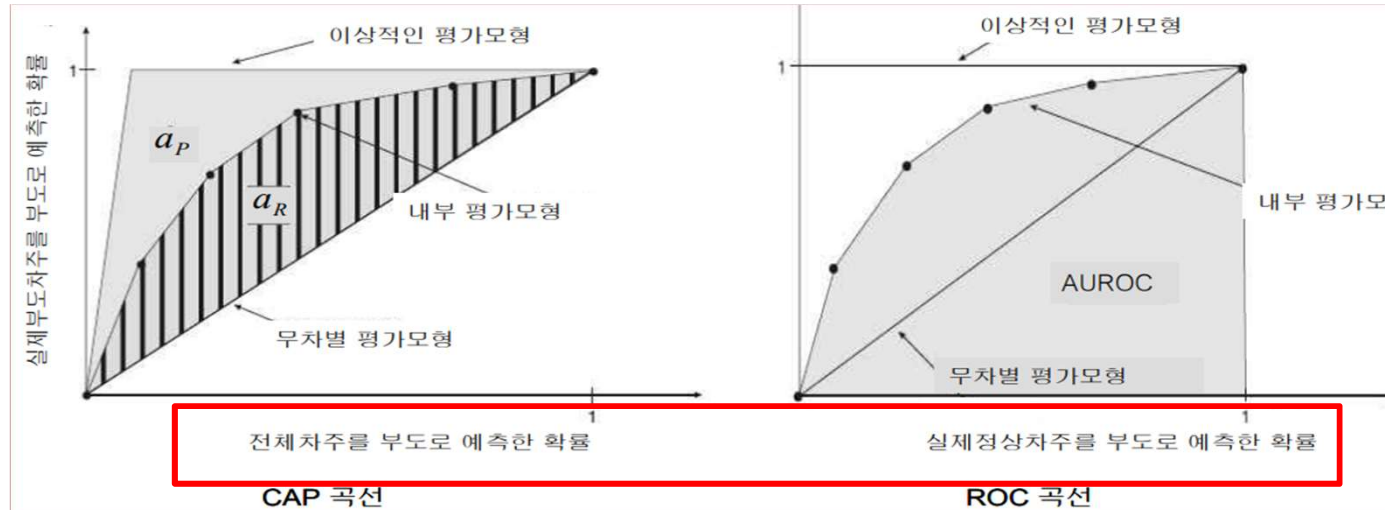
AR(accuracy ratio)

실제 정상기업들 중 부도로 예측된 기업이 차지하는 비중을 나타낸 곡선

→ ROC(receiver operating characteristics) curve



AUROC(area under ROC)



자료: 재무비율을 이용한 부도예측에 대한 연구 한국의 외부감사대상기업을 대상으로 박종원, 안성만

** 로짓 모형의 결과 값

두 집단의 표본으로 짝을 지어 Pairs를 생성

총 (정상기업 개수 × 부도기업 개수) 만큼의 Pairs가 생성 가능 이후 다음과 같은 과정을 거쳐 **Concordant, Discordant, Tied의 값을 산출**

▪ **Concordant(%)**

총 검증표본 중에서 표본내의 2개 집단에 대하여 정상을 정상으로, 부도를 부도로 옳게 분류한 총표본수에서 차지하는 비율(산술평균) → Concordant 값이 높을수록 정확도가 높다.

$$\rightarrow \left(\frac{\text{정상기업예측}}{\text{총정상기업수}} + \frac{\text{부도기업예측}}{\text{총부도기업수}} \right) / 2$$

→ 문제점 ?? (데이터의 불균형의 고려하지 못한다 : 문제점을 **보완한 값이 FI score**)

▪ **Discordant (%)**

총 검증표본 중에서 표본내의 2개 집단에 대하여 정상을 부도로, 부도를 정상으로 잘못 분류한 표본수가 총표본수에서 차지하는 비율 → Discordant 값이 낮을수록 분류 정확도 높다.

→ 제1종오류와 제2종오류의 합

▪ **Tied(%)**

총 검증표본 중에서 표본내의 2개 집단에 대하여 분류 정확도가 50%
(표본 수/ 총 검증표본 수)

* Cohen's Kappa 계수

- 코헨(1968)이 제안
- 두 관찰자 간의 측정 범주 값에 대한 평가 결과가 얼마나 일치(agreement) 하는지를 측정하는 방법
- **두 관찰자 간의 일치도를 Cohen's Kappa**
- 세 관찰자 이상의 일치도를 Fleiss Kappa
- **카파 상관계수 : 0~1 사이의 값**
- 카파계수가 0과 0.2 사이의 값을 가질 때 약간의 일치도를 갖는 것으로 판단
- 0.8 이상의 값을 가질 때 완벽한 일치도를 갖는 것으로 판단

Confusion Matrix

		실제 값	
		참	거짓
예측 값	참	True Positive (TP)	False Positive (FP)
	거짓	False Negative (FN)	True Negative (TN)

Guidelines of Landis and Koch

Kappa Statistic	Strength of Agreement
< 0	Poor
0 — 0.2	Slight
0.2 — 0.4	Fair
0.4 — 0.6	Moderate
0.6 — 0.8	Substantial
0.8 — 1	Almost perfect

사례 보기

https://ko.wikipedia.org/wiki/%EC%B9%B4%ED%8C%8C_%EC%83%81%EA%B4%80%EA%B3%84%EC%88%98

$$P(\bar{Y}) = \frac{(TP + FN) \times (TP + FP)}{(TP + TN + FP + FN)^2}$$

$$P(\bar{N}) = \frac{(TN + FN) \times (TN + FP)}{(TP + TN + FP + FN)^2}$$

$$P(E) = P(\bar{Y}) + P(\bar{N})$$

$$K = \frac{\text{Accuracy} - P(E)}{1 - P(E)}$$

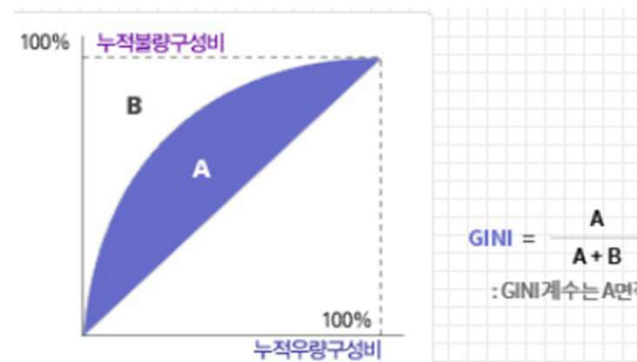
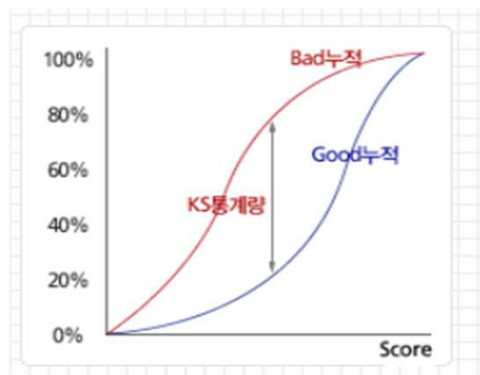
* 참고 (신용등급/점수 산정)

자료 : https://www.niceinfo.co.kr/creditrating/bi_score_4.nice

K-S 통계량 (Kolmogorov-Smirnov Statistics)

= (정상기업 각 등급별 누적확률분포 - 부도기업 각 등급별 누적확률분포) 변별력을 평가

•→ K-S 통계량이 40% 이상일 경우 양호한 수준이라고 할 수 있으며, 20% 미만이면 재검토 필요 모형



PSI (Population Stability Index)

- PSI는 모집단의 안정성을 나타내는 지수
- 기준시점 대비 현재 분포의 차이
- 수치가 클수록 모집단의 변화가 크다는 것을 의미

$$PSI = \sum_{\text{점수대별}} (\%O - \%E) \cdot \ln \frac{\%O}{\%E} \quad (\%E : \text{기준시점 구성비}, \%O : \text{현재 구성비})$$

AR (Accuracy Ratio)

AR은 신용등급에 따른 우량 누적분포와 불량 누적분포를 이용하여 모형의 변별력을 판별하는 지표

** 참고 (신용점수 산정)

information value:

특정 범위(구간)로 모수를 나누었을 때 해당 범위에서의 feature 선택에 사용되며 예측력 측정

<https://medium.com/henry-jia/how-to-score-your-credit-1c08dd73e2ed>

$$IV = \sum_{i=1}^k (Dist\ Good_i - Dist\ Bad_i) * WOE_i$$

Dist Good/Bad : 전체 조사건수에서 등급구간별로 비연체건, 연체건 비율을 의미

Information Value	예측력
0 to 0.02	무의미
0.02 to 0.1	낮은 예측
0.1 to 0.3	중간 예측
0.3 to 0.5	강한 예측
0.5 to 1	너무 강한 예측 (의심되는 수치)

Cramer's V

- ✓ 일반적으로 0.10을 이상 예측력이 있다고 봄.
- ✓ 카이제곱 통계량에 기반을 둔 값

WOE: Weights of Evidence

= 각 범주형 범주 그룹에 고유값 할당

$$\left[\ln \left(\frac{Distr\ Good}{Distr\ Bad} \right) \right] \times 100$$



$$Cramer's\ V = \sqrt{\frac{\chi^2}{n}}$$

(Regression Model) 빅데이터 모델 평가 검증하기

(1) Mean Absolute Error; MAE

- 잔차의 절대값들 평균을 나타내는 방법
- (+)와 (-) 잔차값이 다양하게 발생하더라도 잔차 간 상쇄효과를 예방
- 계산이 쉽고 이해가 용이
- underperformance or overperformance 구분 안됨
- Scale에 의존적 (실제값 100 → 예측값 110, 실제값 10,000 → 예측값 10,010)

잔차 (Residual)
표본의 회귀식에 의한 예측치와
관측치의 차이

$$MAE = \frac{1}{n} \sum_{y_i \in L} |y_i - \hat{y}_i|$$

(2) Mean Squared Error; MSE

- 잔차를 제곱하여 이를 합한 뒤 n으로 나누어 계산된 평균한 값
- underestimates or overestimates 구분 안됨
- 1미만의 에러는 더 작아지고, 1 이상의 에러는 더 커진다.
- MSE값은 항상 MAE보다 크다.

$$MSE = \frac{1}{n} \sum_{y_i \in L} (y_i - \hat{y}_i)^2$$

(3) Root Mean Square Error; RMSE

RMSE: MSE의 제곱근 값
→ MSE로 예측오차를 평가하게 될 때 그 수치가 커지는 것을 제곱근을 취함으로써 보정 한 값

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

(4) Mean Absolute Percentage Error; MAPE

- MAE를 비율(%)로 표현한 것
- 실제 종속변수 값 대비 예측오차 비율의 절대값 들을 평균한 값으로 실제 데이터에서 오차가 어느 정도의 비율로 발생했는지 확인하려는 방법
- 비율 변수이므로 MAE, MSE, RMSE에 비하여 비교에 용이
- 비율로 해석이 의미 있는 값에만 적용

$$MAPE = \frac{1}{n} \sum_{y_i \in L} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

빅데이터 모델 평가 검증하기 비교

구분	Supervised learning				Unsupervised learning
	Regression			Classification	
	MAE	MSE	RMSE		
지표	오차의 절대값 합의 평균	오차 제곱합에 대한 평균	MSE의 루트		Mutual information
장점	이상치 영향 적다→ 이상치가 있는 경우 사용	미분 가능 이상치까지 고려하여 일반화할 때	MSE 단점 해소		
단점	오차 상태 판단 불가 미분 불가능	이상치 영향			
비교	각 오차에 동일한 가중치를 부여 Loss 값이 낮게 나옴	모델 학습 불안정	각 오차가 다른 가중치를 갖는 것 Loss 값이 크게 나옴		

Thank you for listening

